

- [Packaging](#)

Translation(s): [English](#) - [Español](#) - [Português \(Brasil\)](#) - [Italiano](#) - [Svenska](#) - [Русский](#)



This is the **packaging portal**, for people who want to create new packages. For commonly-installed packages, see [Software](#). Or to install and remove packages, see [package management](#).

Contents

1. General guides

1. [Find your feet](#)
2. [Further reading](#)

2. More granular guides

1. [Language-specific guides](#)
2. [Topic-specific guides](#)
3. [Tool guides](#)
4. [Job guides](#)
5. [File guides](#)
6. [Working with other developers](#)

3. Training Sessions

4. External links

1. [See also](#)

5. Wiki pages

General guides

There are no shortcuts to learning good packaging practices - you can't just throw a trivial packager like [DebianPkg: equivs](#) at the problem and hope for the best.

The links in this section will help you gain a deep understanding of the problems you need to solve if you want to create or maintain a package.

Find your feet

The first step is to find a basic approach that works for you. [The Debian mentors FAQ](#) advises you to re-consider, clarifies why and how to start, and provides a short overview of the process.

New tools are invented every few years, and the best way to use them depends on the specific projects you want to package and the way you like to work. Here are some guides you can get inspiration from:

- [create a .deb for private use](#) shows how to make a basic package in 5 minutes
- a tutorial series from 2010-2011 helps you learn the theory:
 - [introduction to Debian packaging](#) explains packaging from scratch
 - [the building tutorial](#) explains applying changes to an existing package
 - [advanced building tips](#) covers some details that didn't fit in the other tutorials
- a tutorial series from 2025 by Otto Kekäläinen shows one good packaging workflow:
 -  [Creating Debian packages from upstream Git](#)
 -  [Debian source packages in git explained](#)
- [packaging with git](#) discusses common git-based workflows
- [DebianMan: the debconf developers guide](#) explains how to ask the user questions during installation
-  [Debian New Maintainers' Guide](#) covers debian source package fundamentals
 -  [Guide for Debian Maintainers](#) covers the same topic, but lets debmake handle some details

 Debian packaging works by example as much as by theory. Find well-maintained packages, and see how they do it!

Further reading

Once you've found a workflow you can live with, you can optimise it for your personal requirements. These pages might give you some ideas:

- [learn Debian Packaging in steps](#) takes you from new to advanced packaging levels (by the JavaScript team)
-  [Debian Packaging Tutorial](#) tells you what you really need to know about Debian packaging
-  [The Developers Reference](#) gives an overview of recommended procedures and available resources
-  [How will my package get into Debian?](#) walks you through the process of getting packages accepted by Debian
- [The packaging FAQ](#) answers some common questions
- [The upstream guide](#) advises upstream maintainers how to support packaging
- [Complex Web Apps](#) discusses creating Debian packages for complex web applications
- [setting up a Debian unstable system](#) provides some options for creating a clean package-building environment

These advanced pages explain how packaging works under the hood:

-  [the Debian Policy](#) - technical requirements that each package must satisfy
- [Projects/DebSrc3.0](#) - details about the "3.0 (quilt)" and "3.0 (native)" source package formats
- [Courses2005/BuildingWithoutHelper](#) - how to make a Debian package without using a helper
- [/HackingDependencies](#) - hacking dependencies
- [Diagrams](#) - packaging-related diagrams and sketches
-  [Debian Trends](#) - how packaging practices have evolved over time

Finally, if you'd like to see the development process for the packaging system itself:

- [PackageConfigUpgrade](#) - proposed way to smoothly handle configuration upgrades during package upgrades
- [DataPackages](#) - brainstorming about huge data packages

More granular guides

Once you're comfortable creating packages generally, you'll need to learn the tools and techniques for your particular problem.

Language-specific guides

Each of Debian's language-specific teams have their own policies and tools:

- [Emacs Lisp](#) team page
-  [Golang](#) packaging documentation
- [Haskell](#) team page
- [Java](#) packaging guide
 - see also [Guidelines for Packages Maintained on salsa.debian.org/java-team](#)
- [JavaScript](#) policy
 - [Node.js](#) packaging guide
-  [Lua](#) group page
- [Mono](#) packaging guide
- [OCaml](#) maintainers page
-  [Perl](#) packaging guide
- [Python](#) library style guide
- [Ruby](#) packaging guide
- [Rust](#) team page

 see also [a comparison of tools that create Debian packages](#)

Topic-specific guides

If your package addresses a specific topic, you may need to read information from people who have been there before:

- [Android tools](#) information
- Debian team guides
 - [Debian GNOME Team](#) guide
 - [KDE Team](#) home page
 - [Debian MATE Packaging Team](#) guide
 -  [Debian Med team](#) policy
 - [Debian Multimedia](#) packaging guide
 -  [Vim team](#) packaging guide
 -  [Debian Science](#) policy manual
 - other [packaging teams](#) may have guides
- [fonts](#) packaging policy
- [repackaging RPM source packages as .deb packages](#)
 - see also [using RPM in Debian](#)

Tool guides

You will probably need to use some of these:

- [pbuilder](#) or [sbuild](#) to create environments to build packages in
- [lintian](#), [piuparts](#), [autopkgtest](#) and [blhc](#) to debug your packages

You might also want to use some of these:

- [Packaging/ruby-team-meta-build](#) - build scripts used by ruby team, helps testing reverse dependencies easily
- [Quilt](#) for managing patches without a version control system
- [DebianPkg: devscripts](#) to make your life easier
- [DebianPkg: dh-make](#) to convert source archives into Debian package source
- [checkinstall](#) lets you build binary .deb packages from installation scripts (make install...)

Job guides

If you're trying to achieve a particular outcome:

- [rename a package](#)
- [transition users from one package to another](#)
- [let files from one package be temporarily replaced by files from another](#)
- [make a package for your config files](#)
- [make a private backport from unstable to stable](#)

- [make a backport suitable for backports.debian.org](#)
- [create a package from an Ubuntu PPA](#)
- [upload source packages without binary packages](#)
 - see also [Debian archive](#)
- [avoid keeping convenience copies of files from one project in another](#)
- [language-specific solutions for versioned -dev packages](#)
- [write a good package description](#)
- [manage differences between your package and upstream](#)

File guides

If you need help with a particular file in your package's `debian/` directory:

- [debian/changelog](#) - changes between versions of your package
- [debian/copyright](#) - copyright information about the upstream package
- [debian/patches](#) - debian-specific patches to the upstream code
- [debian/upstream](#) - files describing the upstream project
 - [debian/upstream/edam](#) - formal categorisation from the  [EDAM ontology](#)
 - [debian/upstream/metadata](#) - miscellaneous information
- [debian/watch](#) - check for upstream updates

To find real-world examples of any `debian/` file, go to  [codesearch.debian.net](#) and search for e.g.  [Reference path:debian/control](#).

Working with other developers

If you want to get involved with the Debian community:

- [Mentors](#) - sponsors/mentors for packages in specific areas of Debian
 -  [introduction for reviewers](#)
- [SponsorChecklist](#) - Sponsor Checklist
- [Software that can't be packaged](#) - projects that have already been found ineligible for inclusion in Debian
- guidelines for processing the  [NEW queue](#)
 - [LucaFalavigna's checklist](#)
 -  [Reject FAQ](#)
- [DEX](#) - improving Debian and its derivatives through cross-community teamwork
- [how developers can help packagers](#)
- [Work-Needing and Prospective Packages](#) - packages that have been requested for packaging, or need a new maintainer
- [Salsa](#) - GitLab-based Debian development server

Training Sessions

External links

-  [What's in a debian/ directory](#)
-  [Managing Debian package files with cme](#)
-  [Ubuntu Packaging Guide](#)
-  [Debian Binary Package Building HOWTO](#) (2005)
-  [How to build and deploy a Debian Package with GitLab CI](#)
-  [fpm](#) can build .deb packages from various other package formats ( [rubygems](#),  [pip](#), pear, tar, npm, pacman...)
-  [Packaging - Ubuntu Wiki](#)
-  [Debian build tools - Russ's Debian Notes](#)
-  [Packaging Scripts for Debian - Russ's Debian Notes](#)

See also

-  [Debian administration - Rolling your own Debian packages \(part 1\)](#)
-  [Autobuilding non-free packages](#)
-  [A Debian packaging howto \(2010\)](#)
-  [Avoid a newbie packager mistake: don't build your Debian packages with dpkg - b](#)
- [Debuginfod](#) eliminates the need for users to install debuginfo packages in order to debug programs using GDB, systemtap or other tools
-  [Browse through the source code of the Debian operating system](#)

Wiki pages

All pages related to packaging in Debian:

1. [AdvancedBuildingTips](#)
2. [Alioth](#)
3. [AndroidTools](#)
4. [AutomaticPackagingTools](#)
5. [BootstrapBootableSystem](#)
6. [BuildingFormalBackports](#)
7. [BuildingWithoutFakeroot](#)
8. [BzrBuildpackage/DesignIdeas](#)
9. [CPEtagPackagesDep](#)
10. [CheckInstall](#)
11. [ConfigPackages](#)
12. [CopyrightReview](#)
13. [CopyrightReviewTools](#)
14. [Courses/MaintainingPackages](#)

15. [Courses2005/BuildingWithoutHelper](#)
16. [CreatePackageFromPPA](#)
17. [Creating signed GitHub releases](#)
18. [CrossBuildPackagingGuidelines](#)
19. [DDPO](#)
20. [DEX](#)
21. [DataPackages](#)
22. [Debhelper](#)
23. [DebianAstro/AstropyPackagingTutorial/Packaging](#)
24. [DebianAstro/AstropyPackagingTutorial/Preparation](#)
25. [DebianChangelog](#)
26. [DebianDevelopment](#)
27. [DebianGNUstep/TODO](#)
28. [DebianMentorsFaq](#)
29. [DebianMultimedia/DevelopPackaging](#)
30. [DebianRepository/Setup](#)
31. [DebugPackage](#)
32. [DevelopersCorner](#)
33. [Diagrams](#)
34. [Distcc](#)
35. [DkmsPackaging](#)
36. [DpkgConfFileHandling](#)
37. [DpkgDiversions](#)
38. [EmacspeakTestingGuide](#)
39. [FTBFS](#)
40. [FastTrack](#)
41. [Fonts/PackagingPolicy](#)
42. [GettingPorted](#)
43. [GitPackaging](#)
44. [GitPackagingSurvey](#)
45. [GitPackagingSurvey/bare debian](#)
46. [GitPackagingSurvey/bare debian monorepo](#)
47. [GitPackagingSurvey/bare template](#)
48. [GitPackagingSurvey/git-debcherry](#)
49. [GitPackagingSurvey/git-debrebase](#)
50. [GitPackagingSurvey/git-dpm](#)
51. [GitPackagingSurvey/manually maintained applied](#)
52. [GitPackagingSurvey/merging](#)
53. [GitPackagingSurvey/modified orig plus further unapplied patches](#)
54. [GitPackagingSurvey/rebasing](#)
55. [GitPackagingSurvey/unapplied](#)
56. [GitPackagingWorkflow](#)
57. [GitPackagingWorkflow/DebConf11BOF](#)
58. [GitSrc](#)
59. [Gnome/Git](#)
60. [Gnome/Rust_Packaging](#)

61. [HardeningWalkthrough](#)
62. [HowToPackageForDebian](#)
63. [Java/Packaging](#)
64. [Javascript/Forwarding-Patches](#)
65. [Javascript/Policy](#)
66. [Javascript/Repacking](#)
67. [Maintainers](#)
68. [MakeAPrivatePackage](#)
69. [ManageUpstreamDifferences](#)
70. [Mapping_package_names_across_distributions](#)
71. [Mentors](#)
72. [Mingw-W64](#)
73. [NonMaintainerUpload](#)
74. [OpenSuseBuildService](#)
75. [PackageConfigUpgrade](#)
76. [PackageSalvaging](#)
77. [PackageTransition](#)
78. [Packaging](#)
79. [Packaging/EmbeddedCopies](#)
80. [Packaging/HackingDependencies](#)
81. [Packaging/Intro](#)
82. [Packaging/Learn](#)
83. [Packaging/Pre-Requisites](#)
84. [Packaging/Pre-Requisites/nspawn](#)
85. [Packaging/Variables](#)
86. [Packaging/ruby-team-meta-build](#)
87. [Packaging/sbuild](#)
88. [PackagingFAQ](#)
89. [PackagingTools](#)
90. [PackagingWithDarcs](#)
91. [PackagingWithDocker](#)
92. [PackagingWithGit](#)
93. [PbuilderTricks](#)
94. [PkgQtKde/BookwormReleasePlans](#)
95. [PkgQtKde/ForkyReleasePlans](#)
96. [PkgQtKde/TrixieReleasePlans](#)
97. [Projects/DebSrc3.0](#)
98. [Python/DbgBuilds](#)
99. [Python/GitPackaging](#)
100. [Python/LibraryStyleGuide](#)
101. [Python/Policy](#)
102. [RPM](#)
103. [RenamingPackages](#)
104. [Repackage_srcrpm](#)
105. [Repacking](#)
106. [ReproducibleBuilds](#)

107. [Salsa](#)
108. [Salsa/support](#)
109. [ServiceSandboxing](#)
110. [Services/wnpp-by-tags.debian.net](#)
111. [SimpleBackportCreation](#)
112. [SimplePackagingTutorial](#)
113. [Software that can't be packaged](#)
114. [SoftwarePackaging](#)
115. [SponsorChecklist](#)
116. [Teams](#)
117. [Teams/DebianHaskellGroup](#)
118. [Teams/DebianMonoGroup/NewPackage](#)
119. [Teams/Dpkg/Spec/DeclarativePackaging](#)
120. [Teams/Foo2zjs](#)
121. [Teams/Games](#)
122. [Teams/MySQL](#)
123. [Teams/MySQL/MySQL-wsrep](#)
124. [Teams/OCamlTaskForce](#)
125. [Teams/Printing](#)
126. [Teams/Ruby/Packaging](#)
127. [UntrustedDebs](#)
128. [UpstreamGuide](#)
129. [UpstreamMetadata](#)
130. [UscanEnhancements](#)
131. [UsingQuilt](#)
132. [WNPP](#)
133. [WritingDebianPackageDescriptions](#)
134. [binNMU](#)
135. [debian/copyright](#)
136. [debian/patches](#)
137. [debian/upstream](#)
138. [debian/upstream/edam](#)
139. [debian/watch](#)
140. [pbuilder](#)
141. [piuparts](#)
142. [pt_BR/AdvancedBuildingTips](#)
143. [pt_PT/Teams](#)
144. [sbuild](#)
145. [tag2upload](#)
146. [udeb](#)
147. [zh_CN/DebianRepository/Setup](#)
148. [zh_CN/sbuild](#)

Search

package names



equivs

[all options](#)

Limit to suite: [bullseye](#) [bullseye-updates](#) [bullseye-backports](#) [bookworm](#) [bookworm-updates](#) [bookworm-backports](#) [trixie](#) [trixie-updates](#) [trixie-backports](#) [forky](#) [sid](#) [experimental](#)

Limit to a architecture: [alpha](#) [amd64](#) [arm](#) [arm64](#) [armel](#) [armhf](#) [avr32](#) [hppa](#) [hurd-i386](#) [i386](#) [ia64](#) [kfreebsd-amd64](#) [kfreebsd-i386](#) [loong64](#) [m68k](#) [mips](#) [mips64el](#) [mipsel](#) [powerpc](#) [powerpcspe](#) [ppc64](#) [ppc64el](#) [riscv64](#) [s390](#) [s390x](#) [sh4](#) [sparc](#) [sparc64](#) [x32](#)

You have searched for packages that names contain *equivs* in all suites, all sections, and all architectures. Found **1** matching packages.

Exact hits

Package equivs

- [bullseye \(oldoldstable\)](#) (admin): Circumvent Debian package dependencies
2.3.1: all
- [bookworm \(oldstable\)](#) (admin): Circumvent Debian package dependencies
2.3.1: all
- [trixie \(stable\)](#) (admin): Circumvent Debian package dependencies
2.3.2: all
- [forky \(testing\)](#) (admin): Circumvent Debian package dependencies
2.3.3: all
- [sid \(unstable\)](#) (admin): Circumvent Debian package dependencies
2.3.3: all

This page is also available in the following languages (How to set [the default document language](#)):

[Български \(Bǎlgarski\)](#), [Deutsch](#), [suomi](#), [français](#), [magyar](#), [日本語 \(Nihongo\)](#), [Nederlands](#), [polski](#), [Português \(br\)](#), [Русский \(Russkij\)](#), [slovensky](#), [svenska](#), [Türkçe](#), [українська \(ukrajins'ka\)](#), [中文 \(Zhongwen, 简\)](#), [中文 \(Zhongwen, 繁\)](#)

See our [contact page](#) to get in touch.

Content Copyright © 1997 - 2026 [SPI Inc.](#); See [license terms](#). Debian is a [trademark](#) of SPI Inc. [Learn more about this site](#).

This service is sponsored by [Hewlett-Packard](#).

- [DebianMentorsFaq](#)

Common questions asked by new posters to the [DebianList: debian-mentors](#) mailing list. There are five sections:

- [General questions](#) - the list, Debian, and whatever else doesn't fit in another section
- [Packaging](#) - creating and maintaining Debian packages
- [Sponsored packages](#) - the purpose and process of package sponsorship
- [Infrastructure Projects](#) - how to contribute to Debian infrastructure projects
- [For sponsors](#) - for developers sponsoring or considering sponsoring a package or two

If in doubt, just read the whole thing. If you still have your question, ask on the [DebianList: debian-mentors](#) list.

Contents

1. General questions

1. [What is the debian-mentors mailing list for?](#)
2. [I want to help Debian. Tell me what I can do.](#)
3. [What's the process of becoming a Debian Developer?](#)
4. [What other resources are there for new and prospective Developers?](#)
5. [How do I adopt an existing package?](#)
6. [What can I do to help Debian other than packaging?](#)

2. Packaging

1. [How do I make my first package?](#)
2. [How do I add a new package to the archive?](#)
3. [How do I install a package from mentors.debian.net?](#)
4. [Where else can I learn about the packaging process?](#)

3. Sponsored Packages

1. [What's a sponsor, why do I want one, and how do I get one?](#)
2. [What's the difference between a sponsor and an advocate?](#)
3. [So how do I get an advocate then?](#)
4. [How do I get a sponsor for my package?](#)
5. [But why should I waste time packaging if there's no guarantee it's going to be uploaded?](#)
6. [Where else can I get a sponsor?](#)
7. [What happens if I can't find a sponsor?](#)
8. [Why is it so hard to find a sponsor?](#)

4. Infrastructure Projects

5. For Sponsors...

1. [Who can sponsor a package?](#)
2. [What is the process for sponsoring a package?](#)
3. [What about recurring sponsorship of the same package?](#)

General questions

What is the debian-mentors mailing list for?

[DebianList: debian-mentors](#) is for mentoring new and prospective Debian maintainers, as well as contributors to Debian infrastructure. Almost any question relating to Debian development is potentially on-topic for d-mentors. Although there are other lists which deal with most aspects of developing for Debian, they're often pretty hardcore and can be daunting. If you think your question might be a little "newbie" for the hardcore developers on debian-devel or infrastructure team lists, then it's probably a good fit for debian-mentors.

In short, we aim to be the "softer, gentler" Debian development forum.

One of the most common uses for the list is to seek someone to sponsor a package you have created (or are intending to create). It's such a common request that it has its [own section](#) in this FAQ.

Things that aren't really appropriate for the debian-mentors forum:

- questions about using the Debian system, either as a regular user or an admin
 - these would tend to belong in [DebianList: debian-user](#) or a list devoted to the particular subsystem you're using ([DebianList: debian-x](#), [DebianList: debian-apache](#), etc.)
 - see [DebianList: all mailing lists](#)
- questions about programming in general
 - this would be a topic for a list or forum dedicated to the language or system used
- hardcore Debian development questions
 - d-mentors will try and help out, but you'll get the big brains working on your problem if you take it next door to [DebianList: debian-devel](#), or to one of the topic-specific Debian lists

I want to help Debian. Tell me what I can do.

This is a very difficult question, because there's so many different things that could be done, and we don't know what you're really interested in. Problems include:

It's hard to know what you actually want to do. Even with a detailed resume, how do we know whether you're going to be interested, long term, in some task? Everyone's a volunteer, and we have no hold over you to tell you to do something you don't want to do (unlike paid employment).

If someone spends a lot of time specifying a task that doesn't speak to you, they've wasted their time. Mentors have lots of calls on their time, and after they've had that experience a few times, they just stop specifying tasks for people.

It's usually better to just dive into something and try and work out how it's done. Hands-on learning is the best type of learning anyway. If you really are keen on getting involved in random places in Debian, there are lots of resources in the question "[What can I do to help Debian other than packaging?](#)", below.

What's the process of becoming a Debian Developer?

See [DebianDeveloper/JoinTheProject](#).

What other resources are there for new and prospective Developers?

Apart from the [DebianList: debian-mentors](#) mailing list, there are a wealth of useful places and documents which the aspiring developer can use to improve themselves:

- the [DebianIRC: #debian-mentors](#) IRC channel
 - for help with any packaging and other Debian-related problems you might be having
-  [The Developer's corner](#)
 - has links to the Debian Policy, Developers' Reference, New Maintainer's Reference etc.
- guides from the debian-women project
 - a [Short BTS howto](#)
 - a  [One-on-one mentoring program](#) for new developers

How do I adopt an existing package?

See  <https://www.debian.org/devel/wnpp/#howto-o>.

What can I do to help Debian other than packaging?

Excellent question! Debian is a lot more than a great big collection of random packages. There's documentation to write and translate, bugs to fix, installers to test, and newbies to harass^H^H^H^H, uh, help. Besides, running and developing the Debian infrastructure also takes a lot of work. Projects like [Debbugs](#) (our bug tracking system),  [dak](#) (which runs our package archive) and many more would love to see new contributors. See the section [Infrastructure Projects](#) below for more information on how to contribute to Debian infrastructure projects.

If you write a language other than English fairly well (either because you're a native speaker or you've spent the time to learn) then consider helping out with translations. Everything in Debian needs to be translated, more or less - package descriptions, debconf questions, all of our documentation, etc. There are a lot of clever people

involved there - be one of them! Translation work is mostly handled through <https://www.debian.org/international/>. Most translation projects can be found from there.

There are several thousand open bugs in Debian packages, ranging from the trivial to the critical. Consider looking in the [Bug Tracking System](#) for packages that you use often. Try and reproduce a bug, or find the cause of the bug and submit a patch. If a bug has one or more patches that haven't been applied in a long time, especially for more serious bugs, and the maintainer hasn't commented, consider bringing the package to the attention of [DebianList: the Quality Assurance team](#). You can find Release Critical bugs for packages installed on your system (in case you might like to help fix them) by running the `rc-alert` script in [DebianPkg: devscripts](#).

Some maintainers have requested some help with their packages, by posting a Request For Help ("RFH") on [the Work-needing and Prospective Packages list](#). A list of current requests for help posted by other people is available from https://www.debian.org/devel/wnpp/help_requested, in case you're feeling helpful.

If you're interested in helping to fix a bug, there is a [list of bugs](#) that have been identified by package maintainers as fairly simple, and good to improve your skills on. For more information, see [wiki page about the tag](#).

Packaging

How do I make my first package?

Co-maintenance may be a better alternative to creating a new package on your own. You'll get a pre-existing package, plus someone to answer your questions and sponsor your uploads while you learn the ropes.

To co-maintain a package, look at [packages for which help was requested](#) for all of Debian, or install [DebianPkg: how-can-i-help](#) and run `how-can-i-help --show RFH --old` for packages on your computer. Either way, find a bug that interests you and reply to it.

Adopting a package is another alternative. There are always packages [up for adoption](#) or [orphaned](#) by their old maintainer, and you can see installed packages with `how-can-i-help --show RFA,0 --old`. These are packages which are in Debian but their previous maintainer(s) have given up maintenance. Packages with a Request For Adoption ("RFA") should have someone to answer your questions and sponsor your uploads while you learn the ropes, but will be more hands-off and will likely expect you to take over in the long-run. You're on your own with orphaned packages, but at least you have a working example to start from.

You can also find packages that need work at [the WNPP bug list](#) ([alternative interface](#)), or by calling `how-can-i-help` without arguments (shows every type of issue, but only issues it's never shown you before).

For "Request For Adoption" packages, contact the current maintainer to ask if they'd be willing to let you maintain it, and to sponsor your uploads. Please respect their judgement if they decline - maintainers usually have a good reason for not handing the package off to someone with no experience.

Once you've selected a package to adopt, retitle the WNPP bug so it starts with "ITA:" (for Intent To Adopt) and start working on the package. Fix bugs, package new versions, and hunt for a sponsor for your new package. Remember to set yourself as the maintainer, and to close the WNPP bug in the changelog of your first upload.

Improving an abandoned package is also possible, even if the maintainer hasn't officially orphaned it. It's usually best to avoid this unless the package has obvious bugs that have been open for a long time and don't have a reply (e.g. asking to fix a typo that would take a few minutes, or to package a new upstream version with important features). A developer that's just having a bad year won't be happy to see someone threatening to hijack their package!

If you use a program and think you'd be a good new maintainer, bring it to [DebianList: debian-mentors](#) or [DebianList: debian-qa](#) with your reasoning.

Answering a 🌐 [Request For Package](#) is a good way to create a new package, because you can prove there's interest, have someone to test your package, and may even get someone to sponsor your uploads.

Creating an unsolicited package is fine too, so long as it's not already in the archive, and not listed as an ITP (Intent To Package) on the 🌐 [WNPP bugs page](#).

For more information about creating a package, see the next question.

How do I add a new package to the archive?

📘 If you don't have a specific package in mind, see the previous question for other ways to scratch your itch.

The basic process is:

1. check your project isn't already 🌐 [being worked on](#)
 - if it is, offer to help out instead
2. check if your project is already 🌐 [requested to be worked on](#)
 - if it is, contact the requester to see if they're willing to help
3. check you can actually create the package
 - is its license compatible with the [Debian Free Software Guidelines](#)?
 - can you reasonably maintain it?
 - how complex and time-consuming would you find it?
 - are you ready to stick around for years maintaining bugs?
 - are you familiar with the programming language(s) used?

- can you expect upstream to fix packaging issues and respond to bugs from Debian users?
 - [the upstream guide](#) discusses what you're volunteering them for
4. file an ITP bug report against WNPP as detailed by [section 5.1 of the developer's reference](#)
 - or if it's already listed as an RFP, retitle the bug as an ITP (see the [BTS manipulation](#) manual)
 5. put a package together, built against a current version of [sid](#)
 - [Packaging](#) has many guides about this
 6. Install and test your package thoroughly
 - ask on the [DebianList: debian-mentors mailing list](#) for people to check your package for errors and common problems
 - if someone else requested the package, ask them to try it out
 7. upload your source package (.orig.tar.gz, .diff, and .dsc), binary package (.deb) and .changes files (sign your .changes if possible)
 - preferably to [salsa.debian.org](#)
 - a [forge](#) like GitHub is OK too
 - there is a "pretend" upload queue at <https://mentors.debian.net/>
 - if you're already a [Debian Developer](#), just upload the package to Debian
 8. [request a sponsor from debian-mentors](#), and anywhere else you think you might find one
 - when you get a sponsor, let everyone else know so they don't duplicate that effort
 9. keep your package maintained after the initial upload - respond to bug reports and new upstream versions

For more information, see [Packaging](#).

How do I install a package from mentors.debian.net?

 packages on `mentors.debian.net` have not been reviewed and could contain malicious code.

If you have reviewed a package to be sure it's clean, and you still want to install it:

1. install [DebianPkg: devscripts](#):

```
# install devscripts:
sudo apt-get install devscripts
```

2. Use `dget` to download the source code (see [DebianMan: dget](#)):

```
dget <link to the .dsc file>
```

3. If you get an error from `dscverify` about the OpenPGP signature check, use your personal OpenPGP keyring and then download the author's OpenPGP public key:

```
cp /etc/devscripts.conf ~/.devscripts
# uncomment DSCVERIFY_KEYRINGS="" and set it to your OpenPGP
# (e.g. DSCVERIFY_KEYRINGS=~/.gnupg/pubring.gpg")
sensible-editor ~/.devscripts
gpg --keyserver pool.sks-keyservers.net --recv-keys <author's
dget <link to the .dsc file>
```

4. `cd` into the directory that has been created:

```
cd /path/to/extracted/package
```

5. Build the source into a `.deb` file:

```
debuild -us -uc
```

6. install the package:

```
sudo dpkg -i <package-name>.deb
```

- you may need to manually install dependencies - `dpkg` can list missing dependencies, but can't do anything about it

Where else can I learn about the packaging process?

More questions are available at [PackagingFAQ](#), and [Packaging](#) has many more links.

Sponsored Packages

What's a sponsor, why do I want one, and how do I get one?

A sponsor is a [Debian Developer](#) ("DD") who uploads packages to the archive on behalf of a non-DD. The sponsor is required to check the quality of the package, that there are no show-stopping bugs in it, and that it is unlikely to harm either the Debian infrastructure during build or users' systems when in use.

The sponsor needs to do all of this because they are ultimately responsible for what gets uploaded by them into the archive.

You need a sponsor because, as a non-DD, you do not have the ability to upload packages directly into the Debian archive. So, you need to route your packages through a sponsoring DD so they can be properly signed and uploaded.

The presence of the sponsor doesn't mean you can neglect your maintainership duties. You are listed as the maintainer - it's your reputation on the line too. You are expected to keep a handle on bugs and new upstream versions, feed information from Debian users back to upstream, and generally uphold Debian's reputation. If you "dump" poor quality packages on sponsors, or do not maintain your packages well, do not expect to get many people willing to sponsor you.

<yoda>Scared? You will be...</yoda>

What's the difference between a sponsor and an advocate?

A *sponsor* is any Debian Developer willing to upload packages on your behalf into the Debian archive. You may have multiple people who act as sponsors for your packages - different people for different types of packages, or maybe you change sponsors over time, or rotate between a few (with the knowledge of all of them) to spread the load.

An *advocate* is a Debian Developer who has worked with you in your capacity as a contributor to Debian, and believes you would be an asset to Debian. When you are ready to apply for New Maintainership, the advocate makes a statement to the effect that they believe you would be a worthwhile addition to Debian, and your application moves forward. Without an advocate, your application cannot proceed.

There is no point asking for an advocate on debian-mentors or any other open mailing list. No developer should be willing to advocate for you without knowing something about you.

So how do I get an advocate then?

Which Debian Developers have you done Debian-related work with? Typically, if you're ready for NM, you should have worked with at least one, if not several. Ask them. If you've done good work, they should be more than happy to say that to the Front Desk via an advocacy. Get them to ask on [DebianList: debian-mentors](#) if they're not sure exactly what advocacy is, when it should be done, and how.

One of your sponsors can be your advocate, and this would probably be the most common way of obtaining an advocate. But if you've had significant contact with any other DD, you can consider asking them.

How do I get a sponsor for my package?

First, be aware there is no guarantee that anyone will sponsor your package. It all depends on the interest level and time available from any prospective sponsor.

You should file a "Request For Sponsor" (RFS) bug report in the Debian BTS. The "sponsorship-requests" pseudo-package in the Debian bug tracking system is designed for receiving these requests. See the  [instructions on filing a correct RFS](#). That will lead you through the steps to create an informative, concise request for a sponsor.

Messages in other forums, or without all the information, are far more likely to be ignored.

Your RFS message is like an ad for your package. It's likely to be the only thing that prospective sponsors will judge your package on. You can have all the extra URLs you like in there where sponsors can get more information, but unless your initial message piques their interest, they'll never look at them.

So, tell us what exactly your package does, and why it should be in Debian. If there is already a program that does a similar thing, say why your one is better. Put a little "hot spice" in there to hold people's interest. In other words, think like an advertising executive. Just remember to wash the slime off afterward.

You'll notice that one of the things to have is where the package can be downloaded from. That implies that you've packaged it already. If you need help with packaging, ask for that, but don't bother asking for a sponsor until you've got the source package (more-or-less) ready for a sponsor to download and build.

Sponsoring a package takes a lot more than just downloading it from your website and uploading it to Debian. The prospective sponsor needs to check over the quality of the packaging and ensure that it meets Debian's quality standards before uploading it. For this reason, you need to provide all of the parts which would be needed for a source package upload (the changes file `foo.changes`, the source control file `foo.dsc`, the upstream source `foo.orig.tar.gz`, the Debian packaging patch `foo.diff.gz`). Also, it may take a few days/weeks to get the whole package checked over, and the sponsor might want to talk to you a bit, just to find out what sort of a person you are.

Think of all this as a smaller-scale New Member check - which is, basically, what it is.

But why should I waste time packaging if there's no guarantee it's going to be uploaded?

You shouldn't be generating a first package "just to get it in Debian". The archive is already well-stocked with poorly-maintained vanity packages. A first package, especially, should be something that you really want to see in Debian but which hasn't been uploaded already for whatever reason. That personal interest will help keep your interest levels up.

It is important to remember that an upload isn't a one-off thing - you're responsible for that package forevermore, unless you request its removal or find another suitably qualified person to maintain it for you. It's like having a child - a big responsibility.

And you can always just [create your own repository](#) and upload it to a static hosting service (like GitHub Pages). One guide to creating a repo is [DebianRepository/SetupWithReprepro](#).

Where else can I get a sponsor?

[DebianList: debian-mentors](#) isn't the only place where you can get a sponsor. In general, you're more likely to get a sponsor who is actually interested in what you've packaged, but doesn't have time to package it themselves. So, if there's a list dedicated to your "niche market", send the RFS there. For RFS bugs, use the  [X-Debbugs-CC](#) field in the pseudo-header. For plain RFS mails, CC the address. For instance, if you're packaging Yet Another Perl Module, try [DebianList: debian-perl](#). If it's YAPython Module, [DebianList: debian-python](#). Apache module? [DebianList: debian-apache](#). Something legal-related? [DebianList: debian-lex](#). Something for kids? [DebianList: debian-edu](#). Getting the picture yet? There's a [pretty comprehensive list of teams](#) elsewhere on the wiki. Teams may sometimes request that they become the Maintainer for your package. Don't worry if this is the case --- it just means that they are interested in helping you out. They may also request that you adopt the team's workflow for the package. Again, don't worry. Team members should help you get everything straight and having packages organised in the same way makes maintenance easier.

Also, if you're packaging something in a particular technological line, you'll probably want to be subscribed to the relevant speciality mailing list anyway, so you can ask technology-specific questions to the people who know all about it.

What happens if I can't find a sponsor?

First off, don't panic. Not having your package in Debian immediately isn't the end of the world.

Solicit feedback on [DebianIRC: #debian-mentors](#) and [DebianList: the mailing list](#). Don't just mention the package name, also provide (at least) the brief description. Typically a developer is more likely to sponsor something they might be interested in using, or which they recognise there might be a need for. Just the package name doesn't really convey the sort of information needed to make that call.

Sometimes RFS' fall through the cracks, so continue updating the package. Mention the package on d-mentors every month or two; people come and go, and their available time varies, so a later mention may catch someone who couldn't help you before.

Meet some Debian folk in your area or at [DebConf](#) and convince them to sponsor you 😊 It's a lot easier for someone to commit some time to helping you if they've got a face and real personality to put to the e-mail address (plus it's a great excuse to get some keysigning action happening).

Increase the quality of your package. Check it against various checklists, and with [package-checking tools](#) like [lintian](#). Look at the PTS page for your package and at the links on that page. See [Working with other developers](#).

Fix lots of bugs, get involved in doing QA work. Do reviews for other folks looking for sponsorship and ask them to review your package in return. Perhaps you will get to know a DD well enough that they will sponsor you.

Why is it so hard to find a sponsor?

Volume

There are not enough Debian contributors to package every piece of software nor enough Debian members to sponsor every package made by Debian contributors. This has always been the case and always will be the case, there is just so much free software out there and probably many more Debian contributors than Debian members.

Time

Sponsorship is hard work and is a large time investment. Some Debian members might not have enough "Debian time" to do it at all and others might prefer to spend their time on doing work that they signed up to do, like maintain infrastructure or packages they are a maintainer for.

Specialization

Debian contributors generally work on stuff they use or are otherwise interested in. This can limit the scope of software that gets sponsored. With well-functioning teams, it can also mean that software for a specific area is well covered with sponsorship, [DebianMed](#) is a good example. Unfortunately this can mean there are no sponsors for particular areas.

Preferences

Different folks have different packaging preferences. Your packaging choices will in part reduce the set of folks who will have experience with and willingness to sponsor your

package. There are folks and teams who do not do sponsorship, but have a strong emphasis on collaboration and instead do co-maintenance or team maintenance.

Responsibility

Sponsors take responsibility for your upload. Some folks might not want to take this responsibility on, if they didn't check the upload quite well enough and later it was discovered to contain malicious or buggy code, it would be their fault. This scares some people away from doing sponsorship. Some folks are scared of uploading new packages in case it turns out the contributor will disappear after one upload.

Infrastructure

In the past we had pretty poor ways of matching packages to be sponsored with potential sponsors based on the above criteria. This is improving, but still needs work.

Emphasis

Sponsorship wasn't emphasised quite as much as other activities in Debian, so fewer folks considered taking it. This may have changed already, or maybe we need to adjust our new-member documents.

Infrastructure Projects

[DebianList: debian-mentors](#) explicitly endorses questions about Debian infrastructure projects. We're more than happy to help new contributors navigate through our infrastructure projects.

Some context: In order to run a project as big as Debian, a lot of infrastructure work is needed. Often, the Debian infrastructure teams had to write custom software for running our infrastructure as no Free Software solution existed beforehand. These software projects need to be maintained, tested, improved and further developed and usually the relevant teams are more than happy to introduce new contributors.

Still, many of our infrastructure teams are chronically understaffed and over-worked, which might lead to brevity and long delays when responding to comprehension questions by new contributors.

If you're interested in contributing to one of our infrastructure projects or have questions about their mode of operation, don't hesitate to ask on [DebianList: debian-mentors](#).

Some examples of Debian infrastructure projects:

-  [Debbugs](#), our bug tracking system
-  [DAK](#), which runs our package archives
-  [debci](#), a system that runs automated tests on our packages

- [distro-tracker](https://distro-tracker.org), which powers <https://tracker.debian.org>
- [all Debian services](#)
- [all debian.net domains](#)

For Sponsors...

Who can sponsor a package?

Any Debian Developer with a key in the keyring can sponsor a package on behalf of another maintainer. There is no special "register" of sponsors. Note that sponsorship is rarely a one-off event; you need to be able to handle regular uploads on behalf of your "sponsee". If you can only do a once-off upload, make your prospective sponsee aware of this up-front, and try to find someone else who will be able to continue the sponsorship process in the future if required.

What is the process for sponsoring a package?

You should first, of course, find a package to sponsor. <https://mentors.debian.net/>, the d-mentors [DebianList: mailing list](#) and [DebianIRC: IRC channel](#), are all good places to find packages.

You need to get the **source** package from your sponsee. That is, the `.dsc`, `.orig.tar.gz`, and `.diff.gz`. You should check the package over carefully, perhaps with reference to [the sponsor checklist](#). If there are problems, provide constructive feedback to the sponsee. Work with them to make the package the best it can be. Build the package on your system, try it out -- does it install/remove properly, does the program run, is it policy-compliant? All of the things you're supposed to do for your own packages. Yes, it's a lot of work, but you're responsible for any crud that lands in the archive under your key.

Immediately before upload, rebuild the package in a clean [sid](#) chroot, sign the `.changes/.dsc` file with your key, and then make a normal upload. Make sure your name isn't in the `Maintainer:` or `Changed-By:` fields of the `.changes` file. You can ensure this by either using the `-k` option to `dpkg-buildpackage/debuild`, or building without signing (`-uc -us`) and then using `debsign` to later sign the `.changes` file (which will sign the `.dsc` as part of the process). Do not use the `-m` or `-e` flags to `dpkg-buildpackage/debuild`, as that actually changes the maintainer/uploader information in the `.changes/.dsc`, which is most certainly not what you want.

Whatever you do, do **not** simply re-sign and upload the sponsee's `.dsc/.changes`, or (worse!) just upload what they send you (since it won't be properly signed).

What about recurring sponsorship of the same package?

You should work out a mutually-agreeable way for your sponsee to contact you to upload new versions of their package. Upload to <https://mentors.debian.net/> (with notification to you, the sponsor) is a good way to go, especially if the sponsee doesn't have web space of their own.

You should do appropriate checks of each new revision your sponsee provides. `debdiff` (from [DebianPkg: devscripts](#)) is excellent for getting an overview of what has changed between Debian revisions.

What if I upload a package, and then my mentee disappears?

As a Debian developer, it might be scary to upload a package for someone, only to have them perhaps disappear. Don't fret; you don't have to take responsibility for it if your sponsee vanishes.

If the sponsee disappears, just follow the same procedure as if a Developer went missing-in-action: do a non-maintainer upload of the package, assigning it to `debian-qa`, and make it available for adoption (or eventually orphanage).

Having said that, this does create some extra work for people who decide to work on QA. It's good to make sure the package that you upload is of decent quality and not a duplicate of an existing package.

To summarise, maintainers and packages enter and exit Debian; it's part of a life cycle. It's okay to upload sponsee packages knowing that you risk going through an orphaning/adoption process later.

What resources are there for sponsors and prospective sponsors?

- <https://mentors.debian.net/>
- [DebianList: mailing list](#)
- [DebianIRC: #debian-mentors](#)
- [Several sponsorship checklists](#)
 - none of the lists are either exhaustive or definitive
- [the REJECT FAQ](#)

Consider setting up a "sponsoring" alias for you and your sponsee by creating a file in your home directory on `master.debian.org` called `.forward-sponsoring-<sponsees-name>` containing your e-mail address and the address of your sponsee separated by commas (see existing entries on master). This will create a mail alias, `<your-name>-sponsoring-<sponsees-name>`, which will send mail to both you and your sponsee. Handy!

Originally created and maintained by Mathew Palmer at 
https://people.debian.org/~mpalmer/debian-mentors_FAQ.html

Copyright © 2003,2004,2005,2006,2007 Matthew Palmer. Released under the terms of
the GNU General Public License version 2.

[CategoryDeveloper](#) [CategoryPackaging](#)

DebianMentorsFaq (last modified 2025-08-19 16:10:04)

- [MakeAPrivatePackage](#)

Create a .deb for private use

This page provides a minimal set of steps to create a .deb file you can install on your local computer. To create packages robust enough to install on other computers, see [Packaging](#).

First, create a directory for your package to go in:

```
mkdir -p mypackage
```

Then add some files for your package:

```
mkdir -p mypackage/usr/bin

cat > mypackage/usr/bin/myprogram.sh <<EOF
#!/bin/sh
echo "Hello, World!"
EOF

chmod 755 mypackage/usr/bin/myprogram.sh
sudo chown root:root mypackage/usr/bin/myprogram.sh
```

Then add some Debian metadata:

```
mkdir -p mypackage/DEBIAN
cat > mypackage/DEBIAN/control <<EOF
Package: mypackage
Architecture: all
Version: 1.0
Section: misc
Maintainer: Local user <root@localhost>
Priority: optional
Standards-Version: 4.7.0
```

Description: My package

EOF

Finally, build and test your package:

```
dpkg-deb -b mypackage
sudo dpkg -i mypackage.deb
myprogram.sh
sudo dpkg --remove mypackage
```

You can now use `mypackage.deb` like any other package! But it might break in surprising ways, because it lacks features you take for granted in packages reviewed by Debian. To learn how to create proper packages, see [Packaging](#).

Better solutions in the real world

Creating a Debian package with [DebianMan: dpkg-deb](#) is like creating a Word document with [DebianMan: zip](#) -  [the underlying format works that way](#), but people use higher-level interfaces for a reason.

[DebianMan: equivs](#) is more advanced than `dpkg-deb` but not as complicated as [git](#). For example, it uses a normal lowercase `debian/` directory instead of `dpkg-deb`'s `DEBIAN/`, lets you put scripts in your `control` file that run during package installation and removal, and requires you to specify the location each file should be installed in.

[CategoryPackaging](#)

MakeAPrivatePackage (last modified 2025-06-18 17:43:49)

- [Packaging](#)
- [Intro](#)

Translation(s): [English](#) - ?Español - [Italiano](#) - [Català](#) - [Português \(Brasil\)](#) - [Română](#) - [Русский](#)

Introductory tutorial for making Debian packages. It doesn't get very deep into the more intricate bits of Debian packaging, but it shows how to make Debian packages for software that is simple to package. For more tutorials, see [Packaging](#).

Contents

1. [What is a "package"?](#)
 2. [Requirements](#)
 3. [Three central concepts](#)
 4. [The packaging workflow](#)
 1. [Step 1: Rename the upstream tarball](#)
 2. [Step 2: Unpack the upstream tarball](#)
 3. [Step 3: Add the Debian packaging files](#)
 4. [Step 4: Build the package](#)
 5. [Step 5: Install the package](#)
 5. [Conclusion](#)
 6. [See also](#)
-



Written in 2010

This tutorial describes recommended practice as of 2010. The theory hasn't changed, but the specific tools are no longer recommended for general use.

For more guides, see [Packaging](#).

What is a "package"?

A Debian package is a collection of files that allow for applications or libraries to be distributed via the package management system. The aim of packaging is to allow the automation of installing, upgrading, configuring, and removing computer programs for Debian in a consistent manner. A package consists of one source package, and one or more binary packages. The Debian Policy specifies the standard format for a package, which all packages must follow.

Binary packages contain executables, standard configuration files, other resources required for executables to run, documentation, data, ...

Source packages contain the upstream source distribution, configuration for the package build system, list of runtime dependencies and conflicting packages, a machine-readable description of copyright and license information, initial configuration for the software, and more.

The goal of packaging is to produce these packages from the unpacked source. The source package (.dsc) and binary packages (.deb) will be built for you by tools such as dpkg-buildpackage.

You can read more about the anatomy of [binary packages](#) or [source packages](#) on their wiki pages.



Packages must comply with Debian policy to be accepted into the package archives. Manually constructed .deb binary packages that are not built from a source package will not be accepted. This is to maintain consistency and reproducibility.

Requirements

This tutorial assumes you understand:

- installation of binary packages
- general command line use
- editing files using a text editor of your choice

That is all you need.

Needed packages:

- [DebianPkg: build-essential](#)
- [DebianPkg: devscripts](#)
- [DebianPkg: debhelper](#) version 13 or higher

Three central concepts

The three central concepts are

- **upstream tarball:**
 - A **tarball** is the *.tar.gz* or *.tgz* file upstream makes (can also be in other compression formats like *.tar.bz2*, *.tb2* or *.tar.xz*).
 - Contains the software **upstream developer** has written.
- **source package:**
 - This is the second step in the packaging process, to build the **source package** from the upstream source.
- **binary package:**

- From this source package you then build the **binary package**, which is distributed and installed.

The simplest [source package](#) consists of three files:

- The upstream tarball, renamed according to the naming convention
- A debian directory containing the changes made to upstream source, plus all the files required for the creation of a binary package.
- A description file (with *.dsc* extension), which contains metadata for the above two files.

The packaging workflow

- Step 1: Rename the upstream tarball
- Step 2: Unpack the upstream tarball
- Step 3: Add the debian.tar.gz files
- Step 4: Build the package
- Step 5: Test the package

After testing, the source and binary packages can be uploaded in the Debian archive.

For this tutorial,  [this tarball](#) is used as an example.

Step 1: Rename the upstream tarball

The packaging tools require that the tarball complies with the naming convention.

The name is as follows: source package name, underscore, upstream version number, followed by *.orig.tar.gz*. The source package name should be all lower case, and can contain letters, digits, and dashes. Some other characters are also allowed. More detailed naming convention can be found in  [debmake doc](#).

Minimal changes should be made to the original name, to make it Debian-compliant. If the original name complies with the standard, then you should use that.

In our example case, upstream has picked a suitable name, "hithere", so there are no changes needed.

We will end up with a file called **hithere_1.0.orig.tar.gz**.

Note that there is an underscore (`_`), not a dash (`-`), in the name. This is important.

```
$ mv hithere-1.0.tar.gz hithere_1.0.orig.tar.gz
```

Step 2: Unpack the upstream tarball

The source will unpack into a directory of the same name and upstream version with a hyphen in between (not an underscore), so the upstream tarball should unpack into a directory called "hithere-1.0".

In this case, the tarball already unpacks into the correct subdirectory, so no changes are required.

```
$ tar xf hithere_1.0.orig.tar.gz
```

Step 3: Add the Debian packaging files

All of the following files will be placed into the *debian/* subdirectory inside the source directory, so we create that directory.

```
$ cd hithere-1.0  
$ mkdir debian
```

Let's take a look at the files we need to provide.

debian/changelog

First file is **debian/changelog**. This is the log of changes to the Debian package.

It does not *need* to list everything that has changed in upstream code, but a summary is helpful for others.

Since we are now making the first version, there is nothing to log. However, we still need to make a changelog entry, because the packaging tools read information from the changelog: most importantly, the package version.

debian/changelog has a standard format. The easiest way to create it is to use the **dch** tool.

```
$ dch --create -v 1.0-1 -u low --package hithere
```

This will result in a file like this:


```
hithere (1.0-1) UNRELEASED; urgency=low
```

```
* Initial release. (Closes: #XXXXXX)
```

```
-- Lars Wirzenius <liw@liw.fi> Thu, 18 Nov 2010 17:25:32 +0000
```

A couple of notes:

- The package name (***hithere***) MUST be the same as the source package name. ***1.0-1*** is the version. The ***1.0*** part is the upstream version. The ***-1*** part is the Debian version: the first version of the Debian package of upstream version 1.0. If the Debian package has a bug, and it gets fixed, but the upstream version remains the same, then the next version of the package will be called 1.0-2. Then 1.0-3, and so on.
- ***UNRELEASED*** is called the upload target. It tells the upload tool where the binary package should be uploaded. ***UNRELEASED*** means the package is not yet ready to be uploaded. It's a good idea to keep the ***UNRELEASED*** there so you don't upload by mistake.
- ***urgency=low*** will not be explained yet.
- The (***Closes: #XXXXXX***) bit is for closing bugs when the package is uploaded. This is the usual way in which bugs are closed in Debian: when the package that fixes the bug is uploaded, the bug tracker notices this and marks the bug as closed. If there are no bugs fixed, this part can be omitted.
- The final line in the changelog tells who built this version of the package, and when. The dch tool tries to guess the name and e-mail address, but you can configure it with the right details. See the dch(1) manual page for details.

debian/control

The control file describes the source and binary package, and gives some information about them, such as their names, who the package maintainer is, and so on. Here is an example of what it might look like:

```
Source: hithere
Maintainer: Lars Wirzenius <liw@liw.fi>
Section: misc
Priority: optional
Standards-Version: 4.7.0
Build-Depends: debhelper-compat (= 13)

Package: hithere
```

```
Architecture: any
Depends: ${shlibs:Depends}, ${misc:Depends}
Description: greet user
    hithere greets the user, or the world.
```

There are several required fields, but you can just treat them as magic, for now. So, in *debian/control*, there are two stanzas.

The first stanza describes the **source package**, with these fields:

Source

The source package name.

Maintainer

The name and e-mail address of the person responsible for the package.

Priority

The priority of the package (one of 'required', 'important', 'standard' or 'optional'). In general, a package is 'optional' unless it's 'essential' for a standard functioning system, i.e., booting or networking functionality. As of  [Debian Policy 4.5.1](#) (or sooner) package priority 'extra' is deprecated.

Build-Depends

The list of packages that need to be installed to build the package. They might or might not be needed to actually use the package.

All stanzas after the first describe the **binary packages** built from this source. There can be many binary packages built from the same source; our example only has one. We use these fields:

The *debhelper-compat* Build-Dependency specifies the " [compatibility level](#)" for the [DebianPackage: debhelper](#) tool level via the version constraint. The example above specifies compat level 13. (This replaces the obsolete *debian/compat* file)

Package

The name of the binary package. The name might be different from the source package name.

Architecture

Specifies which computer architectures the binary package is expected to work on: i386 for 32-bit Intel CPUs, amd64 for 64-bit, armel for ARM processors, and so on. Debian works on about a dozen computer architectures in total, so this architecture support is crucial. The "Architecture" field can contain names of particular architectures, but usually it contains one of two special values.

any

(which we see in the example) means that the package can be built for any architecture. In other words, the code has been written portably, so it does not make too many assumptions about the hardware. However, the binary package will still need to be built for each architecture separately.

all

means that the same binary package will work on all architectures, without having to be built separately for each. For example, a package consisting only of shell scripts would be "all". Shell scripts work the same everywhere and do not need to be compiled.

Depends

The list of packages that must be installed for the program in the binary package to work. Listing such dependencies manually is tedious, error-prone work. To make this work more automatic, the `_${shlibs:Depends}` magic bit needs to be in there for packages that contain built shared libraries and executables. The other magic stuff is there for debhelper, which will fill in the `_${misc:Depends}` bit. For other dependencies, you need to add them manually to Depends or Build-Depends. Note that the `_${...}` magic bits only work in Depends, not Build-Depends.

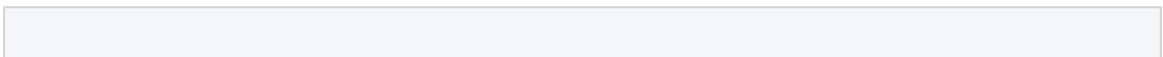
Description

The full description of the binary package. It is meant to be helpful to users. The first line is used as the short synopsis (summary) description, and the rest of the description must be an independent longer description of the package.

The command `cme edit dpkg` provides a GUI to edit most packaging files, including `debian/control`. See [🌐 Managing Debian packages with cme](#) page. The `cme` command is shipped in Debian in the [DebianPkg: cme](#) package. You can also edit **only** `debian/control` with `cme edit dpkg-control` command.

debian/copyright

It is quite an [🌐 important file](#), but for now we will be happy enough with an empty file.



For Debian, this file is used to keep track of the legal, copyright-related information about a package. However, it is not important from a technical point of view. For now, we'll concentrate on the technical aspects. We can get back to *debian/copyright* later, if there's interest.

debian/rules

It should look like this:

```
#!/usr/bin/make -f

%:

    dh $@
```

🚨 **Note:** The last line should be indented by one TAB character, not by spaces. The file is a makefile, and TAB is what the make command wants.

debian/rules can actually be quite a complicated file. However, the `dh` command in debhelper version 7 has made it possible to keep it this simple in many cases.

debian/source/format

The final file we need is *debian/source/format*, and it should contain the version number for the format of the source package, which is "3.0 (quilt)".

```
3.0 (quilt)
```

Step 4: Build the package

First try

Now we can build the package.

There are many commands we could use for this, but this is the one we'll use. If you run the command, you'll get an output similar to this:

```
$ debbuild -us -uc
```

```
make[1]: Entering directory `/home/liw/debian-packaging-tutorial/x/
install hithere /home/liw/debian-packaging-tutorial/x/hithere-1.0/d
install: cannot create regular file `/home/liw/debian-packaging-tut
make[1]: *** [install] Error 1
make[1]: Leaving directory `/home/liw/debian-packaging-tutorial/x/h
dh_auto_install: make -j1 install DESTDIR=/home/liw/debian-packagin
make: *** [binary] Error 29
dpkg-buildpackage: error: fakeroot debian/rules binary gave error e
debbuild: fatal error at line 1325:
```

```
dpkg-buildpackage -rfakeroot -D-us -uc failed
```

Something went wrong. This is what usually happens. You do your best creating *debian/** files, but there's always something that you don't get right.

So, the thing that went wrong is this bit:

```
install hithere /home/liw/debian-packaging-tutorial/x/hithere-1.0/d
```

The upstream Makefile is trying to install the compiled program into the wrong location.

There are a couple of things going on here: first is a bit about how Debian packaging works.

Correction

When the program has been built, and is "installed", it does not get installed into **/usr** or **/usr/local**, as usual, but somewhere under the **debian/** subdirectory.

We create a subset of the whole file system under **debian/hithere**, and then we put that into the binary package. So the **.../debian/hithere/usr/local/bin** bit is fine, except that it should not be installing it under **usr/local**, but directly under **usr**.

We need to do something to make it install into the right location (**debian/hithere/usr/bin**).

The right way to fix this is to change **debian/rules** so that it tells the Makefile where to install things.

```
#!/usr/bin/make -f
%:
    dh $@

override_dh_auto_install:
    $(MAKE) DESTDIR=$(pwd)/debian/hithere prefix=/usr install
```

It's again a bit of magic, and to understand it you'll need to know how Makefiles work, and the various stages of a debhelper run.

For now, I'll summarize by saying that there's a command debhelper runs that takes care of installing the upstream files, and this stage is called ***dh_auto_install***.

We need to override this stage, and we do that with a rule in **debian/rules** called ***override_dh_auto_install***.

The final line in the new **debian/rules** is a bit of 1970s technology to invoke the upstream Makefile from **debian/rules** with the right arguments.

Let's try again

```
$ debuild -us -uc
```

It still fails!

This time, this is the failing command:

```
install hithere /home/liw/debian-packaging-tutorial/x/hithere-1.0/d
```

We are now trying to install into the right place, but it does not exist. To fix this, we need to tell the packaging tools to create the directory first.

Ideally, the upstream Makefile would create the directory itself, but in this case the upstream developer was too lazy to do so.

Another correction

The packaging tools (specifically, debhelper) provide a way to do that.

Create a file called **debian/hithere.dirs**, and make it look like this:

```
usr/bin  
usr/share/man/man1
```

The second line creates the directory for the manual page. We will need it later. You should be careful to maintain such *.dirs files because it can lead to empty directories in future versions of your package if the items listed in those files aren't valid anymore.

Let's try once more

```
$ debuild -us -uc
```

Now the build succeeds, but there are still some small problems.

debuild runs the *lintian* tool, which checks the package that has been built for some common errors. It reports several for this new package:

```
Now running lintian...
W: hithere source: out-of-date-standards-version 3.9.0 (current is
W: hithere: copyright-without-copyright-notice
W: hithere: new-package-should-close-itp-bug
W: hithere: wrong-bug-number-in-closes 13:#XXXXXX
Finished running lintian.
```

These should eventually be fixed, but none of them look like they'll be a problem for trying the package. So let's ignore them for now.

Look in the parent directory to find the package that was built.

```
$ ls ..
```

```
hithere-1.0                hithere_1.0-1_amd64.deb  hithere_1.0.o
hithere_1.0-1_amd64.build  hithere_1.0-1.debian.tar.gz
hithere_1.0-1_amd64.changes hithere_1.0-1.dsc
```

Step 5: Install the package

The following command will install the package that you've just built.

Do NOT run it on a computer unless you don't mind breaking it.

In general, it is best to do package development on a computer that is well backed up, and that you don't mind re-installing if everything goes really badly wrong.

Virtual machines are a good place to do development.

```
$ sudo dpkg -i ../hithere_1.0-1_amd64.deb
```

```
[sudo] password for liw:  
Selecting previously deselected package hithere.  
(Reading database ... 154793 files and directories currently instal  
Unpacking hithere (from ../hithere_1.0-1_amd64.deb) ...  
Setting up hithere (1.0-1) ...  
Processing triggers for man-db ...  
liw@havelock$
```

How do we test the package? We can run the command.

```
$ hithere
```

It works!

But, it's not perfect. Remember, lintian had things to say, and **debian/copyright** is empty, etc.

We have a package that works, but it isn't yet of the high quality that is expected of a Debian package.

Conclusion

Once you've built your own packages, you'll eventually want to learn how to set up your apt repository, so your package is easy to install. The best tool for that I know of is [DebianPkg: reprepro](#).

For more testing of your package, you may want to look at the tool called [DebianPkg: piuparts](#). (I wrote it originally so it is perfect and never has any bugs. er...)

And, finally, if you start making changes to the upstream source, you'll want to learn about the [DebianPkg: quilt](#) tool.

Other things you might want to read are listed in the [🌐 Debian Developers' Corner](#).

See also

- the [packaging FAQ](#) answers some common questions
 -  [Debian Developers' Corner](#)
 -  [Debian Policy Manual](#)
 -  [Writing manual pages](#) by Lars Wirzenius
 - the [rebuilding tutorial](#)
 - [Packaging](#) is the page that gathers everything about packaging on this wiki
 -  [Debian Women IRC log: making Debian packages](#)
 -  [Debian Women IRC log: taking an existing package, re-building it, applying changes to it, and preparing those changes so as to send them as a bug patch.](#)
-

[CategoryPackaging](#)

Packaging/Intro ([last modified 2025-06-18 15:16:58](#))

- [BuildingTutorial](#)

[Translation\(s\)](#): English - [Español](#) - [Italiano](#) - [Português \(Brasil\)](#)- [Русский](#)

Take an existing package, re-build it, apply changes, and prepare those changes to send a patch to the bug-tracker. Also useful if you want to rebuild a testing/unstable package (e.g. as a [backport](#)). For more tutorials, see [Packaging](#).

Contents

1. Introduction

2. Requirements

1. configure apt

2. create a working directory

3. Choose the package

3. The packaging workflow

1. Get the source package

1. Method 1: apt source

2. Method 2: dget

3. Method 3: debcheckout/origtar.gz

2. Get the build dependencies

3. Rebuild without changes

4. Edit the source code

5. Building the modified package

6. Installing and testing the modified package

7. Building the source package

8. Sending your changes to the BTS

4. Conclusions

1. That's it, get ready for the next package

2. Feedback

3. Questions and Answers

4. See also



Written in 2011

This tutorial describes recommended practice as of 2011. The theory hasn't changed, but the specific tools are no longer recommended for general use.

A simplified version of this guide suitable for newbies and updated in 2025 is available at [BuildingTutorialSimplified](#)

For more guides, see [Packaging](#).

Introduction

This tutorial is based on a  [Debian Women Build it Event](#) held by Margarita Manterola (in collaboration with the  [OpenHatch project](#)) on 07-May-2011

This tutorial is now available as a video from Debian's Peertube instance in  [English](#) and  [Malayalam](#)

Requirements

You need very little previous knowledge for this tutorial, just no fear of the command line 😊

Technical requirements:

- You should have a working Debian distribution or a Debian based distribution. You can follow [Packaging/Pre-Requisites](#) if you need help setting up a Debian Unstable environment.
- You should have administration rights in this computer (either root or [DebianPkg: sudo](#)). Every time that admin rights are needed, we'll include `sudo` in front. If you don't use `sudo`, just get the rights whatever way you like.
- [DebianPkg: build-essential](#)
- [DebianPkg: fakeroot](#)
- [DebianPkg: devscripts](#)

Remember:

```
sudo apt-get install build-essential fakeroot devscripts
```

configure apt

Once you have installed the needed packages, the next thing that you need to do, is make sure that you have some **source repositories** configured in your computer.

Open your `/etc/apt/sources.list` file and check if you have one or more lines that start with **deb-src**.

```
deb-src http://deb.debian.org/debian unstable main
```

This line is needed in order to work with source packages.

If you don't have any `deb-src` lines, you'll need to add at least one . This is usually achieved by copying one of the existing `deb` lines and changing it to `deb-src`. You can do that by running an editor with admin rights (`sudo gedit`, `sudo kate` or `sudo vim`).

Usually, it's a good idea to use **unstable** as the repository, since you'll be working with the latest version of the package. But if you intend to modify a package as it is in **stable** or **testing** you could use those distributions as well.

If you use stable/testing/whatever as your running distribution, getting source from unstable won't affect it.

Once you've added the line, you'll need to do

```
sudo apt-get update
```

in order to update the list of packages available for installation.

create a working directory

With the sources URL added to your apt repositories, you'll now be able to get the source of **any Debian package that you like**.

For this particular tutorial, we are going to download the source of one package and make a small modification to it, so that it works better.

It's always a good idea to have a directory that you use to work with source software, separated from other directories used for other stuff. In case you don't already have one, I'd suggest that you create a directory *src* with another one called *debian* inside it:

```
mkdir -p src/debian/; cd src/debian
```

Inside this directory we will get the source of the package that we want to work with.

Choose the package

Note: node-pretty-ms instructions is recent so fdupes example below can be skipped (it is kept as historic artifact only).

Example 1: node-pretty-ms

In this example, we'll use a package called [DebianPkg: node-pretty-ms](#), a library to print time in human readable format.

Example 2: fdupes

In this example, we'll use a package called [DebianPkg: fdupes](#), a tool to detect duplicate files, and we will be fixing the [Closed: #585426: fdupes: Strange help message: Debian bug 585426](#).

You should install the package (or check if you have it installed and up to the latest version) before proceeding, since you'll need to have the dependencies sorted up when you want to install the modified one.

If you don't have [DebianPkg: fdupes](#) installed, you can do this by doing:

```
sudo apt-get install fdupes
```

and check that the bug is still present. You can do that by running

```
fdupes --help
```

and checking that the second line of info for the *--debug* option still doesn't make any sense.

The packaging workflow

Get the source package

Method 1: apt source

In order to **get the source** of any package, what you need to do is go to your chosen directory (*src/debian* in this example) and do (as normal user):

Example 1: node-pretty-ms

```
apt-get source node-pretty-ms
```

Example 2: fdupes

```
apt-get source fdupes
```

.

Method 2: dget

You can also find link to .dsc file from <https://tracker.debian.org> (or for older versions from <https://snapshot.debian.org>) and use `dget` command to download the source.

Example 1: node-pretty-ms

```
dget https://deb.debian.org/debian/pool/main/n/node-pretty-ms/node-pretty-ms_7.0.1-1
```

Example 2: fdupes

```
dget http://snapshot.debian.org/archive/debian/20100529T162939Z/pool/main/f/fdupes/fdu
```

You have now downloaded the 3 files (1. `.dsc`, 2. `.orig.tar.*`, 3. `.diff.gz` or `.debian.tar.*`), composing the **Debian source package**.

Once the package is downloaded, you can check the directory where you are (typing `ls`), and you'll find that apart from the 3 files that were downloaded you also have a directory, called **node-pretty-ms-7.0.1** or **fdupes-1.50-PR2**. This is the **unpacked source/source directory** of the Debian package.

Note: In case the extraction of .dsc file failed with an error like this,

```
dscverify: node-pretty-ms_7.0.1-1.dsc failed signature check:
gpg: no valid OpenPGP data found.
gpg: the signature could not be verified.
Please remember that the signature file (.sig or .asc)
should be the first file given on the command line.
Validation FAILED!!
```

Then you can extract the .dsc file manually with,

```
dpkg-source -x node-pretty-ms_7.0.1-1.dsc
```

This will create the directory mentioned above.

To enter that directory, type:

Example 1: node-pretty-ms

```
cd node-pretty-ms-7.0.1
```

Example 2: fdupes

```
cd fdupes-1.50-PR2/
```

Method 3: debcheckout/origtargz

You can also get the source repo from the version control system (for example, git).

Example 1: node-pretty-ms

```
debcheckout node-pretty-ms  
cd node-pretty-ms  
origtargz
```

Example 2: fdupes

```
debcheckout fdupes  
cd fdupes  
origtargz
```

Note: in case *origtargz* fail with error, use *gbp export-orig --pristine-tar* or other methods explained above (*apt source* or *dget*).

When you check the contents of this directory (typing `ls` again), you'll see quite a number of files of different sorts, and a **debian** directory.

Every Debian (or Debian derivative) package includes a *debian* directory, where all the information related to the Debian package is stored. Anything that's outside of that directory, is the **upstream code**, i.e. the original code released by whoever programmed the software.

Go into the **debian** directory, by typing

```
cd debian
```

This is the directory that the package maintainer has added to the source code to build the package.

In this directory you'll usually find lots of files related to Debian's version of the program, Debian specific patches, manpages, documentation, and so on. We won't be going any deeper about these files here. Look at its contents by typing `ls`.

Just keep in mind that

- the **rules** file is the executable file that we will be running in order to build the package.
- in the **patches** directory, there are also a number of patches applied by the maintainer

Let's move one directory back, by doing

```
cd ..
```

You should be again at **node-pretty-ms-7.0.1** or **fdupes-1.50-PR2**, the main directory of the source code.

Get the build dependencies

In order to build almost any program, you will need some **dependencies** installed.

The dependencies are the programs or libraries needed to compile your program. Usually it's a bunch of packages that end in *-dev*, but it might also be other things like [DebianPkg: automake](#) or [DebianPkg: gcc](#), depending on how many development tools you've ever installed in that machine.

[DebianPkg: apt](#) provides a way of easily installing all the needed dependencies:

Example 1: node-pretty-ms

```
sudo apt build-dep node-pretty-ms
```

Example 2: fdupes

```
sudo apt-get build-dep fdupes
```

Once you've downloaded these tools, you'll be ready to build the package.

Rebuild without changes

Before making any changes to the code, let's **build the package** as it is right now, just to make sure that it builds and it installs properly. Do:

```
debuild -b -uc -us
```

The extra parameters are to build a binary only package (.deb) and prevent it from signing the package, since we don't need to sign it right now.

This command will probably take a while to run, since usually it first has to run `./configure`, then it has to compile the source code and then build the packages. In fact, it will run the commands that are listed in the *debian/rules* file and will hopefully end with a message like:

Example 1: node-pretty-ms

```
dpkg-deb: building package 'node-pretty-ms' in '../node-pretty-ms_7.0.1-1_all.deb'.
```

Example 2: fdupes

```
dpkg-deb: building package `fdupes' in `../fdupes_1.50-PR2-3_<your arch>.deb'
```

In your own language. <your arch> can be *i386*, *amd64*, depending on which architecture you are running your machine on. Architecture indepent packages can be *all*.

Once the package has correctly built, the next step is to **install this file** with:

Example 1: node-pretty-ms

```
apt install ../node-pretty-ms_7.0.1-1_all.deb
```

Remaining sections are relevant to Example 2 only.

Example 2: fdupes

```
sudo dpkg -i ../fdupes_1.50-PR2-3_<your arch>.deb
```

After that, **check that the bug is still present**, running

```
fdupes --help
```

Edit the source code

Now, we want to actually **fix this bug**.

Here comes the fun part. When you are trying to fix a package bug, sometimes it will be located in the upstream source, sometimes it will be related to how the program was packaged for Debian. So you'll be editing different files depending on where the problem is.

In this particular case, the package uses the [DebianPkg: dpatch](#) tool, a tool to manage patches for the package, so we will make use of that tool. Other packages use a different tool, called [DebianPkg: quilt](#), to manage patches, but we won't be covering that one in this tutorial.

To create a new patch, you'll need to do the following. Type:

```
dpatch-edit-patch 80_bts585426_fix_help 70_bts537138_disambiguate_recurse.dpatch
```

This will start a new shell inside a special environment, where you can edit your files and *dpatch* will afterwards take care of getting the differences with the original.

- the first parameter is the name assigned to the new patch : 80_bts585426_fix_help
- the second parameter is the last patch that should be applied before applying the new one : 70_bts537138_disambiguate_recurse.dpatch

The name of the patch was chosen to match the pattern already established by the maintainer.

Now we need to edit the **fdupes.c** file. Go to the line 1066 and delete it. The line says:

```
printf("                \teach set of duplicates without prompting the user\n");
```

You can edit the file with your preferred editor (`vi fdupes.c`, `gedit fdupes.c`, `kate fdupes.c`, etc).

Once you are done, you should type into the console :

```
exit
```

It will end the special environment that *dpatch* created for us, and you'll have a new patch in the **debian/patches/** directory. Check it out with:

```
cat debian/patches/80_bts585426_fix_help.dpatch
```

In order for this patch to get applied, you'll need to edit the **debian/patches/00list** file, and add after the last line :

```
80_bts585426_fix_help
```

The *00list* file is the *dpatch* file that lists all the patches that will be applied. They are applied in order, from the one appearing in the first line, till the one appearing in the last line.

Before rebuilding the package with this patch, we want to **make our package different from the original one**, so that we can afterwards extract the changes in order to send them as a patch to the bug. In order to do this, type:

```
dch -n
```

This will

- add a new entry in the *changelog* file, maybe with your name (depending on other configurations that we are not going to cover), with the current date,
- and open the changelog with the configured command-line editor.
 - In case this is *vi*, and it's your first time with *vi*, you can start editing by pressing the *Insert* key, and after you are finished, you can save and close by pressing: *ESC :wq*

So, you are editing the **changelog** file now. What you have to enter in this file is some description of the change that we've made. For example:

```
Added a patch to fix the --help issue with the --debug option. Closes: #585426".
```

Do this in the line with the empty `*`.

Building the modified package

Once this is done, just build the package again with the same command as before:

```
debuild -b -uc -us
```

Add `-tc` option if you want to clean build artifacts.

In the case that you need to debug the compiled package, particularly if it's a *Segmentation Fault* that you are trying to fix, you might want to compile it like this, so that the code is not optimized and not stripped and it's easier to debug:

```
DEB_BUILD_OPTIONS='nostrip noopt debug' dpkg-buildpackage -b -uc -us
```

You'll see a some compiling output on screen. This is usually not very interesting, unless you are looking for a bug that is related to the compilation of the package itself. Normally, I just let this go while I do something else (grab some cookies for your coffee, for example).

This time, the package created should be: `../fdupes_1.50-PR2-3.1_<your arch>.deb`, the version changed because `dch` changed it for us in the changelog (`-3.1` instead of `-3`).

Installing and testing the modified package

Install it with:

```
sudo dpkg -i ../fdupes_1.50-PR2-3.1_<your arch>.deb
```

and **test** that the help is now correct. 😊

If in any case what you've done has made things worst, you can always revert to Debian's version by doing:

```
apt-get install --reinstall fdupes=<previous_version>
```

Building the source package

Once a bug is fixed, you might want to also **build the source package**. This means not only the `.deb` file, but the other files that we downloaded at the beginning. This is done with:

```
debuild -us -uc
```

If you've built the source package, go to the previous directory with `cd ..` and check the files there with `ls`. You'll see that you have more files there now, including 2 *dsc* files, one for the original package and one for the one you just made.

Sending your changes to the BTS

Once you've built any source package, you can **find out the difference between your package and the original one**, by using [DebianPkg: debdiff](#):

```
debdiff fdupes_1.50-PR2-3.dsc fdupes_1.50-PR2-3.1.dsc > my_first_debdiff.diff
```

In this particular case, since we used the *dpatch* tool, what we would send to the BTS as a patch is the *dpatch file* that we created, because the change that we made is enclosed there.

But if we hadn't used *dpatch*, we could use the output of that `debdiff` and send that to the BTS.

Conclusions

That's it, get ready for the next package

You're done with modifying the package, you can now keep fixing bugs in other Debian packages! These are the important commands you'll need to remember:

```
apt-get source the-package
apt-get build-dep the-package
(...)
fakeroot debian/rules binary
dpkg -i the-package-blabla.deb
dpkg-buildpackage -us -uc
debdiff package-blabla.dsc package-blabla.1.dsc > another_debdiff.diff
```

Feedback

I hope you enjoyed the tutorial, please  [send me your feedback](#) if you feel like it, or modify this wiki if you want to add some useful information I might have left out.

Questions and Answers

QUESTION: Is the number 80_ in the name of the patch a cosmetic thing?

ANSWER: It's so that you can get them in order when you list them with `ls`. The order of application is the order in which they appear in the `00list` file. Some maintainers choose to give them numbers, some don't. But when you are patching a package, you should try to follow whichever pattern the maintainer chose.

QUESTION: About `dpatch-edit-patch` params, where did you get your params? I mean the `bts****`

ANSWER: The first one is the name of the patch. I chose a name similar to the other ones. "bts" stands for "bug tracking system", the number is the bug number that we are closing.

QUESTION: How does dch pick the number "3.1"?

ANSWER: We used `dch -n`, because we are not the maintainer of the package (n stands for non-maintainer). Then it's 3.1. If you were the maintainer, you'd have done `dch -i`, and it would have chosen -4

QUESTION: When sending a patch to the BTS, do we send anything about the changelog?

ANSWER: Not necessarily, but you can send that too, it depends on the bug that you are fixing. Sending the debdiff output is always acceptable.

If you're sending only a small and simple patch it's normal to let the maintainer write the changelog; if it's a big NMU update then yes you should send the changelog as part of your big patch to the BTS.

QUESTION: Sending a changelog with a NMU is considered not polite?

ANSWER: What some (not all) maintainers don't like is if when someone actually uploads an NMU. Sending a patch to the BTS is always/immer/siempre/sempré well received!

QUESTION: What is the quickest/easiest way to find out which tool (if any) is used to manage patches?

ANSWER: If there's a build dependency on quilt or dpatch in debian/control. Or if there's a debian/README.source file.

QUESTION: Does this workflow work for all packages?

ANSWER: The workflow is quite general. The only thing that might change is the patch management system (dpatch/quilt/none). The rest is basically always the same.

See also

- [the packagaing introduction](#) covers packaging basics
- [AdvancedBuildingTips](#) takes you a bit further in your package making
- [Packaging](#) is the page that gather everything about packaging on this wiki

- [AdvancedBuildingTips](#)

[Translation\(s\)](#): [English](#) - [Italiano](#) - [Português \(Brasil\)](#)

Packaging details that aren't covered in [the packaging intro](#) or [building tutorial](#). For more information, see [Packaging](#).



Written 2006-2010

This page focusses mainly on packaging as it was understood in the late 2000's. The theory hasn't changed, but many of the specific tools are no longer recommended for general use.

For more guides, see [Packaging](#).

Once you've downloaded, modified and built a Debian package, you usually feel that you could keep doing it, since **it's so easy**.

And it's just like that, but even if the [Building Tutorial](#) is a good introduction to modifying Debian packages, there were some things that were left out on purpose, to keep it simple. So, here you'll find some extra tips to take full advantage of some of the many Debian tools.

Useful packages

Although they might not be build dependencies, there's a bunch of useful packages that anyone interested in helping Debian development should have installed:

- **build-essential**: includes the essential packages for building your own ones, this is probably already installed in your system, but it never hurts to check.
- **devscripts**: lots of useful scripts for Debian Development (including: dch, debuild, uscan, uupdate and many others).
- **lintian**: package checker, that verify the built package is in good shape regarding Debian Policy.
- **diff**, **patch**, **patchutils** and **quilt**: utilities for working with patches.
- **git**, **dggit**: utilities for working with packages in git

And here's a useful command line, to copy and paste:

```
apt install build-essential devscripts lintian diffutils patch patchutils
```

Using the changelog

The `debian/changelog` file is one of the important files in the `debian` directory. Let's take one changelog entry as an example:

```
control-center (1:2.8.1-3) unstable; urgency=low
* debian/rules:
  - Corrected erroneous line responsible for not including the .desktop
    files (Closes: #274401)
* debian/patches:
  - Suppressed 'Text Editor' in the "preferred applications" as it's not
    with the new mime type system.
-- Arnaud Patard <arnaud.patard@rtp-net.org> Thu, 25 Nov 2004 21:16:00
```

There are many things to notice in this entry: in the first line you have the name and version of the source package, also the distribution to which it is initially uploaded to (unstable), and the urgency level (this is used by Debian bots to determine when the package is ready to enter testing).

Then, we have a bulleted list with the files that were changed and a very brief summary of the reasons for the changes. Note the [Closed in control-center/1:2.8.1-2, control-center/1:2.8.1-3: #274401: gnome-font-viewer isn't registered for application/x-font-ttf and x-font-type1: 274401](#), this is used by Debian bugs bot to automatically close bugs.

Finally, we can see the name and email of the last person to change the package and the date on which it was changed.

Now, to edit the changelog, if you have installed **devscripts** you'll have a nifty command `dch` that prefills a lot of this information and opens your preferred editor. Although you can edit everything manually if you want, I recommend using `dch`.

Important: when you first start modifying a package, you'll want to make a new version of it, so that you can differentiate it from the original one. So, the first time you are about to edit a changelog entry, you should do

```
dch -i
```

This will add a whole new entry, incrementing the version number and letting you write which files you modified. After that, you can always add more info about your changes by just using `dch`.

Building the source package

When you do `debian/rules binary` you just build the binary package, the one that you want to install on your system. But in many cases you want to also build the source package (which is actually composed of three files: `package_version.orig.tar.gz`, `package_version.diff.gz` and `package_version.dsc`).

To do this there are lots of tools. Here, we'll be using `dpkg-buildpackage` that comes with the **dpkg-dev** package (part of **build-essential**).

Provided you already modified the package and the package's changelog, you'll build it by doing:

```
dpkg-buildpackage -rfakeroot
```

(There are many options to pass to `dpkg-buildpackage`, `-S` for example, will only build the source package, `-tc` will clean the compiled files after making the package, etc. See `man dpkg-buildpackage` for more info)

Once you've done this, you'll find a new trio of files in your parent directory, that will have almost the same name as the ones you downloaded but with a change in the version number.

Using interdiff

If you've built a package, incremented the version number in the changelog file, and now you have two sets of three files (two source packages) you can gather what the changes between versions were very easily by using `interdiff`:

```
interdiff -z package_oldversion.diff.gz package_newversion.diff.gz >
```

After running this command you can inspect the contents of `your.patch`, it should contain everything that you changed to the package. If you are solving a bug in the Debian BTS, this is what you should send to the bug number.

Note that `interdiff` only makes sense when you haven't changed the upstream source (this is to say, the `package_version.orig.tar.gz` has to be the same).

Using lintian

Not all Debian packages comply perfectly with Debian's policy. Sometimes this is due to the policy getting quickly updated, sometimes this is because of lack of time from the maintainer, sometimes because it's not easy for certain software to be totally compliant.

In any case, these tools provide us with an easy way to find problems in Debian packages. Using them is easy: you just run them against the `.diff.gz` and `.dsc` files of your source package.

Depending on the package, you could get no messages, some messages or a lot of messages. These messages are usually brief and not always will you know what they mean, but many times they'll point you to the problematic part of the package.

Of course, these tools are just an aid, not a replacement to **actually reading** Debian's policy.

Getting help

If you get stuck in what you are doing, there's lots of places where you can get help.

- You can go to the  [Debian Developer's corner](#) where you can find lots of information online.
- You can ask online at `irc.debian.org` (OFTC), on `#debian-women`, `#debian-mentors`, `#debian-bugs` or `#debian-devel` (choose the one that is closer to what you are doing).
- Or you can ask on one of the many  [Debian Mailing Lists](#).

[CategoryPackaging](#)

AdvancedBuildingTips (last modified 2025-06-18 15:18:28)

- [PackagingWithGit](#)

Translations: [English](#) - [Deutsch](#) - [Italiano](#) - [Indonesian](#) - [Português \(Brasil\)](#) - [Русский](#)

This page describes how to create debian packages using the `git` version control system. For general information about the version control system, see [git](#). For general information about creating packages, see [Packaging](#).

The attached image shows the packaging model as described by this page. You may prefer to think about it a different way.

 The most common git packaging suite is "*git-buildpackage*". The most common command in that suite is *gbp* buildpackage.

Contents

1. [Configure git-buildpackage](#)

2. [Import stage](#)

1. [Import from tarballs](#)
2. [Import from an upstream git repository](#)
3. [Import from both tarballs and git](#)
4. [If there is no upstream](#)

3. [Patch stage](#)

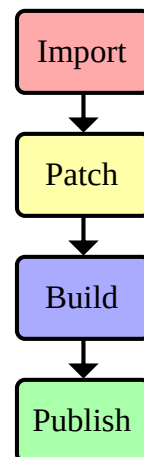
1. [Create the debian/ directory](#)
2. [Manage debian/changelog](#)
 1. [Common policy questions](#)
 2. [Helpful tools](#)
3. [Manage debian/copyright](#)
4. [Manage files outside of debian/](#)

4. [Build stage](#)

1. [Build script](#)

5. [Publish stage](#)

6. [See also](#)



Configure git-buildpackage

[DebianPkg: git-buildpackage](#) is the most popular suite of tools to create Debian packages, but its default configuration should be modified before use. Edit your `~/ .gbp.conf` to contain at least the following:

```
[DEFAULT]

# Manage a "pristine-tar" branch:
pristine-tar = True
pristine-tar-commit = True

# Use Debian-style branch names:
upstream-branch=upstream/latest
debian-branch=debian/latest

# Build packages with pbuilder:
pbuilder = True

# Or use sbuild:
#builder = sbuild

# Sign tags (optional):
```

```
#sign-tags = True
#keyid = <fingerprint>
```

pristine-tar and pristine-tar-commit

If you need to import a project from an upstream tarball, you might some day be asked to prove that a commit maps to a tarball with a specific checksum. These options keep track of e.g. the order that files appeared in the tarball (see [pristine-tar](#), [--git-pristine-tar](#) and [--git-pristine-tar-commit](#)).

upstream-branch and debian-branch

[DEP-14](#) recommends using `upstream/latest` instead of `master`, and using `debian/latest` for the Debianised version of `upstream/latest`. These options tell `git-buildpackage` to use those names (see [--git-upstream-branch](#) and [--git-debian-branch](#)).

pbuilder and builder

Packages should be built in a clean chroot environment, so your local configuration doesn't contaminate the package. `pbuilder=True` uses the [pbuilder](#) stack, and is a good default choice. Or `builder=sbuild` uses [sbuild](#), which provides more flexibility for complex use cases. Your stack will need to be configured before use - see the relevant page for details.

sign-tags and keyid

If you want to PGP-sign your git tags, set `sign-tags=True` and set `keyid` to the fingerprint for the key marked [SC] in the output of `gpg --list-secret-key`. This will attach a PGP signature when you do `gbp buildpackage --git-tag`, proving you were the one who created that tag.

`/etc/git-buildpackage/gbp.conf` lists many more useful options - you might like to come back and review it once you're comfortable with your workflow. For details, see [the man pages for each command](#).

[i](#) `git-buildpackage` has [DebianMan: several configuration files](#). `~/ .gbp.conf` is for configuration that applies to all your projects. For branch- and project-specific configuration, use your project's `debian/gbp.conf` or `.git/gbp.conf`.

Import stage

The *import* stage involves creating a git repository based on whatever upstream provides, and optionally describing how to get updates from them. That usually means downloading/scanning tarballs on an FTP server, cloning/pulling changes from a [forge](#), or branching/merging commits in your own repo.

Import from tarballs

The traditional way to get upstream code is to import their latest release tarball:

```
git init --initial-branch="$(gbp config DEFAULT.debian-branch)" some-project.git
cd some-project.git
gbp import-orig https://some-project.example.com/releases/some-project_1.0.tar.gz
```

This will pull the tarball into the upstream and debian branches you specified in your [~/gbp.conf](#), create a new `upstream/1.0` tag, and update the `pristine-tar` branch (if you set that option).

You should also create a `debian/watch` file to tell `git-buildpackage` how to pull new tarballs. For details, see [debian/watch](#).

`git import-orig` discards any top-level `debian/` directory from your tarball. In the unlikely event this is a problem, see [ManageUpstreamDifferences](#).

Once you have created your `debian/watch` and [debian/changelog](#), you can get new versions of the tarball with `gbp import-orig --uscan`. Or if you can't create a `debian/watch` file, you'll have to download a new tarball and call `gbp import-orig <tarball>` every time.

[i](#) you might like to add a [git alias](#) for your import command. For example `git config --local alias.get '!gbp import-orig --uscan'` makes `git get` work in this repository

Import from an upstream git repository

If your upstream doesn't provide tarballs but does put their source code in a git [forge](#) with [release tags](#) that have a useful pattern (like vMAJOR.MINOR.PATCH or release-MAJOR.MINOR.PATCH), adapt the following to suit your requirements:

```
# Clone the upstream repository and `cd` into it:
git clone https://forge.example.com/some-project.git
cd project.git

# Use DEP-14 branch names, but don't bother with an upstream branch:
git branch -m debian/latest

# Describe what upstream tags look like (see below for details):
mkdir -p debian
cat >> debian/gbp.conf <<EOF
[DEFAULT]
upstream-tag=v%(version)s

# No tarballs to track (upstream uses git):
pristine-tar = False
pristine-tar-commit = False
EOF
git add debian/gbp.conf
```

By default, `git-buildpackage` assumes upstream tags are of the form `upstream/%(version)s`. Set `upstream-tag` in `debian/gbp.conf` if your upstream uses a different format. For details, see [the manual](#).

Disabling `pristine-tar` and `pristine-tar-commit` avoids creating a redundant extra branch, and makes the project configuration clearer for anyone reviewing your work.

If your upstream doesn't create useful release tags, consider asking them to start. If they're not willing to create tags, but do provide release tarballs, you'll have to [import from tarballs](#) - Git and Debian both have excellent alternatives to `git pull`, but can't replicate the information in a release tarball.

If your upstream contains a top-level `/debian` directory, you will need to delete it from the upstream branch. For details, see [ManageUpstreamDifferences](#).

To get new versions from an upstream git repository, just do a normal `git pull` from `upstream/latest`.

Import from both tarballs and git

If your upstream provides tarballs *and* a git repository, you can import both and compare them:

```
# Import the tarball first:
git init --initial-branch="$(gbp config DEFAULT.debian-branch)" some-project.git
cd some-project.git
gbp import-orig https://some-project.example.com/releases/some-project_1.0.tar.gz

# Then import the git repo:
git remote add origin https://forge.example.com/some-project.git
git fetch

# Compare the tarball and the repo (remember to use the right tags):
git diff upstream/1.0 v1.0
```

If `git diff` shows one source or other is definitely correct, delete `some-project.git` and start over with the appropriate set of instructions. Or if you need to track both, configure [debian/watch](#) and [debian/changelog](#), then get new versions with something like `gbp import-orig --uscan && git fetch`.

If there is no upstream

Some projects don't have an upstream. For example, Debian-specific projects often have a single repository with upstream code in one branch and Debian packaging in another.

If possible, set your main development branch name to `upstream/latest`:

```
# Set branch name when creating the repo:
git init --initial-branch="$(gbp config DEFAULT.upstream-branch)" project.git
# Or rename the branch when you start to Debianise:
git branch -m master "$(gbp config DEFAULT.upstream-branch)"
```

The rest of this page assumes your project is ready to package, so you may want to pause here and come back once you finish your first version.

Once you've finished your first version, check out a new branch for Debian-specific changes:

```
git checkout -b "$(gbp config DEFAULT.debian-branch)" \
    --track "$(gbp config DEFAULT.upstream-branch)"
```

Projects without an upstream repository can usually choose their own tag pattern and ignore upstream tarballs. Consider doing:

```
mkdir -p debian
cat >> debian/gbp.conf <<EOF
[DEFAULT]
upstream-tag=v%(version)s

# No tarballs to track (no upstream repo):
pristine-tar = False
pristine-tar-commit = False
EOF
git add debian/gbp.conf
```

Because `debian/latest` tracks `upstream/latest`, you can get upstream changes with `git pull` just like you would with a remote repository.

 Debian-specific packages that will *never* have an upstream repository can use the **native** patch system instead of the default `quilt` model - see [Projects/DebSrc3.0](#).

Patch stage

The *patch* stage involves creating files in your `debian/` directory, and optionally changing other directories as required by Debian. For example, Debian requires programs to have a `man` page, which you should create if it's missing upstream.

 This section is a language-neutral guide to patching a project. Prefer [language-specific guides](#) where available.

Create the `debian/` directory

Debian metadata is stored in `debian/`, which is usually initialised with [DebianPkg: debmake](#) or [DebianPkg: dh-make](#). Unfortunately, they only work with tarballs:

```

# Specify your project name and version:
PROJECT_ID=<project> # e.g. "myproject"
PROJECT_VERSION=<version> # e.g. "1.0"

# Remember to run this in your debian branch:
git checkout debian/latest

# Create a release tarball and associated directory:
mkdir -p tmp debian
git archive HEAD --prefix="${PROJECT_ID}-${PROJECT_VERSION}"/ \
    -o tmp/"${PROJECT_ID}_${PROJECT_VERSION}".orig.tar.gz
cd tmp
tar xzf "${PROJECT_ID}_${PROJECT_VERSION}".orig.tar.gz

# Run debmake and move debian/ into your repo:
cd "${PROJECT_ID}-${PROJECT_VERSION}"
debmake # or dh_make
mv -n debian/* ../../debian/
git add ../../debian/

# Tidy up:
cd ../../
rm -rf tmp/"${PROJECT_ID}-${PROJECT_VERSION}"
rm -rf tmp/"${PROJECT_ID}_${PROJECT_VERSION}".orig.tar.gz
rmdir tmp

# Commit auto-generated files before making manual changes (optional):
git commit debian/ -m 'Create initial debian/ directory with debmake'

```

This will create all the files you're likely to need, plus .ex examples for files you might need to create.

Read through all the files in the `debian/` directory. [Debian policy](#) explains which of these you need and what they should look like. Tests during the build stage (e.g. [lintian](#)) will tell you anything you missed. `debmake` and `dh-make` have different opinions about which defaults are best - adapt your favourite suggestions from each of them.

[DebianPkg: dh-make](#) (with a dash) is the name of a package, `dh_make` (with an underscore) is the name of the main program in that package. See [DebianMan: dh_make\(1\)](#) for information about the questions it asks.

Manage debian/changelog

This file acts both as a human-readable history of the project, and as a machine-readable configuration file for e.g. the version number and target distribution. It will be installed to `/usr/share/doc/<your-package>/changelog.Debian.gz`.

You need to decide a policy about what to include in your changelog. This section discusses some policy questions to help decide your policy, and some tools to help automate your workflow.

See also [DebianChangelog](#).

Common policy questions

What will your version numbers look like?

Most projects use [semantic versioning](#), but some use calendar versioning, or don't have version numbers at all. You'll need a way to extract version numbers from the upstream project, and may need to reformat the numbers for Debian.

Do you want to include upstream changes in the changelog?

Some projects go to great lengths to tell users what changed in each version, others just keep their man pages up-to-date and expect users to re-read them now and then. It's good to Debianise upstream changes if they're available; but it's not worth trying to write new information that wasn't recorded upstream.

Do you want to generate your changelog from the git commit history?

If your upstream project has a neatly-maintained `CHANGES` file, you might prefer to write your own script to convert it to Debian format. Otherwise, adapt your workflow so you can generate a changelog from git commits.

If your changelog is based on the commit history, which commits should you include?

For example, your upstream might use [topic branches](#), so only their merge commits belong in the changelog. Or if they don't have any useful information about changes, you might want to exclude upstream commits altogether.

Helpful tools

Guidance about Debianising version numbers is available at [DebianMan: deb-version](#), but also be aware that Debian repositories require every `.deb` to have a unique filename. Different architectures get different filenames automatically, but different distributions need unique [DebianMan: Debian revision numbers](#). For example, you might like to use odd numbers for stable and even numbers for testing, or you could use decorated revision numbers like `-ubuntu1` or `-forky2`.

If you want to write a changelog by hand, use [DebianMan: dch](#) to create and edit entries. The changelog format is quite precise and sometimes unintuitive, and `dch` will help find mistakes. `dch --edit` uses [DebianMan: sensible-editor](#), so you can use [DebianMan: select-editor](#) to configure your preferred editor.

If you want to write a script that generates a changelog, read [DebianMan: the changelog specification](#). You might like to use `dch` to actually edit the file, or just to check you've got the format right.

If you want to generate `debian/changelog` from your git changelog, use [DebianMan: gbp-dch](#). You may be able to exclude upstream commits by calling `gbp dch debian/`, or you may need to use [--git-log](#) to specify [git log](#) parameters. If the latter, consider saving them in your `debian/gbp.conf`:

```
[dch]
# Only include merge commits:
git-log = --merges
# Exclude merge commits:
git-log = --no-merges
# Exclude upstream commits:
git-log = --first-parent
```

`gbp dch --new-version` adds a new entry to the changelog, unless the current entry has `distributions` set to `UNRELEASED`, in which case it merges changes into the current entry. For example, you may want to make several [DebianMan: snapshot commits](#) during development, then merge them together in the public changelog.

Manage debian/copyright

This is a machine-readable file that specifies users' legal rights to use the package, and the files in it. `debmake` should have provided an initial version, but there are many [copyright review tools](#). You can use whichever you prefer, but [licensecheck](#) is a good default, and [cme](#)'s interactive mode may help you learn how the file works.

Getting this right can be very time-consuming, but Debian won't host packages unless they know it's legal to do so. If you have a good relationship with upstream, consider asking them to read [the licensing section of the upstream guide](#).

Manage files outside of debian/

Changes in your `debian` branch but outside your `debian/` directory need special handling. For example, you might need to patch the build system to install files in the right locations, and Debian will refuse to host even an upstream tarball that violates the [Debian Free Software Guidelines](#). For details, see [ManageUpstreamDifferences](#).

Build stage

The *build* stage involves generating and testing your package files, including one or more `.deb` files.

To do a test-build of your package, make sure your `debian/changelog` has the correct version number, then do:

```
gbp buildpackage --git-dist=<dist> --git-arch=<arch>
# Or if you configured pbuilder=True and want to use pbuilder instead of cowbuilder:
BUILDER=pbuilder gbp buildpackage --git-dist=<dist> --git-arch=<arch>
```

If your build succeeds, it should produce a collection of `../<project>_<version>*` files.

[Package checkers](#) look for issues that don't cause build failures. It's particularly important to run [lintian](#), [piuparts](#) and [autopkgtest](#), as Debian will usually reject packages that fail these tests.

`lintian` looks for common issues and violations of Debian policy. For example, boilerplate text that hasn't been modified since you ran `debmake`. You can run it with most tests enabled by doing `lintian --info --display-info --pedantic /path/to/your/package.changes`, or choose your preferred options from the [DebianMan: lintian man page](#).

`piuparts` creates a minimal chroot, then tries to install and uninstall your package in it. It needs `pbuilder-style .tgz` files - it can't use `cowbuilder's .cow` directories. It's often run as `sudo piuparts --log-level info --basetgz /path/to/your/base-environment.tgz /path/to/your/package.deb`, and its options are explained in the [DebianMan: piuparts man page](#).

`autopkgtest` runs any integration tests you've added to `debian/tests`. This usually isn't necessary for simple packages - learn more at [autopkgtest](#).

 if you did not configure `pbuilder=True`, `lintian` will run without arguments during the build. You may want to run it again with your preferred arguments anyway.

Build script

You may want to write a script to automate your build process. Here's an example that shows off several options:

```
#!/bin/sh

# Exit if any programs fail:
set -e

# Can be /var/cache/pbuilder/result for some workflows:
RESULT_DIRECTORY=..

# Get some information from the changelog
SOURCE="$(dpkg-parsechangelog --show-field=Source)"
VERSION="$(dpkg-parsechangelog --show-field=Version)"

# Read "--git-dist=foo" and "--git-arch=bar" from the command-line:
DIST=sid
ARCH="$(dpkg --print-architecture)"
for ARG
do
    case "$ARG" in
        "--git-dist=*" )
            DIST="${ARG#--git-dist=}"
            ;;
        "--git-arch=*" )
            ARCH="${ARG#--git-arch=}"
            ;;
        "--git-export-dir=*" )
            RESULT_DIRECTORY="${ARG#--git-export-dir=}"
            mkdir -p "$RESULT_DIRECTORY"
            ;;
    esac
done
```

```

        -h|--h|--he|--hel|--help)
            cat <<EOF
Usage: $0 [--git-dist=<distribution>] [--git-arch=<architecture>]"

Build a Debian package.
See https://wiki.debian.org/PackagingWithGit#Build\_script
EOF

        exit 1
    ;;

esac
done

if grep -q '^Architecture: all' debian/control
then PKG_ARCH=all
else PKG_ARCH="$ARCH"
fi

CHANGES_FILE="$RESULT_DIRECTORY/${SOURCE}_${VERSION}_${ARCH}.changes"
DEB_FILE="$RESULT_DIRECTORY/${SOURCE}_${VERSION}_${PKG_ARCH}.deb"
LOG_FILE="$RESULT_DIRECTORY/${SOURCE}_${VERSION}_${ARCH}.log"

# Calculate the pbuilder base tarball/path:
BASE=/var/cache/pbuilder/base
if [ -n "$DIST" -a "$DIST" != sid ]
then BASE="$BASE-$DIST"
fi
if [ -n "$ARCH" -a "$ARCH" != "$(dpkg --print-architecture)" ]
then BASE="$BASE-$ARCH"
fi

# Run pbuilder with arguments from the command-line:
echo -n "[$(date)] Running gbp buildpackage" >&2
set -o pipefail
BUILDER=pbuilder gbp buildpackage "$@" \
    | tee "$LOG_FILE" \
    | while read LINE ; do echo -n . >&2 ; done
set +o pipefail
echo " OK" >&2
echo >&2

echo "[$(date)] Running piuparts..." >&2
sudo piuparts --log-level info --basetgz "$BASE.tgz" "$DEB_FILE"
echo "[$(date)] piuparts: OK" >&2
echo >&2

echo "[$(date)] Running autopkgtest..." >&2
set +e
sudo autopkgtest --no-built-binaries --apt-upgrade "$CHANGES_FILE" \
    -- unshare --release "$DIST" --arch "$ARCH"
RESULT="$?"
set -e
case "$RESULT" in
    0) echo "autopkgtest: OK" >&2 ;;
    8) echo "autopkgtest: no tests in this package, or all non-superficial tests were skipped" >&2 ;;
    *) exit "$RESULT" >&2
esac
echo >&2

```



```
echo "[$(date)] Running lintian..." >&2
lintian --info --display-info --pedantic "$CHANGES_FILE"
echo "[$(date)] lintian: OK" >&2
echo >&2

echo "[$(date)] Success!" >&2
```

 if you would rather call `gbp buildpackage` directly, you can call your script from the `buildpackage.postbuild` option in a `gbp.conf` file.

Publish stage

The *publish* stage involves either putting your built files in a repository you control, or requesting someone else (e.g. the Debian project) put it in a repository they control.

If you want to publish your package to Debian itself,  [register for a Salsa account](#) and check the  [user registration section of the FAQ](#). While you're waiting, file an [Intent To Package](#) ("ITP") bug and add an entry to your `debian/changelog` that ends with Closes: `#<itp-bug-number>`.

Change UNRELEASED in `debian/changelog` to the distribution(s) your package should be in. The Debian archive  [only allows one distribution](#), so you may need to publish multiple times, updating the version number each time. Other repositories may accept a single version with a space-separated list of distributions.

Commit your change, then build and tag your release:

```
gbp buildpackage --git-tag
```

This will create a tag based on the `debian-tag` configuration option. By default, it resembles `debian/X.Y-Z`.

If you want to publish your package to Debian itself, push to [salsa.debian.org](#) and follow  [the sponsoring process](#).

Finally, as a precaution, add a new temporary [DebianMan: snapshot commit](#):

```
gbp dch --snapshot
git commit -m 'WIP: next version' debian/changelog
```

If all goes well, you can undo this commit before your next merge, by doing `git commit --reset HEAD^`. But if you accidentally push changes to Salsa, the tests will fail harmlessly instead of releasing unwanted changes to a public repository.

See also

- the git-buildpackage manual is available  [online](#) or at `/usr/share/doc/git-buildpackage/manual-html/`
- other guides to packaging with git
 -  [from the Guide for Debian Maintainers](#)
 -  [from Russ Allbery](#)
 -  [from Philipp Huebner](#)
 -  [Creating Debian packages from upstream Git](#)
- discussions about including the upstream git in the package's git
 -  [by Russ Allbery](#)
 -  [by Joey Hess](#)
- [switching from svn-buildpackage](#)
- general resources about git
 -  [homepage](#)
 -  [wiki](#)

-  [manual](#)

[CategoryDeveloper](#) [CategoryPackaging](#) [CategoryGit](#)

PackagingWithGit ([last modified 2026-01-26 07:14:42](#))

NAME

debconf - developers guide

DESCRIPTION

This is a guide for developing packages that use debconf.

This manual assumes that you are familiar with debconf as a user, and are familiar with the basics of debian package construction.

This manual begins by explaining two new files that are added to debian packages that use debconf. Then it explains how the debconf protocol works, and points you at some libraries that will let your programs speak the protocol. It discusses other maintainer scripts that debconf is typically used in: the postinst and postrm scripts. Then moves on to more advanced topics like shared debconf templates, debugging, and some common techniques and pitfalls of programming with debconf. It closes with a discussion of debconf's current shortcomings.

THE CONFIG SCRIPT

Debconf adds an additional maintainer script, the config script, to the set of maintainer scripts that can be in debian packages (the postinst, preinst, postrm, and prerm). The config script is responsible for asking any questions necessary to configure the package.

Note: It is a little confusing that dpkg refers to running a package's postinst script as "configuring" the package, since a package that uses debconf is often fully pre-configured, by its config script, before the postinst ever runs. Oh well.

Like the postinst, the config script is passed two parameters when it is run. The first tells what action is being performed, and the second is the version of the package that is currently installed. So, like in a postinst, you can use dpkg --compare-versions on \$2 to make some behavior happen only on upgrade from a particular version of a package, and things like that.

The config script can be run in one of three ways:

1

If a package is pre-configured, with dpkg-preconfigure, its config script is run, and is passed the parameters "configure", and installed-version.

2

When a package's postinst is run, debconf will try to run the config script then too, and it will be passed the same parameters it was passed when it is pre-configured. This is necessary because the package might not have been pre-configured, and the config script still needs to get a chance to run. See HACKS for details.

3

If a package is reconfigured, with dpkg-reconfigure, its config script it run, and is passed the parameters "reconfigure" and installed-version.

Note that since a typical package install or upgrade using apt runs steps 1 and 2, the config script will typically be run twice. It should do nothing the second time (to ask questions twice in a row is annoying), and it should definitely be idempotent. Luckily, debconf avoids repeating questions by default, so this is generally easy to accomplish.

Note that the config script is run before the package is unpacked. It should only use commands that are in essential packages. The only dependency of your package that is guaranteed to be met when its config script is run is a dependency (possibly versioned) on debconf itself.

The config script should not need to modify the filesystem at all. It just examines the state of the system, and asks questions, and debconf stores the answers to be acted on later by the postinst script. Conversely, the postinst script should almost never use debconf to ask questions, but should instead act on the answers to questions asked by the config script.

THE TEMPLATES FILE

A package that uses debconf probably wants to ask some questions. These questions are stored, in template form, in the templates file.

Like the config script, the templates file is put in the control.tar.gz section of a deb. Its format is similar to a debian control file; a set of stanzas separated by blank lines, with each stanza having a RFC822-like form:

```
Template: foo/bar
Type: string
Default: foo
Description: This is a sample string question.
This is its extended description.
.
Notice that:
- Like in a debian package description, a dot
on its own line sets off a new paragraph.
- Most text is word-wrapped, but doubly-indented
text is left alone, so you can use it for lists
of items, like this list. Be careful, since
it is not word-wrapped, if it's too wide
it will look bad. Using it for short items
is best (so this is a bad example).
```

```
Template: foo/baz
Type: boolean
Description: Clear enough, no?
This is another question, of boolean type.
```

For some real-life examples of templates files, see
/var/lib/dpkg/info/debconf.templates, and other .templates files in that directory.

Let's look at each of the fields in turn..

The name of the template, in the 'Template' field, is generally prefixed with the name of the package. After that the namespace is wide open; you can use a simple flat layout like the one above, or set up "subdirectories" containing related questions.

Type

The type of the template determines what kind of widget is displayed to the user. The currently supported types are:

string

Results in a free-form input field that the user can type any string into.

password

Prompts the user for a password. Use this with caution; be aware that the password the user enters will be written to debconf's database. You should probably clean that value out of the database as soon as is possible.

boolean

A true/false choice.

select

A choice between one of a number of values. The choices must be specified in a field named 'Choices'. Separate the possible values with commas and spaces, like this:

Choices: yes, no, maybe

multiselect

Like the select data type, except the user can choose any number of items from the choices list (or choose none of them).

note

Rather than being a question per se, this datatype indicates a note that can be displayed to the user. It should be used only for important notes that the user really should see, since debconf will go to great pains to make sure the user sees it; halting the install for them to press a key. It's best to use these only for warning about very serious problems, and the error datatype is often more suitable.

error

This datatype is used for error messages, such as input validation errors. Debconf will show a question of this type even if the priority is too high or the user has already seen it.

title

This datatype is used for titles, to be set with the SETTITLE command.

text

This datatype can be used for fragments of text, such as labels, that can be used for cosmetic reasons in the displays of some frontends. Other frontends will not use it at all. There is no point in using this datatype yet, since no frontends support it well. It may even be removed in the future.

Default

The 'Default' field tells debconf what the default value should be. For multiselect, it can be a list of choices, separated by commas and spaces, similar to the 'Choices' field. For select, it should be one of the choices. For

boolean, it is "true" or "false", while it can be anything for a string, and it is ignored for passwords.

Don't make the mistake of thinking that the default field contains the "value" of the question, or that it can be used to change the value of the question. It does not, and cannot, it just provides a default value for the first time the question is displayed. To provide a default that changes on the fly, you'd have to use the SET command to change the value of a question.

Description

The 'Description' field, like the description of a Debian package, has two parts: A short description and an extended description. Note that some debconf frontends don't display the long description, or might only display it if the user asks for help. So the short description should be able to stand on its own.

If you can't think up a long description, then first, think some more. Post to debian-devel. Ask for help. Take a writing class! That extended description is important. If after all that you still can't come up with anything, leave it blank. There is no point in duplicating the short description.

Text in the extended description will be word-wrapped, unless it is prefixed by additional whitespace (beyond the one required space). You can break it up into separate paragraphs by putting " ." on a line by itself between them.

QUESTIONS

A question is an instantiated template. By asking debconf to display a question, your config script can interact with the user. When debconf loads a templates file (this happens whenever a config or postinst script is run), it automatically instantiates a question from each template. It is actually possible to instantiate several independent questions from the same template (using the REGISTER command), but that is rarely necessary. Templates are static data that comes from the templates file, while questions are used to store dynamic data, like the current value of the question, whether a user has seen a question, and so on. Keep the distinction between a template and a question in mind, but don't worry too much about it.

SHARED TEMPLATES

It's actually possible to have a template and a question that are shared among a set of packages. All the packages have to provide an identical copy of the template in their templates files. This can be useful if a bunch of packages need to ask the same question, and you only want to bother the user with it once. Shared templates are generally put in the shared/ pseudo-directory in the debconf template namespace.

THE DEBCONF PROTOCOL

Config scripts communicate with debconf using the debconf protocol. This is a simple line-oriented protocol, similar to common internet protocols such as SMTP. The config script sends debconf a command by writing the command to standard output. Then it can read debconf's reply from standard input.

Debconf's reply can be broken down into two parts: A numeric result code (the first word of the reply), and an optional extended result code (the remainder of the reply). The numeric code uses 0 to indicate success, and other numbers to indicate various kinds of failure. For full details, see the table in Debian policy's debconf specification document.

The extended return code is generally free form and unspecified, so you should generally ignore it, and should certainly not try to parse it in a program to work out what debconf is doing. The exception is commands like GET, that cause a value to be returned in the extended return code.

Generally you'll want to use a language-specific library that handles the nuts and bolts of setting up these connections to debconf and communicating with it.

For now, here are the commands in the protocol. This is not the definitive definition, see Debian policy's debconf specification document for that.

VERSION number

You generally don't need to use this command. It exchanges with debconf the protocol version number that is being used. The current protocol version is 2.0, and versions in the 2.x series will be backwards-compatible. You may specify the protocol version number you are speaking and debconf will return the version of the protocol it speaks in the extended result code. If the version you specify is too low, debconf will reply with numeric code 30.

CAPB capabilities

You generally don't need to use this command. It exchanges with debconf a list of supported capabilities (separated by spaces). Capabilities that both you and debconf support will be used, and debconf will reply with all the capabilities it supports.

If 'escape' is found among your capabilities, debconf will expect commands you send it to have backslashes and newlines escaped (as `\\` and `\n` respectively) and will in turn escape backslashes and newlines in its replies. This can be used, for example, to substitute multi-line strings into templates, or to get multi-line extended descriptions reliably using METAGET. In this mode, you must escape input text yourself (you can use `debconf-escape(1)` to help with this if you want), but the confmodule libraries will unescape replies for you.

SETTITLE question

This sets the title debconf displays to the user, using the short description of the template for the specified question. The template should be of type title. You rarely need to use this command since debconf can automatically generate a title based on your package's name.

Setting the title from a template means they are stored in the same place as the rest of the debconf questions, and allows them to be translated.

TITLE string

This sets the title debconf displays to the user to the specified string. Use of the SETTITLE command is normally to be preferred as it allows for translation of the title.

INPUT priority question

Ask debconf to prepare to display a question to the user. The question is not actually displayed until a GO command is issued; this lets several INPUT

commands be given in series, to build up a set of questions, which might all be asked on a single screen.

The priority field tells debconf how important it is that this question be shown to the user. The priority values are:

low

Very trivial items that have defaults that will work in the vast majority of cases; only control freaks see these.

medium

Normal items that have reasonable defaults.

high

Items that don't have a reasonable default.

critical

Items that will probably break the system without user intervention.

Debconf decides if the question is actually displayed, based on its priority, and whether the user has seen it before, and which frontend is being used. If the question will not be displayed, debconf replies with code 30.

GO

Tells debconf to display the accumulated set of questions (from INPUT commands) to the user.

If the backup capability is supported and the user indicates they want to back up a step, debconf replies with code 30.

CLEAR

Clears the accumulated set of questions (from INPUT commands) without displaying them.

BEGINBLOCK

ENDBLOCK

Some debconf frontends can display a number of questions to the user at once. Maybe in the future a frontend will even be able to group these questions into blocks on screen. BEGINBLOCK and ENDBLOCK can be placed around a set of INPUT commands to indicate blocks of questions (and blocks can even be nested). Since no debconf frontend is so sophisticated yet, these commands are ignored for now.

STOP

This command tells debconf that you're done talking to it. Often debconf can detect termination of your program and this command is not necessary.

GET question

After using INPUT and GO to display a question, you can use this command to get the value the user entered. The value is returned in the extended result code.

SET question value

This sets the value of a question, and it can be used to override the default value with something your program calculates on the fly.

RESET question

This resets the question to its default value (as is specified in the 'Default' field of its template).

SUBST question key value

Questions can have substitutions embedded in their 'Description' and 'Choices' fields (use of substitutions in 'Choices' fields is a bit of a hack though; a better mechanism will eventually be developed). These substitutions look like "\${key}". When the question is displayed, the substitutions are

replaced with their values. This command can be used to set the value of a substitution. This is useful if you need to display some message to the user that you can't hard-code in the templates file.

Do not try to use SUBST to change the default value of a question; it won't work since there is a SET command explicitly for that purpose.

FGET question flag

Questions can have flags associated with them. The flags can have a value of "true" or "false". This command returns the value of a flag.

FSET question flag value

This sets the value of a question's flag. The value must be either "true" or "false".

One common flag is the "seen" flag. It is normally only set if a user has already seen a question. Debconf usually only displays questions to users if they have the seen flag set to "false" (or if it is reconfiguring a package). Sometimes you want the user to see a question again -- in these cases you can set the seen flag to false to force debconf to redisplay it.

METAGET question field

This returns the value of any field of a question's associated template (the Description, for example).

REGISTER template question

This creates a new question that is bound to a template. By default each template has an associated question with the same name. However, any number of questions can really be associated with a template, and this lets you create more such questions.

UNREGISTER question

This removes a question from the database.

PURGE

Call this in your postrm when your package is purged. It removes all of your package's questions from debconf's database.

X_LOADTEMPLATEFILE /path/to/templates [owner]

This extension loads the specified template file into debconf's database. The owner defaults to the package that is being configured with debconf.

Here is a simple example of the debconf protocol in action.

```
INPUT medium debconf/frontend
30 question skipped
FSET debconf/frontend seen false
0 false
INPUT high debconf/frontend
0 question will be asked
GO
[ Here debconf displays a question to the user. ]
0 ok
GET no/such/question
10 no/such/question doesn't exist
GET debconf/frontend
0 Dialog
```

LIBRARIES

Setting things up so you can talk to debconf, and speaking the debconf protocol by hand is a little too much work, so some thin libraries exist to relieve this minor drudgery.

For shell programming, there is the `/usr/share/debconf/confmodule` library, which you can source at the top of a shell script, and talk to debconf in a fairly natural way, using lower-case versions of the debconf protocol commands, that are prefixed with `"db_"` (ie, `"db_input"` and `"db_go"`). For details, see [confmodule\(3\)](#)

Perl programmers can use the `Debconf::Client::ConfModule(3pm)` perl module, and python programmers can use the `debconf` python module.

The rest of this manual will use the `/usr/share/debconf/confmodule` library in example shell scripts. Here is an example config script using that library, that just asks a question:

```
#!/bin/sh
set -e
. /usr/share/debconf/confmodule
db_set mypackage/reboot-now false
db_input high mypackage/reboot-now || true
db_go || true
```

Notice the uses of `"|| true"` to prevent the script from dying if debconf decides it can't display a question, or the user tries to back up. In those situations, debconf returns a non-zero exit code, and since this shell script is set `-e`, an untrapped exit code would make it abort.

And here is a corresponding postinst script, that uses the user's answer to the question to see if the system should be rebooted (a rather absurd example..):

```
#!/bin/sh
set -e
. /usr/share/debconf/confmodule
db_get mypackage/reboot-now
if [ "$RET" = true ]; then
shutdown -r now
fi
```

Notice the use of the `$RET` variable to get at the extended return code from the `GET` command, which holds the user's answer to the question.

THE POSTINST SCRIPT

The last section had an example of a postinst script that uses debconf to get the value of a question, and act on it. Here are some things to keep in mind when writing postinst scripts that use debconf:

* Avoid asking questions in the postinst. Instead, the config script should ask questions using debconf, so that pre-configuration will work.

Always source `/usr/share/debconf/confmodule` at the top of your `postinst`, even if you won't be running any `db_*` commands in it. This is required to make sure the config script gets a chance to run (see HACKS for details).

Avoid outputting anything to `stdout` in your `postinst`, since that can confuse `debconf`, and `postinst` should not be verbose anyway. Output to `stderr` is ok, if you must.

If your `postinst` launches a daemon, make sure you tell `debconf` to STOP at the end, since `debconf` can become a little confused about when your `postinst` is done otherwise.

Make your `postinst` script accept a first parameter of `"reconfigure"`. It can treat it just like `"configure"`. This will be used in a later version of `debconf` to let `postinsts` know when they are reconfigured.

OTHER SCRIPTS

Besides the config script and `postinst`, you can use `debconf` in any of the other maintainer scripts. Most commonly, you'll be using `debconf` in your `postrm`, to call the `PURGE` command when your package is purged, to clean out its entries in the `debconf` database. (This is automatically set up for you by `dh_installdebconf(1)`, by the way.)

A more involved use of `debconf` would be if you want to use it in the `postrm` when your package is purged, to ask a question about deleting something. Or maybe you find you need to use it in the `preinst` or `prerm` for some reason. All of these uses will work, though they'll probably involve asking questions and acting on the answers in the same program, rather than separating the two activities as is done in the config and `postinst` scripts.

Note that if your package's sole use of `debconf` is in the `postrm`, you should make your package's `postinst` source `/usr/share/debconf/confmodule`, to give `debconf` a chance to load up your templates file into its database. Then the templates will be available when your package is being purged.

You can also use `debconf` in other, standalone programs. The issue to watch out for here is that `debconf` is not intended to be, and must not be used as a registry. This is unix after all, and programs are configured by files in `/etc`, not by some nebulous `debconf` database (that is only a cache anyway and might get blown away). So think long and hard before using `debconf` in a standalone program.

There are times when it can make sense, as in the `apt-setup` program which uses `debconf` to prompt the user in a manner consistent with the rest of the debian install process, and immediately acts on their answers to set up `apt's sources.list`.

LOCALIZATION

`Debconf` supports localization of templates files. This is accomplished by adding more fields, with translated text in them. Any of the fields can be translated. For example, you might want to translate the description into Spanish. Just make a field named `'Description-es'` that holds the translation. If a translated field is not available, `debconf` falls back to the normal English field.

Besides the 'Description' field, you should translate the 'Choices' field of a select or multiselect template. Be sure to list the translated choices in the same order as they appear in the main 'Choices' field. You do not need to translate the 'Default' field of a select or multiselect question, and the value of the question will be automatically returned in English.

You will find it easier to manage translations if you keep them in separate files; one file per translation. In the past, the `debconf-getlang(1)` and `debconf-mergetemplate(1)` programs were used to manage debian/template.ll files. This has been superseded by the `po-debconf(7)` package, which lets you deal with debconf translations in .po files, just like any other translations. Your translators will thank you for using this new improved mechanism.

For the details on po-debconf, see its man page. If you're using debhelper, converting to po-debconf is as simple as running the `debconf-gettextize(1)` command once, and adding a Build-Dependency on po-debconf and on debhelper ($\geq 4.1.13$).

PUTTING IT ALL TOGETHER

So you have a config script, a templates file, a postinst script that uses debconf, and so on. Putting these pieces together into a debian package isn't hard. You can do it by hand, or can use `dh_installdebconf(1)` which will merge your translated templates, copy the files into the right places for you, and can even generate the call to PURGE that should go in your postrm script. Make sure that your package depends on debconf (≥ 0.5), since earlier versions were not compatible with everything described in this manual. And you're done.

Well, except for testing, debugging, and actually using debconf for more interesting things than asking a few basic questions. For that, read on..

DEBUGGING

So you have a package that's supposed to use debconf, but it doesn't quite work. Maybe debconf is just not asking that question you set up. Or maybe something weirder is happening; it spins forever in some kind of loop, or worse. Luckily, debconf has plenty of debugging facilities.

DEBCONF_DEBUG

The first thing to reach for is the DEBCONF_DEBUG environment variable. If you set and export `DEBCONF_DEBUG=developer`, debconf will output to stderr a dump of the debconf protocol as your program runs. It'll look something like this -- the typo is made clear:

```
debconf (developer): <-- input high debconf/frontend
debconf (developer): --> 10 "debconf/frontend" doesn't exist
debconf (developer): <-- go
debconf (developer): --> 0 ok
```

It's rather useful to use debconf's readline frontend when you're debugging (in the author's opinion), as the questions don't get in the way, and all the debugging output is easily preserved and logged.

DEBCONF_C_VALUES

If this environment variable is set to 'true', the frontend will display the values in Choices-C fields (if present) of select and multiselect templates rather than the descriptive values.

debconf-communicate

Another useful tool is the `debconf-communicate(1)` program. Fire it up and you can speak the raw debconf protocol to debconf, interactively. This is a great way to try stuff out on the fly.

debconf-show

If a user is reporting a problem, `debconf-show(1)` can be used to dump out all the questions owned by your package, displaying their values and whether the user has seen them.

.debconfrc

To avoid the often tedious build/install/debug cycle, it can be useful to load up your templates with `debconf-loadtemplate(1)` and run your config script by hand with the `debconf(1)` command. However, you still have to do that as root, right? Not so good. And ideally you'd like to be able to see what a fresh installation of your package looks like, with a clean debconf database.

It turns out that if you set up a `~/.debconfrc` file for a normal user, pointing at a personal `config.dat` and `template.dat` for the user, you can load up templates and run config scripts all you like, without any root access. If you want to start over with a clean database, just blow away the `*.dat` files.

For details about setting this up, see `debconf.conf(5)`, and note that `/etc/debconf.conf` makes a good template for a personal `~/.debconfrc` file.

ADVANCED PROGRAMMING WITH DEBCONF

Config file handling

Many of you seem to want to use debconf to help manage config files that are part of your package. Perhaps there is no good default to ship in a `conffile`, and so you want to use debconf to prompt the user, and write out a config file based on their answers. That seems easy enough to do, but then you consider upgrades, and what to do when someone modifies the config file you generate, and `dpkg-reconfigure`, and ...

There are a lot of ways to do this, and most of them are wrong, and will often earn you annoyed bug reports. Here is one right way to do it. It assumes that your config file is really just a series of shell variables being set, with comments in between, and so you can just source the file to "load" it. If you have a more complicated format, reading (and writing) it becomes a bit trickier.

Your config script will look something like this:

```
#!/bin/sh
CONFIGFILE=/etc/foo.conf
set -e
. /usr/share/debconf/confmodule

# Load config file, if it exists.
```

```

if [ -e $CONFIGFILE ]; then
. $CONFIGFILE || true

# Store values from config file into
# debconf db.
db_set mypackage/foo "$FOO"
db_set mypackage/bar "$BAR"
fi

# Ask questions.
db_input medium mypackage/foo || true
db_input medium mypackage/bar || true
db_go || true

```

And the postinst will look something like this:

```

#!/bin/sh
CONFIGFILE=/etc/foo.conf
set -e
. /usr/share/debconf/confmodule

# Generate config file, if it doesn't exist.
# An alternative is to copy in a template
# file from elsewhere.
if [ ! -e $CONFIGFILE ]; then
echo "# Config file for my package" > $CONFIGFILE
echo "FOO=" >> $CONFIGFILE
echo "BAR=" >> $CONFIGFILE
fi

# Substitute in the values from the debconf db.
# There are obvious optimizations possible here.
# The cp before the sed ensures we do not mess up
# the config file's ownership and permissions.
db_get mypackage/foo
FOO="$RET"
db_get mypackage/bar
BAR="$RET"
cp -a -f $CONFIGFILE $CONFIGFILE.tmp

# If the admin deleted or commented some variables but then set
# them via debconf, (re-)add them to the conffile.
test -z "$FOO" || grep -Eq '^ *FOO=' $CONFIGFILE || \
echo "FOO=" >> $CONFIGFILE
test -z "$BAR" || grep -Eq '^ *BAR=' $CONFIGFILE || \
echo "BAR=" >> $CONFIGFILE

sed -e "s/^ *FOO=.* /FOO=\"\$FOO\"/" \
-e "s/^ *BAR=.* /BAR=\"\$BAR\"/" \

```

```
< $CONFIGFILE > $CONFIGFILE.tmp  
mv -f $CONFIGFILE.tmp $CONFIGFILE
```

Consider how these two scripts handle all the cases. On fresh installs the questions are asked by the config script, and a new config file generated by the postinst. On upgrades and reconfigures, the config file is read in, and the values in it are used to change the values in the debconf database, so the admin's manual changes are not lost. The questions are asked again (and may or may not be displayed). Then the postinst substitutes the values back into the config file, leaving the rest of it unchanged.

Letting the user back up

Few things are more frustrating when using a system like debconf than being asked a question, and answering it, then moving on to another screen with a new question on it, and realizing that hey, you made a mistake, with that last question, and you want to go back to it, and discovering that you can't.

Since debconf is driven by your config script, it can't jump back to a previous question on its own but with a little help from you, it can accomplish this feat. The first step is to make your config script let debconf know it is capable of handling the user pressing a back button. You use the CAPB command to do this, passing backup as a parameter.

Then after each GO command, you must test to see if the user asked to back up (debconf returns a code of 30), and if so jump back to the previous question.

There are several ways to write the control structures of your program so it can jump back to previous questions when necessary. You can write goto-laden spaghetti code. Or you can create several functions and use recursion. But perhaps the cleanest and easiest way is to construct a state machine. Here is a skeleton of a state machine that you can fill out and expand.

```
#!/bin/sh  
set -e  
. /usr/share/debconf/confmodule  
db_capb backup  
  
STATE=1  
while true; do  
case "$STATE" in  
1)  
# Two unrelated questions.  
db_input medium my/question || true  
db_input medium my/other_question || true  
;;  
2)  
# Only ask this question if the  
# first question was answered in  
# the affirmative.  
db_get my/question  
if [ "$RET" = "true" ]; then  
db_input medium my/dep_question || true  
fi
```

```
;;
*)
# The default case catches when $STATE is greater than the
# last implemented state, and breaks out of the loop. This
# requires that states be numbered consecutively from 1
# with no gaps, as the default case will also be entered
# if there is a break in the numbering
break # exits the enclosing "while" loop
;;
esac
```

```
if db_go; then
STATE=$((STATE + 1))
else
STATE=$((STATE - 1))
fi
done
```

```
if [ $STATE -eq 0 ]; then
# The user has asked to back up from the first
# question. This case is problematical. Regular
# dpkg and apt package installation isn't capable
# of backing up questions between packages as this
# is written, so this will exit leaving the package
# unconfigured - probably the best way to handle
# the situation.
exit 10
fi
```

Note that if all your config script does is ask a few unrelated questions, then there is no need for the state machine. Just ask them all, and GO; debconf will do its best to present them all in one screen, and the user won't need to back up.

Preventing infinite loops

One gotcha with debconf comes up if you have a loop in your config script. Suppose you're asking for input and validating it, and looping if it's not valid:

```
ok=''
do while [ ! "$ok" ];
db_input low foo/bar || true
db_go || true
db_get foo/bar
if [ "$RET" ]; then
ok=1
fi
done
```

This looks ok at first glance. But consider what happens if the value of foo/bar is "" when this loop is entered, and the user has their priority set high, or is using a non-interactive frontend, and so they are not really asked for input. The value of foo/bar is not changed by the db_input, and so it fails the test and loops. And loops ...

One fix for this is to make sure that before the loop is entered, the value of foo/bar is set to something that will pass the test in the loop. So for example if the default value of foo/bar is "1", then you could RESET foo/bar just before entering the loop.

Another fix is to check the return code of the INPUT command. If it is 30 then the user is not being shown the question you asked them, and you should break out of the loop.

Choosing among related packages

Sometimes a set of related packages can be installed, and you want to prompt the user which of the set should be used by default. Examples of such sets are window managers, or ispell dictionary files.

While it would be possible for each package in the set to simply prompt, "Should this package be default?", this leads to a lot of repetitive questions if several of the packages are installed. It's possible with debconf to present a list of all the packages in the set and allow the user to choose between them. Here's how.

Make all the packages in the set use a shared template. Something like this:

```
Template: shared/window-manager
Type: select
Choices: ${choices}
Description: Select the default window manager.
Select the window manager that will be started by
default when X starts.
```

Each package should include a copy of the template. Then it should include some code like this in its config script:

```
db_metaget shared/window-manager owners
OWNERS=$RET
db_metaget shared/window-manager choices
CHOICES=$RET

if [ "$OWNERS" != "$CHOICES" ]; then
db_subst shared/window-manager choices $OWNERS
db_fset shared/window-manager seen false
fi

db_input medium shared/window-manager || true
db_go || true
```

A bit of an explanation is called for. By the time your config script runs, debconf has already read in all the templates for the packages that are being installed. Since the set of packages share a question, debconf records that fact in the owners field. By a strange coincidence, the format of the owners field is the same as that of the choices field (a comma and space delimited list of values).

The METAGET command can be used to get the list of owners and the list of choices. If they are different, then a new package has been installed. So use the SUBST command to change the list of choices to be the same as the list of owners, and ask the question.

When a package is removed, you probably want to see if that package is the currently selected choice, and if so, prompt the user to select a different package to replace it.

This can be accomplished by adding something like this to the preinst scripts of all related packages (replacing <package> with the package name):

```
if [ -e /usr/share/debconf/confmodule ]; then
. /usr/share/debconf/confmodule
# I no longer claim this question.
db_unregister shared/window-manager

# See if the shared question still exists.
if db_get shared/window-manager; then
db_metaget shared/window-manager owners
db_subst shared/window-manager choices $RET
db_metaget shared/window-manager value
if [ "<package>" = "$RET" ] ; then
db_fset shared/window-manager seen false
db_input high shared/window-manager || true
db_go || true
fi

# Now do whatever the postinst script did
# to update the window manager symlink.
fi
fi
```

HACKS

Debconf is currently not fully integrated into dpkg (but I want to change this in the future), and so some messy hacks are currently called for.

The worst of these involves getting the config script to run. The way that works now is the config script will be run when the package is pre-configured. Then, when the postinst script runs, it starts up debconf again. Debconf notices it is being used by the postinst script, and so it goes off and runs the config script. This can only work if your postinst loads up one of the debconf libraries though, so postinsts always have to take care to do that. We hope to address this later by adding explicit support to dpkg for debconf. The [debconf\(1\)](#) program is a step in this direction.

A related hack is getting debconf running when a config script, postinst, or other program that uses it starts up. After all, they expect to be able to talk to debconf right away. The way this is accomplished for now is that when such a script loads a debconf library (like /usr/share/debconf/confmodule), and debconf is not already running, it is started up, and a new copy of the script is re-exec'd. The only noticeable result is that you need to put the line that loads a debconf library at the very top of the script, or weird things will happen. We hope to address this later by changing how debconf is invoked, and turning it into something more like a transient daemon.

It's rather hackish how debconf figures out what templates files to load, and when it loads them. When the config, preinst, and postinst scripts invoke debconf, it will

automatically figure out where the templates file is, and load it. Standalone programs that use debconf will cause debconf to look for templates files in `/usr/share/debconf/templates/progname.templates`. And if a postrm wants to use debconf at purge time, the templates won't be available unless debconf had a chance to load them in its postinst. This is messy, but rather unavoidable. In the future some of these programs may be able to use `debconf-loadtemplate` by hand though.

`/usr/share/debconf/confmodule`'s historic behavior of playing with file descriptors and setting up a fd #3 that talks to debconf, can cause all sorts of trouble when a postinst runs a daemon, since the daemon ends up talking to debconf, and debconf can't figure out when the script terminates. The STOP command can work around this. In the future, we are considering making debconf communication happen over a socket or some other mechanism than stdio.

Debconf sets `DEBCONF_RECONFIGURE=1` before running postinst scripts, so a postinst script that needs to avoid some expensive operation when reconfigured can look at that variable. This is a hack because the right thing would be to pass `$1 = "reconfigure"`, but doing so without breaking all the postinsts that use debconf is difficult. The migration plan away from this hack is to encourage people to write postinsts that accept "reconfigure", and once they all do, begin passing that parameter.

SEE ALSO

[debconf\(7\)](#) is the debconf user's guide.

The debconf specification in debian policy is the canonical definition of the debconf protocol. `/usr/share/doc/debian-policy/debconf_specification.txt.gz`

[debconf.conf\(5\)](#) has much useful information, including some info about the backend database.

AUTHOR

Joey Hess <joeyh@debian.org>

- [Packaging](#)
- [Learn](#)

Translations: [English](#) - [Español](#) - [Português \(Brasil\)](#) - [\(+\)](#)

Note: Taken from  [Software Freedom Camp](#).

Contents

1. [Level 0: Basics of release process and setup a development environment](#)
2. [Level 1: Learn basics of Packaging](#)
3. [Level 2: Update existing packages to new upstream minor or patch versions](#)
4. [Level 3: Packaging more complicated modules](#)
5. [Level 4: Pick an unpackaged but useful module and upload to archive](#)

Level 0: Basics of release process and setup a development environment

1.  [How to Install a .Deb File via Command-Line](#)
2.  [Lifecycle of a Release](#) (the journey of a package from first upload to reaching a user, included in a supported release)
3. [Manage repositories/sources.list](#) (managing the list of websites that serve Deb packages)
4.  [How to install packages from stable-backports](#) (there is some extra steps required to get new upstream versions of popular software on a stable system)
5. `apt -file find` command to find which package includes a file (usually to find a missing command or header file) -  [apt-file tutorial](#)
6. Setup Debian Unstable/Sid using [Incus](#) (hopefully you are now familiar with Lifecycle of a release from previous steps and know new packages or new upstream versions of existing packages are first uploaded to Sid or Unstable, so we do development on this branch of debian)
7. [Building existing packages from source](#). Learning to build from source is useful when you want to backport a package (for example you want to rebuild a new version of a package from unstable or testing on your stable system). This is also useful if you want to modify an existing package, for example to fix a bug or cherry pick a commit from upstream.

By this time you should be familiar with

1. `debbuild`
2. `apt build-dep`
3. `apt source -b`

commands to rebuild an existing debian package from source.



Advanced/Extra/Reference: [Different options for setting up a Debian Sid environment](#)

Level 1: Learn basics of Packaging

Understand the basic concepts using debmake/dh_make (getting source tarballs, creating source package, building the binary package, making it lintian clean)

A high level overview of packaging involves,

1. Downloading source code of the software to be packaged (commonly release tarballs)
2. Creating a debian directory template (using tools like debmake, npm2deb or gem2deb depending on the programming language used)
3. Building the .deb file, combining the above two (using dpkg-buildpackage)

Hands on steps,

1. [Setup your name and email address to be used for packaging](#)
2. Then follow [this tutorial](#) to create a package from scratch



Extra tutorial with more detailed outputs. [Abraham Raji's simple packaging tutorial](#)

Once you understand the basic concepts, use npm2deb to automate some of those tasks like getting source tarball, a better debian directory template than the ones created by dh_make/debmake as [DebianPkg: npm2deb](#) knows more details specific to node modules. You will still have to fix the remaining issues flagged by lintian.

1. [npm2deb Tutorial](#)

By this time you should know,

1. creating lintian clean packages for simple modules and
2. building it in a clean environment like [sbuild](#).
3. You should also know to [import a dsc file to a git repo](#) (gbp import-dsc --pristine-tar) and
4. push your work to a public git hosting service like <https://salsa.debian.org> (git push -u --all --follow-tags)

Level 2: Update existing packages to new upstream minor or patch versions

Once you get a clear picture of packaging a simple module, we can move to the next stage of updating existing packages

1. Understand  [Semantic Versioning scheme](#) and know when a new upstream release can be a breaking change. Pay special attention to software with 0.x versions - there is no guarantee about compatibility for minor updates as well.
2. Follow this tutorial to [update a package to new upstream version](#). Make sure you have your  [SSH key added to your salsa profile](#).
3. You can use  [gbp pq](#) or [Quilt](#) in case you need to modify any patch or create a patch. Notes/Tips: If you are already familiar with git branches and rebase, gbp pq would be easier for you. Quilt will take some practice to not miss any steps - if you miss any step, it can be very difficult to recover from a mistake, so better to start over if you see many errors when building the package. A very common mistake is commit upstream changes directly (if you forgot to run quilt pop -a before committing) and dpkg-buildpackage will complain about modified files. Always run debclean and quilt pop -a before committing any changes and make sure you are only committing changes inside debian directory (all upstream changes will be saved as patch files in debian/patches when you run quilt pop -a).
4. Start  [signing your git commits](#) and upload your gpg public key to  [OpenPGP.org key server](#) (this is default key server in gpg and thunderbird so people can easily search your keys using email address. You also need to verify your email address after you upload your public key for the discovery with email to work.)

By this time you should know,

1. How to send RFS mails
2. Using Quilt to modify upstream source if required
3. Sign your git commits

Once you are comfortable with simple updates (say you have done 5-6 updates), ask someone to suggest more challenging updates, like a major update.

Level 3: Packaging more complicated modules

Next step is packaging more complicated modules that will involve things like, modifying some upstream files, removing some files from source tarball, generating some files from source, getting the source tarball from a git commit etc.

1. [Advanced tutorial for more complicated modules](#)

By this time you should know,

1. Creating patches with quilt
2. Repacking orig.tar and exclude specific files
3. Use pkg-j-s-tools options to build from source files

4. Build packages with typescript sources

Level 4: Pick an unpackaged but useful module and upload to archive

1.  [List of node dependencies for gitlab](#)
2. Join  [Packaging teams](#) that you find interesting. join their mailing list/irc channels and tell them you like to help with packaging and follow their documentation.

By this time you should know,

1. How to file ITP

[CategoryPackaging](#)

Packaging/Learn ([last modified 2025-12-04 11:54:59](#))

Debian Developers' Manuals

- Debian Policy Manual
- Debian Developer's Reference
- Guide for Debian Maintainers
- Introduction to Debian packaging
- Debian Menu System
- Debian Installer internals
- dbconfig-common documentation
- dbapp-policy
- How to package for Debian

Debian Policy Manual

This manual describes the policy requirements for the Debian GNU/Linux distribution. This includes the structure and contents of the Debian archive, several design issues of the operating system, as well as technical requirements that each package must satisfy to be included in the distribution.

Authors:	Ian Jackson, Christian Schwarz, David A. Morris
Maintainer:	The Debian Policy group
Status:	ready
Availability:	Debian package debian-policy Latest version: English: [HTML] [PDF] [EPUB] [plain text]

The latest restructuredText source is available through the git repository.

- Web interface: <https://salsa.debian.org/dbnpolicy/policy>
- VCS interface: `git clone https://salsa.debian.org/dbnpolicy/policy.git`

[Proposed amendments](#) to Policy

Supplemental Policy documentation:

- [Filesystem Hierarchy Standard](#) [\[PDF\]](#) [\[plain text\]](#)
- [Upgrading checklist](#)
- [Virtual package names list](#)
- [Menu policy](#) [\[plain text\]](#)
- [Perl policy](#) [\[plain text\]](#)
- [debconf specification](#)
- [Emacsen policy](#)
- [Java policy](#)

- [Python policy](#)
- [copyright-format specification](#)

Debian Developer's Reference

This manual describes procedures and resources for Debian maintainers. It describes how to become a new developer, the upload procedure, how to handle our bug tracking system, the mailing lists, Internet servers, etc.

This manual is thought as a *reference manual* for all Debian developers (newbies and old pros).

Authors: Ian Jackson, Christian Schwarz, Lucas Nussbaum, Raphaël Hertzog, Adam Di Carlo, Andreas Barth

Maintainer: Lucas Nussbaum, Hideki Yamane, Holger Levsen

Status: ready

Availability: Debian package [developers-reference](#)
Latest version:

- **English:** [\[HTML\]](#) [\[HTML \(single page\)\]](#) [\[PDF\]](#) [\[plain text\]](#) [\[EPUB\]](#)
- German: [\[HTML\]](#) [\[HTML \(single page\)\]](#) [\[PDF\]](#) [\[plain text\]](#) [\[EPUB\]](#)
- French: [\[HTML\]](#) [\[HTML \(single page\)\]](#) [\[PDF\]](#) [\[plain text\]](#) [\[EPUB\]](#)
- Italian: [\[HTML\]](#) [\[HTML \(single page\)\]](#) [\[PDF\]](#) [\[plain text\]](#) [\[EPUB\]](#)
- Japanese: [\[HTML\]](#) [\[HTML \(single page\)\]](#) [\[PDF\]](#) [\[plain text\]](#) [\[EPUB\]](#)
- Russian: [\[HTML\]](#) [\[HTML \(single page\)\]](#) [\[PDF\]](#) [\[plain text\]](#) [\[EPUB\]](#)

The latest restructuredText source is available through the git repository.

- Web interface: <https://salsa.debian.org/debian/developers-reference>
- VCS interface: `git clone https://salsa.debian.org/debian/developers-reference.git`

Guide for Debian Maintainers

This tutorial document describes the building of the Debian package to ordinary Debian users and prospective developers using the `debmake` command.

It is focused on the modern packaging style and comes with many simple examples.

- POSIX shell script packaging
- Python3 script packaging
- C with Makefile/Autotools/CMake
- multiple binary packages with shared library etc.

Authors:	Osamu Aoki
Maintainer:	Osamu Aoki
Status:	ready
Availability:	Debian package debmake-doc Latest version:

- **English:** [\[HTML\]](#) [\[plain text\]](#) [\[PDF\]](#)
- Russian: [\[HTML\]](#) [\[plain text\]](#) [\[PDF\]](#)
- Chinese (China): [\[HTML\]](#) [\[plain text\]](#) [\[PDF\]](#)
- German: [\[HTML\]](#) [\[plain text\]](#) [\[PDF\]](#)
- Japanese: [\[HTML\]](#) [\[plain text\]](#) [\[PDF\]](#)

The latest XML source is available through the git repository.

- Web interface: <https://salsa.debian.org/debian/debmake-doc>
- VCS interface: `git clone https://salsa.debian.org/debian/debmake-doc.git`

Introduction to Debian packaging

This tutorial is an introduction to Debian packaging. It teaches prospective developers how to modify existing packages, how to create their own packages, and how to interact with the Debian community. In addition to the main tutorial, it includes three practical sessions on modifying the `grep` package, and packaging the `gnujump` game and a Java library.

Authors:	Lucas Nussbaum
Maintainer:	Lucas Nussbaum
Status:	ready

Availability: Debian package [packaging-tutorial](#)
Latest version:

- **English:** [\[PDF\]](#)
- German: [\[PDF\]](#)
- Spanish: [\[PDF\]](#)
- French: [\[PDF\]](#)
- Japanese: [\[PDF\]](#)
- Portuguese: [\[PDF\]](#)
- Russian: [\[PDF\]](#)
- Chinese: [\[PDF\]](#)
- Chinese: [\[PDF\]](#)

The latest LaTeX source is available through the git repository.

- Web interface: <https://salsa.debian.org/debian/packaging-tutorial>
- VCS interface: `git clone https://salsa.debian.org/debian/packaging-tutorial.git`

Debian Menu System

This manual describes the Debian Menu System and the **menu** package.

The menu package was inspired by the install-fvwm2-menu program from the old fvwm2 package. However, menu tries to provide a more general interface for menu building. With the update-menus command from this package, no package needs to be modified for every X window manager again, and it provides a unified interface for both text- and X-oriented programs.

Authors: Joost Witteveen, Joey Hess, Christian Schwarz

Maintainer: Joost Witteveen

Status: ready

Availability: Debian package [menu](#) [HTML online](#)

Debian Installer internals

This document is intended to make Debian Installer more accessible to new developers and as a central location to document technical information.

Authors: Frans Pop

Maintainer: Debian Installer team

Status: ready

Availability:

[HTML online](#)

[DocBook XML source online](#)

dbconfig-common documentation

This document is intended for package maintainers who maintain packages that require a working database. Instead of implementing the required logic themselves they can rely on dbconfig-common to ask the right questions during install, upgrade, reconfigure and deinstall for them and create and fill the database.

Authors: Sean Finney and Paul Gevers

Maintainer: Paul Gevers

Status: ready

Availability: Debian package [dbconfig-common](#)
Latest version:

- **English:** [[HTML](#)] [[plain text](#)] [[PDF](#)]

The latest SGML source is available through the git repository.

- Web interface: <https://salsa.debian.org/debian/dbconfig-common>
- VCS interface: `git clone https://salsa.debian.org/debian/dbconfig-common.git`

Additional also the [design document](#) is available.

dbapp-policy

A proposed policy for packages that depend on a working database.

Authors: Sean Finney

Maintainer: Paul Gevers

Status: draft

Availability: Debian package [dbconfig-common](#)
Latest version:

- **English:** [[HTML](#)] [[plain text](#)] [[PDF](#)]

The latest SGML source is available through the git repository.

- Web interface: <https://salsa.debian.org/debian/dbconfig-common>
- VCS interface: `git clone https://salsa.debian.org/debian/dbconfig-common.git`

How to package for Debian

The book describes how Debian packages are created using a git repository using the programs from the package **git-buildpackage** and other useful programs. Afterwards, the reader should be able to build production quality Debian packages with this program and the descriptions of the individual steps.

Authors: Dipl.-Ing. Mechtilde Stehmann, Dr. Michael Stehmann

Maintainer: Debian Documentation Project

Status: ready

Availability: Debian package [debian-package-book](#)
Latest version:

- **English:** [\[PDF\]](#)
- **German:** [\[PDF\]](#)

The latest LaTeX source is available through the git repository.

- Web interface: <https://salsa.debian.org/ddp-team/dpb>
- VCS interface: `git clone https://salsa.debian.org/ddp-team/dpb.git`

Please send all comments, criticisms and suggestions about these web pages to our [mailing list](#).

Back to the [Debian Project homepage](#).

This page is also available in the following languages:

Select your language ▼

How to set [the default document language](#)

[Home](#)

[About](#)

[Social Contract](#)
[Code of Conduct](#)
[Free Software](#)
[Legal Info](#)
[Help Debian](#)

[Getting Debian](#)

[Network install](#)
[CD/USB ISO images](#)
[Pure Blends](#)
[Debian Packages](#)
[Developers' Corner](#)

[News](#)

[Project News](#)
[Events](#)
[Documentation](#)
[Release Info](#)
[Debian Wiki](#)

[Support](#)

[Debian International](#)
[Security Information](#)
[Bug reports](#)
[Mailing Lists](#)

[Site map](#)

[Search](#)
[The Debian Blog](#)
[Debian Micronews](#)
[Debian Planet](#)

See our [contact page](#) to get in touch. Web site source code is [available](#).

Last Modified: Wed, Jul 9 14:03:32 UTC 2025 Last Built: Thu, Mar 19 23:29:40 UTC 2026

Copyright © 1997-2025 [SPI](#) and others; See [license terms](#)

Debian is a registered [trademark](#) of Software in the Public Interest, Inc.

- [PackagingFAQ](#)

Frequently asked questions about creating packages. For general information, see [Packaging](#).

Contents

1. General questions

1. What do people mean by "the package" vs. "the source package" or "the binary package"?
2. What is a good packaging workflow?
 1. How do I keep track of my packaging changes?
3. Can I make a package that just configures other packages?
4. Can I make a package that updates files in /home?
5. How do I make a package if upstream includes a /debian subdirectory?
6. How do I make a package if upstream only provides binaries (e.g. Nvidia blobs)?
7. What is a non-maintainer upload?
 1. If I do a non-maintainer upload, which debian/control field should I put my name in?
8. What is the Debian equivalent of Ubuntu PPAs?
9. What's the difference between a "native" and "quilt" patch system?
10. What does "dfsg" or "ds" in the version string mean?

2. Manipulating package files

1. How do I access the contents of a package?
2. How do I find out who uploaded a package?
3. How do I find the .dsc files etc. for packages in /var/cache/apt/archives?
 1. How do I get the changelog for packages in /var/cache/apt/archives?

3. Building packages

1. What format should orig tarball filenames have?
2. Should I repack the original tarball if it doesn't have a /<packagename>-<version> folder?
3. What does the --pristine-tar option for gbp commands do?
4. How do I check the upstream source for DFSG-compliance?
5. Where should I run build commands from?
6. My upstream uses a strange build system, how do I support it?
 1. Should I use --with \$ADDON?
7. Where is the documentation for debian/rules targets like dh_auto_install?
8. How do I sign packages with an OpenPGP signature?
9. What's the difference between Depends: and Build-Depends: in debian/control?
 1. Can I have a Build-Depends without a corresponding Depends?
10. How do I know which packages to (build-)depend on?
11. How do I build a package for e.g. arm64 on my PC?
12. Should Architecture: all builds mention the architecture in the output?
13. Does Architecture: foo affect the build process or the binary package?
14. How do I track upstream changes?
15. Is it OK to edit old changelog entries?
16. What does "dpkg-source: info: local changes detected" mean?

General questions

What do people mean by "the package" vs. "the source package" or "the binary package"?

Saying "the package" can refer to any of:

1. a source `.dsc` package, which you use to build one or more binary packages
2. a binary `.deb` package, which you install on your computer
3. the source `.dsc` package and the complete set of `.deb` packages generated by it
4. the most interesting `.deb` package in the current conversation
 - e.g. if `foo` depends on `foo-doc`, you might talk about "the package" and "the doc package"

You can usually work out which one people are referring to from the context. For example, newbies often talk about "the package" but mean "the binary package" because they don't know source packages exist; whereas packaging guides often talk about "the package" but mean "the source package" because they haven't told you to build any binary packages yet. If in doubt, ask which package someone is referring to.

What is a good packaging workflow?

It depends on your personal preference, and the specific package you're working on. Every packaging guide will give you a different recommendation, and most of them will insist theirs is the only correct choice. Just try stuff and find out what works for you.

The guides in [Packaging](#) might give you some inspiration, and the [debian-mentors' packaging questions](#) might point you in a good direction.

How do I keep track of my packaging changes?

Most people do their [packaging with git](#) or some other version control system. If you've adopted a package that's been around since before git, [🌐 https://snapshot.debian.org/](https://snapshot.debian.org/) lets you download packages from older versions of Debian, which you can compare using `debdiff` and `interdiff` (in [DebianPkg: devscripts](#) and [DebianPkg: patchutils](#)).

Can I make a package that just configures other packages?

It's technically possible, but such a package will only be accepted in Debian if it uses a documented interface of the other package.

Editing configuration files creates a lot of weird problems. For example, say the packages `foo` and `bar` both add the same message to `/etc/motd`. If you install both

packages, should the message be added twice? If not, what happens when you uninstall one package but not the other?

Safely editing other packages' configuration files can only be done on a case-by-case basis, so it's against  [Debian policy](#) unless the package in question specifically supports it. For example, [DebianPkg: logrotate](#) lets other packages put files in `/etc/logrotate.d/`, which it then uses to configure log rotations.

If you're doing [WikiPedia: infrastructure as code](#), you probably want a [WikiPedia: continuous configuration automation tool](#) like  [Ansible](#) instead.

Can I make a package that updates files in `/home`?

No. Files in `/home` aren't yours to update - see  [debian-mentors thread](#) and  [Debian Policy entry](#).

Work with your upstream to find another solution. These include:

- if a user has a deprecated setting, have your program display a warning or refuse to run until it's fixed
- read from more than one file
 - e.g. read from both `~/.config/foo.conf` and `/etc/foo/foo.conf`
- provide a mechanism for users to include other files
 - e.g. let `~/.config/foo.conf` have a line like
`include /etc/foo/foo.conf`

If you're writing a private package and are really convinced this is the right solution, you're probably using packages to do [WikiPedia: infrastructure as code](#). Consider using a [WikiPedia: continuous configuration automation tool](#) like  [Ansible](#) instead.

How do I make a package if upstream includes a `/debian` subdirectory?

Debian packaging information goes in the `debian/` subdirectory of the package's top-level directory. Upstreams are [asked not to include a /debian directory](#), but some do anyway.

Modern packaging tools will delete this directory automatically, or you'll need to delete it manually if you're just `git` pulling an upstream repository. For details, see [If upstream has a /debian directory or file](#).

Historically, this used to be a bigger problem. You may need to do some extra work if you're adopting a really old package that still uses the `1.0` format.

How do I make a package if upstream only provides binaries (e.g. Nvidia blobs)?

Treat the tarball with the blobs as the source package, avoid compiling them from source.

You'll have difficulty getting your package accepted into Debian, for a mixture of practical reasons (how do you do a security review on a binary blob?) and philosophical ones ( [Debian is about software freedom](#)).

For information about Nvidia, see [NvidiaGraphicsDrivers](#).

What is a non-maintainer upload?

It's when a package is uploaded to the Debian archive by someone other than its maintainer.

For more information, see [NonMaintainerUpload](#).

If I do a non-maintainer upload, which debian/control field should I put my name in?

None of them, but please make the first line of the changelog "Non-maintainer upload".

`debian/control` has a  [Maintainer field](#) for the primary maintainer, and an  [Uploaders field](#) for co-maintainers. Despite the name, these are not required for [non-maintainer uploads](#).

The  [changelog](#) should explain what changes you've made. Putting "Non-maintainer upload" here clarifies that you made the change without implying you're involved in ongoing maintenance.

What is the Debian equivalent of Ubuntu PPAs?

Debian does not run a PPA service like Ubuntu, but it's easy enough to [create your own repository](#) and upload it to any static hosting service.

One guide to creating a repo is [DebianRepository/SetupWithReprepro](#).

What's the difference between a "native" and "quilt" patch system?

Prefer `quilt` unless you're certain the package will *never* be useful outside of Debian. For example, [DebianPkg: dpkg](#) is a native package, [DebianPkg: libc6](#) is non-native.

Putting `3.0 (native)` in `debian/source/format` disallows any differences between upstream and Debian outside of the `debian/` directory, `3.0 (quilt)` manages such changes using the "quilt" patch system, which was pioneered by

[DebianPkg: the program of the same name](#) and is now used by all common tools. If in doubt, prefer 3.0 (quilt).

Most packages have versions like 1.2.3-4, where -4 is the Debian revision. Native packages versions just look like 1.2.3, because native format doesn't support separate Debian revisions. Native format is only useful for Debian-specific packages like [DebianPkg: dpkg](#), where the flexibility of the quilt format would just add needless complexity.

For more information, see [Projects/DebSrc3.0](#) and [If upstream content needs patching](#).

What does “dfsg” or “ds” in the version string mean?

+dfsg.N and +ds.N are a conventional way of extending a version string, when the Debian package's upstream source tarball is actually different from the source released upstream.

+dfsg in a version string stands for [Debian Free Software Guidelines](#), and means upstream's source release contains elements that must not be hosted by Debian.

+ds in a version string stands for “Debian Source”, and means upstream's source release was modified for non-DFSG reasons (e.g. it contains huge optional files that would waste space on Debian's servers).

.N is the number of times you've repacked since changing the part of the version number before the +.

The changes should be documented in README.source.

Manipulating package files

How do I access the contents of a package?

Show the contents of an installed package:

```
dpkg -L <package-name>
```

Access the contents of a package you have downloaded but not installed:

```
# List the contents of a binary package:
dpkg-deb -c <package-name>.deb

# Extract the contents of a binary package:
dpkg-deb -x <package-name>.deb /tmp/package-name
```

```
# Extract the contents of a source package:
```

```
dpkg-source -x <source-package-name>.dsc /tmp/source-package-name
```

Download a package without installing it:

```
sudo apt-get download <package-name>
```

List the contents of a package in Debian without downloading it (using [DebianPkg: apt-file](#)):

```
sudo apt-get install apt-file  
sudo apt-file update  
apt-file list <package-name>
```

.debs are [binary packages](#), .dscs are [source packages](#). See also [DebianMan: dpkg-deb](#) and [DebianMan: dpkg-source](#).

How do I find out who uploaded a package?

```
who-uploads <package-name>
```

(who-uploads is in [DebianPkg: devscripts](#))

How do I find the .dsc files etc. for packages in /var/cache/apt/archives?

Go to <https://packages.debian.org/<distro>/<packagename>> (e.g. <https://packages.debian.org/sid/hello>) and search for "Download Source Package". The files are linked from a section on the right of the page.

.debs are [binary packages](#). To save bandwidth, the associated [source packages](#) aren't downloaded by default.

For alternative solutions, see [Get the source package](#).

How do I get the changelog for packages in /var/cache/apt/archives?

If you have installed the package, it should be in `/usr/share/doc/<package-name>/changelog.gz`. Otherwise, do:

```
dpkg-deb -x /var/cache/apt/archives/<package-name> /tmp/package-name  
ls /tmp/package-name/usr/share/doc/*/
```

Building packages

What format should orig tarball filenames have?

`<packagename>_<version>.orig.tar.gz` (or `.bz2` etc.).

For a list of valid compression schemes, see [DebianMan: dpkg-source --compression](#).
For upstream guidance about source tarballs, see [Provide a well-named source tarball](#).

Should I repack the original tarball if it doesn't have a `/<packagename>-<version>` folder?

No. Upstream developers often have their tarball's root directory contain a single subdirectory named after the project and version, but modern tools can work around this.

For more upstream developer recommendations, see [Provide a well-named source tarball](#).

What does the `--pristine-tar` option for gbp commands do?

These manage a "pristine" tarball - one that is byte-for-byte identical every time (files stored in the same order etc.).

For details, see [DebianPackaging--pristine-tar-option-explained](#).

How do I check the upstream source for DFSG-compliance?

One solution is to install [DebianPkg: devscripts](#), then do:

```
licensecheck -r /path/to/upstream/source/
```

For more solutions, see [CopyrightReviewTools](#).

Where should I run build commands from?

Wherever upstream wants you to run them from. For git projects, it's usually the directory with the `.git` subdirectory. For raw tarballs, it's usually the `<project>-<version>/` directory created by the tarball.

Upstream maintainers are encouraged to [support out-of-tree builds](#), but not all of them do.

My upstream uses a strange build system, how do I support it?

At the time of writing, Debian's build system has modules to deal with ant, autoconf, cmake, make, meson, ninja, Perl (`Module::Build` and `ExtUtils::MakeMaker`), Python (`distutils`) and qmake (normal and qt4). These should all work without you needing to configure anything. You can edit `debian/rules` if you need to add something special, but that's rare nowadays.

Some build systems have proven difficult for packaging. See [Avoid Scons](#) and [Avoid waf](#).

For upstream guidance about build systems, see [Document your build system](#). For a list of build system modules, see  /usr/share/perl5/Debian/Debhelper/Buildsystem/*.pm.

Should I use `--with $ADDON`?

That works, but it's better to put `Build-Depends: dh-sequence-$ADDON` in `debian/control` instead.

Older guides often recommend putting e.g. `dh $@ --with python3` in `debian/rules`. Specifying addons this way was a big advance when it was introduced, but people gradually found edge cases where it caused problems. Adding e.g. `Build-Depends: dh-sequence-python3` does the same thing while solving those edge cases.

For details, see [dh-sequence-* dependencies](#).

Where is the documentation for `debian/rules` targets like `dh_auto_install`?

Find the rules your package uses with `grep dh_ /path/to/your/package.build`, then look for documentation with `man dh_something` or `zgrep 'dh_something' /usr/share/man/man?/*`. For example, [DebianMan: man dh_auto_install](#).

`dh` uses a sequence of targets to build a package; each is called `dh_something` and can be overridden by a target in `debian/rules` called `override_dh_something`.

There's no central list, but many targets have man pages.

For information about dh itself, see [DebianMan: dh](#).

How do I sign packages with an OpenPGP signature?

To sign *a whole package* yourself, you'll need to [create a key](#) and get it [signed by an existing Debian Developer](#). Consider  [finding a sponsor](#) instead.

Debian uses OpenPGP signatures to *authenticate* you are who you say you are, but uses the [Debian keyring](#) to *authorise* you to upload packages. Getting on the keyring is a difficult process, and often not necessary.

To sign *git repo tags*, see [Configure git-buildpackage](#). To sign *upstream releases*, see [Sign releases](#).

What's the difference between Depends: and Build-Depends: in debian/control?

Depends: foo means "install foo before installing the package".

Build-Depends: foo means "install foo before building the package from source".

Packages can be both build-dependencies and normal dependencies, but are usually only one or the other.

Consider an upstream tarball with a PHP script in it. If the script is copied into the binary package and used when the program is running, that's **Depends:** php. Or if the script is used by the build system to run some tests, that's **Build-Depends:** php. If it's used in both places, add both lines.

Can I have a Build-Depends without a corresponding Depends?

Yes. **Build-Depends** and **Depends** are entirely separate. It's common to have dependencies in one but not the other.

How do I know which packages to (build-)depend on?

With difficulty! Solutions include:

- check the README file
 - [we ask upstream to put dependencies there](#)
- build the package in a build environment using [pbuilder](#) or [sbuild](#)
 - missing build-dependencies will now cause build failures
 - you can [tell git-buildpackage to do this automatically](#)
- run [autopkgtests](#) in a test environment
 - missing runtime-dependencies will now cause test failures

- install [DebianPkg: devscripts](#) and run the program with [DebianMan: dpkg-depcheck](#)
- read the program's source code
- send upstream a patch with everything you think should be in the README
 - helps people packaging for other distributions
 - often said to be  [the best way to get the right answer](#)
- wait for bug reports to come in
 - this feels bad, but it happens to all of us

How do I build a package for e.g. arm64 on my PC?

If you're using [git-buildpackage](#):

```
gbp buildpackage --git-dist=<dist> --git-arch=<arch>
```

For more information, see [CrossCompiling](#).

Should Architecture: all builds mention the architecture in the output?

Yes, this is fine.

Architecture: all means your package can be *installed* on all architectures, but it still needs to be *built* on a specific architecture. Your `.build`, `.buildinfo` and `.changes` files log that architecture in case you need to e.g. debug a problem that only happens when someone builds your package on a Raspberry Pi.

Does Architecture: foo affect the build process or the binary package?

Both. The build process only builds **Architecture: all** packages once, other packages need to be rebuilt for each supported architecture. The install process allows **Architecture: all** packages to be installed on any system, other architectures are only allowed if they're supported by the current system.

For information about allowing more than one architecture on the same system, see [Multiarch/HOWTO](#).

How do I track upstream changes?

See [debian/watch](#) for ways to configure Debian's upstream-tracking system. Examples for common [forges](#) like GitHub are on that page and in the [DebianMan: uscan manual](#).

Is it OK to edit old changelog entries?

You should refrain from doing so, but it's allowed if you have a good reason.

See [!\[\]\(c8d96c8885d3000a912c2582004aed63_img.jpg\) debian-mentors discussion](#).

What does "dpkg-source: info: local changes detected" mean?

If your build system finds differences between your project directory and your upstream `orig.tar.gz` file, it will print something like this:

```
dpkg-source: info: building your-package using existing ./your-package_
dpkg-source: info: local changes detected, the modified files are:
  your-package/your-file
dpkg-source: error: aborting due to unexpected upstream changes, see /
dpkg-source: hint: make sure the version in debian/changelog matches th
dpkg-source: hint: you can integrate the local changes with dpkg-source
```

Changes outside `debian/` need to be tracked specially, for legal and security reasons. Check the file(s) mentioned in the message - it might just be a temporary file you forgot to delete, or you might need to [manage differences with upstream](#).

See also

- [the debian-mentors FAQ](#) has [a packaging section](#)
- Many of these are based on questions from two Debian Women IRC logs:
 - [!\[\]\(f2fdbbba686c1099e6b2b8779766e2d3_img.jpg\) making Debian packages](#)
 - [!\[\]\(b3cfbfd04368a71f4c64e073908d25d7_img.jpg\) taking an existing package, re-building it, applying changes to it, and preparing those changes so as to send them as a bug patch](#)

[CategoryPackaging](#)

PackagingFAQ (last modified 2025-08-25 20:58:21)

- [UpstreamGuide](#)

Advice for upstream developers who want their software to be in Debian. To learn how to actually create Debian packages, see [Packaging](#).

Even if you don't want your software to be in Debian, much of the advice on this page will make your software easier to deploy, maintain, and package for other distributions.

Contents

1. [Why get your software into Debian?](#)

2. [Make changes based on feedback](#)

1. [Bootstrapping \(compilers/interpreters only\)](#)

3. [Choose a standard open source license](#)

1. [Use the same license for all files](#)

2. [Maintain a COPYING file](#)

3. [Maintain an AUTHORS file \(optional\)](#)

4. [Put copyright attribution at the top of every file](#)

5. [Keep copyright information up-to-date](#)

6. [Actually follow license rules!](#)

4. [Register new media types](#)

5. [Provide a well-named source tarball](#)

1. [Specify dependencies and versions](#)

2. [Do not include automatically-generated files](#)

3. [Do not include a /debian directory](#)

4. [Do not include third-party code](#)

5. [Do not hardcode distribution-specific values](#)

6. [Include all documentation](#)

1. [Write manual pages](#)

2. [Provide upstream metadata](#)

7. [Use standard paths](#)

1. [Remove language extensions from executable filenames](#)

2. [Support /etc/your-project.d for configuration](#)

3. [Support XDG Base Directory paths](#)

4. [Use the right home directory](#)

6. [Release early, release often](#)

1. [Use a public git repository \(optional\)](#)

1. [Add git tags with a common format \(optional\)](#)

2. [Add unique version numbers](#)

3. [Sign releases](#)

4. [Explain support lifetimes](#)

5. [Ensure backwards-compatibility](#)

7. [Use a bug tracker](#)

1. [Distinguish between bugfix and feature changes](#)

2. Handle security issues transparently

8. Document your build system

1. Autoconf and Automake
2. Make
3. Java build systems
4. Perl build systems
5. Python build systems
6. OCaml build systems
7. Avoid SCons
8. Avoid waf

9. Maintain a flexible build process

1. Run tests while building
2. Support verbose building
3. Support offline building
4. Support out-of-tree builds
5. Support out-of-VCS builds
6. Support cleaning the tree
7. Support other compiler commands
8. Support staged installs
9. Split the "make" and "install" stages
10. Create directories during the "install" stage
11. Ignore your project's top-level directory name
12. Do not modify existing files
13. Do not target the machine being built on

10. See also

1. Debian resources for your package
2. Producing software
3. Interacting with distributions
4. Managing a community
5. Legal and philosophical

Why get your software into Debian?

You'll have more, happier users:

- people can find and install your software more easily
 - including users of Debian [derivatives](#) like Ubuntu
- organisations can streamline their approval process
- they're protected from supply chain attacks (see  [paper](#) and  [examples](#))

You'll get higher-quality software:

- you'll get a review from an external developer who can share best practices from other projects
- your Debian Maintainer will ensure your software plays nicely with system startup, logging, file types, network setup, desktop environment, sandboxing, and so on

- your dependencies' Debian Maintainers will ensure compatibility, security etc. for those projects
- CI tests will alert you about problems with specific hardware architectures (including ones you may not have access to)
- QA tests will alert you about changes in your dependencies

Make changes based on feedback

Be prepared for packagers to put significant effort into reviewing your code, and to suggest changes you see as trivial or problematic. Make time for their questions, and take their arguments as constructive criticism. Where possible, use feedback as an opportunity to write unit tests.

There is a lot of advice on this page, and Debian developers are responsible for ensuring their packages follow the  [Debian Policy Manual](#). This sometimes means making all Debian packages behave the same, because users would rather have one rule they can rely on, even if your way *is* better. And it sometimes means mitigating theoretical security concerns, because they'd rather be too secure for 99% of packages than not be secure enough for the several hundred edge cases in the Debian archive.

It's fine to put changes behind a configuration flag you disable by default, or to discuss alternative solutions instead of accepting the first answer - just engage constructively, and understand that your packager is working on a slightly different problem than you are. Debian can maintain downstream patches to your project if necessary (as can other distributions), but a one-line patch to a configuration file creates less of a maintenance burden than having to change behaviour throughout your codebase.

Debian will run your unit tests as part of every build, so every unit test you write becomes a free source of bug reports. If Debian upgrades a dependency, adds a new architecture, or makes some other seemingly-trivial change, a unit test will immediately tell you if it breaks your program somehow.

Debian supports [a wide variety of architectures](#), and your packager may well ask for changes that aren't relevant to any architecture you actually use. For details, see [CategoryMultiarch](#). Once the packaging process is underway, build logs on  buildd.debian.org will show failures and compiler warnings for all supported architectures.

Bootstrapping (compilers/interpreters only)

If your project is a self-hosting compiler/interpreter for a programming language, explain how to compile the project on a new architecture that does not have an existing compiler for your language. Solutions include:

- support cross-compilation
 - for example, teach your x86_64 compiler to create armhf code

- write a compiler in a language that already exists for that platform, such as...
 - a reference implementation of your language in Python
 - a mini-compiler that only supports the subset of your language necessary to compile the compiler
 - a fork of the last version of your project before self-hosting

This process will very rarely be used in practice, so it's OK for it to be quite baroque. For example, you might start with the last version before self-hosting, then use that to create a mini-compiler, then use that to create a non-optimising modern compiler, and so on.

For another perspective on why this is important, see Ken Thompson's famous [reflections on trusting trust](#) attack, and David A. Wheeler's [diverse double-compiling](#) defence.

Choose a standard open source license

License your project using one of the [licenses compatible with the Debian Free Software Guidelines](#). Popular choices include the [GPL](#), [LGPL](#) and [3-clause BSD license](#).

Writing your own license without training as a lawyer is like writing your own encryption algorithm without training as a cryptographer - you'll probably create something vulnerable to an attack that's widely known in the community but too obscure to think of yourself.

Use the same license for all files

Choose a single license and stick to it throughout the project. If some files need different licenses, consider splitting them into a separate project.

Debian needs to ensure it uses software in accordance with the license - multi-license projects are supported, but are significantly harder to maintain. And if a future legal judgement renders those licenses incompatible somehow, we won't be able to distribute your package at all.

For ways packagers can discard optional files with non-DFSG-free licenses, see ["If upstream has content that is disallowed" in ManageUpstreamDifferences](#).

Maintain a COPYING file

Include a file in the top-level directory called `COPYING`, and if necessary other files with individual licenses.

If all files have the same license, `COPYING` should include the full text of that license.

If some files have different licenses, `COPYING` should specify which files have which licenses, and individual files called e.g. `GPL` or `LGPL` should include the full text of each license.

Some ecosystems use a different filename (e.g. [Python suggests variations on "LICENSE"](#)). If you're not sure a packager would find the file right away, mention the location in the `README`. It's OK to also create a symbolic link from `COPYING` to that file, although that might cause issues for Windows users who try to clone your repository. It's best not to maintain a second copy of the file, because it may become impossible to distribute your package if the two copies fall out of sync.

Maintain an `AUTHORS` file (optional)

If you want to provide detailed information about individual authors (e.g. their e-mail addresses, who they work for, and what they work on), put that in a file called `AUTHORS` in the project's top-level directory.

For an example, see [GNU Hello's `AUTHORS` file](#).

This file is not related to git's [mailmap](#) file, but projects that need one are likely to need the other.

Put copyright attribution at the top of every file

Put a copyright attribution statement at the top of every file, including a statement about the author(s) and license. E-mail addresses and company names can only be omitted if they are listed in `AUTHORS`.

Debian needs to track the copyright information for each file you release, so our [copyright review tools](#) treat copyright statements as metadata. Without that metadata, reviewers have to spend a lot (often a majority) of their time manually confirming the license of each file. See also [On the Importance of Per-File License Information](#).

For an example, see [GNU Hello's `src/hello.c` file](#).

Keep copyright information up-to-date

Change the names and dates in all your copyright statements as necessary.

If you're taking over an abandoned project, add your name everywhere the previous maintainer's name went.

You'll need to update all the year numbers in the copyright statements next January. Consider writing the year as e.g. `2020-2020` instead of just `2020`, so you can use a command like:

```
sed -i -e "s/\(20[0-9][0-9]-\)20[0-9][0-9]/\1$(date +%Y)/" \
$(git grep -l 20[0-9][0-9]-20[0-9][0-9])
```

Actually follow license rules!

License statements specify what you can do with the licensed project. Violating those rules means Debian can't package your project.

One common issue is choosing a BSD license but including GPL libraries. GPL licenses override BSD ones, applying the GPL to the whole package. You'll need to use LGPL libraries instead.

Some libraries have licenses that can't be used together at all. For example, OpenSSL used to have a license that did not allow linking against GPL code.

Register new media types

If you create a new [WikiPedia: media type](#) ("MIME type"), consider  [registering it with the IANA](#). This ensures that applications distributed in Debian and other operating systems recognise your file type.

Provide a well-named source tarball

Provide your source code in a file called something like `$PROJECT_NAME-$PROJECT_VERSION.tar.{gz,bz2,lzma,xz}` (e.g. `hello-1.0.tar.gz`). If you want to provide precompiled binaries for users, put them in a separate binary tarball instead.

Debian needs you to provide complete source code so it can build its own binaries from your source code. That's how it knows the binaries it distributes actually match the source. Debian's standard tools expect to import sources from tarballs with filenames that specify the project name and version number.

Debian needs you *not* to provide binary files so it can distribute the source code itself. That's how other people can confirm that Debian is actually building the binaries from the specified source.

If you include binaries in your source tarball, your packager will need to spend time removing those files because they can't prove you didn't e.g. include an icon with unclear copyright attribution. If you want ordinary users to have a copy of the source code, add it to a subdirectory of the binary tarball instead of adding binaries to a subdirectory of the source tarball.

For an example of Debian distributing source and binary files, see [DebianPkg: the GNU Hello .deb file](#), and [DebianPkg: the source code to build it](#).

For information about proving binaries were created from a given source, see [ReproducibleBuilds](#).

Specify dependencies and versions

Include a list of other projects necessary to build your project, and a separate list of projects necessary to run it. Each dependency should include a name, homepage, license and (where relevant) a range of supported version numbers. For dependencies that require a specific range of versions, describe the range of supported versions. For optional dependencies, mention that they're optional and how to work around their absence.

Debian needs to install packages on their servers while building your project, and needs to tell users which packages to install alongside your package. Listing the relevant projects makes that much easier.

If your project depends on a feature that was only added in a given version of a dependency, or is broken by a change in a given version, Debian needs to know that so it can install your package and know which versions of the dependency to ship.

Depending on a version that's only in [Debian unstable](#) might make it harder to release your package, and depending on an unreleased version or a patch that hasn't been accepted upstream will make things harder still. If you can't avoid such dependencies, explain the issue and link to a post or issue tracker where you sent the patch upstream - a packager may be willing to keep an eye on the issue and package your project when the upstream issue is resolved.

If a dependency needs to be removed from Debian for some reason (e.g. a license dispute), disabling the dependency may be the only way to keep your package available. This is only possible for optional dependencies.

Dependencies are usually specified as e.g. `project-name >= 1.0.0`. You can use `=` instead of `>=` if you need exactly `1.0.0`, but if another package requires `1.0.1`, Debian may be forced to remove your package from the archive.

For details about specifying versions, see  [Syntax of relationship fields](#). For architecture-specific dependencies, see discussions of the `<!nocheck>` profile in [BuildProfileSpec](#). For tricks packagers use to find dependencies, see [How do I know which packages to \(build-\)depend on?](#).

Do not include automatically-generated files

Avoid putting files in your tarball that are automatically generated from other files. Your build system should create these files, and your documentation should explain the dependencies necessary to create them. For artwork, audio etc., see [🌐 How to be a good upstream for games](#).

For security and licensing reasons, Debian needs to prove all files really can be built from the original sources. The only way to do that is to build them, so your package needs to make that possible. For example, compiled documentation, lexers and parsers should all be buildable from source.

This rule is sometimes waived, mainly for historical reasons. For example, most packages do not rebuild the files generated by autoconf and automake, because they used to break during automatic builds.

See also [AutoGeneratedFiles](#).

Do not include a /debian directory

Never put a top-level subdirectory called `debian/` in your tarball. If you want to provide a rough initial packaging directory, consider using `contrib/debian/` instead.

The Debian packaging process involves maintaining a `debian/` directory with its own history separate from your project. Providing your own version of that directory will cause merge conflicts, so most packagers will delete it without looking at it.

If you create your own `.deb` packages, consider keeping your `debian/` directory in a separate branch. Or if you want to nudge packagers to use particular metadata, put example files in `contrib/debian/` and co-ordinate with them when you want to change it.

For information about managing upstream and packaging in a single repository, see ["If there is no upstream repository" in PackagingWithGit](#). For workarounds used by packagers, see ["If upstream has content that is disallowed" in ManageUpstreamDifferences](#).

Do not include third-party code

Do not put code in your tarball where the canonical version is hosted elsewhere. List other people's libraries as build-dependencies, and build against the library versions that are installed system-wide.

When a security issue is detected, Debian's security team needs to quickly identify and fix all the packages that include the vulnerable software. This can involve checking hundreds of projects in a short time, which is only possible if dependencies are declared in a standard way, and if completely automated rebuilds are available.

After the security team resolve a problem, the build servers need to rebuild affected packages, and users need to update their packages. If your package links against a library instead of including it, your package itself won't be part of that problem - your users can just download the library and restart your program.

If your project can only be built against a local copy of a dependency (e.g. because it contains patches you haven't yet pushed upstream), ensure the dependency is optional so Debian can just exclude it. If you can't make a dependency optional and can't get the authors to accept your patches, you'll need to fork that project and become its permanent maintainer.

If you want to specify the exact version of a third-party project because you've experienced version-specific bugs, state the version number in the list of build-dependencies and provide tests that fail on non-conforming versions of the third-party project. Debian will then have enough information to maintain the package.

For details, see  [debian policy 4.13](#).

Do not hardcode distribution-specific values

Do not hardcode messages or settings that are specific to any particular distribution. Instead, talk with your packager about adding build flags, paths to put logos in, or other hooks for them to hang distro-specific content on.

If you hardcode a distro-specific message in your man page or logo in your GUI, packagers for other distributions will need to replace them. That means maintaining a downstream fork, and checking each new release for new distro-specific additions.

Changing distro-specific content (e.g. a updating a logo) involves a lot of distro-wide co-ordination. If you make that configurable, distributions can make the change without needing any of your time.

Include all documentation

Put all your documentation in your source package. Do not hotlink to online versions, or rely on online contents (e.g. JavaScript libraries, images etc.). Avoid third-party libraries and non-free material altogether.

Many users prefer local documentation for various reasons: it works behind a restrictive company firewall, still exists when the online documentation has been updated to a newer version, and lets you grep for a regular expression.

If you use your documentation to generate your project website, consider documenting a build flag to disable site creation. Debian can then set that flag, and remove any build-dependencies that are only needed when making the site.

Write manual pages

Write a manual page for each executable, and for libraries in languages that usually provide them (like C and Perl). If possible, write a manual page for every configuration file. Include an EXAMPLES section wherever possible.

Users expect that typing `man foo` will tell them how to run `foo`. Note that generating a man page from `--help` output (with a tool such as `help2man`) usually don't look like users expect, and generating such a page during the build process can cause your package not to cross-compile.

Users can copy examples from an EXAMPLES section if they're busy or confused. For example, a page for an executable might show some common usages, a page for a library might show examples a user can paste into their own program, or a page for a config file might show the default configuration.

For details, see [!\[\]\(cbe80b694ebd74fcfe136a095b608235_img.jpg\) debian policy 12.1](#).

Provide upstream metadata

Include a list of project-related URLs, security contacts, etc. For scientific projects, include references to the work your project is based on.

Your packager will put this in a [debian/upstream/metadata](#) file - see that page for information and a suggested YAML format.

Use standard paths

Follow the [Filesystem Hierarchy Standard](#) for file locations. If possible, use variables like autotools' [!\[\]\(5361750c22c4e047a52f4eac1ec2d4cc_img.jpg\) GNU Directory Variables](#) or cmake's [!\[\]\(f276343e5e0d2402c20fdc9e8443c0dd_img.jpg\) GNUInstallDirs](#). Other build systems should provide equivalents.

Upstream packages are often installed in `/usr/local`, but Debian packages must not touch that directory. If you use standard environment variables, packagers can change the location without patching the source.

See also [Support staged installs](#).

Remove language extensions from executable filenames

Call your main program e.g. `myprogram`, not `myprogram.py`. If you feel a filename benefits from a language-specific extension, that usually suggests it should be an example program, and should go in an `examples` subdirectory.

Ordinary users shouldn't need to know what language programs are written in, so that information shouldn't be encoded in filenames.

Ordinary users may well want to know what language an example program is written in, but packagers need to put them in `/usr/share/doc/package/examples` instead of `/usr/bin`. If you put examples in an `examples` subdirectory, your packager can use [DebianMan: `dh\_installexamples`](#) to install them.

For details, see:

- [🌐 Debian policy section 10.4](#)
 - see also [🌐 a thread discussing the addition of this policy](#)
- [WikiPedia: Wikipedia's discussion of command names](#)
- [🌐 Commandname extensions considered harmful](#)

Support `/etc/your-project.d` for configuration

Read configuration files from `$(sysconfdir)/your-project.d/*.conf` or `$(sysconfdir)/your-project.d/your-project.conf`, not just `$(sysconfdir)/your-project.conf`.

Supporting multiple configuration files often makes configuration easier. For example, `logrotate` reads anything in `/etc/logrotate.d`, so packages can register themselves with it just by putting a file in that directory.

Even if you only ever expect to support a single configuration file, putting that file in a separate directory can help with some use cases. For example, security-conscious users may want to run your program in a sandbox, which is easier if they can bind-mount a configuration directory (bind-mounting files is supported, but tends to behave in surprising ways).

Support XDG Base Directory paths

Choose directories for caching, configuration, data, sockets etc. based on the [🌐 XDG Base Directory Specification](#). There are [libraries available in Debian](#).

Users increasingly expect programs to follow the XDG Base Directory Specification, and will know to look for files there. And [🌐 systemd uses these directories](#) to configure user units.

Use the right home directory

Choose the user's home directory by checking the `HOME` environment variable (using [DebianMan: `getenv\(3\)`](#) or similar), or fall back on information from the password database (using [DebianMan: `getpwuid\(3\)`](#) or similar).

Hardcoding `/home/$USER` fails for commands run with `sudo`, and for people who choose to put an account's home directory in e.g. `/mnt/memory-stick`.

Release early, release often

Release a new tarball every time you reach a point where everything generally works and you've fixed a variety of bugs or implemented some interesting new features.

Releases signal to Debian that a version is considered stable. Otherwise Debian will have to guess the version from the latest commit to the upstream repo, or an automatic snapshot, or some other heuristic - these all tend to cause nasty surprises.

[Adding a git tag for a release](#) is a useful addition, but a [source tarball](#) is the baseline that is supported no matter the upstream or packaging workflow.

Use a public git repository (optional)

Publish your code in a  [git](#) repository, hosted in a [forge](#) (or at least somewhere online).

 [Debian's packaging forge](#) and  [Fedora's packaging forge](#) both require packages to use git. Using git in your own project makes it easier for most Debian maintainers to pull your changes and send you their patches.

Debian can handle projects that just release tarballs, but putting your repository online lets package maintainers (and other contributors) `pull` your changes in a more natural way.

Forges provide a more familiar interface for ordinary users, and can include features that help automate the packaging process. For example, you might be asked to create a webhook on your forge that  [triggers a pipeline](#) on Salsa whenever you release a new version.

Add git tags with a common format (optional)

If you manage your project with `git`, consider making a new tag of the form `v$VERSION` for each release (e.g. `v1.2.3` for release 1.2.3). If you use a different pattern (e.g. `release-$VERSION`), explain the pattern so your packager can write a regular expression for it. Please *avoid* tags of the form `upstream/$VERSION`, as some Debian automation software generates this pattern by default.

Many packagers who use `git` will automate their process by looking for tags that match a known pattern. Guaranteeing that pattern makes their life easier.

For more information, see ["Import from an upstream git repository" in PackagingWithGit](#).

Add unique version numbers

Give each release a version number greater than the previous release, even if the previous release was an accident that only went up for a few minutes. If version numbers are supposed to mean something (e.g. [semantic versioning](#)), ensure numbers follow the policy even if you have to bump features to a later milestone than you promised.

If two tarballs have the same release number, packaging scripts might behave surprisingly because they only saw one version. Or worse, they might think your website has been compromised and malicious code added to your package. No matter how tiny the change, you should never reuse a version number. Nobody will mind if you release version 1.2.4 a few minutes after 1.2.3 with a message about how the previous version was unusable for some boring reason.

Be bold about increasing version numbers - don't get trapped in sub-sub-sub-versions or scared about crossing magic numbers. If you promised yourself version 1.0 would have some feature, bump the feature back to version 2.0 instead of releasing a version 0.9.8.93 that's 1.0 in all but name. You won't run out of numbers, and anyway you'll be on to release version 1.1 the moment a user finds a bug.

[i](#) Debian went straight from version 0.93R6 to 1.1, [Wikipedia: there is no Debian 1.0](#).

Sign releases

Sign your release tarballs, tags, and announcements with your OpenPGP key. Include digests for all artifacts in your announcement, using a strong digest such as SHA2-256, SHA2-512 or SHA3, not e.g. MD5, SHA-1 or RIPEMD-160. If your project is especially security-sensitive, consider [signing every commit](#).

Package maintainers try to review your work, but their specialism is packaging, so they often have to accept your code changes are correct. Signing your packages makes it harder for an attacker to get malicious code into Debian by guessing some password.

Signing everything and including the digests in the announcement creates chain of trust for the entire release. If you only sign the release announcement, an attacker can replace the source tarball with a malicious copy. If you sign the release, source tarball, and any other artifacts but don't include the digests in the announcement, an attacker can replace the source tarball with a (signed) older version that still had a known vulnerability.

For information about signing tags, see [git tag -s](#). For details about key-signing, see [creating and signing OpenPGP keys](#) and [finding somebody to sign your key](#).

Explain support lifetimes

Explain the level of support for the current and older versions of your project. Consider committing to long-term support for versions of your software that wind up in stable releases of major distributions.

Distributions need to support all packages for the lifetime of a release, which is often several years. It's OK to leave that to distros if you don't have the resources to do it yourself, but they need to prepare for that during the packaging process.

If you're not sure how much support you can provide, talk it through with package maintainers and agree who will provide what level of support.

Ensure backwards-compatibility

Design your API/ABI with backwards-compatibility in mind, and update the project's major version every time you break compatibility. Don't let private symbols leak into public interfaces.

If you're writing a library that's compiled to a `.so` file, assign an SONAME like `libfoo.so.N`, where `N` is the project's current major version number. Consider using the [!\[\]\(83f22ed94ec5517769dd76d702c6bfd8_img.jpg\) ABI Compliance Checker](#) to make sure you don't accidentally make an incompatible change.

If you're writing a library, hide symbols by default unless you explicitly want to make them part of the public API. For C and C++, see [!\[\]\(8d0f0e0fe25b320c33272c52aec1fbca_img.jpg\) Visibility](#) (page starts talking about C++ in GCC, then moves on to C and other compilers). For other languages, look for discussions about "visibility", "hidden" or "private" symbols. For languages that lack a built-in "private" flag, there is often a convention of prepending an `_` to indicate that a symbol is private.

If your project puts some functionality in library files for internal use only, make those libraries private whenever possible. For C and C++, use `-fvisibility=hidden` (discussed in [!\[\]\(642aa997563f9a325b310230bb5078b7_img.jpg\) Visibility](#)). For other languages, search online for discussions about linkage. For interpreted languages, your build system might need to concatenate your libraries and executable into a single file.

Use a bug tracker

Explain how people should report issues. If possible, explain how to privately report security issues. For example, your `README` file could link to your GitHub issue URL and ask people to e-mail you privately about security issues.

Users often report issues to [!\[\]\(d0262bbe9d2356661a2e89321dfcc781_img.jpg\) `bugs.debian.org`](#), even for non-packaging-related problems. Linking to your own bug tracker makes it easier for people triaging bugs to contact you, or for users to find your URL if they don't get a response from Debian.

It's particularly important to put your security procedure somewhere people will find it, because security issues are likely to come from people who haven't interacted with your package before (e.g. the Debian security team), and need to quickly mitigate problems with a large set of vulnerable packages.

Distinguish between bugfix and feature changes

Keep features and bugfixes in separate commits, especially if you rely on Debian for some of your project's [support lifetime](#).

When you fix a bug in the latest version of your project, there may be a need to backport it to an older version. For small bugs in a well-maintained git repository, this is usually as simple as 🌐 [cherry-picking](#) the commit and creating a new patch-level release.

Handle security issues transparently

🌐 [Request a CVE identifier](#) for every security issue, and include the CVE identifier(s) in commit logs, NEWS, changelogs and announcements. Provide patches for each version you support, and help security teams and maintainers who need to backport that fix to older versions you don't support. 🌐 [Contact the Debian security team](#) if you have questions.

It's natural to take a moment to blame yourself when a security issue arises, but everyone else will judge you more on your actions after the event than before.

As a rule of thumb, transparency about security issues helps defenders more than attackers. Sneaking an undocumented fix into an unrelated patch might increase the time before evil hackers find it, but it also increases the risk overworked maintainers will ignore it completely. Announcing the issue on a mailing list probably won't give those hackers enough time to exploit it before you can patch it, let alone before your users can roll out temporary mitigations.

Ideally, you should provide a patch for all versions of your program in the stable versions of all major distros. It's OK to rely on distributions to do that maintenance work, but be aware they'll need your help cherry-picking the relevant commit(s) into their version.

Document your build system

Explain which build system your project uses, and which commands are necessary to build it.

Debian adds a Makefile called `debian/rules` that can wrap any build system, but well-supported build systems like those below tend to make packaging easier.

For a list of build systems explicitly supported by Debian, see [Debhelper/Buildsystem/*.pm](#).

Autoconf and Automake

Update `config.guess` and `config.sub` before each release - use `autoreconf`, [download them from GNU](#), or copy them from `/usr/share/misc/` (they're part of [DebianPkg: autotools-dev](#)).

Autoconf and automake are mature enough that most users can make do with older versions. But changes often affect some edge case that's relevant to Debian, so staying up-to-date can avoid build failures.

Make

Do not set variables used by `dpkg-buildflags`. For a list, see [DebianMan: supported flags](#) or do `dpkg-buildflags --list`. Common flags include `CFLAGS`, `CPPFLAGS`, `CXXFLAGS` and `LDFLAGS`.

Some compilation flags are reserved for the user. The [Automake manual](#) and the [GNU coding standards](#) advise never to use them for switches that are required for proper compilation. `dpkg-buildflags` sets these variables during compilation, and overriding them can cause unexpected behaviour.

If you need to set compilation flags in your Makefile, use another name instead. For example:

```
LOCAL_CFLAGS=$(CFLAGS) -DF00=1 -DBar=2
```

Java build systems

Consider using [Maven](#). Debian can redirect `pom.xml` files to dependencies available locally, making maintenance much easier.

Some language-neutral issues that are particularly common in Java packages:

- do not ship prebuilt `.class` or `.jar` files, or any other generated files without accompanying source code
 - see [Do not include automatically-generated files](#)
- do not download files during the build process
 - see [Support offline building](#)
- document each dependency, especially where its source code is available from
 - see [Specify dependencies and versions](#)

Perl build systems

Use `Module::Build` or `ExtUtils::MakeMaker`, and please don't modify it in too many odd ways. Provide a way to set defaults or bypass prompts with known answers, to make automated builds possible.

For details, see [!\[\]\(3dfb8d66e81160ad61421a3452093d1b_img.jpg\) the Perl group's package policy](#).

Python build systems

Use one of Python's standard build systems. [!\[\]\(0f848bbd71cef6b345273b16f905912a_img.jpg\) Debian's python policy specifically mentions setuptools](#).

Some language-neutral issues that are particularly common in Python packages:

- do not break API/ABI unless you really have to
 - see [Ensure backwards-compatibility](#)
- do not bundle local copies of third-party modules, we will remove them anyway
 - see [Do not include third-party code](#)
- do not depend on unstable or unreleased versions of third-party libraries
 - see [Specify dependencies and versions](#)

OCaml build systems

Support installing your project on architectures where the native-code compiler (`ocamlopt`) is not available.

OCaml provides [!\[\]\(6059a5aa8b4ca7bb793408023d6c6e42_img.jpg\) native-code compilation](#) for some platforms, and [!\[\]\(d293b9aef7d8767760396289fbc64e8a_img.jpg\) bytecode compilation](#) for the rest.

The easiest solution is to provide `make` targets (or your build system's equivalent) called `all` and `install` to build native code, plus targets called `opt` and `install-opt` to build bytecode. Alternatively, have `all` and `install` behave differently depending on whether `ocamlopt` is present at build-time.

Avoid SCons

Avoid SCons - it is hard to use correctly. For example, it ignores environment variables like `CFLAGS` and `DESTDIR` by default, and doesn't support [Wikipedia: SONAMEs](#). Projects that use SCons often don't have a working `install` target. Projects also tend to have their own workarounds, so there's no way to just use a "standard" SCons project in Debian.

If you use SCons anyway, ensure that the usual environment compiler variables (`CC`, `CFLAGS`, etc.) and path variables (`DESTDIR`, `BINDIR`, `LIBDIR`, etc.) are honoured. There is a [!\[\]\(e3275251d0893157c3584e20c81dc3ba_img.jpg\) recipe](#) to address some of these issues.

For more information, see the Gentoo wiki's [arguments against using SCons](#).

Avoid waf

Avoid waf - it recommends shipping a waf executable in your source tarball (i.e. [shipping an auto-generated file](#)).

Debian's FTP team consider shipping just the waf binary non-compliant with the Debian Free Software guidelines, creating more work for your packager. For details, see [Closed in postler/0.1.1+dfsg-0.1: #645190: postler: doesn't contain source for waf binary code: #645190](#). If you ship a waf executable anyway, see [UnpackWaf](#).

Debian supports leaving the waf executable out of your source tarball and compiling with the system-provided waf instead. But upstream waf explicitly discourages that.

Maintain a flexible build process

Run tests while building

Make a test suite that runs during the build process. If possible, make another test suite for the actual installed program.

Unit tests are a great way to ensure your program works. Running them with a command like `make test` lets Debian check your program works on every architecture it's built for. It's quite common for programs that have only ever run on an x86_64 PC to fail on e.g. armhf.

System integration tests are a useful way to check for packaging errors. For example, Debian can often detect missing dependencies just by installing your package and calling your program with `--help`. Programs that interact with the system in more complex ways (e.g. editing the firewall) may find integration tests even more useful than unit tests.

For test results from recently-tested packages, see <https://ci.debian.net>.

Support verbose building

Disable your build system's "verbose" mode by default, enable it with a flag that is either well-known or documented in your README file.

Your packager will build your package a few times, then hand it over to Debian's build servers, which will rebuild automatically from time to time (e.g. when your dependencies are updated). Packagers usually prefer a quiet build log during testing, ignore the log for successful automatic builds, then suddenly want all the information they can get when they find an intermittent error that only occurs on riscv64. That will all just work if you provide a standard way for the build servers to enable verbosity.

These well-known flags are currently recommended:

```
# if you use autoconf:
./configure --disable-silent-rules
# if you use Makefiles:
make V=1
```

Support offline building

Ensure that your project can be built without Internet access. You should only need the dependencies listed in your README file.

Debian guarantees every binary package can be built from the available source packages for licensing and security reasons. For example, if your build system downloaded dependencies from an external site, the owner of the project could release a new version of that dependency with a different license. An attacker could even serve a malicious version of the dependency when the request comes from Debian's build servers.

Support out-of-tree builds

Ensure that your software can be built from specific subdirectories of the source tree, and from entirely different locations.

Debian may need to create multiple packages for your project. For example, if your `/server` directory contains a C program and your `/client` directory contains a shell script, Debian will probably `cd ../server` and build it for every architecture, then `cd ../client` and just build it once.

Debian may also need to build your software for multiple architectures at the same time. For example, if your project needs to be compiled for `i386` and `x86_64`, the build system might extract the project contents into two separate directories and run the compilations in parallel.

If you use automake,  [make distcheck](#) will check your package supports this.

Support out-of-VCS builds

Ensure that your software can be built in a directory without files from your version control system (e.g. `.git` or `.svn`).

Debian guarantees every binary package can be built from the available source packages for licensing and security reasons. We can't do that if your package requires version control information that isn't present in the source tarball.

If your project can't be built without VCS metadata, you will need to include it in your source tarball. Consider exporting VCS logs with  [git2cl](#),  [svn2cl](#) or  [cvs2cl](#). Or consider  [autorevision](#) to export VCS metadata.

Support cleaning the tree

Delete all generated files and reset the build tree to its initial state with a command that is either well-known or documented in your README file.

Debian (and your users) may want to reuse an extracted directory instead of deleting it. This is usually done with `make clean`, but other options can be supported if necessary.

Support other compiler commands

Call your compiler using a variable like `$(CC)` or `$(CXX)`, not a specific program name like `gcc` or `g++`. Either use standard variables names or document the variables. If possible, try your code in another compiler like `clang`.

Your project will usually be compiled *on* an amd64 PC, but can be [cross-compiled](#) for all of Debian's other architectures. Cross-compilation builds of `gcc` have filenames like `aarch64-linux-gnu-gcc`, so Debian needs to call the right command when cross-compiling.

Debian's build system sets the well-known `CC` and `CXX` environment variables when cross-compiling, but packagers can add non-standard variables in `debian/rules` if necessary.

Compiling your code in another compiler can be a good way to find subtle bugs. For example, `gcc` and `clang` often adapt each other's code tests, so fixing a clang-only warning today could avoid a gcc error tomorrow.

The following standard variables are available by default:

Replace...	... with
<code>ar</code>	<code>\$(AR)</code>
<code>as</code>	<code>\$(AS)</code>
<code>cc</code>	<code>\$(CC)</code>
<code>ctangle</code>	<code>\$(CTANGLE)</code>
<code>cweave</code>	<code>\$(CWEAVE)</code>
<code>g++</code>	<code>\$(CXX)</code>
<code>f77</code>	<code>\$(F77)</code>
<code>ld</code>	<code>\$(LD)</code>
<code>lex</code>	<code>\$(LEX)</code>
<code>lint</code>	<code>\$(LINT)</code>
<code>m2c</code>	<code>\$(M2C)</code>

make	\$(MAKE)
cc	\$(OBJC)
pc	\$(PC)
tangle	\$(TANGLE)
tex	\$(TEX)
texi2dvi	\$(TEXI2DVI)
weave	\$(WEAVE)
yacc	\$(YACC)

Other  [binutils commands](#) (like `strip` or `readelf`) need to be replaced with variables, but do not have standard names.

Support staged installs

Support installing in a subdirectory with the `DESTDIR` environment variable.  [GNU Directory Variables](#),  [GNUInstallDirs](#) etc. do this automatically.

When building a package, your project will be installed to a staging directory, then everything in that directory will be packaged. This is usually accomplished by setting the `DESTDIR` environment variable, and expecting the build system to prefix that to all paths.

For details, see  [DESTDIR: Support for Staged Installs](#).

If you use automake,  [make distcheck](#) will check your package supports this.

Split the "make" and "install" stages

Provide a "make" step that builds all necessary files, and an "install" step that just moves those existing files into their correct location.

If you modify files during the install stage, you risk embedding temporary staging paths in the final output. This is ugly at best, and can be a security issue for [Wikipedia: RPATH](#) values.

Create directories during the "install" stage

Ensure your "install" stage works whether or not your destination directories exist.

Most users who run `make install` on their local system will already have a `/usr/local` directory, but most automated build systems will set `DESTDIR` to an empty directory. The `mkdir -p`, `install -d` and `installdirs` commands all succeed whether or not the destination path exists, as does the `$(MKDIR_P)` Autoconf/Automake macro. Your build system may also provide its own solution.

Ignore your project's top-level directory name

Don't assume your project's top-level directory will have any particular name.

When building packages, Debian might unpack your project into a temporary directory. So for example, a `cmake` script that sets `PROJECT_NAME` based on `CMAKE_SOURCE_DIR` could end up calling your project `tmp.HjH9buvNvx`.

Do not modify existing files

Treat files in your source tarball as read-only. If you need to generate a file based on them, create a new file with a different name instead of overwriting the original.

Files with different contents between the source tarball and the final build are confusing, and can make it look like your packager has introduced a Debian-local modification. Such modifications are frowned upon, which makes your packager look bad.

Autoconf solves this problem well. It reads files of the form `foo.in`, substitutes some variables, then writes an equivalent `foo` file. `foo` should be treated like any other [generated file](#), and should be deleted by `make clean` and/or `make distclean`.

Do not target the machine being built on

Avoid compilation options that assume the CPU being built on is the same as the CPU the program will run on. If you want to provide such options, disable them by default and mention how to enable them in the README. Or if you want to provide optimised versions of libraries, use [DebianMan: hardware capabilities directories](#).

Packages are usually compiled on machines that are relatively modern, but Debian supports decades-old CPUs so long as they support certain baseline features. If you create binaries that require all the features of the build system's processor, your program will crash on most users' machines. Even if you just *optimise* for the current build system, you will reduce performance for the users who most need it.

See also

Debian resources for your package

- <http://tracker.debian.org/your-package>
 - a portal to all kinds of information
 - click *subscribe* in the top-right to be notified about releases and bug reports
 - recommended but optional - the Debian maintainer will forward you bugs that need your help or interest
- <http://bugs.debian.org/your-package>
 - all bugs that have been reported by Debian users
 - we would be delighted if you helped out, proposed patches or fixed issues upstream

- [DebianList: debian-devel](#)
 - discussion about technical development topics
- [DebianList: debian-project](#)
 - non-technical topics related to the Debian Project

Producing software

-  [Producing Open Source Software](#) by Karl Fogel
-  [Packaging Unix software](#) by Adam Sampson
-  [Releasing FLOSS for Source Installation](#) by David A. Wheeler
-  [Ten Simple Rules for the Open Development of Scientific Software](#) by Andreas Prlić and James B. Procter ( [LWN comments](#))
-  [FOSS Build and Install Common Practices](#) by Matt Taggart
-  [The Fundamental Theorem of Developing FLOSS](#) by Máirín Duffy ( [followup](#))
-  [Expectations of free software developers](#) by Lars Wirzenius
-  [What is source code?](#) by Francesco Poli ⚠️ Possibly controversial
-  [Best Practices Criteria for Open Source Software \(OSS\)](#) by the Linux Foundation's Core Infrastructure Initiative ( [LWN article](#))
-  [Release management in Open Source projects](#) by Martin Michlmayr
-  [What is Left to do After your Open Source Project is Done](#) by Loup Vaillant
-  [Our Software Dependency Problem](#) by Russ Cox
-  [Writing a C library](#) by David Zeuthen ⚠️ (includes some advice that can be considered controversial)
-  [How you know your Free or Open Source Software Project is doomed to FAIL](#) ( [wiki version](#)) by Tom Callaway
 - see also  [This is why you FAIL](#) at SCALE 2011 and  [This is why you fail: The avoidable mistakes open source projects STILL make](#) at OSCON 2015
 - ⚠️ includes some advice that large successful projects like Linux, Qt, and GTK do not heed
- A blog series by François Marier
 -  [Choosing the right license for your new Free Software/Open Source project](#)
 -  [Promoting your Free Software Project](#)
 -  [Growing the community around your new Free Software Project](#)
 -  [Getting your project included into Free Software distributions](#)
 -  [Starting a Free Software project](#)
 -  [Getting involved in an Open Source project](#)
 -  [Handling security bugs in your Free Software project](#)
 -  [Writing the perfect patch](#)
 -  [3 ways to improve your source control history](#)
 -  [Keeping track of what others are saying about your project](#)

Interacting with distributions

- Ubuntu's [UbuntuWiki: Upstream Guide](#)
-  [How to be a good upstream](#), a FOSDEM 2010 talk by Petteri Rätty of Gentoo
- "Distribution-friendly projects" by Diego Pettenò -  [part 1](#),  [part 2](#),  [part 3](#)
-  [Why not bundle dependencies](#) by Sebastian Pipping and others
-  [How to be a good upstream for games](#) by Debian/Fedora/etc games teams
-  [The java packaging nightmare...](#) by Vincent Fourmond
-  [The real problem with Java in Linux distros](#) by Thierry Carrez ( [LWN comments](#))

Managing a community

-  [Free Software Project Management HOWTO](#) by Benjamin Mako Hill
-  [The cost of going it alone](#) by Dave Neary ( [LWN comments](#))
-  [Open Source Community, Simplified](#) by Max Kanat-Alexander
-  [How to spread the word about your code](#) by Peter Cooper and Robert Nyman
-  [Considerations on a non-profit home for your project](#) by Bradley M. Kuhn
-  [Spread the word: Marketing your FOSS project](#) by VM Brasseur
-  [The open source way](#) by Red Hat

Legal and philosophical

-  [Managing Copyright Information within a Free Software Project](#) by the  [Software Freedom Law Center](#) ( [LWN comments](#))
-  [Community Distribution Patent Policy FAQ](#) by the  [Software Freedom Law Center](#) with input from Stefano Zacchiroli ( [LWN comments](#))
-  [It isn't open source if it doesn't pass "The patch test"](#) by Andy Oliver
-  [Don't call it "open source" unless you mean it](#) by Christian Heilmann ( [LWN comments](#))
-  [Choosing a license](#) by Dave Neary
-  [Source required for art licensed under the GPL](#) discussion started by Guus Sliepen
-  [Missing source code for non-software works in free GNU/Linux distributions](#) by Michał Masłowski

[CategoryPackaging](#)

- [Packaging](#)
- [ComplexWebApps](#)

Example 1: Greenlight

- ruby on rails project.
- it starts a web service and needs a systemd unit file
- Home page:  <https://github.com/bigbluebutton/greenlight>

Package each dependency separately

1. start with packaging dependencies in Gemfile
2. you could make a tracker page like  <http://debian.fosscommunity.in/status/gitlab-v12.6.0/> to see which of those are packaged already
3. you can look at gitlab, diaspora, redmine and open-build-system as examples
4. vendor/assets/javascripts will need to be replaced with their packaged versions
5. you should target unstable and later you can try to get it to stable-backports, trying to run it directly on stable will not be a good idea
6. look at diaspora-installer as a quicker PoC to get a working package fast, just runs a bundle install to install all dependencies from rubygems.org instead of using packaged gems
7. you could embed all gems in the package too. bundle install --path vendor/bundle and include that directory in source
8. download source tarball from tag, rename as required 2. extract and run bundle install 3. Include vendor/bundle in the orig.tar, may be rename to +debian.orig.tar 3. debmake 4. dpkg-buildpackage. See  <https://wiki.debian.org/SimplePackagingTutorial>
9. rake assets:precompile where to run it? Option 1. in postinst like in gitlab or Option 2 like in rainloop during build

Packaging/ComplexWebApps (last modified 2020-05-12 18:01:11)

- [Packaging](#)
- [Pre-Requisites](#)

Translation(s): [English](#) - [Português \(Brasil\)](#)

Contents

1. [Option 1: Incus](#)
2. [Option 2: Systemd Nspawn and Debspawn](#)
3. [Option 3: Schroot and Sbuild](#)
4. [Option 4: LXC](#)
5. [Option 5: Docker](#)
6. [Option 6: Virtual Machine](#)
7. [Option 7: Direct/Bare metal install](#)
8. [Option 8: Window Subsystem for Linux \(WSL 2\)](#)

You MUST have a [Debian unstable](#) environment (physical/dual boot, virtual machine, container or a chroot with [schroot](#)) to create packages suitable for uploading to Debian. See instructions given below to setup Debian Sid. You can use lxc, incus, docker or a virtual machine for development.

Note: The target audience is people just discovering Debian.

Readers note: This page lists 8 different approaches. It doesn't list 8 steps of one approach. You can choose any one approach at a time and follow the instructions under it. For example, if you want to use docker, there is no need to run the commands listed under Schroot/Incus/LXC/Virtual Machine.

If you want to use osx on x86_64 platform for packaging, refer to docker or virtual machines sections and for arm64 platform use  [UTM](#)

If you don't know about these options, Incus is the recommended option.

Option 1: Incus

Incus is a fork of LXC/LXD by the original LXD developers. It can create a container or a virtual machine.

If you already setup a sid development environment using one of the methods given above, you can skip this step.

- [Using Incus](#) for creating a sid development environment.

Option 2: Systemd Nspawn and Debspawn

If your host system already has systemd as init system, this would be the best option for you.

- [Using Systemd Nspawn](#) for creating a sid development environment.
- [Using debspawn](#) for clean build. sbuild can be used as an alternative to debspawn.

Option 3: Schroot and Sbuild

If you already have a Debian stable or Debian-based distribution (Arch also has schroot package, though if you have a different distro, you need to check if it has schroot package), this option is best for you.

-  [Using schroot](#) for creating a sid development environment.

Option 4: LXC

If you have trouble setting it up, skip this section and see docker section below.

If you already setup a sid development environment using one of the methods given above, you can skip this step.

- [Using LXC](#) for creating a sid development environment.

Option 5: Docker

If you already setup a sid development environment using one of the methods given above, you can skip this step.

- [Using Docker](#) for creating a sid development environment.

Option 6: Virtual Machine

If you already set up a sid development environment using one of the methods given above, you can skip this step.

- [Using Virtual Box or Vagrant](#) to create a sid development environment.

Option 7: Direct/Bare metal install

You can also install Debian unstable in your machine under dual boot configuration or even replacing existing OS. See [DebianUnstable](#)

Option 8: Window Subsystem for Linux (WSL 2)

If you already set up a sid development environment using one of the methods given above, you can skip this step.

Debian unstable can also be installed in your local Windows OS machine using  [WSL2 \(install info\)](#).

Note that this requires you be running Windows 10 version 2004 and higher (Build 19041 and higher) or Windows 11. See  [installing WSL2](#).

The installed WSL2 doesn't come with an "out of the box" setup for schroot. You will need to setup schroot (from [Option 3](#)) after  [installing a Debian distro](#) on WSL2.

[CategoryPackaging](#)

Packaging/Pre-Requisites ([last modified 2026-03-19 17:03:46](#))

- Projects
- [DebSrc3.0](#)

Contents

1. Summary

2. FAQ

1. Does a 3.0 (quilt) source package need to build-depend on quilt?
2. Is the README.source file needed with 3.0 (quilt) source packages?
3. How to use multiple upstream tarballs in 3.0 (quilt) format?
4. Why should I use 3.0 (quilt) format for my package?
5. My 3.0 (quilt) package fails to build or fails to unpack on all builds
6. Why aren't -p0 patch supported in 3.0 (quilt) format?
7. How many packages have already been converted?
8. Are there docs?
9. Are there more docs?
10. Are there plans for a 4.0 format? Will I need to switch again soon?
11. How to convert back to 1.0?
12. Can I avoid the multiple patches? All my patches are in version control
13. How to convert a source package?

3. Validation of all tools with new source formats

4. Contributors

Summary

This page summarizes information concerning the "**3.0 (quilt)**" and "**3.0 (native)**" source package formats that have become the de-facto standard for non-trivial packages in Debian. [DebianMan: dpkg-source\(1\)](#) recommends using one of those formats for new packages. All packages should explicitly declare their source format in `debian/source/format` (the default is still "1.0" which should not be used for new packages).

The 3.0 formats support

- gzip, bzip2, lzma and xz
 - Note: ftp-master.debian.org may disallow formats (such as lzma) that the packaging software supports.
- multiple upstream tarballs (only for the **3.0 (quilt)** format).
- inclusion of binary files. There is no need, for example, to uuencode a Debian specific PNG icon.

The debian directory is automatically removed from the upstream tarball. It is not necessary to repack upstream sources just because they contain a debian directory.

Debian-specific changes are stored in multiple patches in `debian/patches/`. This is often called a **patch queue**. It is compatible with quilt (hence its name) but does not require its usage as `dpkg-source` is able to do everything needed by itself. It applies patches at extraction time and update the patch series at build time.

These source formats are supported in Debian since the early 2010 years. You can expect universal support.

[DebianMan: dpkg-source\(1\)](#) has some documentation about all source formats in the "SOURCE PACKAGE FORMATS" chapter.

FAQ

Does a 3.0 (quilt) source package need to build-depend on quilt?

No. Don't invoke quilt explicitly in `debian/rules`. You CAN optionally use quilt to maintain your patch queue in `debian/patches`, but there are tools like `gbp pq` that allow you to maintain your patch queue without quilt.

Is the README.source file needed with 3.0 (quilt) source packages?

No. It's the de facto default, people know how to handle this type of package.

How to use multiple upstream tarballs in 3.0 (quilt) format?

Simply put a file `<source>_<version>.orig-<addon>.tar.{gz,bz2,lzma,xz}` near the traditional `.orig` tarball. The content of that tarball should also be already unpacked in the `addon` subdir of the unpacked source package. The next `dpkg-source -b` will pick it up automatically.

More docs can be found in this article: [🌐 How to use multiple upstream tarballs in Debian source packages](#) and in [DebianMan: dpkg-source\(1\)](#).

Why should I use 3.0 (quilt) format for my package?

Pick the reasons that matter to you:

- it's the de-facto standard, recommended by dpkg, and everyone knows how to use this format.
- it keeps patches separate from each other and properly documented (with  [DEP-3](#)):
 - the Debian changes are more likely to be reviewed
 - upstream contributors are more likely to find and merge the patches (if you haven't forwarded them properly)
 - other distributions can reuse (or ignore) our patches
- even if you don't have any upstream patch right now, next time that someone must NMU your package, they can cleanly add a patch (with a proper DEP-3 header) without having to modify the build system
- same applies for derivative distributions that have to modify your packages... you're more likely to be able to find something valuable to merge
- debian/rules is simpler without patching/unpatching code
- it is easier to add binary Debian-specific data
- the upstream tarball can be compressed with other compressors than gzip while keeping pristine upstream sources in Debian's archive.
- the upstream tarball can contain a debian directory without bothering your work.
- the upstream author releases the software in multiple tarballs

My 3.0 (quilt) package fails to build or fails to unpack on all buildds

If you get a failure like:

```
dpkg-source: info: applying XXXX
1 out of 1 hunk FAILED -- saving rejects to file xxx.rej
```

You probably have quilt patches that only applies with fuzz (i.e. they only apply if patch ignores context lines). quilt applies those without failing but dpkg-source doesn't. The solution is to refresh the patches, for example with those commands :

```
quilt pop -a
while quilt push; do quilt refresh -p ab; done
```

If you use gbp pq or a similar tool to maintain your patch queue, use their mechanisms.

dpkg-dev prevents you from building source packages with such patches.

Why aren't -p0 patch supported in 3.0 (quilt) format?

See sub-thread starting  [here](#).

How many packages have already been converted?

Many. Less than 5 % of Debian's source packages are not in 3.0 (quilt) format.

Are there docs?

To actually take advantage of this new format, see

- [DebianMan: dpkg-source\(1\)](#)
- the  [Quilt for Debian Maintainers guide](#),
- the  [Guide for Debian Maintainers](#), and
- Raphaël's very helpful article:  [How to use quilt to manage patches in Debian packages](#) (August 8, 2012).

They explain how to add and edit patches and integrate things in your workflow.

Are there more docs?

Try:

-  [Raphaël's Article about maintainer 3.0 quilt packages in a VCS](#) and
-  [Colin Watson's Article about how he uses 3.0 quilt with bazaar with patches applied in bazaar](#)

Are there plans for a 4.0 format? Will I need to switch again soon?

At the time of this writing (2025/05), there are no immediate plans for a 4.0 format. But there are lists of grievances with the **3.0** formats that might be the base for future formats (such as [Teams/Dpkg/TimeTravelFixes#dsc](#), and some of the items in [Teams/Dpkg/Spec/ChangesFormat2.0](#) related to digests).

How to convert back to 1.0?

Please consider not doing so. We have no idea why you should want this, but [DpkgV3toV1](#) has the instructions from the old backports.org site.

Can I avoid the multiple patches? All my patches are in version control

Yes. Just create a file `debian/source/local-options` with content:

```
single-debian-patch
```

This option creates a single "debian-changes" patch in `debian/patches`, similar to the `.diff.gz` of the v1 source format. The `local-options` file is **not** shipped with the source package, so this means that any NMUs or downstream changes will still use a separate patch file. This is a feature, not a bug; it allows you to use your preferred workflow while not impacting others.

How to convert a source package?

You need to put "3.0 (quilt)" or "3.0 (native)" in `debian/source/format` to indicate the desired format to `dpkg-source` (see [DebianMan: dpkg-source\(1\)](#) for more information):

```
mkdir debian/source ; echo '3.0 (quilt)' > debian/source/format ; dch 'Switch to dpkg-source 3.0 (quilt) format'
```

Native packages should not need any other change.

When you switch to "3.0 (quilt)", there are other changes that you might want to do:

- If your `.diff.gz` modifies upstream files, you should really put those changes in separate quilt patches first, otherwise all those changes will be merged in a single quilt patch named `debian/patches/debian-changes-<version>`. Consider documenting those patches by following [DEP-3](#).
- If you use another patch system (`dpatch`, `db`s, `cdb`s, etc.; with `-p0`, `-p1`, or `p2`), consider switching to the "3.0 (quilt)" format (see [Closed in quilt/0.60-3-#581186: quilt: add conversion script to new 3.0 \(quilt\) format: 581186](#) for a conversion helper script). (You can then simplify your `debian/rules` with `"dh $@"`, see next point).
- You **should** remove everything related to quilt in `debian/rules` (`patch/unpatch` logic, cleanup of quilt stamp file and its `.pc` directory).

Validation of all tools with new source formats

For all applications that work with 3.0 packages, please be sure to test the 3 important cases:

- 3.0 (native)
- 3.0 (quilt)
- 3.0 (quilt) with multiple upstream tarballs

It's also interesting to test with various compressions methods (`gz`, `bz2`, `lzma`, `xz`).

Some sample source packages are available [here](#). You can add it to `/etc/apt/sources.list` with `deb-src http://people.debian.org/~hertzog/packages/ debsrc3.0/`. They have the following characteristics:

- sample1: 3.0 (quilt) with `orig.tar.gz` / `debian.tar.gz` and additional tarballs with all compressions schemes
- sample2: 3.0 (quilt) with `orig.tar.bz2` / `debian.tar.bz2`
- sample3: 3.0 (quilt) with `orig.tar.lzma` / `debian.tar.lzma`
- sample4: 3.0 (native) with `tar.gz`
- sample5: 3.0 (native) with `tar.bz2`
- sample6: 3.0 (native) with `tar.lzma`

Please consider also testing with more current packages from the archive. (Contributors, please consider adding example packages here).

Current test results:

[Teams/Dpkg/DscSupport](#)

As of 2025, none of the mainstream tools that handle source packages have known issues with 3.0 packages.

Possible improvements (please consider research or asking on `-devel` before spending work on those, those ideas are a decade old):

- other devscripts tools ?

- A new debreview tool would be welcome to replace zless *.diff.gz: [Open in devscripts/2.10.61: #575394: \[new\] deb{review,inspect} showing Debian packaging files and Debian changes: 575394](#)
- devscripts dqilt
 - Wishlist: wrapper for quilt. [Open in devscripts/2.10.72: #624556: devscripts: dqilt for 3.0 \(quilt\) format: 624556](#)

Please try using new source packages with all your usual tools (in particular if you maintain them) and file bugs if you encounter problems and usertag them appropriately (user  hertzog@debian.org / tag 3.0-quilt-by-default). Also update the list above with any relevant information (including pointers to bugs).

Contributors

[RaphaelHertzog](#) and others.

[CategorySpec](#) [CategoryDeveloper](#) [CategoryPackaging](#) [CategoryPermalink](#)

Projects/DebSrc3.0 (last modified 2026-01-21 11:21:33)

- Courses2005
- [BuildingWithoutHelper](#)

[Translation\(s\)](#): none

How to make a Debian package without using a helper

by Miriam Ruiz Version 0.3, 09-Jan-2005

Contents

1. [How to make a Debian package without using a helper](#)
2. [Introduction](#)
3. [Downloading and unpacking the source](#)
4. [Testing the upstream building system](#)
5. [Fixing some upstream bugs](#)
6. [Creating Debian specific files](#)
7. [Understanding the rules file](#)
8. [Building the package](#)
9. [Final words](#)
10. [Full files](#)

Introduction

This document is written for people who want to understand the insides of debian packaging. It is not assumed that the reader knows how to package a program, but it would be much better if they did.

This is not supposed to be the best document to learn to make packages from zero. If that is your case, I suggest you to read "Debian New Maintainers' Guide" at  <http://www.debian.org/doc/maint-guide/>

Thanks to Helen Faulkner and Dafydd Harries for their help, their support and for helping me with this document, and Manoj Srivastava for his interest in helping me to learn and for his code, which was an important reference for me.

Special thanks go to Debian-Women and  [ChicasLinux](#) for their support.

Note: This document shall not be understood as "here's an easy/good way to maintain your packages", but as "here is how the insides of debian package building works". I don't know if it can be useful for anyone. If that were the case, I would be glad.

Downloading and unpacking the source

We're going to build a Debian package from scratch without using debhelper, yada or anything like that, just with the help of "dpkg" and "dpkg-dev" packages.

The program I've selected for this task is called "roaddemo". It is available at <http://rhk.dataslab.com/roaddemo/> and is distributed under a GNU GPL License. It depends on OpenGL and SDL and is built using the standard "./configure && make && make install" procedure (with a minor trick, as "make install" doesn't seem to work properly as it is, but it will do anyway for the tutorial).

The first step is to make a new directory in which we will work, and to download the program:

```
$ mkdir roaddemo
$ cd roaddemo
$ wget -t 0 -c http://rhk.dataslab.com/roaddemo/roaddemo-1.0.1.tar.gz
```

We'll expand the tar.gz archive and rename it to the proper .orig.tar.gz file name that will be needed in Debian's building system (<package>_<version>.orig.tar.gz):

```
$ tar xvfz roaddemo-1.0.1.tar.gz
$ mv roaddemo-1.0.1.tar.gz roaddemo_1.0.1.orig.tar.gz
$ cd roaddemo-1.0.1
```

The code expands itself in a directory called roaddemo-1.0.1, which is consistent with the <package>-<version> nomenclature. Otherwise we'd have to rename it properly using "mv" command.

Testing the upstream building system

We'll see if it compiles properly before attempting to package it:

```
$ ./configure --prefix=/usr --mandir=\${prefix}/share/man --infodir=\${prefix}/share/info
```

We'll use those switches for a start. It might depend on how the autoconf stuff is done whether they might need some tweaking or not. If it is well done, typing "./configure --help" should show the available options.

Then we'll try to build the program using "make":

```
$ make
```

As that seems to work, let's try to install it using "make install".

We don't really want the program to be installed in our main directory tree, so we'll add the parameter "DESTDIR" to tell make where the program should be installed. We'll make a temporary directory for that:

```
$ mkdir tmp
```

We'll use the "pwd/tmp" trick to give the program the absolute directory instead of a relative one.

```
$ make install DESTDIR=`pwd`/tmp
```

This command seems to give an error, but normally it should work perfectly, so don't worry about this. In this case, the problem is solved by adding a new parameter telling "make" what "LIBTOOL" is, which should be defined inside it anyway. This is a bug in the upstream source.

```
$ make install DESTDIR=`pwd`/tmp LIBTOOL=libtool
```

Now we have the program we wanted to install inside "tmp/".

It is important that you have a look at the files the building system has installed. If you are working from the console, there is a command called "tree" which is very useful for this sort of thing. The Debian package is called "tree".

It is not uncommon to have some kind of problem in the building process. Sometimes "autoconf" or "automake" files are not done as they should, sometimes the parameters to "./configure" are not standard and you'll have to find them out. Sometimes only a conventional "Makefile" is available and you have no "./configure" stuff. In the latter case, you might need to fix it so that the "DESTDIR" parameter works. There is no global solution for every problem, so that's the creative part of it.

Let's clean everything we have created so far:

```
$ make distclean  
$ rm -rf tmp
```

Fixing some upstream bugs

Now, if the program was well done, you shouldn't need to do what I'm doing now, but as it is quite a common mistake, I'll solve it here just in case. The program seems to open a file called "road.bmp", which is not installed by "make install", so we'll need to install it by hand.

Inside the code, "road.bmp" is open without an absolute address, so when the program is run, it'll try to load it from the current working directory, whatever that is. According to Debian Policy, the file should go somewhere under "/usr/share/<package name>", so we'll put it right in "/usr/share/roaddemo/road.bmp".

We'll have to modify the code. First we try to find where in the source code that file is used:

```
$ grep road.bmp *.h *.c *.cpp *.cc *.hpp
```

That command gives us the following result:

```
roaddemo.cc:    if (load_texture("road.bmp", &road_tex_id) < 0)
```

Great, that's what we need to modify. Change "road.bmp" in that file to "/usr/share/roaddemo/road.bmp" and don't touch anything else.

This sort of thing is best fixed by sending a patch upstream that lets you define paths at compile time, or at least notifying the upstream author.

You usually need to modify something in the upstream code when developing a package, so be aware of it. What is uncommon is the program that goes smoothly as is. You will likely have to find and fix similar problems yourself.

Even though you can modify anything you need, don't change the files more than necessary. That means, don't change the indentation, carriage return styles or whatever is not absolutely needed. When the developing of the package ends, you are going to get a .diff.gz file with the changes between the original files and yours, and modifying those kind of things makes it unnecessarily big, and increases the chances that you will not be able to apply them to newer releases of the upstreamer.

Creating Debian specific files

Now let's go to the package building part. We'll have to create "./debian" directory and, at least, the following files: "debian/control", "debian/copyright", "debian/changelog" and "debian/rules".

```
$ mkdir debian
```

According to Debian's Policy, man pages are required for every binary in your package. It would also be nice to add an icon and a "debian/menu" file with the data needed to add the program to the menu, but that are not strictly needed. As this is not a tutorial about man pages, I'll not cover that issue here.

Control file

"debian/control" should have an structure similar to this:

```
Source: <package name>
Section: <section>
Priority: optional
Maintainer: <maintainer>
Build-Depends: <build dependencies>
Standards-Version: 3.6.1

Package: <package name>
Architecture: any
Depends: ${shlibs:Depends}
Description: <insert up to 60 chars description>
    <insert long description, indented with spaces>
```

Copyright file

Our "debian/copyright" file, as it is a program distributed under the GNU GPL License:

```
This package was debianized by <maintainer> on
<date and time>.
```

```
It was downloaded from <upstream URI>
```

```
Upstream Author: <upstream author>
```

```
License:
```

```
This package is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; version 2 dated June, 1991.
```

This package is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this package; if not, write to the Free Software Foundation, Inc., 59 Temple Place - Suite 330, Boston, MA 02111-1307, USA.

On Debian GNU/Linux systems, the complete text of the GNU General Public License can be found in ``/usr/share/common-licenses/GPL'`.

Changelog file

Now for "debian/changelog":

```
<package name> (<package version>-<release>) unstable; urgency=low

* Initial Release.

-- <maintainer> <date and time>
```

I'm not going to explain here how this files are done. Have a look at the "Debian New Maintainers' Guide" ( <http://www.debian.org/doc/maint-guide/>).

Rules file

Now for the "debian/rules" part, where the rules to build the package are defined.

"debian/rules" is in fact a "Makefile", and is run by `"/usr/bin/make"`. You can go do "info make", "man make" or have a look at  <http://www.gnu.org/software/make/manual/make.html> if you need more information on "make" building system.

"debian/rules" can be called with diferent parameters: "build", "clean", "binary", so we'll have to implement all of them.

Understanding the rules file

Initial definitions

First of all, we'll add a line so that the shell knows it is a "make" file in case it is directly executed:

```
#!/usr/bin/make -f
```

Now a useful definition of the package's name for simplifying "debian/rules" management:

```
# Name of the package
package=roaddemo
```

At some point of the execution, we'll need to make sure we're in the appropriate directory, and that we're running the script as "root" (or with the command "fakeroot"), so we'll define a couple of functions:

```
# Function to check if we're in the correct dir
define checkdir
    @test -f debian/rules -a -f roaddemo.cc || \
    (echo Not in correct source directory; exit 1)
endef
```

```
# Function to check if we're root
define checkroot
    @test $$ (id -u) = 0 || (echo need root priviledges; exit 1)
endef
```

"checkdir" checks that a "debian/rules" file exists, and also a file called "roaddemo.cc" in the current directory. "checkroot" checks that we're "root". These functions were taken 'as is' from Manoj Srivastava's building system and can be used under the GNU GPL License.

We'll then define our working directories:

```
# Top directory of the source code (thanks Manoj)
SRCTOP    := $(shell if [ "$$PWD" != "" ]; then echo $$PWD; else pwd)
# Destination directory where files will be installed
DESTDIR    = $(SRCTOP)/debian/$(package)
```


"SRCTOP" is the top directory of our source code, and "DESTDIR" is where the package will be temporarily installed, that is, "debian/roaddemo/" directory.

We'll define more directories to make our life simpler later:

```
# Definition of directories
BIN_DIR = $(DESTDIR)/usr/bin
GAMES_DIR = $(DESTDIR)/usr/games
SHARE_DIR = $(DESTDIR)/usr/share/roaddemo
DOCS_DIR = $(DESTDIR)/usr/share/doc/roaddemo
MAN_DIR = $(DESTDIR)/usr/share/man/man1
MAN_GAMES_DIR = $(DESTDIR)/usr/share/man/man6
MENU_DIR = $(DESTDIR)/usr/lib/menu
PIXMAPS_DIR = $(DESTDIR)/usr/share/pixmaps
```

"BIN_DIR" and "GAMES_DIR" is where binary files for conventional programs and games should respectively go. "SHARE_DIR" is where the platform independent files for the package should be installed. "DOCS_DIR" is for documentation, "MAN_DIR" and "MAN_GAMES_DIR" are for man pages, "MENU_DIR" is where we would put the menu management files and "PIXMAPS_DIR" where icons should go.

Rules

Now for the implementation of the commands. You can look at <http://www.debian.org/doc/debian-policy/ch-source.html#s-debianrules> to see what rules need to be implemented:

```
# Stamp Rules
```

```
configure-stamp:
```

```
    $(checkdir)
```

```
    ./configure --host=$(DEB_HOST_GNU_TYPE) --build=$(DEB_BUILD_GNU)
```

```
    touch configure-stamp
```

```
build-stamp: configure-stamp
```

```
    $(checkdir)
```

```
    -rm -f build-stamp
```

```
    $(MAKE)
```

```
    touch build-stamp
```

```
# Debian rules
```

```
build: build-stamp
```

```
clean: configure-stamp
```

```
$(checkdir)
-rm -f *-stamp
$(MAKE) distclean
-rm -rf debian/$(package)
-rm -f debian/files
-rm -f debian/substvars
```

```
binary-indep: build
```

```
# Definitions for install
```

```
INST_OWN = -o root -g root
MAKE_DIR = install -p -d $(INST_OWN) -m 755
INST_FILE = install -c $(INST_OWN) -m 644
INST_PROG = install -c $(INST_OWN) -m 755 -s
INST_SCRIPT = install -c $(INST_OWN) -m 755
```

```
binary-arch: build
```

```
$(checkdir)
$(checkroot)
```

```
# Install Program
```

```
$(MAKE) install DESTDIR=$(DESTDIR) LIBTOOL=libtool
```

```
# Install Program Resources
```

```
$(MAKE_DIR) $(SHARE_DIR)
$(INST_FILE) road.bmp $(SHARE_DIR)
```

```
$(MAKE_DIR) $(DESTDIR)/DEBIAN
```

```
# Install Docs
```

```
$(MAKE_DIR) $(DOCS_DIR)
$(INST_FILE) debian/copyright $(DOCS_DIR)/copyright
$(INST_FILE) debian/changelog $(DOCS_DIR)/changelog.Debian
$(INST_FILE) README $(DOCS_DIR)/README
```

```
# Install Manpages
```

```
$(MAKE_DIR) $(MAN_GAMES_DIR)
```

```

#$(INST_FILE) roaddemo.6 $(MAN_GAMES_DIR)

# Install Menu and Icon
#$(MAKE_DIR) $(MENU_DIR)
#$(INST_FILE) debian/menu $(MENU_DIR)/roaddemo
#$(MAKE_DIR) $(PIXMAPS_DIR)
#$(INST_FILE) debian/roaddemo.xpm $(PIXMAPS_DIR)

# Install Package Management Scripts
#$(INST_SCRIPT) debian/postinst $(DESTDIR)/DEBIAN
#$(INST_SCRIPT) debian/postrm $(DESTDIR)/DEBIAN

# Compress Docs (thanks Helen)
gzip -9 $(DOCS_DIR)/changelog.Debian
#gzip -9 $(MAN_GAMES_DIR)/roaddemo.6

# Strip the symbols from the executable (thanks Helen)
strip -R .comment $(BIN_DIR)/roaddemo

# Work out the shared library dependancies (thanks Helen)
dpkg-shlibdeps $(package)

# Generate the control file (thanks Helen)
dpkg-gencontrol -isp -P$(DESTDIR)

# Make DEBIAN/md5sums (thanks Helen)
cd $(DESTDIR) && find . -type f ! -regex '.*DEBIAN/.*' -printf

# Create the .deb package (thanks Helen)
dpkg-deb -b $(DESTDIR) ../

```

Finally, we end with a fairly generic part:

```

# Below here is fairly generic really

binary: binary-indep binary-arch

.PHONY: binary binary-arch binary-indep clean build

```

Phony rules are any rules which don't produce a file with the same name as the rule -- i.e. if you type "make foo", and it doesn't create a file/directory "foo", then it's a phony rule (thanks, daf).

Explanation

Now for the explanation of each part:

"configure-stamp" checks that we're in the correct directory and does the configuration stuff of the package.

The purpose of the *-stamp files is to avoid repeating things. When "make" is deciding what to do, it looks at the timestamps of the files in question. It is obvious that this doesn't work when the rules in question don't produce files. If you run `./debian/rules build`, and `./debian/rules binary` afterwards, "make" doesn't know the second time that you already did the first one. If, instead, you create a "build-stamp" rule, and make "build" depend on that, and "touch" the "build-stamp" when you are finished building, then "make" has a way of knowing that the build is finished (thanks again, daf).

"build" depends on "configure-stamp" being executed first. If it wasn't, it is automatically called before starting to build ("info make" for more information). After that, the building part is done.

"clean" cleans the directories from previous builds. As it uses "make distclean" for that, it depends on "configure-stamp" being called before it. After making "make distclean", we still have to clean some of the files the package creation systems temporarily makes inside `./debian` directory, that is, the whole `debian/roaddemo/` temporary installation tree, and the files `debian/files` and `debian/substvars`.

"binary" makes, installs and builds all the packages. It depends on "binary-indep" and "binary-arch".

"binary-indep" makes, installs and builds the platform independent packages, such as perl or python scripts, which do not need to be compiled for every platform. We do not need it for anything in this case.

"binary-arch" makes, installs and builds the platform dependent packages, such as the compiled programs, as ours. In first place, we check that we're in the appropriate directory and that we are "root" (or using "fakeroot"). Then we call "make install" for installing the program. As the installation script doesn't handle the file `road.bmp`, we move it by hand into its destination directory: `/usr/share/roaddemo/`. Remember that we modified the source code to get it from there.

The control part of the package needs to go inside a `debian/roaddemo/DEBIAN` directory, so we make it, then we copy the doc files into `/usr/share/doc/roaddemo`,

including "debian/copyright" and "debian/changelog" (the latter renamed to "changelog.Debian", just in case there's a "changelog" file from upstream author).

In case we have man pages, we install them. We also install menu scripts and icons if we have them. A note on icons: there's no problem in adding a new .xpm icon file, as it is just a text format. Debian packaging system does not support adding new binary files, such as .bmp, .png or .ico for example. If you needed to do so, you'd have to use "uuencode" and "uudecode" accordingly.

Then we add installation/removal scripts. In case we are adding a menu script, we need at least the necessary commands in "postinst" and "postrm" to update the menus afterwards. In our program, if you are not adding menu items, you do not need them.

Now the final part. We compress "changelog.Debian" as well as the man pages (if we had any), strip the debugging symbols from binary executables (with "strip"), work out the shared library dependencies (with "dpkg-shlibdeps") and generate the control file (with "dpkg-gencontrol"). After that we create a file "debian/roaddemo/DEBIAN/md5sums" with the "md5sum" of every file in the package (for checking afterwards), and finally we create the .deb package in "../" directory.

Don't forget that "debian/rules" is an executable, so remember to do:

```
$ chmod +x debian/rules
```

Adding a menu

If you want your program to be automatically added to the Debian menu system, you'll need to add a "debian/menu" file, which should be something like:

```
?package(roaddemo):needs="x11" section="Games/Toy" \
    title="Road Demo" command="roaddemo" \
    icon="/usr/share/pixmaps/roaddemo.xpm"
```

Or, if you don't have an icon:

```
?package(roaddemo):needs="x11" section="Games/Toy" \
    title="Road Demo" command="roaddemo"
```

If you add a menu, take into account that you also have to add the necessary command lines to update the menu system in the package's management scripts "debian/postinst" and "debian/postrm":

If you add a menu, be careful to also add a "debian/postinst" and a "debian/postrm" files:

"debian/postinst" file:

```
#!/bin/sh
set -e
if [ "$1" = "configure" ] && [ -x /usr/bin/update-menus ]; then update
```

"debian/postrm" file:

```
#!/bin/sh
set -e
if [ "$1" = "configure" ] && [ -x /usr/bin/update-menus ]; then update
```

You can read additional info about menu system in `"/usr/share/doc/menu/html/index.html"` inside your Debian system. You also can find there the list of possible sections you can use.

Building the package

Now we're done, lets check it:

We clean the files resulting from previous tries:

```
$ debian/rules clean
```

And we make the package:

```
$ debuild
```

If everything goes OK, we should have new files in `"../"` directory:

```
$ ls ..
roaddemo-1.0.1                roaddemo_1.0.1-1_i386.changes
roaddemo_1.0.1-1.diff.gz      roaddemo_1.0.1-1_i386.deb
roaddemo_1.0.1-1.dsc          roaddemo_1.0.1.orig.tar.gz
roaddemo_1.0.1-1_i386.build
```

Now you should check if there's something strange in the package using "lintian":

```
$ lintian ../roaddemo_1.0.1-1_i386.deb
```

Final words

Even though this started as a personal exercise to learn the insights of the Debian packaging building system, it has become a full tutorial that might be useful for other people. I couldn't have done that without the help of Debian-Women and Debian-Mentors. The same goes to Helen and Dafydd, I wouldn't have done this without your help. I don't mean just writing this document, but, most important for me, learning and understanding it. Thanks.

Full files

debian/control

-- CUT HERE --

```
Source: roaddemo
Section: games
Priority: optional
Maintainer: Miriam Ruiz <little_miry@yahoo.es>
Build-Depends: libsdl1.2-dev, libgl-dev
Standards-Version: 3.6.1

Package: roaddemo
Architecture: any
Depends: ${shlibs:Depends}
Description: Simple demo using opengl and SDL
    Bezier-road demo ported from glut to SDL.
.
    Use the right mouse button to select one of the
    endpoints or control points, then click and drag
    with the left mouse button to move the point.
```

-- CUT HERE --

debian/copyright

-- CUT HERE --

This package was debianized by Miriam Ruiz <little_miry@yahoo.es> on
Sat, 8 Jan 2005 20:09:28 +0000.

It was downloaded from <http://www.newimage.com/~rhk/roaddemo/>

Upstream Author: Ray Kelm <rhk@newimage.com>

License:

This package is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; version 2 dated June, 1991.

This package is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this package; if not, write to the Free Software Foundation, Inc., 59 Temple Place - Suite 330, Boston, MA 02111-1307, USA.

On Debian GNU/Linux systems, the complete text of the GNU General Public License can be found in `/usr/share/common-licenses/GPL'.

-- CUT HERE --

debian/changelog

-- CUT HERE --

roaddemo (1.0.1-1) unstable; urgency=low

* Initial Release.

-- Miriam Ruiz <little_miry@yahoo.es> Sat, 8 Jan 2005 20:09:28 +0000

-- CUT HERE --

-- CUT HERE --

```
#!/usr/bin/make -f

# Debian Rules by Miriam Ruiz <little_miry@yahoo.es>
#   January 2005
#
# Thanks for their help and for some of their code go to:
#   * Manoj Srivastava
#   * Helen Faulkner
#   * Dafydd Harries
#   * Gregory Pomerantz

#####

##
## This program is free software; you can redistribute it and/or modify
## it under the terms of the GNU General Public License as published by
## the Free Software Foundation; either version 2 of the License, or
## (at your option) any later version.
##
## This program is distributed in the hope that it will be useful,
## but WITHOUT ANY WARRANTY; without even the implied warranty of
## MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
## GNU General Public License for more details.
##
## You should have received a copy of the GNU General Public License
## along with this program; if not, write to the Free Software
## Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307
##
#####

# Name of the package
package=roaddemo

# Function to check if we're in the correct dir (thanks Manoj)
define checkdir
    @test -f debian/rules -a -f roaddemo.cc || \
        (echo Not in correct source directory; exit 1)
endef
```

```

# Function to check if we're root (thanks Manoj)
define checkroot
    @test $$ (id -u) = 0 || (echo need root priviledges; exit 1)
endef

# Top directory of the source code (thanks Manoj)
SRCTOP      := $(shell if [ "$$PWD" != "" ]; then echo $$PWD; else pwd;
# Destination directory where files will be installed
DESTDIR      = $(SRCTOP)/debian/$(package)

# Definition of directories
BIN_DIR = $(DESTDIR)/usr/bin
GAMES_DIR = $(DESTDIR)/usr/games
SHARE_DIR = $(DESTDIR)/usr/share/roaddemo
DOCS_DIR = $(DESTDIR)/usr/share/doc/roaddemo
MAN_DIR = $(DESTDIR)/usr/share/man/man1
MAN_GAMES_DIR = $(DESTDIR)/usr/share/man/man6
MENU_DIR = $(DESTDIR)/usr/lib/menu
PIXMAPS_DIR = $(DESTDIR)/usr/share/pixmaps

# Stamp Rules

configure-stamp:
    $(checkdir)
    ./configure --host=$(DEB_HOST_GNU_TYPE) --build=$(DEB_BUILD_GNU
    touch configure-stamp

build-stamp: configure-stamp
    $(checkdir)
    -rm -f build-stamp
    $(MAKE)
    touch build-stamp

# Debian rules

build: build-stamp

clean: configure-stamp
    $(checkdir)
    -rm -f *-stamp
    $(MAKE) distclean

```

```
-rm -rf debian/$(package)
-rm -f debian/files
-rm -f debian/substvars
```

binary-indep: build

Definitions for install

```
INST_OWN = -o root -g root
MAKE_DIR = install -p -d $(INST_OWN) -m 755
INST_FILE = install -c    $(INST_OWN) -m 644
INST_PROG = install -c    $(INST_OWN) -m 755 -s
INST_SCRIPT = install -c  $(INST_OWN) -m 755
```

binary-arch: build

```
$(checkdir)
$(checkroot)
```

Install Program

```
$(MAKE) install DESTDIR=$(DESTDIR) LIBTOOL=libtool
```

Install Program Resources

```
$(MAKE_DIR) $(SHARE_DIR)
$(INST_FILE) road.bmp $(SHARE_DIR)
```

```
$(MAKE_DIR) $(DESTDIR)/DEBIAN
```

Install Docs

```
$(MAKE_DIR) $(DOCS_DIR)
$(INST_FILE) debian/copyright $(DOCS_DIR)/copyright
$(INST_FILE) debian/changelog $(DOCS_DIR)/changelog.Debian
$(INST_FILE) README $(DOCS_DIR)/README
```

Install Manpages

```
##$(MAKE_DIR) $(MAN_GAMES_DIR)
##$(INST_FILE) roaddemo.6 $(MAN_GAMES_DIR)
```

Install Menu and Icon

```
##$(MAKE_DIR) $(MENU_DIR)
##$(INST_FILE) debian/menu $(MENU_DIR)/roaddemo
##$(MAKE_DIR) $(PIXMAPS_DIR)
##$(INST_FILE) debian/roaddemo.xpm $(PIXMAPS_DIR)
```

```

# Install Package Scripts

#$(INST_SCRIPT) debian/postinst $(DESTDIR)/DEBIAN
#$(INST_SCRIPT) debian/postrm $(DESTDIR)/DEBIAN


# Compress Docs (thanks Helen)
gzip -9 $(DOCS_DIR)/changelog.Debian
#gzip -9 $(MAN_GAMES_DIR)/roaddemo.6


# Strip the symbols from the executable (thanks Helen)
# TODO: the stripping part in binary-arch should honor the DEB_
strip -R .comment $(BIN_DIR)/roaddemo


# Work out the shared library dependancies (thanks Helen)
dpkg-shlibdeps $(package)


# Generate the control file (thanks Helen)
dpkg-gencontrol -isp -P$(DESTDIR)


# Make DEBIAN/md5sums (thanks Helen)
cd $(DESTDIR) && find . -type f ! -regex '.*DEBIAN/.*' -printf

# Create the .deb package (thanks Helen)
dpkg-deb -b $(DESTDIR) ../


# Below here is fairly generic really

binary: binary-indep binary-arch

.PHONY: binary binary-arch binary-indep clean build

```

-- CUT HERE --

[CategoryPackaging](#)

Courses2005/BuildingWithoutHelper ([last modified 2019-09-07 17:55:58](#))

- [Packaging](#)
- [HackingDependencies](#)

[Translation\(s\)](#): none

How to create a dummy package

You can read ?how to create a dummy package with [DebianPkg: equivs](#)

and also how to create a ?package with some files with [DebianPkg: equivs](#)

Hacking dependencies

Some window managers (like [GNOME](#)) comes with a lot of packages. But what do you do when you don't want a special package installed, but your system depends it?

Please note that this is a crude hack and if thoughtlessly used, it might possibly do damage to your packaging system. And please note as well that using it is not the recommended way of dealing with broken dependencies. Better file a bug report instead.

Anyway, if you still want to keep reading, the answer is to make a dummmmy package. This text will give a small example on how to create a dummy package for *gnome-games*. The idea is to give the dummy a version number so high that the real package will never reach it.

For this job you need [DebianPkg: equivs](#). If you haven't installed this package before, you should do so now.

Open up your favourite editor, name it something like *anti-gnome-games.equivs* and write the following lines:

```
Package: gnome-games
Version: 99:99
Maintainer: Your Name <mail@domain.com>
Architecture: all
Description: dummy gnome-games package
    A dummy package with a version number so high that the real gnome pac
will never reach it.
```

And then in your console:

```
equivs-build anti-gnome-games.equivs
```

You will then have a *gnome-games_99_all.deb* which you can install with *dpkg -i*.

The 99:99 version means  [epoch](#) 99, version 99 – if you don't need an intentionally-absurd version number, you should avoid the epoch and have a version like *1*.

Documentations

- [DebianMan: equivs-build\(1\)](#) - make a Debian package to register local software
- [DebianMan: equivs-control\(1\)](#) - create a configuration file for equivs-build

See Also

- [tasksel](#) - a user interface for installing tasks

[CategoryPackaging](#)

Packaging/HackingDependencies ([last modified 2023-12-28 05:36:30](#))

- [Diagrams](#)

Here are some Debian related diagrams and sketches, please add more. Graphs and other statistics should be added [elsewhere](#).

- Maintainer scripts  [1](#)  [2](#) ( [src](#))  [3](#) ( [src](#))
-  [Debian development process](#) ( [new src](#),  [old src](#)  [old Spanish src](#))
-  [Ubuntu & Debian development](#) ( [src](#))
-  [Debian package breakdown](#) ( [SVG](#),  [plain SVG](#),  [Portugese src](#))
-  [Debian build system](#)
- Debian  [buildd](#),  [wanna-build](#) systems
-  [Debian & neuroscience](#)
-  [Understanding Debian infographic](#) ( [source](#))
-  [Release timeline](#),  [another release timeline](#),  [General timeline](#)
- [Debian package cycle](#), also on  [Wikipedia commons](#)
- [Archive software workflow](#)
-  [Flow of autotools artefacts](#) ( [src](#))
-  [apt state files](#) (src  [1](#)  [2](#))
-  [maintenance problems & Debian](#)
-  [Relationships between apt, aptitude, dpkg etc](#)
-  [Debian package build tools](#)
-  [Package diagram](#)
-  [Suites diagram](#)
-  [Diagram of the NEW process](#)
-  [Diagram of the upload process](#)
-  [dgit design process](#)
-  [BTS system](#) (from [DebConf17](#)  [debbugs](#))  [talk](#)
-  [The Debian DNS universe](#)
- [Debian contributor pathways](#)
-  [Debian Fun Statement](#) infographic

See also:

- [Statistics and graphs](#)

[CategoryPackaging](#)

- [PackageConfigUpgrade](#)

[Translation\(s\)](#): none

This page contains a *proposal* for a way to handle upgrades where a user's data and the package maintainer data are merged with minimal interaction with user. The package *approx* will be used as an example.

Configuration upgrade by package

Contents

1. [Configuration upgrade by package](#)

1. [Status](#)

2. [Introduction](#)

3. [Upgrades with cme and Config::Model](#)

4. [Apply configuration upgrade using an existing model: lcdproc example](#)

1. [Current lcdproc situation](#)

2. [What will change for user](#)

3. [lcdproc package changes for automatic upgrades](#)

5. [If you want to apply config-model based upgrades to your favorite package](#)

6. [Approx example](#)

1. [configuration analysis](#)

2. [Upgrade scenario](#)

3. [Approx configuration model](#)

4. [Configuraion upgrade during package upgrade](#)

7. [cme commands to manage configuration upgrade](#)

8. [Working with Debconf](#)

9. [Links](#)

10. [Thanks](#)

Status

Automatic configuration merge is provided with lcdproc >= 0.5.6

Introduction

For a casual user (like you mother-in-law or your neighbor), the questions asked during package upgrades can be intimidating. Casual users barely know that some files exist outside of their home directory and are completely lost when looking at the `/etc` directory. Unfortunately, during package upgrade, user is faced with cryptic questions whether to keep his configuration, use the maintainers configuration or look at a *diff*. The casual user will seldom know what to do in this case.

This situation can be made worse if user's configuration was edited with a tool (e.g. cups configuration is edited through a web browser): the user has no knowledge of the syntax of the configuration file.

This proposal aims to provide a way for Debian package to minimize the number of questions raised to the user when upgrading packages. If questions are necessary, an interface will be provided to the user. Hopefully, manual editing of configuration files will not be necessary.

Upgrades with cme and Config::Model

 [cme](#) and  [Config-Model](#) provide a framework for editing and validating the content of any configuration file or data. With a configuration model (expressed in a data structure), Config-Model provides a user interface and a tool to validate configuration.

A typical configuration model contains the following specifications :

- The structure of the configuration model. This structure is a tree that can be flat (for simple files like `approx.conf`) or quite deep (for complex configuration like `xorg.conf`). The configuration tree nodes are instances of  [configuration class](#) and the tree leaves contain the configuration  [values](#).
- The  [constraints](#) of the leaf values (i.e. integer, enum type, min values, max values ...)
- the default values where the default value **must** be written in the configuration file.
- the *built-in* default value where the value need not be written in the configuration file. This default value is *built in* the application.
- Help text displayed with interactive configuration editor.

This notion of default value versus built-in default value is important to distinguish:

1. the values that were customized by the user (or sysadmin)
2. the value that are recommended by the package maintainer (and specified as `default` in the model)
3. the value that are provided upstream (specified as `upstream_default`)

When creating a model just for upgrades, a model does not need to feature:

- constraints. Because the configuration file to be upgraded is supposed to be correct
- help text, because the upgrade is done in non-interactive mode
- built-in default values because they are not used during upgrades

During a package upgrade we want to:

- propagate values from the old configuration file to new configuration file.
- discard obsolete data (if any)

- introduce new mandatory values (if any)
- migrate (or translate) data from old format to new format

So the idea is:

- Upgrade the packages files as usual (including upgrade of configuration model)
- In `postinst`, run [cme](#) to load old configuration file using new configuration model and write back the updated data in new configuration file (i.e. merged values from user's data and data from new configuration model).
- If the last step fails, inform the user to launch `cme` in interactive mode

The same principles can be applied to downgrade with some limitations (see below for more details)

Apply configuration upgrade using an existing model: lcdproc example

This is a real use case implemented for [lcdproc](#)

Current lcdproc situation

Up to version 0.5.5, `lcdproc` is shipped with several configuration files, including `/etc/LCDd.conf`. This file is modified upstream at every `lcdproc` release to bring configuration for new `lcdproc` drivers. On the other hand, this file is always customized to suit the specific hardware of the user's system. So upgrading a package will **always** lead to a conflict during upgrade. User will always be required to choose whether to use current version or upstream version.

[DebianPts: libconfig-model-lcdproc-perl](#) provides a [configuration model](#) for `lcdproc`. This [blog](#) explains how this model is generated from upstream `LCDd.conf`. But this is off topic for this page.

What will change for user

When `lcdproc` is upgraded to the new version featuring automatic upgrade, the following changes will be visible: ** `lcdproc` will depend on [libconfig-model-lcdproc-perl](#)* ** user will be asked *once* by [debconf](#) whether to use automatic configuration upgrades or not.* ** no further question will be asked (no [DebianMan: ucf](#) style questions).*

Note: the automatic upgrade currently applies only to `LCDd.conf`. The other configuration files of `lcdproc` are handled the usual way.

lcdproc package changes for automatic upgrades

`lcdproc` package was changed the following way:

- `dh` is now run with `--with cme_upgrade` option (see [debian/rules](#))
- `/etc/LCDd.conf` is no longer used by `lcdproc` or by `cme`. The upstream file is now delivered in `/usr/share/doc/lcdproc` as a reference file. See commit [1](#) and [2](#)
- A new file is added to `debian` directory to specify upgrade instructions: [debian/lcdproc.config-model](#)

This file contains the following lines:

- `cme-model-package: libconfig-model-lcdproc-perl`: specifies the debian package containing `lcdproc` configuration model
- `cme-model-version: 2.040` specifies the minimum version of the above package. These 2 parameters are used to construct a dependency which is added to `lcdproc` package.
- `cme-app-name: lcdproc` specifies the application name passed to [cme](#)
- `cme-command: fix` specifies the command used by `cme`. By default, `cme` will use `migrate` command during upgrade. Due to multi arch, `DriverPath` may contain library path without multi-arch triplet. Using `fix` will ensure that `DriverPath` is set to the correct multi-arch path.
- `cme-options: -force` is required to upgrade `LCDd.conf` from upstream. Upstream config contains configuration for driver which are removed from Debian. This configuration must also be removed from the file. Using `-force` will ensure that.
- `cme-purge: /etc/LCDd.conf*`: with this option, `dh_cme_upgrade` will insert a `postinst` script that will remove the specified file on package purge.

For more information, see [DebianMan: dh_cme_upgrade man page](#).

If you want to apply config-model based upgrades to your favorite package

`lcdproc` did start from an existing configuration model. Let's see what is required to create a minimal configuration model so that your package can also feature automatic configuration upgrade.

You will have to:

- Create a minimal model that describe:
 - items that need special treatment during upgrades
 - how to handle other parameters
- Find a [backend](#) that supports the syntax of the configuration file of your package. If none exists, ask on `config-model-users` at `lists.sourceforge.net`, we'll find a solution that won't require you to learn Perl.

Well, that was the theory. Let's experiment with Approx configuration file and work out the details.

Approx example

configuration analysis

From [DebianMan: approx.conf\(5\) man page](#), we find that `approx` has a very simple configuration. Approx will need one configuration class that will have:

- a bunch configuration parameters, either boolean, integer or string. All of them have default built in `approx`.
- a hash containing distribution names and their URL. These will be modeled with a [hash](#) of [uniline](#) values

The syntax of `approx.conf` is fairly simple but there's no Perl module available to read and write it. A dedicated parser/writer will be required. I.e. a Perl class that will inherit <http://search.cpan.org/dist/Config-Model/lib/Config/Model/Backend/Any.pm>. Well, you can cheat and look at the source of the actual [Approx backend](#).

Upgrade scenario

For the sake of the example, let's assume that `offline` parameter was named `nointernet` in previous versions of Approx. This is of course not true. Apologies to Eric Cooper for such a ludicrous idea.

So the upgrade model must feature:

- instructions to migrate `nointernet` content to `offline` parameter
- instructions to handle other leaf parameters (the one that will not be described in your minimal model)
- instructions to handle distributions as we must avoid collisions between distribution names and `approx` parameter names (the syntax of `approx.conf` was changed with version 2.9.0 for the same reason)

Approx configuration model

To create the Approx configuration model, you can use the GUI provided by [DebianPkg: libconfig-model-itself-perl](#).

In a empty directory, run `config-edit-model -model Approx` and fill the fields as required.

You will have to:

- Create the `Approx` configuration class

- handle the old hypothetical `nointernet` parameter:
 - create a `leaf` parameter with `value_type` set to `boolean`
 - set a `deprecated` status as this parameter is to be migrated
- handle the new `offline` parameter:
 - create a `leaf` parameter with `value_type` set to `boolean`
 - specify migration instruction where:
 - the old parameter is specified in a variable (`- nointernet`)
 - the formula specifies how is used the old value. In this case, it's a simple substitution.
- handle the distribution parameter (a hash of `unilines`)
- specify all mandatory parameters with their default values (fortunately, there's no such parameter for `Approx`)
- specify what to do with unknown parameters. The `accept` instruction will match any other parameter (with the `'.*'` regexp) and use them as uniline values

In the end you will get :

```
[
  {
    'name' => 'Approx',
    'element'
    => [
      nointernet => { type => 'leaf', value_type => 'boolean', status => 'deprecated' },
      offline    => { type => 'leaf', value_type => 'boolean',
                     migrate_from => {
                                   variables => { old => '- nointernet' },
                                   formula  => '$old',
                                   }
                     },
      'distributions', { type => 'hash', index_type => 'string' ,
                         cargo => { value_type => 'uniline', type => 'uniline' },
                         }
    ],
    'accept' => [
      '.*' => { type => 'leaf', value_type => 'uniline',
                }
    ],
  }
] ;
```

You can view the actual model on [🌐 Approx model on github](#). This model is more complex as all parameters and help text are provided.

⚠️ Any Debian specific requirement (e.g. default value) will have to be implemented in the model and not in the template configuration file provided to the user (e.g. not in `/etc/approx/approx.conf`)

Configuraion upgrade during package upgrade

This can use modification similar to the `lcdproc` example shown above.

cme commands to manage configuration upgrade

For reference, `cme` can also be used outside of package script to manually migrate a configuration file. These commands are not specific to Debian and can be applied to other distros or OS.

Let's start with a simple `approx.conf` file:

```
debian          http://ftp.debian.org/debian
$max_wait       12
```

In most cases, simply running `cme migrate` will be enough to upgrade a configuration files. Even if the configuration model is changed from package version N to version N+1, the upgrade will work most to the time. `cme` will exit on error if some data to upgrade are not recognized.

```
cme migrate approx
```

`cme` will:

- parse the old configuration file (or create one in case of first installation)
- validate all values (possibly discarding or migrating obsolete parameters) **FIXME**
add link to details
- add new values declared in N+1 model (if any)
- write back the configuration file (comments are preserved)

After upgrade, `approx.conf` has:

```
# This file was written by Config::Model with Approx model
# You may modify the content of this file. Configuration
# modification will be preserved.

$max_wait 12
```

If you want to keep a backup of the old configuration file, you can run `cme migrate approx -backup`. The old configuration will be in `approx.conf.old`

Working with Debconf

Warning: just a bunch of ideas, no working implementation

Main ideas:

- Run `config-edit -ui debconf -savein debconf` in pre-inst
- Run `config-edit -ui debconf -loadfrom debconf -save` in postinst
- Translate data from model into debconf template. Debconf template will be used by `C::M::Debconf::Wizard` and [Config::Model::WizardHelper](#) to scan configuration tree and ask question in case of error or missing mandatory values (or may be different level of scanning to match debconf's priority. See [Config::Model::DebConfUI](#) (still coding...))

Fortunately, the semantic of `Config::Model value_type` mostly matches debconf's type:

Config::Model	Debconf
uniline	string
integer	string
number	string
string	string (problem with multi-line ?)
boolean	boolean
enum	select
checklist	multiselect
help stored in model	note
<code>\$@ from eval{die ...}</code> 😊	error
title	not necessary

Links

- [Config::Model wiki](#)
- [Discussion on debian-devel and debian-perl](#)
- What is a [conffile](#) ?

Thanks

- To Damyan Ivanov (first review)

- To Jonas Smedegaard for the constructive exchange and the helper scripts and /var/lib/ storage ideas
 - The Gregor Hermann for sponsoring all the libconfig-model-* packages
-
-

[CategoryDeveloper](#) [CategoryPackaging](#)

PackageConfigUpgrade (last modified 2019-11-23 18:10:47)

- [DataPackages](#)

This pages tries to summarise the brainstorm started on -devel about handling huge data packages.

 <https://lists.debian.org/debian-devel/2007/06/msg00273.html>

Solution	Resource-hungry	Official	data in .deb format	Easy to apt	Local processing
Package as usual	yes	yes	yes	yes	no
Package data and provide processing scripts to other packages	yes, but less duplication	yes	yes	not fine-grained	automated
Dummy package	no	yes	no	yes	manual
Packaged cron script	no	yes	no	yes	automated
Wrapper	no	yes	yes	yes	automated
Fink	no	yes (.info files)	no	no, but fink can manage dependencies	automated
New branch separated from main	optional mirroring	yes	yes	yes	no
Unofficial repository	not for Debian	no	yes	needs a key?	no
dpkg --admindir=\$HOME/somedirectory	depends	depends	yes	no	no

Other tools to create Debian binary packages are cmake, checkinstall, alien and equivs.

Some other ideas

- [Projects/DebSrcData](#).
- Regular Debian package from a "dummy" orig source.  <https://lists.debian.org/debian-devel/2007/06/msg00298.html>
- Per user package management.  <https://lists.debian.org/debian-devel/2007/06/msg00371.html>
- Script to download/update, unpack and index public databases  <http://svn.debian.org/wsvn/debian-med/trunk/community/infrastructure/getData/?rev=0&sc=0>
-  data.debian.org
- For CDs / DVDs, move these data files to multi-arch  <https://lists.debian.org/debian-devel/2007/06/msg00317.html>

Biggest space consumers

- Games data,
- Scientific data,
- -dbg packages.

Further discussion threads

- Nexuiz was temporarily bigger than a CD: [🌐 debian-cd](#) [🌐 debian-devel](#) ~~Closed in nexuiz-data/2.5.2-3: #557218: nexuiz-data does not fit on a single CD: bug tracker~~ (each location has some non-crossposted mails)
 - Discussion on debian-devel: [🌐 https://lists.debian.org/msgid-search/87tzgm6yee.fsf@vorlon.ganneff.de](#)
 - Discussion two years later on the [DebianScience](#) list: [🌐 https://lists.debian.org/msgid-search/20100508140545.GA19587@meiner](#)
-

[CategoryDeveloper](#) [CategoryPackaging](#)

DataPackages (last modified 2019-11-25 12:29:09)

- [Teams](#)
- [DebianEmacsenTeam](#)

Debian Emacsen team

Contact

-  [debian-emacsen@lists.debian.org mailing list](mailto:debian-emacsen@lists.debian.org)
- #debian-emacs on irc.debian.org

Scope

- Emacs Lisp addon packages
- GNU Emacs backports

i.e. does not (yet) include any main archive Emacsen themselves.

Membership

Defined by our  [salsa group](#) (not (yet) by tracker.debian.org).

We typically grant full membership to non-DDs who

- are already in the Uploaders field for several team packages, at least some of which they prepared for NEW themselves; and
- know when to ask for review before pushing, especially when it comes to importing new upstream releases.

Addons packaging policy

- All projects must have a real upstream. This can be a release tarball, VCS, or webpage with a bare file-version.el. The important point is that Upstream-Contact must be in control of releases. This means that projects that only exist on "emacswiki.org" should not be part of Debian. If you find a package that has *Emacswiki* as the source, and doesn't have a new project upstream, please bring it to the attention of the team.
- All binary packages installing into site-lisp/elpa (in particular all packages using `dh_elpa`) should be named `elpa - foo` where `foo` is the ELPA package name.
 - + Note that this is often different to the upstream repository name. E.g. Binary package `elpa - f` is called 'f.el' by upstream.
- Source package names are upstream project names. Prefix 'emacs-' or suffix '-el' if and only if that name is too generic. As a special case, if the upstream project name

ends in '.el', replace this with '-el'. I.e. Do not add an `el`pa- prefix to the source package name.

- We use one git repository for each upstream package.
- We keep the upstream source in the packaging git repository.
- Versions should be based on those used by the upstream author, not e.g. MELPA
 - + Though if there is a discrepancy, try to get it resolved with upstream before uploading.
- The ELPA version declared in a package's headers should match its actual version in all but unusual cases; This is needed for proper dependency resolution, but many upstreams rely on MELPA infrastructure to generate correct headers.
 - + The `dh_elpa(1)` man page documents the most elegant method to generate this metadata at build time.
- Maintainer field should be
`Debian Emacsen team <debian-emacsen@lists.debian.org>`
- We generally expect to be able to work on packages as a [team](#)
- See [el](#)pa-hello for sample debian/* files snippets that use `dh_elpa` and follow policy.

`:core` ELPA packages

- ":core ELPA packages" is upstream's name for Emacs Lisp packages that are included in Emacs releases but which also see independent new releases to GNU ELPA, more frequently than Emacs as a whole gets released.
- For some of these packages, we package them twice: there is the copy installed when you install Emacs, and there is also an `el`pa- package. This is so that we can offer users newer releases than they would otherwise get in our stable releases, given how our release cycle and Emacs's do not line up well.
- This scheme only works if **addon packages are never allowed to fall behind the version shipped with Emacs**. That is, after Emacs is updated in Debian, but before our freeze begins, **all :core ELPA packages must be updated to at least the version shipped with Emacs**.
- Otherwise, the version shipped with Emacs can take precedence over the version shipped separately, which is bad for users. See also Debian bug#1035757.

Meetings

- [Teams/DebianEmacsenTeam/Meetings/DC15](#)
- [Teams/DebianEmacsenTeam/Meetings/DC16](#)
- [Teams/DebianEmacsenTeam/Meetings/DC17](#)

Subpages

- [Tips](#)

Useful links

-  [Old team wiki content](#) (not yet fully migrated to here)
-  [Tracker overview of all team packages](#)

Teams/DebianEmacsenTeam (last modified 2024-08-25 22:49:06)

- [Teams](#)
- [DebianHaskellGroup](#)

 [Haskell](#) is a pure functional language.

The Debian Haskell Group (DHG for friends!) is the team committed to packaging and maintaining a working and up-to-date Haskell environment and library in Debian; this wiki pages is mostly of interested to those who contribute to the DHG.

Team contacts

- **General discussion mailing list:**  <https://lists.debian.org/debian-haskell/>
- **Maintainer and commit lists:**  <https://alioth-lists.debian.net/cgi-bin/mailman/listinfo/pkg-haskell-maintainers> and  <https://alioth-lists.debian.net/cgi-bin/mailman/listinfo/pkg-haskell-commits>
- **IRC:**  [#debian-haskell](#)

Note that the maintainer address list ( pkg-haskell-maintainers@lists.alioth.debian.org) tends to be very noise due to automatic mail (e.g. from the Debian archive), so discussions are easier on  debian-haskell@lists.debian.org.

Team infrastructure

-  [Our git repositories](#) -- to contribute to the DHG, register on Salsa and then use this link to ask for project membership; of course, you also need to subscribe to the mailing lists
-  [The Package Plan](#) -- our tool to verify the consistency of our package selection. Update this before uploading packages! See the  [README](#) for documentation.
- QA pages (good to check for problems)
 -  [binNMUs to be scheduled](#) -- triage Failed builds, let the Release Team know if there are NMUs to schedule!
 -  [Build status of our package](#) with unsupported arches hidden, compact mode.
 -  [permanent Haskell transition tracker](#)
 - Status of reproducibility of our packages, for  [unstable](#) and  [testing](#).
-  [Our jenkins jobs:](#)
 - jessie:   https://jenkins.debian.net/view/chroot-installation/job/chroot-installation_jessie_install_haskell/

- sid:  build  disabled
- stretch → buster upgrade: 
https://jenkins.debian.net/job/chroot-installation_stretch_install_haskell_upgrade_to_buster/
- package-plan:  build  passing
- Tables, graphs and statistics
 -  [Comparison of Version numbers in Debian and on Hackage](#)
 -  [Dependency tree of our packages](#) -- updated only manually; colors indicate installability
 -  [Graphs](#) of our build usage
 - Information about ?GHC on different architectures.
- Related, external resources
 -  [Ubuntu wiki page](#)
 - Git repositories for all Hackage packages, for a quick review of new versions:
 <http://hdiff.luite.com/>

Haskell packaging documentation

- [/CollabMaint/GettingStarted](#) gives basic information on how we manage our DHG package collection.
- [/CollabMaint/Processes](#) -- information on our group-internal processes

TODO list

A raw list of things to do, hoping that someday someone will start working on them...

- Fix policy, which is very old
 - Adapt it to the current best-practises and haskell-devscripts workflow
 - Description guidelines, see for example 
<http://lists.debian.org/4C2FA27F.50303@poisson.phc.unipi.it>
- Fix wiki pages, which are very old too (some examples are completely deprecated)
- Create some Lintian test specific to Haskell packages
- Libraries worth packaging: ?/PackageTODO
- Document  <http://anonscm.debian.org/darcs/pkg-haskell/tools/all-packages/> and related processes.

[CategoryPackaging](#)

- [Java](#)
- [Packaging](#)

[Translation\(s\)](#): English - [Español](#)

Contents

1. Introduction	
2. Contributing to Java packaging	
3. Java Policy	
4. Javahelper	
5. Java compilation 101	
6. When a project has no build system	
7. Build systems	
1. Ant	
2. Maven	
3. Gradle	
8. autopkgtest	
9. Frequently asked questions and solutions	

Introduction

In addition to the normal debhelper packaging tools there are some additional tools specifically for dealing with Java packages.

- [DebPkg: javahelper](#) for packages without sane build systems or in combination with build systems to modify and make them Debian compliant,
-  [maven-repo-helper](#) for adding Maven artifacts to a non-Maven package,
-  [maven-debian-helper](#) as the primary helper tool for any Maven project and
-  [gradle-debian-helper](#) for Gradle-based projects.

This page will inform you about common Java packaging workflows in Debian and point you to real-life examples which can be used as a starting point for your own packages.

Contributing to Java packaging

We welcome any help with packaging new applications or libraries and keeping existing ones and their reverse-dependencies up-to-date. These days the easiest way to submit a contribution is via a pull request on  <https://salsa.debian.org/java-team>. Just

1. make your changes,
2. document everything in debian/changelog,
3. set it to UNRELEASED,

4. test that reverse-dependencies continue to work and then
5. make a pull request.

Alternatively you can follow the instructions from mentors.debian.net.

 <https://mentors.debian.net/intro-maintainers>

which is useful if you also want to learn the general process of uploading a package to Debian. The only change is that you don't have to ask for sponsorship on the mentors mailing list, just use  debian-java@lists.debian.org for that.

Java Policy

Java packages in Debian abide like all other packages by the  [Debian Policy](#). Java-specific Policy elements are documented in the  [Debian policy for Java](#).

Javahelper

Javahelper is Debian's helper tool to perform Java packaging tasks like

- installing jar files into the canonical `/usr/share/java` directory,
- building packages from source without a sane build system,
- manipulating MANIFEST files or
- setting the CLASSPATH.

It can also help with avoiding patches by creating automatic symlinks to system libraries.

Javahelper is documented in the javahelper package in `/usr/share/doc/javahelper/tutorial.txt.gz`, also  [replicated here](#).

In addition to the tutorial, a talk was given at [DebConf10](#) covering the use of javahelper. The talk is available as:

- Video ( [high res](#);  [low res](#))

The idea behind javahelper is that most Java packages should be trivial. The integration with debhelper and the dh sequencer means that you shouldn't need anything in `debian/rules`, just a few files under the `debian` directory.

Using javahelper and DH7 ideally your rules file will look like this:

```
export JAVA_HOME=/usr/lib/jvm/default-java
```

%:

```
dh $@ --with javahelper
```

All of the work is done by javahelper working from other files under debian/.

Java compilation 101

 [This section](#) explains the basic terms and methods to compile Java source files into  [bytecode](#) and the creation of jar files.

When a project has no build system

There are some upstream projects that don't provide a proper build system. In some cases it is useful to download source packages from Maven Central which will come without a build system, just source code. For all those reasons Javahelper is the perfect tool to compile the code and create a proper Debian package. 

[NoBuildSystem examples](#)

Build systems

There are a few build systems in use by Java programs.

Ant

If your package uses Ant then debhelper and dh sequencer should automatically detect the build.xml and call things appropriately.  [Ant examples](#)

Maven

Use [maven-debian-helper](#) for standard Maven projects, use [maven-repo-helper](#) in combination with other build systems to provide Maven artifacts and maven-ant-helper to fallback the build to Ant - usefull mostly for building a library which is part of the Build-Depends of Maven itself.  [Maven examples](#)

Gradle

Use [DebianPackage: gradle-debian-helper](#). A Gradle project often requires a patch to the build scripts anyway because the build logic varies from project to project. The gradle-debian-helper is currently in an early stage and mostly helps with the substitution of Maven coordinates, so that Gradle will use Debian's local `/usr/share/maven-repo` instead of downloading build-dependencies from the internet. See also  [Gradle examples](#)

autopkgtest

For [DebianPackage: autopkgtest](#) is HELP and or information needed ...

Example:  <https://salsa.debian.org/java-team/apk-parser/-/blob/debian/sid/debian/tests/control>

Above is bad example, which discussed by:

-  <https://lists.debian.org/debian-java/2020/12/msg00048.html>
- ~~Closed in apk parser/2.6.10+ds-4: #978684: autopkgtest should test the installed package: https://bugs.debian.org/978684~~

Frequently asked questions and solutions

The [Java Packaging FAQ](#) gives answers to common Java packaging questions, explains some common error messages and provides solutions to fix them.

[CategoryPackaging](#)

Java/Packaging (last modified 2020-12-31 15:52:07)

- [Java](#)
- [JavaGit](#)

Contents

1. [Guidelines for Packages Maintained on salsa.debian.org/java-team](#)
2. [Converting a Subversion repository to Git](#)
3. [Using git-buildpackage](#)
 1. [Create the git repository on git.debian.org](#)
 2. [Import upstream](#)
 3. [Typical git workflow](#)
4. [Miscellaneous](#)
 1. [Obtaining the master repository](#)

Guidelines for Packages Maintained on salsa.debian.org/java-team

This chapter adds additional guidelines for packages maintained in Salsa's Git repositories. Please read the page [Java/JavaVcs](#) first if you have not done yet. Most of the guidelines are inspired by git-buildpackage.

Git repositories can be created under the java-team group on Salsa. Using the  <https://salsa.debian.org/java-team/pkg-java-scripts/blob/master/setup-salsa-repository> script is RECOMMENDED for creating a new repository because it uses the correct options and sets up the notifications and hooks.

The upstream code SHOULD be imported. Every new upstream version SHOULD be tagged as upstream/VERSION after cleaning it up. Using pristine-tar is RECOMMENDED. The pristine-tar branch SHOULD be called *pristine-tar*. Having an explicit *upstream* branch is OPTIONAL but it SHOULD be called *upstream* if it exists.

Every repository should have a *master* branch with the upstream code, the Debian code and patches. The newest version of README.source MUST be available in the master branch. The patches SHOULD be unapplied before committing changes.

You MAY push upstream tags to Alioth if upstream uses git. But you SHOULD always create an explicit tag upstream/VERSION even if it is identical to some upstream tag. You MAY skip adding a get-orig-source target in debian/rules if you create the upstream/VERSION tag directly in git without any intermediate tarball. You MUST document the workflow for upstream updates in README.source in such cases. The tarball used in the successful upload SHOULD be committed with pristine-tar.

Every successful upload MUST be tagged as debian/VERSION.

Branches and tag schemas not mentioned here MUST be documented in README.source. Any required tools except git-buildpackage and pristine-tar MUST be documented in README.source.

QUESTION: do we need guidelines about .gitignore, .gitattributes, and .gbp.conf?

Converting a Subversion repository to Git

Clone the repository containing the migration script at <https://anonscm.debian.org/cgit/pkg-java/pkg-java-svn2git.git> and then run:

```
./migrate-svn-repo-to-git <package>
```

Using git-buildpackage

This chapter gives some instruction about using git-buildpackage. The full manual can be found at <https://honk.sigxcpu.org/piki/projects/git-buildpackage/>.

Create the git repository on git.debian.org

```
<login>@vasks:~$ cd /git/pkg-java
<login>@vasks:/git/pkg-java$ ./setup-repository <package>
Initialized empty shared Git repository in /srv/git.debian.org/git/pkg
Repo <package>.git has been created
Commit messages for <package>.git have been forwarded to pkg-java-comm:
```

Import upstream

You **might** have to create the *upstream* branch if you converted from svn: See the commands below *Alternatively, if you are only importing source from original tarballs...* in <http://honk.sigxcpu.org/projects/git-buildpackage/manual-html/gbp.import.html#GBP.IMPORT.CONVERT>.

```
git-import-orig --filter "*.jar" --filter "*.war" \
    --filter apache-solr-1.4.0/docs/api \
    --filter apache-solr-1.4.0/dist --filter-pristine-tar \
    --pristine-tar -u1.4.0 ../apache-solr-1.4.0.tgz
```

Note: Make sure you use the --filter-pristine-tar option if you use --filter options (otherwise your *pristine-tar* branch will be broken)!

This command

- filters out (repackages) unwanted files from the upstream tarball,
- imports the (filtered) upstream tarball into the git branch "upstream",
- tags the import with the version number,
- merges the updated upstream into the packaging branch "master"
- and creates a pristine-tar import to restore the (filtered) upstream tarball.

The --filter options are translated directly to tar --exclude options. See man tar for details.

--filter-pristine-tar makes sure that the same filtering is also applied to the pristine tarball. Therefor the tarball saved with prisitine-tar isn't pristine anymore, but rather repackaged.

So the shellscrip debian/orig-tar.sh from the solr package which manually removes all *.jar and repackages the upstream tarball isn't necessary anymore.

You may want to make two imports with git-import-orig for each new upstream tarball. The first import is without any filtering and saves the original tarball as it for later reference. The second import adds a "+dfsg" suffix to the tarball, filters all jar files and is the one actually uploaded to debian.

You may need to manually remove .git directories from the orig.tar.gz

Typical git workflow

git workflow is quite straightforward:

- fetch tarball
- git-import-orig <tarball> (updates upstream and pristine-tar branches)
- update changelog and other files
- do package improvements, remove patches applied upstream, etc.
- debcommit -a (does a local commit in the master branch)
- git-buildpackage (which uses cowbuilder in my setup)
- local testing
- dch -r
- git commit -a -m "Release"
- git-buildpackage --git-tag --git-sign-tags (produces and tags the release for upload)
- dput <...>.changes
- git push --all (pushes all local changes to the repo at git.d.o)
- git push --tags (pushes the tags)

Miscellaneous

We use git together with mr (Multiple Repository management tool) to track the git repositories for each package.

Obtaining the master repository

```
git clone <alioth-username>@git.debian.org:/git/pkg-java/pkg-java.git
```

NOTE that your public key must be known to alioth in order for this to work, see note below.

Then, to obtain all the team packages, do:

```
cd pkg-java
# look at .mrconfig
apt-get install mr
mr checkout
```

To enable parallel checkouts, use

```
mr -j 5 checkout
```

To avoid typing your username and your password see this [🌐 page](#).

WARNING: as of December 2011, the pkg-java repository uses 5.5 GB of space.

To get the latest sources from all packages, do

```
mr -j 5 update
```

[CategoryJava](#) [CategoryPackageManagement](#) [CategoryGit](#)

Java/JavaGit (last modified 2018-05-04 20:31:28)

- [Javascript](#)
- [Policy](#)



This document is still work in progress.

Please check the [🌐 mailing list archives](#) for the latest discussions about it.

Introduction

This page describes the policy that packages with Javascript libraries should follow.

Policy

See also [Transition Policy](#).

1. given a **foo** library, binary package **must** be called **libjs-foo** and the *source* package name **should** be called **foo.js**;
2. **should** recommend *javascript-common* in *debian/control*
3. **should** be installed to */usr/share/javascript/foo/*
4. **should** ship a *minified* version for each script, generated at build time (use [UglifyJS](#) or [terser](#) for this purpose)
5. **should** generate a **node-foo** binary package if the script is usable also for [Nodejs](#).
6. as per [🌐 Debian Policy § 3.5](#), **node-foo** packages **should not** issue a **nodejs** dependency, just because the library can be used with **nodejs**. Other reasons can result in such a dependency though.

Please remember that it's preferable if you name the repository after the source package name (i.e. *foo.js.git* for *foo.js*). This isn't a hard requirement though.

Proposal

Today most of JS packages does not follow the naming policy. I suggest this:

1. Naming policy. Given a **foo** library
 - a browser-only package **must** be called **libjs-foo**
 - a node-only package **must** be called **node-foo**
 - for a both browser/node binary, browser binary package **must** be called **libjs-foo**, node binary package **must** be called **node-foo** and the *source* package name **should** be called **foo.js**;

Exclude auto-generated files from source

Strict application of **DFSG** requires files generated from source in upstream tarball to be excluded, unless it is possible to regenerate the files and prove they are identical to the ones in the tarball.

Minified files and browserified files are examples of such files that **could** be excluded for that reason. A convenient way to achieve this is to use the **Files-Excluded** field in **debian/copyright**, for more information please see [repacking](#) and [uscan enhancements](#).

[CategoryPackaging](#)

Javascript/Policy ([last modified 2021-02-08 13:08:47](#))

- [Javascript](#)
- [Nodejs](#)

Contents

1. [Node.js](#)
2. [Debian packages to install](#)
3. [Packaging modules](#)
 1. [debian/watch policy](#)
4. [Complex modules](#)
5. [Generated Files](#)
 1. [Browser vs Node](#)
 2. [Minified javascript](#)
 1. [uglifyjs](#)
 2. [terser](#)
 3. [Module bundling or browserification or webpacking \(What is Rollup, Browserify or Webpack?\)](#)
 4. [ES6 and transpiling](#)
 5. [ES modules vs Commonjs](#)
6. [Using build tools like grunt](#)
 1. [Grunt](#)
 2. [Gulp](#)
 3. [Babel](#)
 4. [Webpack](#)
 5. [Rollup](#)
 6. [Browserify](#)
 7. [Cake](#)
7. [Best Practices for Upstreams](#)

Node.js

Node.js is a platform built on Chrome's JavaScript runtime for easy building of fast, scalable network applications. Node.js uses an event-driven, non-blocking I/O model that makes it lightweight and efficient, perfect for data-intensive real-time applications that run across distributed devices.

Debian packages to install

2018-01-22: please enumerate exactly which Debian packages to install, to get the following commands to work correctly

Packaging modules

On Debian systems up to and including Stretch and Buster (Debian 9 and 10), Node.js modules are located in:

```
/usr/lib/nodejs/<module_name>
```

For Debian Bullseye (11), Node.js modules might be located (but this is yet to be decided):

```
/usr/share/node/<module_name>
/usr/lib/<arch triplet>/node/<module_name>
/usr/lib/node/<module_name>
```

For more information about packaging a node module for Debian (and the reasons for the change of location for nodejs modules) please take a look at these pages:

- [npm2deb](#): an automatic tool to easily and quickly deploy debian packages starting from *npm* information
- [manual](#): an overview about best practice and policies in packaging nodejs modules in debian systems

debian/watch policy

Some upstream authors don't publish the same versions in Npmjs and Github tags. To be sure to be consistent between node modules, it is recommended to use Npmjs versions for modules that don't have any build steps. With uscan $\geq 2.18.6$ (*stable backport at least*), you can use `searchmode=plain` to parse npmjs versions. Example:

```
version=4
opts="searchmode=plain" \
https://registry.npmjs.org/aes-js https://registry.npmjs.com/aes-js/-/aes-js-(\d
```

When npm registry tarballs have generated files and missing source files, using source from version control repository (either tag or commit snapshot) is preferred. If npm registry has source and build configurations and generated files, you can just remove the generated files from source package and regenerate them during build or use the version control repository snapshot.

Complex modules

For complex modules, which have many dependencies to be satisfied, you may want to track your work in this wiki in order to keep javascript team updated about.

For more information, please take a look at [Tasks](#) page. Take also a look at [How to group many modules in one package](#).

Generated Files

In debian we take the actual source file and build it instead of the generated file because the source file is what will be modified by the upstream developer (preferably taken from version control system instead of npmjs.com.) As most code on version control system like Github are in different languages like Typescript, coffeescript etc. So the code that people write and share in their version control system cannot be used directly in nodejs or browsers unless compiled into javascript.

If generated files exists in the source tarballs, you should remove all generated files and generate them during debian package build process using tools in debian. See [Javascript/Repacking](#) for steps to exclude generated files and [Javascript/Nodejs#Using_build_tools_like_grunt](#) for using different build tools in debian during debian package build process.

Browser vs Node

Javascript was initially used for client side scripting on the browsers. Then nodejs came, which made using javascript on the servers also possible. Most widely used version of javascript today is called ECMA Script 5 or ES5.

Minified javascript

Javascript code targeted web browsers is usually minified to reduce size (further than possible with lossless zlib or brotli compressors alone).

UglifyJS 3 and terser are generally recommended among the [many JavaScript minification tools](#),

In debian, if we ship a minified js file, we should include the corresponding non minified code also. To guarantee the non minified code is corresponding to the minified code is, to run uglifyjs during build.

uglifyjs

UglifyJS 3 is the conservative choice: Has fewer reverse dependencies so more likely to be kept up-to-date and less likely to be problematic to backport.

terser

Terser is the fancier choice: Works directly with more modern dialects of JavaScript (UglifyJS 3 covers only ECMAScript 5 so newer code needs to be "transpiled e.g. using Babel).

Module bundling or browserification or webpacking (What is Rollup, Browserify or Webpack?)

Minification is the simplest code transformation we have. There are other transformations too. nodejs supports modules, but it is very inefficient in a browser (http/2 will change this situation which allows parallel requests). Also there are some functionality only present in nodejs but not in browser environment. So using tools like browserify, webpack or rollup, code written for nodejs can be transformed to run on the browser. We need to run this transformation also during build. [How To Bring Node.js Modules to the Browser ?](#). See also this [simple explanation of different module formats](#).

ES6 and transpiling

There is one more type of transformation that is common. They have created a new version of Javascript with new syntax, more functionality built in, it is called ES6 or ES2015. But since browsers and nodejs still don't support es6 fully, we have to convert ES6 to ES5. babel and buble are tools to do that. webpack and rollup has plugins to do ES6 to ES5 transformation.

ES modules vs Commonjs

ES modules use import and export statements, but node (current version 12.7) only support this with --experimental-modules flag ([ES modules documentation](#)). Rollup and Webpack supports ES modules but their implementation currently differs in how to specify an ES module. Rollup and Webpack accepts "module" field in package.json as ES module entry point but node core supports only "type": "module" and "main" fields or .mjs file extension. [Read more about native ES module support](#).

Commonjs uses require and only support default export. Rollup and Webpack can convert ES modules to Commonjs.

Read more about different module formats from following blob posts

1. [🌐 What Are CJS, AMD, UMD, and ESM in Javascript?](#).
2. [🌐 CommonJS vs. ES modules in Node.js](#)

Using build tools like grunt

You can check in `package.json` under `scripts` section for the exact commands to be used for building.

Grunt

- grunt package has a patch to load global modules. Just call `grunt` in `debian/nodejs/build`.

[Toggle line numbers](#)

```
___1 grunt build
```

Gulp

- gulp package has a patch to load global modules by default. Just call `gulp` in `debian/nodejs/build`.

[Toggle line numbers](#)

```
___1 gulp build
```

Babel

- `babeljs` command is provided by `babeljs`. See [🌐 node-d3-color](#) for an example of using babel. Just call `babeljs` in `debian/nodejs/build`.

[Toggle line numbers](#)

```
___1 babeljs src -d lib
```

- Note: Check ?Babel 7 transition page if the module is using an older version of Babel.

Webpack

- webpack package needs using a `webpack.config.js` to resolve global modules, see gitlab and [🌐 node-mocha](#) for config files.

Use this as `debian/webpack.config.js`

[Toggle line numbers](#)

```
__1 'use strict';
__2
__3 var fs = require('fs');
__4 var path = require('path');
__5 var webpack = require('webpack');
__6
__7 var config = {
__8   resolve: {
__9     modules: ['/usr/lib/nodejs', '/usr/share/nodejs', '.'],
__10  },
__11  resolveLoader: {
__12    modules: ['/usr/lib/nodejs', '/usr/share/nodejs'],
__13  },
__14  node: {
__15    fs: 'empty'
__16  },
__17  output: {
__18    libraryTarget: 'umd'
__19  },
__20  module: { rules: [ { use: [ 'babel-loader' ] } ] }
__21 }
__22
__23 module.exports = config;
```

and call webpack in debian/nodejs/build.

[Toggle line numbers](#)

```
__1 webpack --config debian/webpack.config.js --entry ./lib/Yaml.js --output .
```

- Note: Check ?Webpack 4 transition page if the module is using an older version of webpack.

Rollup

- Rollup: call rollup -c or build script in debian/nodejs/build.

[Toggle line numbers](#)

```
__1 rollup -c
```

You will need to use rollup-plugin-commonjs module if you get Unresolved dependencies in rollup output like in the example below

```
index → build/d3.js...
(!) Unresolved dependencies
https://github.com/rollup/rollup/wiki/Troubleshooting#treating-module-as-external-
d3-array (imported by index.js)
```

Make sure these are added as build dependencies and you use commonjs plugin (if target module is only available as commonjs) or node-resolve plugin if target (if target module is ES module).

See [node-redux](#) as an example of adding custom path for node-resolve plugin (with minimum version of node-rollup-plugin-node-resolve 11).

Browserify

- browserify can be replaced by webpack or rollup or browserify-lite. See node-chart.js as an example.
 - In case of browserify-lite use case, packages installed via apt will be installed in the /usr/share/nodejs directory, while browserify-lite checks only the node_modules/ directory. We create a debian/nodejs/extlinks or debian/nodejs/extcopies files respectively (if either one do not work as expected) and add the installed package-name in the file. pkg-js-tools will read extlinks or extcopies files create necessary symbolic link or copy of modules or packages listed in these files so they are available in node_modules/ directory, extlinks or extcopies will make it available in node_modules/ directory during package build which is also done by by pkg-js-tools.
 - If extlinks is used, it will add symbolic link (ln -s command) if extcopies is used it will copy (cp -r command) so adding <package-name> to debian/nodejs/extlinks is same as mkdir -p node_modules && ln -s /usr/share/nodejs/<package-name>/node_modules

Cake

- use cake.coffeescript command if you have Cakefile. Just call cake.coffeescript in debian/nodejs/build.

[Toggle line numbers](#)

1 cake.coffeescript build

Best Practices for Upstreams

Documenting things that makes it easy for upstream who wants to support packaging [/BestUpstreamPractices](#)



Search (3 character minimum)

		<u>dh-lua</u>	1	Created 5 Mar 2018
		<u>Lua Cjson</u>	0	Created 5 Dec 2022
		<u>lua-alien</u>	0	Created 5 Mar 2018
		<u>lua-ansicolors</u>	0	Created 5 Mar 2018
		<u>lua-apr</u>	0	Created 5 Mar 2018
		<u>lua-argparse</u>	0	Created 5 Mar 2018
		<u>lua-basexx</u>	0	Created 5 Mar 2018
		<u>lua-bit32</u>	0	Created 5 Mar 2018
		<u>lua-bitop</u>	0	Created 5 Mar 2018
		<u>lua-busted</u>	0	Created 5 Mar 2018
		<u>lua-cgi</u>	0	Created 5 Mar 2018
		<u>lua-check</u>	0	Created 5 Mar 2018
		<u>lua-cliargs</u>	0	Created 5 Mar 2018
		<u>lua-cmsgpack</u>	0	Created 28 Mar 2024
		<u>lua-compatible53</u>	0	Created 5 Mar 2018
		<u>lua-copas</u>	0	Created 5 Mar 2018
		<u>lua-cosmo</u>	0	Created 5 Mar 2018
		<u>lua-coxpcall</u>	0	Created 5 Mar 2018
		<u>lua-cqueues</u>	0	Created 5 Mar 2018
		<u>lua-curl</u>	0	Created 23 Feb 2022

- [Teams](#)
- [DebianMonoGroup](#)
- [NewPackage](#)

How to get a new CLI/Mono package into Debian

This is a *quick* rundown of the steps required to get a new package sponsored by the Debian CLI team. The most important step is the first one. If you get onto IRC then you can ask questions and we'll be able to help you out.

- Join  [#debian-cli @ OFTC](#)
- The  [Debian policy](#),  [New Maintainers' guide](#) and  [CLI policy](#) are all key reference documents
- Package your app using `git-buildpackage` (see  <file:///usr/share/doc/git-buildpackage/manual-html/gbp.import.html#GBP.IMPORT.FROMSCRATCH>)
 - Usually short form `dh` (see `dh(1)` for more)
 - `cli-common-dev` (`--with cli`) (using `cli-common-dev` is **essential**; refer to the CLI policy to find out how this works)
 - Set the Maintainer in `debian/control` to the appropriate team. One of:
 - Debian CLI Applications Team <pkg-cli-apps-team@lists.alioth.debian.org>
 - Debian CLI Libraries Team <pkg-cli-libs-team@lists.alioth.debian.org>
- Sign up on  [alioth.debian.org](#)
- Request to join the `pkg-cli-apps` / `pkg-cli-libs` projects on alioth.
- When approved, `ssh` to `git.debian.org` and
`cd /git/pkg-cli-{apps,libs}; ./setup-repository mypkgname 'packaging for mypkg'`
- Prepare your git repository for pushing to `git.debian.org`:
`git remote set-url origin git+ssh://yourusername@git.debian.org/git/pkg-cli-{apps,libs}/packages/mypkgname.git`
- push all branches
- Ask for sponsorship: (on IRC) `%learn rfs mypkgname <NS>`
- Stay on IRC to receive feedback, also check `%rfs-feedback`
- Get uploaded by a DD
- When the package is accepted, subscribe to bugs on PTS — mail "subscribe mypkgname" to  pts@qa.debian.org. This step is very important. As a Debian maintainer, you need to be responsive when bugs are filed against your package.

[CategoryPackaging](#)

Teams/DebianMonoGroup/NewPackage (last modified 2019-09-07 17:54:26)

- [Teams](#)
- [OCamlTaskForce](#)

Debian OCaml Task Force

A collaborative effort to maintain Debian packages related to the [OCaml](#) programming language

- [TODO list](#)
- [Forthcoming transitions](#)

Infrastructure

- **Website:** [OCamlTaskForce](#) (this wiki page)
- **Salsa group:**  <https://salsa.debian.org/ocaml-team>
- **Documentation:** see [more stuff](#) below

Interacting with the team

- **Email contact:** `<debian-ocaml-maint@lists.debian.org>`
- **Public IRC channel:** `#debian-ocaml` on `irc.debian.org` (OFTC)
- **Joining:** wanna join us? you are more than welcome! please send an *introductory mail* to our email contact above, and then request to join the Alioth Project using the legacy GForge mechanism

Task description

The *Debian OCaml Maintainers Task Force* is up to package [OCaml](#)-related programs and libraries in Debian, to make as easy as possible the usage of such pieces of software.

Our usual activities include both routine packaging and maintenance of OCaml-related software and formalization in our policy of best practice for the OCaml packaging in Debian.

Get involved / future work

- policy related [stuff](#)
- OCaml backports coordination page: [OCamlBackports](#)
- what about a [camlp4-misc](#) package?

Past work

-  [essay](#) on OCaml link time compatibility and Debian dependencies superseded by JFLA 2010 paper:  [Enforcing Type-Safe Linking using Inter-Package Relationships](#)
- [migrating our repository to git](#)

More stuff

Resources

OCaml-related packages in Debian distribution are quite a lot, here you can find some resources to monitor them:

-  [Package Entropy Tracker](#) for OCaml Team's packages
-  [source package list](#) ; formats:  [`.html`](#),  [`.txt`](#)
-  [build dependency graph](#) ; formats:  [`.dot`](#),  [`.ps`](#),  [`.png`](#)
 -  [build order](#) (`.txt` only)
- [transitions](#)
-  [Reproducible builds for the OCaml team](#)
-  [Potentially missing binNMUs for OCaml packages](#)
-  [Bugs in packages maintained by debian-ocaml-maint@lists.debian.org](#)

Policy

We wrote a  [policy](#) establishing best practices for packaging OCaml-related software in Debian. It is available both in HTML and plain text formats. It can also be found in the  [`dh-ocaml`](#) package  [on your filesystem](#).

-  [Debian OCaml Packaging Policy](#) ; formats:  [`.html`](#),  [`.txt`](#)

Version Control System

Most of our packages are maintained using Git, more specifically with git-buildpackage (for building), git-import-orig+pristine-tar (for upstream tarballs) and git-import-dsc (to acknowledge NMUs). Our repositories are available on Salsa. You can  [browse online](#) our repositories, or check one out using the dom-git-checkout script (from the dh-ocaml package).

[CategoryPackaging](#)

- [Python](#)
- [LibraryStyleGuide](#)



This wiki page can be edited by anyone, but has long been stewarded by the [Debian Python Packaging Team](#) to reflect the experience and consensus surrounding best practices they've arrived at. It is therefore requested that you first submit any proposed changes to these recommendations at the [DebianList: debian-python](#) mailing list for discussion and review before making them, unless they are sure to be uncontroversial (e.g. fixing typos, updating dead links, etc.). Thank you!

This document is intended as an approachable **style guide to the packaging of new Python modules for Debian**, presenting the guidelines published in the official [Debian Python Policy](#) using less formal language. If you have not read that document, *stop now and please read it*; this information is designed to augment, not supplant, it and builds on information defined there.

This page captures in example form the best practices for *simple* Python modules as can be found on the [Python Package Index](#) (PyPI). A few assumptions about the Python package you are working on are being made, namely that it:

- Is [PEP 517](#)-compliant, with a working declarative [pyproject.toml](#) or [setup.cfg](#) file (and formerly `setup.py`) and should be generally installable without modification into a [DebianPkg: virtualenv](#).
- Is a "Pure-Python" or simple extension module that can be successfully installed using only `pip install` (or the older `python3 ./setup.py install` incantation),
(For instructions on building more complex modules or extensions, you will need to adapt these guidelines to their needs.)
- Supports at least Python 3,
- Has an upstream-provided test suite, and
- Has some Sphinx-based documentation.

The design of this document is meant to present widely-held best practices in a format that facilitates easy adoption into your own packages, and is by its very nature a permanent work-in-progress.

NOTE: The [Python App Style Guide](#) describes an alternative approach useful for crafting a single package `debian/rules` file.

Contents

1. Overview

2. Packaging files

1. `debian/control`

2. debian/rules	
3. debian/links	
4. debian/watch	
3. Sphinx documentation	
4. Overrides	
5. Building debug packages	
6. autopkgtest	
7. Gotchas	
1. Executables and library packages	
2. MANIFEST.in and missing files	
8. TODO	
9. Notes	
10. See also	

Overview

There are many ways to build Python libraries for Debian, and each package presents its own corner cases and challenges. In modern Debian (e.g. unstable as of November 2013) and derivatives, the easiest way to build Python library packages is with the [pybuild](#) debhelper build system. This link has details on pybuild, including more examples for your cargo culting pleasure.

If you can't use pybuild, say because you want to be able to backport your package to distribution versions before pybuild was introduced, you may be interested in [the historical version](#) of this document, which contains all the gory details you had to implement manually, but which pybuild now takes care of for you.

The examples below are taken from the package for the popular [urllib3](#) module, with modifications for generality.

Note: It is highly recommended you explicitly set the `dh` build system to `pybuild`. By default, if there is a `setup.py`, `dh` will set the build system to `python_distutils` which only supports Python 2 and you'll get errors during package build as it tries to invoke some nonexistent Python 2 tools.

Packaging files

debian/control

Build-Depends

One very important thing to get right is the `Build-Depends` line in the source package stanza. `setuptools/distribute`-based packages have the nasty habit of downloading dependencies from PyPI if they are needed at `python3 setup.py build` time. If the package is available from the system (as would be the case when `Build-Depends` is up-to-date), then `distribute` will not try to download the package,

otherwise it will try to download it. This is a huge no-no, and pybuild internally sets the `http_proxy` and `https_proxy` environment variables (to 127.0.0.1:9) to prevent this from happening.

`dh_python3` will correctly fill in the installation dependencies (via `${python3:Depends}`), but it cannot fill in the build dependencies. Take extra care in getting this right, and double check your build logs for illegal access to `pypi.org`.

You'll want to have at least the following build dependencies:

- `debhelper-compat (= 13)`

[DebianPkg: package](#), [DebianMan: man page](#)

- `dh-python`

[DebianPkg: package](#), [DebianMan: man page](#)

...or after version 3.20190307, its far more convenient alias:

`dh-sequence-python3`

- `python3-all`

[DebianPkg: package](#), [DebianMan: man page](#)

- And at least one—and sometimes more—of:

- `python3-build`

[DebianPkg: package](#), [DebianMan: man page](#)

- `python3-setuptools`

[DebianPkg: package](#), [official documentation](#)

- `python3-wheel`

[DebianPkg: package](#), [official documentation](#)

If the package has documentation in [reStructuredText](#) format, often processed by way of [Sphinx](#), these dependencies are also needed:

- `python3-docutils`
- `python3-sphinx`

Python versions

You may want this line in the first stanza, which defines the source package...

```
X-Python3-Version: >= 3.X
```

...where X is the minimum version of Python interpreter the package requires. The Debian stable distribution (trixie, as of 2024-12-11) includes [DebianPkg: python3.11](#), so if the minimum version is less than 3.11, there is no need to define `X-Python3-Version`. This is to support proper dependency generation for backports.

If you're building upstream documentation, it's best to put those files into a separate `python-foo-doc` package, like:

```
Package: python-foo-doc
Architecture: all
Section: doc
Depends: ${sphinxdoc:Depends}, ${misc:Depends}
Description: Python frobnicator (common documentation)
 This package frobnicates a Python-based doohicky so that you no lo
 need to kludge a dingus to make the doodads work.
.
 This is the common documentation package.
```

debian/rules

The contents of this file depends on whatever build system you choose, however we highly recommend the [pybuild](#) build system for its simplicity (at least for setup-based packages, i.e. the majority of those on PyPI).

Here is a `debian/rules` file for you to start with. First, I'll show you the whole thing, then I'll explain it line by line.

```
Toggle line numbers
__1__#!/usr/bin/make -f
__2__
__3__#export DH_VERBOSE = 1
__4__export PYBUILD_NAME = foo
__5__
__6__%:
__7__    dh $@ --with python3 --buildsystem=pybuild
```

The file starts with the standard `#!/` "shebang" line. I like adding the (commented out before uploading) `DH_VERBOSE` variable seen on line 3 because it can make build problems easier to debug.

Line 4 then defines the important `PYBUILD_NAME` variable. `pybuild` supports a number of environment variables which control such things as the destination directory for build artifacts. The defaults generally do the right thing, but you do typically need to at least

tell pybuild the name of your package. Here, that the name is `foo`. This should match the module name, so for example, in [DebianPkg: python-urllib3](#), examining its `debian/rules` file reveals the presence of [this line \(#3\)](#)...

```
export PYBUILD_NAME = urllib3
```

...even though the name of the binary package that it produces is `python-urllib3`.

The final two lines (nos. 6–7) are the standard [debhelper](#)-based Makefile [match-anything.pattern rule](#) used to catalyze the entire package build process:

```
%:
    dh $@ --buildsystem=pybuild --with python3
```

What's important to note here is that the `dh_python3` helper is being invoked, and also that the *build system* that `dh` will use is `pybuild`. There's a lot of magic packed into this line, and the `pybuild` manpage goes into more detail, but essentially what this does is:

- Builds the Python 3 package for all supported Python 3 versions.
- Automatically detects the command to run the test suite.
- Runs the Python 3 test suite.

Once all that's done, it properly installs all the files into binary packages.

And that's it! You usually won't need a `debian/python3-foo.install` file even if you have multiple binary packages, because again, `pybuild` does the magic for you. You might need these if there are some non-standard installation tricks you need to implement.

debian/links

You may or may not need this file. It can be useful for Sphinx-based documentation, by providing the original `reST` files used to build the HTML documentation. Here is an example `debian/python-foo-doc.links` file.

```
usr/share/doc/python-foo-doc/html/_sources usr/share/doc/python-foo-doc/
```

debian/watch

Make sure that you have a working [debian/watch](#) file, for referencing upstream distribution tarballs. Usually, `uscan` is sufficient to download tarballs from the declarations in `debian/watch` (or, if you're using `svn-buildpackage`, this can be done automatically with `svn-buildpackage --svn-download-orig`), but if the source needs to be repackaged, you should then provide a `get-orig-source` ([policy](#)) target in `debian/rules` which does the job.



The PyPI URLs used by many existing packages are now broken, or very soon will be. PyPI changed the way index listings for package directories work and they are expected to disappear at some point in the future. Thus, it is highly recommended that you update your existing packages to the new format described here.

Since probably for most of our packages, the original tarball comes from the [Python Package Index](#) (a/k/a PyPI, or the ["Cheese Shop"](#)¹), here is a sample `debian/watch` file that you might be able to adapt for your own package:

```
version=4
opts="uversionmangle=s/(rc|a|b|c)/~$1/" \
https://pypi.debian.net/urllib3/urllib3-@ANY_VERSION@@ARCHIVE_EXT@
```

Notice the use of `pypi.debian.net` instead of `pypi.org`. [Debian Services](#) now operates a public [redirector service for PyPI](#) to react to changes in the site's listing format so there's no need to change all of the `debian/watch` files each time in response. In fact, it's trivial to get a valid `debian/watch` file for your package! Just invoke this shell one-liner in the `debian` directory of your source package, substituting the module's name *on PyPI* for `<module_name>` (be advised that this will overwrite or "clobber" any existing watch file there, so rename it first if you wish to keep it):

```
wget -HN "https://pypi.debian.net/<module_name>/watch"
```

Sphinx documentation

The Sphinx documentation section has been moved to its own page, to better accommodate new contributors who were confused by the Python specificity of its directions, and by how it was difficult to discover this information when learning how to build Sphinxdocs for a non-Python package. The new page is found at [SphinxDocumentation](#).

Overrides

If you need to override the `dh_python3` defaults, do it with an override method in your `debian/rules` file. Something like this will work:

```
override_dh_python3:  
    dh_python3 --shebang=/usr/bin/python3
```

If you need to override the default testing regime (e.g. to use `pytest`), you can do it one of two ways, either:

```
export PYBUILD_TEST_PYTEST = 1
```

or you can override the `dh_auto_test` rule with:

```
override_dh_auto_test:  
    dh_auto_test -- --test-pytest
```

Building debug packages

As of 2022, you should not manually build separate `-dbg` packages for Python libraries. No special configuration is needed, and [AutomaticDebugPackages](#) will handle everything when you use `dh`; see [🌐 "Re: Python3 -dbg packages"](#) (2021-09-03), sent by Matthias Klose <doko AT debian DOT org> on the [DebianList: debian-python](#) mailing list for more information.

If you need details on how this was done before, please consult prior revisions of this document available in [its history](#).

autopkgtest

It's good idea to supply CI ([🌐 DEP-8](#)) tests for package. These tests check to make sure your built module can be imported, and that they are working in case any runtime dependencies change. `autodep8` have support for Python packaging. To enable it, just add to `debian/control`:

```
Testsuite: autopkgtest-pkg-python
```

This will check if a Python module, named the same as the package, can be imported from the system. You could write more complex tests yourself.

You should check these tests locally by installing `autopkgtest` and `autodep8` package. Then run `autopkgtest` command (see the manpage for details).

Gotchas

Executables and library packages

Let's say you have a Python package which results in libraries and an executable. What is the best practices for naming and organizing your binary packages?

Clearly you want to at least have:

- `python3-foo` -- the Python 3 library

but where should the `/usr/bin/foo` script go? You could put it in `python3-foo`, but that might not be ideal: users may not expect to find executables in packages that look like library package names, and it would make it difficult to support things like `?PyPy3` packages in future.

Here are some recommendations. We do not have a standard (though maybe we should):

- `foo` - this is nice because it parallels the `/usr/bin/foo` executable. In all likelihood, if the `/usr/bin` script name doesn't collide, then the `foo` binary package won't either.
- `python3-foo` - if the script is Python 3.x compatible.

MANIFEST.in and missing files

If the upstream package requires some non-standard files to be installed, it must include a `MANIFEST.in` file. Sometimes these files are omitted from the original tarball because they're mostly useful for creating the sdist. Case in point: a package contained a bunch of doctests in `.rst` files but `python3 setup.py sdist` [does not include](#) most `.rst` files by default. Thus, when `pybuild` went to build the package locally, the `.rst` files were omitted and the test suite failed because it could not find the doctest.

The solution for this is to either ensure that upstream includes the `MANIFEST.in` file, or that you add one before `pybuild` runs.

TODO

- describe egg related information, what should go in the binary package and what shouldn't.
- when to build-depend against `(lib|)python3(-all|)(-dev|)`

- build-depending against `lib*` should never ever be needed.
- the `-dev` variants are for when you need to compile C extension: if your project is pure python you don't want this
- Using the `-all` variants will compile/test against all current Python(3) versions, rather than only the default one (i.e. binary `.so` modules will be compiled multiple times).
- A bare dependency on `python3-all-dev` breaks cross building. Do use `python3-all-dev:native`, `libpython3-all-dev` instead. In a similar vein, rather than using `python3-dev`, it should be `python3-dev:native`, `libpython3-dev`.
- The non `-all` variants are to be used when your project can't and is not building against all supported versions.

Notes

1. See the [🌐 CheeseShop](#) page on the Python Wiki and this [🌐 Python FAQ entry](#) for more context. ([1](#))

See also

- [Python/Pybuild](#)
- [Python/GitPackaging](#)

[CategoryPackaging](#)

Python/LibraryStyleGuide ([last modified 2025-12-19 01:22:12](#))

- [Teams](#)
- [Ruby](#)
- [Packaging](#)

This page describes the current packaging practices for Ruby in Debian. For more general information, see [this wiki page](#). Some information no longer relevant is moved to [Teams/Ruby/Packaging/Archive](#)

Another effort to document the workflow of packaging ruby-gems for Debian is made by the debian-diaspora Team at: [this wiki page](#), which is targeting beginners and very detailed.

Contents

1. [Joining the Debian Ruby team](#)

2. [Using Git](#)

1. [Git workflow](#)

2. [Obtaining the master repository](#)

3. [Creating a new Git repository and pushing an existing package to salsa.debian.org](#)

4. [Packaging new gems from scratch](#)

5. [Known Issues](#)

3. [Guidelines for Ruby packaging](#)

1. [gem2deb as the preferred packaging tool for ruby software](#)

2. [Switching to github tar and other changes to orig-source](#)

3. [Guidelines for Packaging Ruby on Rails Applications](#)

4. [Multiple gemspec files](#)

5. [Handling Rails Engines](#)

4. [Tasks for team-maintained Ruby packages](#)

1. [Updating a package to a newer version](#)

2. [Updating packages with API breaking changes](#)

5. [Requesting Sponsorship](#)

Joining the Debian Ruby team

We always welcome people to help us out. If you want to, please follow the following guidelines.

- Contact us first on our  [mailing list](#) or on IRC - #debian-ruby on irc.oftc.net (If you are using Matrix or XMPP, there are bridges to IRC which you can use to join IRC channels from your matrix or xmpp clients).
-  [Register an salsa account](#).
- Send contributions via merge requests on salsa. One or more of the existing team members will review your contributions.

- once you have made enough contributions and you are confident that you will stay around, request to be added as team member on salsa.
- Read all the docs (sorry).
- Enjoy!

Using Git

We use git together with myrepos (Multiple Repository management tool) to track the git repositories for each package.

Before you continue, ensure you have installed the tools used by the team:

```
apt-get install myrepos git-buildpackage pristine-tar gem2deb
```

Git workflow

The following workflow is used for (almost) all non-native packages maintained by the Ruby team:

- git-buildpackage as a building tool
- 3 main branches:
 - `pristine-tar` branch to store tarballs generated by gem2deb or by the [gemwatch service](#)
 - `upstream` branch tracking unpacked upstream tarballs
 - `master` branch containing upstream + the debian/ repository (with patches unapplied).
- possibly other branches for backports or stable/testing updates, with specific `debian/gbp.conf` configuration
 - `wheezy-backports` replacing master for backports
 - `master-<codename>`, `upstream-<codename>` for updates targetted to the particular version `<codename>`

Obtaining the master repository

```
git clone git@salsa.debian.org:ruby-team/meta.git ruby-team
```

To obtain one of the packages, do:

```
cd ruby-team  
./checkout $PACKAGE
```

If you want to obtain *all* of the team packages (have in mind that the team has *a lot* of packages, so this will probably take a little long), do:

```
# look at .mrconfig
mr --force checkout
```

To clone all team packages with parallel checkouts, use

```
mr --force -j 5 checkout
```

To avoid typing your username and your password see this [?page](#).

If you don't want to use mr, you can clone individual repos:

```
gbp clone --pristine-tar git@salsa.debian.org:ruby-team/<pkg-name>.git
```

Creating a new Git repository and pushing an existing package to salsa.debian.org

Note: please do not leave empty repositories on salsa; only create a repository there just before pushing your package.

Suppose you want to package the latest 0.7 version of the *foo* Ruby gem, to obtain a *ruby-foo* package. See [the naming conventions](#) for packages.

On your local workstation:

```
cd ruby-team
./setup-project
```

Packaging new gems from scratch

Note: See [Teams/Ruby/Packaging/gem2deb](#) if you are new to ruby packaging

On your local machine, get a basic .dsc from the library you want to package, using:

```
gem2deb foo
```

gem2deb will then try to download the *foo* gem from rubygems.org, and convert it to a primitive Debian package *ruby-foo_0.7-1.dsc*. You can also run *gem2deb* on a local *.gem* file *foo-0.7.gem* or a tarball (e.g. from Github).

At this point, please do not make any change to the package generated by *gem2deb*. It is important to track all the changes in git. Then:

```
cd where-you-cloned-pkg-ruby-extras.git
./update-mrconfig # will generate the config for your new repo
./checkout ruby-foo # will clone your new repo
cd ruby-foo

gbp import-dsc --pristine-tar path-to/ruby-foo_0.7-1.dsc
git tag -d debian/0.7-1 # because the package is not ready yet
git push --all
git push --tags
```

Then, make the needed changes to the packaging, and then

```
git push
```

After upload, tag the package with

```
git tag debian/0.7-1
```

And don't forget to

```
git push --tags
```

Known Issues

If *mr checkout* gives an error like **mr: illegal section "[DEFAULT]" in untrusted:**

```
Add .mrconfig path to ~/.mrtrust
```

If your local username is different from your Alioth username (that's probably the case for non-DD developers with -guest accounts), add the following to your *~/.ssh/config*:


```
Host alioth.debian.org svn.debian.org git.debian.org
User alioth-username
```

Guidelines for Ruby packaging

gem2deb as the preferred packaging tool for ruby software

 [gem2deb](#) is the current packaging tool for Debian Ruby packages. It:

- does almost everything automatically
- uses dh
- is easier to adapt to our needs, since there's no dependency on an external tool
- runs the test suite as part of the build, for each ruby implementation (yes, it's possible to override this). [See more about running tests](#)
- uses a single binary package for native libraries, instead of one binary package for each ruby implementation

It serves two goals:

- it can be used by users to generate .debs from gems locally
- it generates Debian source package from which we can do the packaging work

gem2deb can be considered to some extent as the reference implementation of the Debian Ruby policy. See [Teams/Ruby/Packaging/gem2deb](#) for using gem2deb to create new gem packages.

Switching to github tar and other changes to orig-source

[see this page](#)

Guidelines for Packaging Ruby on Rails Applications

- [Guidelines and tips for rails applications](#)

Multiple gemspec files

You can create `debian/gemspec` as a symbolic link to the gemspec file you want to use. You can also set `DH_RUBY_GEMSPEC` variable in `debian/rules`. For `autopkgtest`, you should remove the `Testsuite` field in `debian/control` and create `debian/tests` manually.

Example packages: `ruby-concurrent`, `ruby-flipper`, `ruby-elasticsearch`

Handling Rails Engines

There are some special ruby gems that provides a rails engine. Purpose of many of those rails engines are providing ?JavaScript libraries to rails applications. jquery-rails gem is such an example, it provides jquery.js to rails applications through a mechanism called rails assets pipeline. These gems usually have a app/assets or vendor/assets or lib/assets directory. In all cases the js files are embedded in the gem.

See [Teams/Ruby/Packaging/RailsEngines](#) for packaging such gems.

Tasks for team-maintained Ruby packages

Updating a package to a newer version

First make sure you got a local repository from salsa which has the pristine-tar branch:

```
gbp clone --pristine-tar git@salsa.debian.org:ruby-team/<pkg-name>.git
```

If you have a working debian/watch file (check by running `uscan --verbose --report`), you can simply import a new version with

```
gbp import-orig --pristine-tar --uscan
```

If you want a different version than latest version, you may have to download it manually using a web browser / `wget` or use

`uscan --verbose --download-version` option and run `gbp import-orig --pristine-tar` separately.

Alternatively, you can fetch the most recent version of a gem, and make a tarball out of it

```
gem2tgz <gemname>
```

where `<gemname>` is the upstream name of the gem, or run `gem2tgz` on a local `.gem` file.

You can also get a specific version and then create the `tgz` by doing

```
gem fetch -v <gemname>  
gem2tgz <gemname>.gem
```

Then import the created tarball in the Git repository with
`gbp import-orig --pristine-tar`.

Edit `debian/changelog`.

Option 1 (gbp dch):

You can use the following command,

```
gbp dch -a
```

Verify the version in changelog is updated, if the version is not updated, update it manually (if there is an UNRELEASED entry when running this command, the upstream version won't be updated automatically).

Option 2 (dch):

Or use `dch` for it, it will give you a template:

```
dch -v <new-upstream-version>-1
```

Insert this as the first entry:

```
* New upstream Release
```

Then try to build the package like usual and make corrections and changes which are needed.

And add the important changes you made for the new release (removing a patch, inserting a new one, etc). Don't insert changes made by upstream in `debian/changelog`.

Once you are done, change UNRELEASED to unstable (or experimental in case of breaking changes) and push the branches and the tags to the team repository

```
git push -u --all --follow-tags
```

Install the package and see if all dependencies are satisfied for rubygems integration by creating a Gemfile for the library and running `bundle install --local`.

```
$ echo "gem '<library name>', '<version>'" > Gemfile
$ bundle install --local
```

See [steps of updating rails apps like diaspora, gitlab and redmine](#)

Updating packages with API breaking changes

Read this [transitions primer](#) if you are new to handling transitions.

At least <https://SemVer.org> compliance should be assumed (even though many upstreams don't follow it and breaking changes are introduced in even security updates, for example rack 2.0.7 -> 2.0.8 is not compatible) as a compromise (as assuming every change breaks is not practical). This means major version updates of upstream with public API (when version ≥ 1.0) and minor updates of upstream without a public API (when version < 1.0). For example ruby-method-source update from 0.8 to 1.0 and pry update from 0.12 to 0.13, both are breaking changes (sometimes we may be lucky and nothing breaks, but we cannot assume that when upstream explicitly gives a warning by appropriate version bumps).

When uploading such API breaking changes these steps must be followed,

1. Verify autopkgtests of all reverse dependencies and rebuilds of all reverse build dependencies are passing. Use [Packaging/ruby-team-meta-build](#); or upload to experimental and watch the [britney pseudo-excuses for experimental](#)
2. There are two paths you can take for uploading to unstable,
 1. Ideally all breaking packages should be fixed before uploading to unstable. Fixed packages should be uploaded along with the package that introduced API breakage.
 2. Upload to experimental and file bug reports with severity important against all failed packages. It is a good idea to document the list of packages that failed in debian/changelog. Give sufficient time for the maintainers (at least a week for simple fixes or more for complex changes) to fix the broken packages.
3. When all the broken packages are fixed or the provided time is passed, upload to unstable along with all the fixed packages. Add Breaks for all the packages that failed. The bugs against unfixed packages should be raised to serious.

Exceptions (be careful with minor updates also/minor updates broke API in the past): rails, ruby-rack, ruby-doorkeeper, ruby-devise, ruby-graphql, ruby-grape

Note: Bigger transitions like ruby or rails may need collaboration from the whole team and new versions may be uploaded to unstable and broken packages may be fixed later. Try to coordinate with the team on the mailing list.

Requesting Sponsorship

Once your package is ready (or at least, once you think it is), a DD will have to review your changes and upload it to the archive. It is important to understand that reviewing and sponsoring packages is a tedious process, and that most DDs would prefer to do something else instead. So, make sure that you make the best possible use of their time, so they will be happy to sponsor you again. 😊

First, make sure that your package is really in good shape. That means answering at least the following questions:

- Does my package build fine in a clean chroot? The easiest way to test that is using [sbuild](#) or `pbuilder`.
- Is my package lintian-clean? If not, did I explain why it isn't in debian/changelog? Note that real problems should not be hidden by lintian overrides.
- Is my debian/watch file correct? That file will be used by your sponsor to fetch the upstream source, so it is very important that it is correct. Please check that `uscan --download-current-version` does the right thing.
- Is the Git repo up-to-date? Did I commit and push my changes and tags?
- Does my package actually work? Please check in a clean chroot, it's easy to miss some dependencies.
- Is the test suite enabled? are all the tests passing?
- Is autopkgtest passing in a clean chroot? (`$ autopkgtest -B ../foo.deb -- schroot unstable-amd64-sbuild`)
- Are you updating an existing package? Did you check if it is a breaking change? If it is a breaking change as per 🌐 <https://semver.org>, follow [Teams/Ruby/Packaging#Updating_packages_with_API_breaking_changes](https://wiki.debian.org/Teams/Ruby/Packaging#Updating_packages_with_API_breaking_changes)

Once your package is ready, you must:

- Change the distribution from UNRELEASED to `unstable`.
- Request sponsorship by mailing ✉️ debian-ruby@lists.debian.org. Here is an example email:

```
Subject: RFS: ruby-foo 1.2-1, ruby-bar 2.0-1
```

```
Hi,
```

```
The following packages are ready to be uploaded (I also verified the
points listed on http://wiki.debian.org/Teams/Ruby/Packaging#Requesting
```

```
<Describe the reason behind updating the package.>
```

```
Could you please sponsor them?
```

ruby-foo 1.2-1

ruby-bar 2.0-1

Thank you!

[PackageManagement Category](#)[Packaging Category](#)[Ruby](#)

Teams/Ruby/Packaging ([last modified 2025-09-07 12:59:37](#))

- [Teams](#)
- [RustPackaging](#)

Debian Rust packaging team

This page is about [Rust](#) packaging.

- **Team project and packaging repos (on salsa):** [🌐 https://salsa.debian.org/rust-team](https://salsa.debian.org/rust-team)
- **Team mailing list:** [🌐 https://lists.debian.org/debian-rust/](https://lists.debian.org/debian-rust/)
- **Public IRC channel:** #debian-rust on irc.oftc.net
- **Public Matrix room:** [🌐 https://matrix.to/#/#debian-rust:matrix.debian.social](https://matrix.to/#/#debian-rust:matrix.debian.social)
- **How to package a Rust application/crates:** [🌐 https://rust-team.pages.debian.net/book/](https://rust-team.pages.debian.net/book/)
- **How to package a GTK Rust application:** [Gnome/Rust_Packaging](#)

TODO

Get involved!

-  *Get in touch with us via ML or IRC and join rust-team on salsa :)*
-  *Contribute to packaging, bug fixing and testing*
-  *Report bugs, QA pitfalls, and stuff missing from drafts and TODO*

Packages

Packaging of all team maintained [Rust](#) crates is done in a single git [repo](#) ("monorepo"), stored in which are only files under debian/ that need manual inspection; we use [debcargo](#) to automatically generate the rest.

To help, `git clone git@salsa.debian.org:rust-team/debcargo-conf` and follow instructions in `README.rst` to set up the packaging, then you can start packaging individual crates. There are also some tips in `rust_hacks.md`.

If you are a Debian Developer, you can directly apply to join the salsa team to obtain push access; otherwise, we'd appreciate a few merge requests so we can review the initial work and help you get familiar in the process.

For more details about what debcargo does, you can read through the [Debian Rust packaging policy](#).

Binary packages (such as applications) should be maintained also with the rust team if they are available as crates. Please file an ITP for those, not for regular crates. Examples: lsd, exa, bat ...

Programs written in Rust using the gtk-rs framework should rather be maintained with the debian gnome team. Examples: shortwave, podcasts, obfuscate

Toolchain (rustc, cargo)

- Staying up-to-date - see "Maintaining this package" section of [rustc's README.source](#) and "Updating the package" section of [cargo's README.source](#)
- Library unbundling - see "Embedded libraries" section of [rustc's README.source](#)
- ?Runtime soname and stability

General info

Bootstrapping a new distribution

rustc and cargo both Build-Depends on each other. In order to break the loop, we use [Build Profiles](#).

For rustc, see "Bootstrapping" section of [rustc's README.source](#).

For cargo, we currently don't have an automated way to do it. If you need to do this, the following steps should work: install upstream's cargo binary to /usr/local, install the *other* Build-Depends, and then `dpkg-buildpackage -d`.

Porting to a new architecture

Use `sbuild --host=$arch --profiles=nocheck $pkg`, it should Just Work. `$pkg` is either `rustc` or `cargo` to use what's already in the FTP archive, or a path to a `.dsc` of your choice.

If it doesn't work, you might need to also give

`--extra-repository="deb http://incoming.debian.org/debian-build build-unstable main"`.

This is the work-around that most-often fixes things, so just try it blindly even if you don't understand the below. Technically, this is required when (e.g.) there was recently a new GCC or binutils upload, but not all architectures were built and uploaded to the main FTP yet so some architectures are out-of-sync version-wise on Debian unstable.

If it still doesn't work, and `sbuild` complains about missing build-dependencies, and these are core packages not directly related to `rustc` such as `gcc`, `binutils` or perl-related packages, you can:

1. check the relevant `buildd` logs (e.g. [gcc-8](#), [binutils](#)). If it says "Building" for either the source or the target architecture, you will need to wait until the builds are done i.e. if both entries say "Installed". Then the `incoming` trick above should work. That's just an unfortunate aspect of how cross-building with `sbuild` works today, sadly.
2. check the transition tracker (e.g. [perl](#)). If there is a transition going on you'll have to wait until it is done, or alternatively use a Debian testing schroot instead of Debian unstable - but be aware that some necessary packages may be out-of-date there (such as the previous version of `rustc` itself) in which case you'll have to download them and supply them manually to `sbuild` using `--extra-package=`.

If it still doesn't work, check that the binary package `crossbuild-essential-$arch` exists for the desired architecture. If it doesn't, add this to your `~/sbuildrc`:

[Toggle line numbers](#)

```
__1 $crossbuild_core_depends = {
__2     sparc64 => ['gcc-sparc64-linux-gnu:native', 'g++-sparc64-linux-gnu:native', 'd
__3 };
```

changing the entries to match your desired architecture. The valid packages are listed [here](#) and [here](#). If using the latter, you'll also need to add [Debian-Ports](#) to your APT sources.`.list`.

If it still doesn't work, then file a bug to `src:rustc`, `src:cargo`, or [debcargo](#) as appropriate. You can also try [glaubitz's method](#) which involves more manual steps, though it is probably out-of-date by the time you read this.

For more details about `rustc` cross-compiling see "Cross-compiling" section of [rustc's README.Debian](#).

People

We always welcome new contributors, and we are mostly available on IRC for quick questions. Feel free to ping us and join the team:

- Luca Bruno (IRC: kaeso)
- Sylvestre Ledru (IRC: sylvestre)
- Ximin Luo (IRC: infinity0)

Teams/RustPackaging ([last modified 2025-08-24 17:23:57](#))

- [AutomaticPackagingTools](#)

These are tools to create basic Debian packages. For packaging projects in different languages, see [Language-specific guides](#). For tools to create other parts of a system, see [PackagingTools](#).

Contents

1. Language-agnostic tools
2. Language-specific tools
1. Notes
2. Usage
3. See also
1. Meetings

Language-agnostic tools

These generate Debian packages in any language. They often wrap language-specific package generators (see below).

Name	Comments
DebianPkg; debdry	Removed in bullseye
DebianMan; debianize	Part of DebianPkg; lintian-brush
DebianPkg; debmake	Widely used

Language-specific tools

These use language-specific features (like documentation standards or software repositories) to choose better defaults.

Language	Any	Perl	Ruby	Python	Python	Node.js	Haskell	Go
Tool (package name)	 dh-make	 dh-make-perl	 gem2deb	 python3-stdeb	 pypi2deb	 npm2deb ¹	 cabal-debian	 dh-make-golang
Upstream repository	N/A	 CPAN.org ,  Alioth	 Rubygems.org	 PyPI	 PyPI	 npm	 Hackage	varies
Debian guide	list	 guide	guide	guide	guide	guide	guide	 guide
Features								
version	1.20150601	0.89-1	0.21.1	0.8.5-1	0.20160809	0.2.2-1	4.31-1	0.6.0-
cdb's or dh?	dh	dh	dh	dh	dh	dh	cdb's	dh
use DEBEMAIL								
Build-Depends		 ²	 ³	partial ⁴			 ³	
Depends		 ²	 ³	 ⁴	 ³		at build time ³	
Homepage								
short/long descriptions						short only		
DEP8 tests		 (autopkgtest- pkg-perl in pkg- perl mode)	only template ⁶			basic (only require)		dh-go autop
.docs / .examples / etc.		docs + examples	docs only		docs + examples	docs		
debian/copyright	only template ⁶		only template ⁶					
debian/watch	only template ⁶	yes						
git repo creation, pristine- tar, etc.								
debian/upstream/metadata								
ITP mail template		 (use dpt gen-ity)						
recursive								no ⁷

Notes

-  [autodeb-tests](#) has example inputs and outputs for several tools

- Several tools use [DebianMan: wrap-and-sort](#) to tidy up debian/ files
-  [DEP-11](#) could end up being the proper way to generate language-specific dependencies
- deb-fix-build from [DebianPkg: ognibuild](#) will parse build logs for missing {build,test} dependencies to add, repeating until the build succeeds
- for other languages, see...
 - **PHP (PEAR packages):** [Packaging PHP Applications](#), [DebianPkg: pkg-php-tools](#) and [DebianPkg: debpear](#)
 - **Java:** [Java/Packaging](#)
 - **OCaml:**  [oasis2debian](#) and  [opam2debian](#) were attempts to create OCaml packaging tools
 - **Common Lisp:**  [gl-to-deb](#) updates Common Lisp libraries debian packages from the current release found at Quicklisp
 - **R:** [Debhelper packaging](#)
 - **Rust:** [Packaging Rust applications and crates](#)

Usage

- Download upstream package from repository
 - **Perl:** `cpan -g Oxford::Calendar` (does nothing here ...; here it works 😊 maybe cpan config needed?)
 - **Python:** `pypi-download NAME --release VERSION`, f.e. `pypi-download Mako --release 1.0.1`
 - **npm:** `npm install PKGNAME`
 - **Ruby:** `gem fetch GEMNAME`
 - **Haskell:** `cabal unpack --pristine PKGNAME`
 - **ELPA:** user is expected to `git clone` upstream repository
- Debianize an unpacked upstream source
 - **Perl:** `dh-make-perl [make] .`
 - **Python:** `python3 setup.py --command-packages=stdeb.command debianize`
 - **Ruby:** `dh-make-ruby`
 - **npm:** not supported
 - **Haskell:** `cabal-debian --official` (see ?the step-by-step guide)
 - **ELPA:** `dh-make-elpa [make]`
- Debianize (without building source)
 - **Perl:** `dh-make-perl [make] {SOURCE_DIR | --cpan MODULE|DIST}`
 - **Python:** not supported
 - **Ruby:** `gem2deb --only-source-dir GEMNAME|GEMFILE`
 - **npm:** `npm2deb create PKGNAME`
 - **Haskell, ELPA:** not supported, need unpacked source
- Debianize and build source package
 - **Perl:** `dh-make-perl [make] {SOURCE_DIR | --cpan MODULE|DIST} --build-source` or `cpan2dsc MODULE`
 - **Python:** `py2dsc DISTFILE`
 - **Ruby:** `gem2deb --only-debian-source GEMNAME|GEMFILE`
 - **npm, Haskell, ELPA:** not supported
- Debianize and build binary packages
 - **Perl:** `dh-make-perl [make] {SOURCE_DIR | --cpan MODULE|DIST} --build` or `cpan2deb MODULE`
 - **Python:** `py2dsc-deb DISTFILE`
 - **Ruby:** `gem2deb GEMNAME|GEMFILE`
 - **npm, Haskell, ELPA:** not supported
- Debianize, build, install
 - **Perl:** `dh-make-perl [make] {SOURCE_DIR | --cpan MODULE|DIST} --install`
 - **Python:** `pypi-install NAME`
 - **Ruby:** not supported
 - **npm, Haskell, ELPA:** not supported
- Refresh an already created package, moving old files to .bak:
 - **All:** `cme migrate dpkg` or `cme fix dpkg`
 - see  [Managing Debian package files with cme](#)
 - **Perl:** `dh-make-perl refresh`
 - **Python:** not supported
 - **Ruby:** `dh-make-ruby -w`
 - **npm:** not supported
 - **Haskell:** `cabal-debian --official --upgrade`
 - **ELPA:** not yet

See also

- [DebianPkg: alien](#) converts RPM/etc binary packages to deb binary packages
- [DebianPkg: checkinstall](#) tracks files created or modified by your installation script
- [game-data-packager](#) converts non-free game data files to Debian binary packages
- [DebianPkg: ognibuild](#): `ogni info --explain --resolve=apt` tries to auto-discover the buildsystem
- [spec2deb](#) was an attempt to convert Fedora spec files to Debian

Meetings

- [DebConf15](#):
 - [Automatic packaging BoF](#): [notes](#), [video](#)
- Debian Reunion 2022:
 - [Automating Debian Packaging](#): [slides](#), [video](#)

[CategoryPackaging](#) [CategorySoftware](#)

1. just generates files from metadata, doesn't download the upstream source or create a proper source package ([1](#))
2. uses local packages and apt-file to identify packages ([2](#) [3](#))
3. just converts upstream package names found in metadata to Debian package names ([4](#) [5](#) [6](#) [7](#) [8](#))
4. uses apt-file with a fallback on module name conversion ([code](#)) ([9](#) [10](#))
5. relies on `dh-elpa`'s dependency detection ([11](#) [12](#))
6. provides a file that is not customized per-package ([13](#) [14](#) [15](#) [16](#))
7. The `estimate` command can tell which parts of the dependency chains are missing from Debian ([17](#))

AutomaticPackagingTools (last modified 2026-01-02 11:10:52)

- [AndroidTools](#)

Contents

1. [Building apps with these packages](#)

2. [Communication Channels](#)

3. [Joining the Android Tools Team](#)

4. [Package naming scheme](#)

5. [Source package structure](#)

6. [Updating the source packages](#)

7. [Upstream repository of tools in Android SDK](#)

8. [Android's upstream version names](#)

9. [Android SDK version declarations](#)

10. [Original android-tools package](#)

11. [Why is this so complicated?](#)

12. [Needs doing](#)

13. [Knowledge dump](#)

14. [See Also](#)

This page is a gathering place for information about the  [android-tools packaging team](#), which is focused on packaging the Android development tools for Debian. There are also some packages which help run Debian in a chroot on Android. The goal of this team is to get as much of the Android SDK and development tools into Debian as possible. There are many advantages to having the SDK and tools in Debian, rather than relying only on the Google distributions:

- easy install and update channel that all Debian users already know
- automatic trustworthy downloads, no need to verify hash sums
- eliminate need for insecure wrapper scripts, like `./gradlew`
- trivial install for specific tools, like `adb`, `fastboot`, etc.

To read more about the rationale behind this work, see this blog post:  <https://guardianproject.info/2015/04/30/getting-android-tools-into-debian/>

To communicate with this team, join our low traffic mailing list,  android-tools-devel@lists.ath.cx and on the IRC channel  [#debian-android-tools](#) ( [webchat](#)). You can also join the IRC Channel through  [Matrix](#)



The binaries for the Android SDK downloadable from Google have a proprietary license but the source code is free software so Debian is packaging it. Not all Android SDK packages can be installed from Debian, some never will be in Debian because they are too specific to Android. Sylvain Beucler's  [libre Android rebuilds](#) and/or Google's non-free binaries can also be used with the Debian Android SDK.

Building apps with these packages



If you're just starting out with building apps, we suggest that you first read the [Introduction to build packages with Debian's Android SDK](#).

If you are already familiar with how to use these tools, you might want to look these brief instructions below. Here are the steps for building Android apps using Debian's Android SDK on Stretch.

1. `sudo apt install android-sdk android-sdk-platform-23`
2. `export ANDROID_HOME=/usr/lib/android-sdk`
3. In `build.gradle`, change `compileSdkVersion` to 23 and `buildToolsVersion` to 24.0.0
4. run `gradle build`

The Gradle Android Plugin is also packaged. Using the Debian package instead of the one from online Maven repositories requires a little configuration before running Gradle. In the `buildscript {}` block:

- add `maven { url 'file:///usr/share/maven-repo' }` to repositories
- use `compile 'com.android.tools.build:gradle:debian'` to load the plugin

Currently there is only the target platform of API Level 23 packaged, so only apps targeted at android-23 can be built with only Debian packages. We will add more API platform packages via backports afterwards. Only Build-Tools 24.0.0 is available, so in order to use the SDK, build scripts need to be modified. Beware that the Lint in this version of Gradle Android Plugin is still problematic, so running the `:lint` tasks might not work. They can be turned off with `lintOptions.abortOnError` in `build.gradle`. Google binaries can be combined with the Debian packages, for example to use a different version of the platform or build-tools.

In stretch-backports (and soon testing), the Gradle Android Plugin is patched to work with Debian's Android SDK. It detects what versions of API Levels and Build-Tools are available and you no longer need to modify the build scripts. In order to build apps, do the following:

1. `sudo apt install android-sdk android-sdk-platform-23 android-sdk-helper`
2. `export ANDROID_HOME=/usr/lib/android-sdk`
3. `gradle build --init-script /usr/share/android-sdk-helper/init.gradle`

Thus, `init.gradle` forces Gradle to use the Gradle Android Plugin in Debian and the plugin will do the rest.

Here's an example project:  <https://gitlab.com/Matrixcoffee/hello-world-debian-android>

Setting up the Android emulator

It is possible to install the proprietary binary packages to run the Android emulator on Debian. These package binaries have a proprietary license, but are built from free software sources. It isn't yet confirmed that they are built from 100% free software source code, but they seem to be.

```
sudo apt install default-jre sdkmanager
export ANDROID_HOME=/opt/android-sdk
sudo mkdir -p $ANDROID_HOME
sudo chown $USER $ANDROID_HOME
sdkmanager "emulator" "cmdline-tools;latest" "platforms;android-31" "platform-to
$ANDROID_HOME/cmdline-tools/latest/bin/sdkmanager "system-images;android-31;defa
```

```
$ANDROID_HOME/cmdline-tools/latest/bin/avdmanager create avd \
--tag default --package "system-images;android-31;default;x86_64" --sdcard 64M
--device "Nexus 5" --name Nexus_5_API_31
/opt/android-sdk/emulator/emulator @Nexus_5_API_31
```

Communication Channels

There are a number of ways that people working on packaging Android Tools communicate. Here is a list:

- Team chat room: `#debian-android-tools`  [OFTC IRC](#)  [\[matrix\]](#) ( [webchat](#)).
- Team email list:  android-tools-devel@lists.aliases.debian.org
- Some Java packages are essential to Android, so we also talk here:  [#debian-java](#)  [\[matrix\]](#) ( [webchat](#)).
- Team blog:  <https://android-tools-team.pages.debian.net/blog/>

Joining the Android Tools Team

We want make it as easy as possible for anyone to get involved in the Android Tools Team, and contribute in ways that they think are important. It is important to note that some of the packages here are quite complicated, and the Android SDK is a very large project. That means that changes must be discussed with the team before they can be committed and pushed. That discussion can happen in anywhere, including merge requests.

So here is the Android Tools Team policy for granting membership to the team, based on the  [GitLab user roles](#):

- any request to become a member is granted at least "Reporter" level
- any Debian Developer or previous contributor is granted at least "Developer" level
- requests from anyone else must be posted to  <https://salsa.debian.org/android-tools-team/admin/issues> and are approved by "lazy consensus"
- specific requests for "Master" level must also be posted to  <https://salsa.debian.org/android-tools-team/admin/issues> and are approved by "lazy consensus"

This is based on "Lazy consensus": if no one objects to a request within one week or so, then any Master/Owner is free to grant the request. So silence is considered approval, but not automatic approval. An existing Master/Owner must actually grant the request after the waiting period.

standalone vs. interdependent packages

Mostly, our procedures are about which packages are standalone and can be freely uploaded, versus which packages are interwoven and must be uploaded in coordinated batches (e.g. the Android SDK). *repo* or *enjarify* is a standalone package, for example. The interwoven ones basically all follow the Android SDK git repo naming scheme for the source package names:

- *android-platform-**
- *android-framework-**
- *android-sdk-meta*

Package naming scheme

The naming scheme for android-tools packages is as follows:

- source package names are named after the git repository, prefixed by *android-* (e.g.  [android-platform-system-core](#),  [android-platform-system-extras](#),  [android-platform-build](#),  [android-platform-frameworks-base](#), etc.)
- binary packages of utilities that run on Debian directly are named after the utility itself (e.g.  [zipalign](#),  [aapt](#), etc.)
- shared libraries that are only used by android-tools packages are named after the library, without a ABI version number in the package name, and prefixed by *android-*. (e.g.  [android-libhost](#),  [android-libcutils-dev](#), etc. *In the Google builds, these are built as static libraries, and linked statically into each binary. In the Debian builds, they are built as shared libraries and installed into /usr/lib/android.*

Source package structure

The structure of each source package is documented in the README.source of each source package for this team:

-  <https://salsa.debian.org/android-tools-team/android-permissions/tree/master/debian/README.source>
-  <https://salsa.debian.org/android-tools-team/android-platform-build/tree/master/debian/README.source>
-  <https://salsa.debian.org/android-tools-team/android-platform-frameworks-base/tree/master/debian/README.source>
-  <https://salsa.debian.org/android-tools-team/android-platform-frameworks-native/tree/master/debian/README.source>
-  <https://salsa.debian.org/android-tools-team/android-platform-system-core/tree/master/debian/README.source>
-  <https://salsa.debian.org/android-tools-team/android-platform-system-extras/tree/master/debian/README.source>

Updating the source packages

The packages in this team are structured somewhat unusually because we are trying to keep the source packages as close as possible to the upstream source organization while still working in a Debian way. Google builds the Android OS and SDK as one giant thing, something like 10 gigs of source code. But the code is broken up into many different git repos that are coordinated using the Android team's tool called `repo`. Also, there are lots of shared libraries used between the various Android SDK tools, but since everything is always built together, those shared libraries are unversioned.

All this means that it is essential that any Android SDK package is only built against the exact same version of all its `Build-Depends` and only uses the exact same version of an Android SDK package as a `Depends`. To achieve that, we use the `substvars` variable in dependency declarations: (`>= ${source:Version}`).

Additionally, because of this and the circular dependencies, it is important to upload updates in the correct order. Some packages also have to be uploaded using a multi-stage method. Here is the estimated upload order:

- Stage 1
 - android-framework-\$apilevel
 - android-platform-external-boringssl
 - android-platform-external-jsilver (Usually no update needed)
 - android-platform-external-libselenium (should be switched to android-platform-external-selenium from 8.0 release)
 - android-platform-external-libunwind
 - android-platform-frameworks-data-binding
 - android-platform-frameworks-native
 - android-platform-libcore
 - android-platform-tools-analytics-library
- Stage 2
 - android-platform-external-doclava
 - android-platform-system-core~stage1
 - android-platform-tools-base
- Stage 3
 - android-platform-art
 - android-platform-development
 - android-platform-frameworks-base
 - android-platform-libnativehelper
 - android-platform-system-extras
 - android-platform-system-tools-aidl
- Stage 4
 - android-platform-art
 - android-platform-build
 - android-platform-dalvik
 - android-platform-system-core
- android-platform-dalvik (Needs `android.jar` of the latest API Level)

Each packages belonging to the same stage are unrelated to each other and can be uploaded in any order.

This order only represents one possible way of updating all of them to a new major version. In practice, you can delay or advance any packages as long as the build-dependencies satisfy.

Upstream repository of tools in Android SDK

Tools in Android SDK come from various repositories. Here is a list of tools consisting the entire Android SDK, with each tool followed with the corresponding upstream repository name.

SDK Tools

- Android Device Monitor
 - ddms: [🌐 platform/tools/swt](https://github.com/android/platform/tools/swt) (deprecated)
 - hierarchyviewer: [🌐 platform/tools/swt](https://github.com/android/platform/tools/swt) (deprecated)
 - monitor (Depends on hprof-conv, systrace.py, logcat)

- traceview:  [platform/tools/swt](#) (deprecated)
- android (Including SDK Manager, AVD Manager) (Deprecated)
- apkanalyzer:
 - git clone  [platform/tools/base](#)
 - source code  [studio-master-dev/apkparser/analyzer/](#)
- avdmanager:
 - git clone  [platform/tools/base](#)
 - source code  [sdklib/internal/avd/](#)
 - D8/R8:
 - git clone  [platform/external/r8](#)
 - source code  [android/tools/r8](#)
- draw9patch:  [platform/tools/base](#)
- emulator:  [platform/sdk](#)
- jobb:
 - git clone  [platform/tools/base](#)
 - source code  [studio-master-dev/jobb/](#)
- hidl-gen:
 - git clone  [platform/system/tools/hidl](#)
 - source code  [refs/heads/master](#)
- lint:
 - git clone  [platform/tools/base](#)
 - source code  [studio-master-dev/lint/](#)
- mksdcard:  [platform/sdk](#)
- monkeyrunner:
 - git clone  [platform/tools/swt](#) (deprecated)
 - source code  [master/monkeyrunner/](#) (deprecated)
- proguard:  <https://github.com/Guardsquare/proguard> (Already in Debian)
- screenshot2:  [platform/tools/base](#)
- sdkmanager:
 - git clone  [platform/tools/base](#)
 - source code  [sdklib/tool/sdkmanager/](#)
- uiautomatorviewer:  [platform/tools/swt](#) (deprecated)

SDK Platform-tools

- adb:  [platform/system/core](#)
- dmtracedump:  [platform/art](#)
- etc1tool:  [platform/development](#)
- fastboot:  [platform/system/core](#)
- hprof-conv:  [platform/dalvik](#)
- systrace.py  [Catapult Project](#)

SDK Build-tools

- aapt/aapt2:  [platform/frameworks/base](#)

- aidl:  [platform/frameworks/base](#)
- apksigner:
- bcc_compat:  [platform/frameworks/compile/libbcc](#)
- dexdump:  [platform/art](#)
- dx:  [platform/dalvik](#)
- jack:  [toolchain/jack](#)
- jill:  [toolchain/jill](#)
- llvm-rs-cc:  [frameworks/compile/slang](#)
- mainDexClasses:  [platform/dalvik](#)
- apksigner:  [platform/tools/apksig](#)
- split-select:  [platform/frameworks/base](#)
- zipalign:  [platform/build](#)

Reverse engineering tools

[DebianPackage: dexdump](#) [DebianPackage: apktool](#) [DebianPackage: gplaycli](#)
[DebianPackage: androguard](#) [DebianPackage: enjarify](#) [DebianPackage: dmtracedump](#)

Other Tools

- Gradle Plugin for Android:  [platform/tools/base](#)
- repo:  [tools/repo](#)
- signapk:  [platform/build](#)
- signtos:  [platform/build](#)
- ziptime:  [platform/build](#)

Android's upstream version names

There are many  [naming](#)  [schemes](#) for versions in Android, and none are used everywhere. For the Android OS itself, there are three schemes: version names, like *Gingerbread* or *Kitkat*; version numbers, like 2.3.7 or 4.4.2; and, "API level" numbers, like `android-10` or `android-19`. None of these line up with each other. For example, the *Jelly Bean* name spans 4.1 through 4.3.1 and `android-16` through `android-18`. The version numbers and API level also do not line up, with `android-14` covering 4.0.1 - 4.0.2, `android-15` covering 4.0.3 - 4.0.4, but `android-16` covers all of 4.1.x.

The madness doesn't end there. Next up we have SDK version numbers, which are like 20, or 23.0.2. Part of the SDK includes the `build-tools` package, which has its own version numbers, which are like 18.1.1 and 20.0.0, but these do not line up with the SDK version numbers. The `platform-tools` seems to have its own versioning scheme also, but it is not really exposed. Then there are the NDK release numbers, which are like `r8b` or `r10`, and they also do not line up with anything else.

In the git repo, there are a mishmash of tags and branches that do not represent all of these various versions. There does seem to be consistent tagging of the OS releases, you can see those listed under  [Source Code Tags and Builds](#). There are a few branches that seem to line up to the SDK version numbers, like `tools_r21` and `tools_r22.2`, but there is not a complete set of those branches.

example versions

Android SDK Tools seems to have it's own versioning scheme, since the major version is ahead of the SDK major version:

- v24.2.0
- v24.1.0
- v23.0.0

Android SDK Build-tools seems to follow the SDK major version, but have its own minor and micro versions:

- v22.0.1
- v21.1.2
- v21.0.0

Android SDK Platform-tools seems to follow the SDK major version only.

- v22
- v21
- v20

You can find the official Google package names and versions in the index XML files that the `android` tool downloads when doing updates:

-  <https://dl.google.com/android/repository/repository-10.xml>
-  <https://dl.google.com/android/repository/repository-11.xml>
-  <https://dl.google.com/android/repository/repository-12.xml>
-  <https://dl.google.com/android/repository/repository2-1.xml>
-  <https://dl.google.com/android/repository/addon.xml>
-  <https://dl.google.com/android/repository/addon-6.xml>
-  <https://dl.google.com/android/repository/sys-img/android/sys-img.xml>
-  <https://dl.google.com/android/repository/sys-img/x86/addon-x86.xml>

Android SDK version declarations

However, the exact version number of Android SDK toolsets can be found in a `source.properties` under each directories. Here are the source locations of each version declaration:

- SDK Tools:  https://android.googlesource.com/platform/sdk/+/master/files/tools_source.properties
- SDK Platform-tools:  https://android.googlesource.com/platform/development/+/master/sdk/plat_tools_source.prop_template
- SDK Build-tools:  https://android.googlesource.com/platform/development/+/master/sdk/build_tools_source.prop_template
- Gradle plugin:  <https://android.googlesource.com/platform/tools/buildSrc/+/master/base/version.gradle>

Which matches the [official release notes](#).

Note that according to the source code the major version number of Platform-tools and Build-tools is simply the API Level, which is defined [here](#). But for SDK Tools it follows its own version pattern and does not relate to the API Level.

Some tools tracks their own version and do not follow the SDK version:

- adb
- dx

Original android-tools package

This team started with the [android-tools](#) package, which is a manually assembled collection of source code needed to build `adb`, `fastboot`, etc. This approach is not maintainable in the long run, especially as we work to add the whole Android SDK. So the *android-tools* package is deprecated, and will be replaced by the collection of packages described above.

Why is this so complicated?

Packaging all of these Android SDK Tools is so complicated because the source code is organized into semi-arbitrary "projects" that are split up between many different git repos. Those git repos are then all checked out at the same time using Google's crazy `repo` tool. Then the entire Android OS and SDK are built using a single, unified build system that requires something like 8 gigs of source code be downloaded. On top of that, one of the more confusing aspects of this whole project is that the same source code base is used to build the OS and the SDK. Those two can start with the same source code files, and end up with very different resulting binaries.

shared libraries within the Android SDK packages

The Android SDK has a number of private libraries that are shared between many different utilities that are part of the Android SDK. Upstream statically links them into each executable. That didn't seem very Debian-ish, so instead they are structured as private shared libraries. Nothing should link to these `android-lib*` shared libraries that is not part of the Android source. This private shared library arrangement allows for security patches in the android shared code to be applied without having to rebuild everything.

All of the `android-*` packages must be updated to the latest version at the same time. It is more unpredictable to have `android-*` packages at different upstream versions since no one is running that configuration, and upstream does not do anything to support that (e.g. no versioned ABI, etc). Therefore, the ABI is guaranteed to be compatible since they'll all be built together. In practice, this means that the process of updating to the latest upstream version has to start from the most core packages, then progress to the ones that depend on it.

A single source package might make some things easier, but it would be about 8-12 gigs in size. That would make it very difficult for many people to work with that source package.

Needs doing

There is lots left to do before someone can do Android development using only official Debian packages. The best way to get started is to [join the email list](#) and ask there. We're also in IRC

at [#debian-android-tools](#). Here are some general topics that need work:

- build an app using only the Debian packages for gradle, gradle plugins, build tools and platform tools (i.e. `apt install android-sdk` then `gradle build`), then file [bug reports](#) on anything that isn't working
- figure out packaging scheme for various android "platforms", i.e. `/opt/android-sdk/platforms/android-*` and `/opt/android-ndk/platforms/android-*`
- build "android support" jars
- figure out how to represent Android NDK clang "toolchains" in Debian, i.e. `/opt/android-sdk/toolchains`
- help Google integrate Android's QEMU into upstream (since Google has started to do this, I think this is the best approach to getting the Android emulator into Debian)

fdroidserver

[fdroidserver](#) is the tool suite for managing FDroid app repos and making release builds. It uses the Android SDK and NDK to make the builds and work with APKs. There are three notable milestones for `fdroidserver` packaging needs:

1. everything to create and manage app repos **DONE!**
2. everything to make pure Java builds
3. everything to make native builds

low-hanging fruit

Here are a couple of easy tasks to do that will further this effort along:

- Continuous integration tests
- build Android apps using `apt install android-sdk` on stretch and newer
- package these existing SDK packages for other distros (Arch, Fedora, NixOS, Homebrew, ? MacPorts, Cygwin, etc)
- get answer from Google on how the SDK versions are marked in the source git repos (i.e. what tag/branch is SDK v23.0.2? And what tag/branch is `build-tools` v20.0.0?)
 - Is <https://source.android.com/source/code-lines.html#terms-and-caveats> any helpful? Among other things, it says:

A release corresponds to a formal version of the Android platform, such as 1.5, 2.1, and so on. Generally speaking, a release of the platform corresponds to the version in the `SdkVersion` field of `AndroidManifest.xml` files and defined within `frameworks/base/api` in the source tree.
 - seems to vaguely contribute to understanding, but no concrete answers for which commit ID to build for SDK release v23.0.2. -- [HansChristophSteiner](#) 2014-10-07 19:51:57
- [android-platform-system-extras](#) package, replacing `android-tools-fsutils` package.
- update all source packages to the latest SDK release (v23.0.2, provided you can find the right tag or branch)
- [Jack](#) Deprecated
- [Android Support Library](#)
- The rest of an "Android platform" ("`android.jar`" is done).

- [DebianPackage: android-platform-dalvik](#) needs update to include "shrunkedAndroid.jar" generated with "android.jar"
- ~~[android-platform-tools-sw](#)~~ and probably though unlikely ~~[android-platform-tools-buildsrc](#)~~
- [android-platform-frameworks-compile-libbcc](#) for bcc_compat, llvm-rs-cc, etc.
- "renderscript" directory under "build-tools"

Knowledge dump

For lack of a better name, this basically contains some of the things we have learnt and some of the general ways in which we work with android packages.

See Also

- [Fedora package for android-tools](#)
- [Fedora package for abootimg](#)
- Arch Linux packages up the Google binary packages <https://wiki.archlinux.org/index.php/Android>

[CategoryPackaging](#)

AndroidTools (last modified 2025-04-07 11:30:21)

- [Gnome](#)
- [Git](#)

Overview

The **Debian GNOME Team** uses **git-buildpackage** to make working with the packaging repositories easier. **git-buildpackage** is not required if you only want to build a package or for more contributions. Please do use **git-buildpackage** to import new versions.

The [package layout](#) is DEP-14 style with full upstream source and upstream history without upstream tags and with **pristine-tar**.

Browse the repos

 <https://salsa.debian.org/gnome-team>

Contents

1. Overview

1. Browse the repos

2. Recommended setup

1. Install tools

2. Set up git urls

3. Do initial clone

3. Use git

4. Use git-buildpackage

5. Repo layout

6. Package new upstream version

7. UNRELEASED

8. debian/changelog

9. Make a release

10. Other information

1. Initial packaging

2. Warning for stable updates

3. History

4. pristine-tar branch not found

5. Archived packages

6. Git hooks

7. Misc. cases

8. Packages named differently from their git repos

9. Our default gbp.conf

11. upstream-vcs-tag

1. To do

Recommended setup

Install tools

```
sudo apt install debhelper git-buildpackage gnome-pkg-tools
```

Set up git urls

Git has a feature to rewrite urls. We can use this to create handy shortcuts that require less characters to type and will make migration easier if your username ever changes (for instance, if you become a Debian Developer and no longer need an Alioth -guest account) or if the repo ever moves (like happened with the 2018 Debian Gitlab migration).

You can set up and change git url handling in your `~/.config/git/config` (or with the `git config` command).

```
[url "git@salsa.debian.org:gnome-team/"]
    insteadof = pkg-gnome:
```

Do initial clone

For this example, we're going to grab the `gnome-text-editor` source.

It's recommended to use a separate directory for your code so that it doesn't clutter your home directory. `pkg-gnome` is a good name for that. (No example shown for this step).

Type the following commands in to your terminal to check out the Debian packaging and add a **remote** named **upstreamvcs** to track the upstream GNOME repo.

```
gbp clone pkg-gnome:gnome-text-editor --add-upstream-vcs
cd gnome-text-editor
```

The `--add-upstream-vcs` option requires `git-buildpackage` 0.9.29 or higher which is available in Debian Testing or Ubuntu 22.10. If you are using Ubuntu 22.04 LTS or Debian 11, use these commands instead:

```
gbp clone pkg-gnome:gnome-text-editor
cd gnome-text-editor
```

```
git remote add upstreamvcs https://gitlab.gnome.org/GNOME/gnome-text-e
```

The upstreamvcs repo is pulled from the Repository field of debian/upstream/metadata in each of our packaging repos. Several of our repos don't yet have debian/upstream/metadata so that will need to be added for the --add-upstream-vcs feature to work.

Use git

A full git tutorial is beyond the scope of this wiki page. See the  [Pro Git book](#) for one popular guide.

Use git-buildpackage

Please read the  [manual](#) to learn how to use git-buildpackage.

Repo layout

The Debian GNOME team does not use the current default git-buildpackage layout for its repositories. Instead, a  [DEP-14 style layout](#) is used. debian/gbp.conf is provided to enable git-buildpackage to work without manual adjustment.

The primary packaging branch is named **debian/latest**. The parallel branch for upstream source without the debian/ directory is named **upstream/latest**

pristine-tar is used. Also, full upstream source with full upstream commit history is integrated (automated with the help of git-buildpackage).

The latest version should always use the **debian/latest** and **upstream/latest** branches. If the latest version is not suitable for upload to unstable, feel free to create a branches like **debian/buster** (where buster is the targeted Debian release even if it is initially upload to unstable). For the corresponding upstream branch, it should generally be named **upstream/3.26.x** if it is tracking upstream's 3.26 series.

Package new upstream version

Make sure you start from the **debian/latest** branch with no uncommitted changes.

```
gbp pull
```

Check the git log for any changes that have been made since the last upload. If there are changes, run `gbp dch; git add debian/changelog; git commit -m "Update changelog"`. Continue with the actual import

```
git fetch --all
gbp import-orig --uscan
```

This will update the `pristine-tar`, `upstream/latest` and `debian/latest` branches.

Make sure the package still builds, then push the branches and the single upstream tag. `gbp push` does all of this for us, except that it [Open in git-buildpackage/0.9.10: #908204: git-buildpackage: cannot use gbp push without tagging a release: only pushes](#) since the last tag so we need to manually push our `debian/latest` branch.

```
git push origin debian/latest
gbp push
```

Note that currently the Debian GNOME team does not wish to push all upstream tags, only tags with the `debian/` and `upstream/` prefix.

UNRELEASED

Please keep the `debian/changelog` series as `UNRELEASED` until the version is actually being uploaded.

debian/changelog

We use `gbp dch -- debian/`. In general, that means you usually shouldn't directly edit `debian/changelog` except before importing a new release and when finalizing the changelog for upload. Another case for updating `debian/changelog` is when adding Breaks. (Breaks and Replaces are required when moving files to a different package but the full file name stays the same.)

We currently use the default short mode for `gbp dch`. That means that only the "subject" line of git commits is used for creating the `debian/changelog` entries. Add `Gbp-Dch: Full` on a blank line at the end of your commit message if you prefer the full commit message be used instead. Or if your commit is too trivial to be included in `debian/changelog`, you can use `Gbp-Dch: Ignore`.

 [gbp-dch manual](#)

Make a release

For this example, the Debian release is named `3.26.2-1`. Here we finalize the changelog, make a release tag, and push the latest packaging and release tag.

```
gbp dch -- debian/
```

Review debian/changelog and adjust if needed.

```
git add debian/changelog
git commit -m "Update changelog"
git push origin debian/latest
```

Then, *if you are the uploader* (skip this step if you are being sponsored), finalise the changelog for the intended distribution:

```
dch -D unstable --release "" # or experimental
gbp buildpackage --git-tag-only
git push origin debian/latest debian/3.26.2-1
```

Finally build the package for upload (🌐 [source-only uploads](#) are preferred except for packages that need to go through the new queue) and upload with dput.

Other information

Initial packaging

This example is about the `gnome-shell-extension-desktop-icons-ng` package whose upstream URL does not start with `git@ssh.gitlab.gnome.org:GNOME/`. Initially we want to grab all commits up to the one represented by the upstream tag `0.15.0`. A corresponding `orig.tar.gz` file already exists.

```
git clone git@gitlab.com:rastersoft/desktop-icons-ng.git -b 0.15.0
cd desktop-icons-ng
git checkout -b upstream/latest
git checkout -b debian/latest

gbp import-orig --pristine-tar ../gnome-shell-extension-desktop-icons-ng-0.15.0
--upstream-vcs-tag=0.15.0 --upstream-branch=upstream/latest --debian-b
```

- Add the `debian/` folder including [debian/gbp.conf](#) and `debian/watch`.
- Create a blank salsa project on the 🌐 [website](#)

```
git remote set-url origin pkg-gnome:shell-extensions/gnome-shell-extensions
git push origin debian/latest upstream/latest pristine-tar
git push origin upstream/0.15.0
```

Warning for stable updates

When branching from an old release, be sure to run a thorough diff against the debian tarball to ensure file permissions are correct and there aren't obsolete files. This is needed because the svn conversion process wasn't 100% precise.

History

The Debian GNOME Team switched most of their packages from svn to git in December 2017. For some background notes on that, see [/SvnConversion](#).

pristine-tar branch not found

If you see this error when trying to run `gbp buildpackage`

```
$ gbp buildpackage
gbp:warning: Pristine-tar branch "pristine-tar" not found
```

Just run these commands:

```
git checkout pristine-tar
git pull pristine-tar
git checkout debian/latest
```

Archived packages

When packages are removed from unstable, log into the Salsa page for the repo. Click Settings > General. Open Advanced and click Archive Project.

Here are the archived packages:

 <https://salsa.debian.org/groups/gnome-team/-/archived>

Git hooks

Commits are announced in the `#debian-gnome` IRC channel and to `tracker.debian.org` subscribers subscribed to the Vcs keyword.

Misc. cases

- intltool upstream uses bazaar so we don't include upstream commit history for it.

Packages named differently from their git repos

These packages have different git repo names than their Debian package names. (In particular, Gitlab does not accept the `+` character in repo names.)

- atk1.0
- atkmm1.6
- bijiben (upstream's git repo is gnome-notes)
- clutter1.0
- clutter-gst-3.0
- gdm3
- glib2.0
- glibmm2.4
- gnome-games-app
- gtk+2.0
- gtk+3.0
- libxml++2.6
- libsigc++-2.0

Our default gbp.conf

```
[DEFAULT]
pristine-tar = True
debian-branch = debian/latest
upstream-branch = upstream/latest

[buildpackage]
sign-tags = True

[dch]
multimaint-merge = True

[import-orig]
postimport = dch -v%(version)s New upstream release; git add debian/changelog
upstream-vcs-tag = %(version%~%. )s

[pq]
patch-numbers = False
```

upstream-vcs-tag

The most common format is 3.26.2. For those packages, use this line in `debian/gbp.conf`:

```
upstream-vcs-tag = %(version%~%. )s
```

Another common format is v3.26.2. For those packages, use the prefix `v`:

```
upstream-vcs-tag = v%(version%~%. )s
```

Some packages use a different prefix like `libdazzle-3.26.2`. That needs just a slight adjustment:

```
upstream-vcs-tag = libdazzle-%(version%~%. )s
```

Some upstreams still use CVS style tags. (CVS did not support using the `.` character in release tags). This format is perhaps a bit "ugly" so you can ask upstream to consider changing to a more modern format. `gbp-import-orig` allows rewriting the `_` characters like this:

```
upstream-vcs-tag = GNOME_SETTINGS_DAEMON_%(version%.%_)s
```

To do

1. Add a  [myrepos](#) config to make it easier to work with a large number of repositories
2. Document workflow for git snapshots. (Basically fetch, tag locally, create a tarball, and then use `gbp import-orig --upstream-vcs-tag= /path/to/tarball`)
3. Add protected tags to prevent pushing tags without a `debian` or `upstream` prefix.

[CategoryPackaging](#)

- [PkgQtKde](#)

Debian Qt KDE packaging team

This wiki page is intended to keep information necessary to the team. If you want to contribute, join the team: visit  <https://qt-kde-team.pages.debian.net/>.

Release Plans

- [Forky \(Debian 14\)](#)
- [Trixie \(Debian 13\)](#)
- [Bookworm \(Debian 12\)](#)

Documentation for Packagers

- [Using dch within kde-pkg](#)

Pages that keep track of current development

- ...

Likely outdated

- ...

ToDo list

- [KDEToDo](#)

- [PkgMate](#)
- [GitPackaging](#)

Contents

- 1. [Packaging guidelines](#)
- 2. [Workflow guidelines](#)
- 3. [Documentation](#)
- 4. [Working on existing packages with Git for newbies](#)
 - 1. [Uploading new upstream version to existing repository](#)

Packaging guidelines

- Packages should use Git, as mentioned above.
- Upstream sources should be obtainable via `debian/rules get-orig-source`
- Packages are build from tarballs that upstream provides at `http://pub.mate-desktop.org/releases/`
- The *master* branch should only contain the `debian/` directory.
- This means no direct changes to the source can be made in Git. Instead a patch manager (`quilt`) must be used for patch management.
- The maintainer field should be set to
MATE Packaging Team <pkg-mate-team@lists.alioth.debian.org>
- The Git repository should be hosted on Alioth, under the pkg-mate project. It should forward commit messages to `pkg-mate-commits@lists.alioth.debian.org` and `package_cvs@packages.qa.debian.org`
- You can use the `./setup-git-repository` script in `/git/pkg-mate` (on host `git.debian.org`) to create the bare repository with commit messages enabled. The repository should be named as the source package name for the message forwarding to work (e.g. the repository for source package `mate-terminal` is named `mate-terminal.git`).

```
$ ssh -l<alioth-user> git.deban.org
$ cd /git/pkg-mate
$ ./setup-git-repository <src:package-name>
```

- The control file should use the `Vcs-Git` and `Vcs-Browser` tags.

Workflow guidelines

- **One change per commit.** This is very important, it eases review, cherry picking, bisecting (and thus debugging) and backporting.

- Do not commit `debian/changelog` along with the changes. This practice makes cherry-picking and backporting changes unnecessarily hard. The changelog is generated with [DebianMan: git-dch\(1\)](#) at the time of upload.
- The commit message should have a short (<78 characters) summary of the change followed by a newline. After that, you can elaborate on the change. The summary is treated specially by various tools like [DebianMan: git-dch\(1\)](#), or the commitdiff mailer.
- When releasing a package, the changelog gets created via

```
$ git dch --auto
```

This command generates a changelog stanza from the Git commits for the current package version. Thus, choose your Git commit messages well--as if they were entries in `debian/changelog`. They actually are!!!

- Then simply run `dch -r` which will mark the package from UNRELEASED to released (normally `unstable`).
- Now commit the generated changelog stanza (editing and fine-tuning it is allowed) to Git and (e.g. for Debian release 1.6.1-1 of the `mate-common` package) use a commit message like

```
git commit -a -m "upload to unstable (debian/1.6.1-1)"
```

- Build (e.g. with `sbuild`) and upload the package to Debian unstable.

```
$ cd <package-dir>
$ debuild -uc -us -S -Zxz
$ cd ..
$ sbuild -sAd unstable <package>.dsc
```

- If you have not used `lintian` for checking the package integrity, then it is really time for it now:

```
$ lintian -iIE --pedantic --show-overrides --color auto <package>_<
```

- Only if `lintian` is considerably silent and content with your package effort, the package is suitable for an upload to Debian unstable. Do not upload a package that `lintian` still reports errors for.
- Now, sign the package (using `debsign` and (if you are a DD or a DM) upload the package (using e.g. `dput`):

```
$ debsign <package>_<arch>.changes
$ dput <package>_<arch>.changes
```

- For uploads to NEW: be patient and wait for the FTP masters to accept the uploaded package... (if they reject it, mark the package as UNRELEASED again and fix what the FTP team requests from you)
- Tags should be created (and signed) by the uploading DD, in the case of the debian/* tags in the master branch, and by the person importing the upstream sources in the case of upstream tags:

```
git tag -s debian/1.6.1-1 -m "Debian release 1.6.1-1"
```

- After each upload, the first commit should be creating a new changelog entry, to ease testing of unreleased packages.

Documentation

-  [Using git on Alioth](#)

Working on existing packages with Git for newbies

- As a very basic introduction, first read [PackagingWithGit](#).
- locate the packaging branch on the  [pkg-mate Git index](#)
- Note the metadata just above the shortlog, it contains 2 URLs, one of them starting with `git://`
- then  [git-clone\(1\)](#) to get a copy of the repository. E.g.

```
git clone git://git.debian.org/pkg-mate/<src:package-name>.git
```

- get the upstream source tarball belonging to that package:

```
$ debian/rules get-orig-source
```

- do changes to that branch, build the package:

```
$ debuild -uc -us -Zxz
```

... and test your changes. Commit early and often! Do regular pushes to the origin Git repos on Alioth.

```
$ git push origin
```

- If someone else has committed while you are working on the branch, you need to integrate his/her changes. For this you have 2 options:
- if your local changes are rather minor and clean, rebase when using  [git-pull\(1\)](#):

```
$ git pull --rebase origin
```

- only if you rather don't want to break your history because you think that your changes are rather large and are likely to interfere with the other changes, better merge them with:

```
$ git pull origin
```

- In case you want to cleanup your local commits before pushing, use  [git-rebase\(1\)](#):

```
$ git rebase -i origin/master
```

You will be presented a textfile where you can specify what of your local commits you want to modify, drop or squish (merge with the previous one). You can also improve your commit message here. Make sure that you don't rewrite already published commits.

- In order to get comments on your (preferably cleaned up) commits, use  [git-format-patch\(1\)](#) to generate patches of your commits and email them to the mailing list. Any committer can do this while preserving attribution.
- None of the steps above require membership in the pkg-multimedia alioth group. All of that can be done totally anonymously!

Uploading new upstream version to existing repository

- Updating a git branch

```
$ git checkout master
```

```
$ git pull
```

- resolve merge conflicts, review your changes e.g. with `gitk`
- pushing new revisions

```
$ git push origin master --tags --dry-run
```

- if happy, remove the `--dry-run` parameter
- **Assumptions in all cases**
- in the source tree
- the remote 'aliath' is setup properly

[CategoryGit](#)

PkgMate/GitPackaging (last modified 2014-01-10 08:28:21)

- [DebianMultimedia](#)
- [DevelopPackaging](#)

Develop Packaging

Packages maintained by the Debian Multimedia Team

Here you can find a list of packages maintained by the Debian Multimedia Team:

 <https://qa.debian.org/developer.php?login=debian-multimedia@lists.debian.org>

 <https://qa.debian.org/developer.php?login=pkg-multimedia-maintainers@lists.alioth.debian.org> ( [deprecated](#))

New packages should set the Maintainer field to

Debian Multimedia Maintainers <debian-multimedia@lists.debian.org>. Existing packages should move to using that address on a best-effort basis.

All source packages must be hosted at  [multimedia-team](#) on Salsa.

How to help with packaging

The Debian Multimedia Maintainers need help to create new Debian packages and maintain the existing ones. If you want to contribute to this effort, but you are new to the Debian packaging systems, here follows some information to get you started. Also see [DebianMultimedia/Sponsoring](#) for Best Practices with regard to obtaining sponsorship for your uploads if you are not a DD or DM.

Packaging guidelines

- Packages should use git, as mentioned above. Desirable is being able to use [DebianPkg: git-buildpackage](#).

Contents

1. [Develop Packaging](#)
2. [Packages maintained by the Debian Multimedia Team](#)
3. [How to help with packaging](#)
 1. [Packaging guidelines](#)
 2. [Workflow guidelines](#)
 3. [Documentation](#)
 4. [Proposing new packages for the Multimedia Team](#)
 5. [Working on existing packages with git for newbies](#)
4. [Common tasks for team members](#)
 1. [Uploading proposed package to the team git repository](#)
 2. [Uploading new upstream version to existing repository](#)
 3. [Removing/Adding debian tags to repository](#)
 4. [Common configuration options](#)
 5. [Bugs for packages found in DMO](#)

- Upstream sources should be kept in the *upstream* branch and the the debian packaging in the *master* branch.
- [DebianPkg: quilt](#) should be used for patch management, and the *master* branch should only differ from the *upstream* branch in files inside the *debian/* directory. This means no direct changes to the source in the master branch.
- In order to keep patches uniform, maintainers should configure [DebianPkg: quilt](#) by means of a `~/ .quilt.rc` file in accordance with the [Debian New Maintainers' Guide](#). Torsten Glaser suggested a more sophisticated `~/ .quilt.rc` file resulting from the team's discussion, the Perl and Maintainer's Guide suggestions in *clean* POSIX shell on the [pkg-multimedia-maintainers list](#).
- Patches should have a [DEP-3](#) compliant header, and should be submitted upstream (if relevant) before uploading to Debian.
- The maintainer field should be set to
Debian Multimedia Maintainers <debian-multimedia@lists.debian.org>
- The git repository should be hosted on salsa, under the multimedia-team project. It should forward commit messages to
`dispatch+package_vcs@tracker.debian.org`.
- The control file should use the Vcs-Git and Vcs-Browser tags.

Workflow guidelines

- **One change per commit.** This is very important, it eases review, cherry picking, bisecting (and thus debugging) and backporting.
- Do not commit `debian/changelog` along with the changes. This practice makes cherry picking and backporting changes unnecessarily hard. The changelog is generated with [DebianMan: git-dch\(1\)](#) at the time of upload. Also, packages should use the changelog release heuristic of [DebianMan: dch\(1\)](#).
- The commit message should have a short (<78 characters) summary of the change followed by a newline. After that, you can elaborate on the change. The summary is treated specially by various tools like [DebianMan: git-dch\(1\)](#), or the commitdiff mailer.
- After each upload to the Debian FTP servers, the first commit should be creating a new changelog entry, to ease testing of unreleased packages.
- Tags should be created (and signed) by the uploading DD, in the case of the debian tags in the master branch, and by the person importing the upstream sources in the case of upstream tags.
- To indicate that the package is ready for upload, update `debian/changelog` to include the target distribution (i.e. 'unstable' or 'experimental'). Otherwise just leave 'UNRELEASED'. If you want a review or help, just ask on this mailing list.

Documentation

- [Debian New Maintainers' Guide](#)
- [Debian Developer's Reference](#)
- [Debian Policy Manual](#)

-  [Using git on Alioth](#)
-  [git-buildpackage online documentation](#) (offline: /usr/share/doc/git-buildpackage/manual-html)
-  [Debian Library Packaging guide](#)

Proposing new packages for the Multimedia Team

- Don't bother with git for packaging new software. Instead, package it properly following [DebianPolicy](#) and the  [DevelopersReference](#).
- If you are confident with your package, propose it for review on the pkg-multimedia-maintainers mailing list. Link to your package preferably in a way that can be easily used with `?dget`. The  [Debian Mentors](#) site is a good place.
- Members of the team will then review your package and give you hints how to improve it. If the package is "good enough" (at the reviewer's discretion), they will setup a repository on `git.debian.org` and integrate your package while preserving attribution.

Working on existing packages with git for newbies

- As a very basic introduction, first read [PackagingWithGit](#).
- We generally follow the workflow and defaults of  [git-buildpackage](#). Besides `gbp-import-orig(1)`, the use of `git-buildpackage` is up to you. Using `git` works as well.
- Locate the packaging branch on  <https://salsa.debian.org/multimedia-team>.
- Use  [gbp-clone\(1\)](#) to get a copy of the repository. E.g.

```
$ gbp clone --pristine-tar git@salsa.debian.org:multimedia-team/jack-a
```

- Do changes to that branch, build the package, test your changes, and commit early and often!
- If someone else has committed while you are working on the branch, you need to integrate his changes. For this you have 2 options:
- If your local changes are rather minor and clean, rebase when using  [git-pull\(1\)](#):

```
$ git pull --rebase origin
```

- If you rather don't want to break your history because you think that your changes are rather large and are likely to interfere with the other changes, better merge them with:


```
$ git pull origin
```

- In case you want to cleanup your local commits before pushing, use  [git-rebase\(1\)](#):

```
$ git rebase -i origin/master
```

You will be presented a textfile where you can specify what of your local commits you want to modify, drop or squish (merge with the previous one). You can also improve your commit message here. Make sure that you don't rewrite already published commits.

- In order to get comments on your (preferably cleaned up) commits, use merge requests on  [salsa](#) or use  [git-format-patch\(1\)](#) to generate patches of your commits and email them to the mailing list.
- None of the steps above require membership in the multimedia-team group. All of that can be done totally anonymously!

Common tasks for team members

Uploading proposed package to the team git repository

- First create a repository on  [Salsa](#).
- Then import the sources locally, register the remote archive and push your commits (i.e. upload changes) with the following commands:

```
$ cd /path/to/sources/  
$ gbp import-dsc --pristine-tar <project>_0.0.1-1.dsc  
$ cd <project>  
$ git remote add origin git@salsa.debian.org:multimedia-team/<project>  
$ git push origin master upstream pristine-tar --follow-tags
```

- If the package hasn't been uploaded yet, then debian tag should be deleted: (thus debian tag can be created on upload)

```
$ git tag -d debian/0.0.1-1
```

Uploading new upstream version to existing repository

- Updating a git branch:

```
$ git checkout master  
$ git pull
```

or by running:

```
$ gbp pull
```

- Updating local branches without switching to them:

```
$ git remote update  
$ git fetch origin upstream:upstream pristine-tar:pristine-tar
```

- Merging a new upstream version:

```
$ gbp import-orig --pristine-tar --uscan
```

or, if you `uscan` does not work for some reasons, you can also import a local tarball:

```
$ gbp import-orig --pristine-tar /path/to/<package>_0.0.1.orig.tar.l
```

- Resolving merge conflicts, review your changes e.g. with `git`:

```
$ git commit
```

- Pushing new revisions:

```
$ git push origin master upstream pristine-tar --follow-tags --dry-
```

If you are happy, remove the `--dry-run` parameter. Alternatively:

```
$ gbp push
```

- **Assumptions in all cases**
- in the source tree
- the remote 'origin' is setup properly

Removing/Adding debian tags to repository

- Removing debian tag:

```
$ git tag -d debian/0.0.1-1
```

- Adding debian tag:

```
$ gbp buildpackage --git-tag-only
```

Alternatively, it's possible to create tags by hand:

```
$ git tag [-a|-s] "debian/0.0.1-1" -m "Debian release 0.0.1-1"
```

- -s (or -u <key>) for an OpenPGP-signed tag
- -a for unsigned tag
- Pushing tags:

```
$ git push origin master --follow-tags
```

- Replacing a previously set tag:

```
$ git tag -f [-a|-s] "debian/1.2.3-1" -m "Debian release 1.2.3-1"
```

Common configuration options

These parameters should be placed in the `debian/gbp.conf` file. (uncomment compression line if upstream tarball(s) are not gzip-compressed)

```
[DEFAULT]
pristine-tar = True
#compression = bzip2
```

If you adopt the dpkg source format 3.0 (*quilt*) you should:

* Add this to debian/source/format file

```
3.0 (quilt)
```

* Add this to .gitignore file

```
.pc
```

Bugs for packages found in DMO

If there is a bug that is not found in the Debian version of a package but found only in the DMO version of the package, it is helpful to mark such bugs with a usertag. The procedure we take to mark such bugs is as follows.

Send an email to close the bug ( NNNNNN-done@bugs.debian.org) and also to the BTS bug control interface ( control@bugs.debian.org). Closing the bug will automatically notify the submitter of the bug as well. The contents of the email must begin with the following lines.

```
user debian-multimedia@lists.debian.org
usertags NNNNNN dmo
tags NNNNNN = unreproducible
notfound NNNNNN <version1>
notfound NNNNNN <version2>
...
notfixed NNNNNN <version1>
notfixed NNNNNN <version2>
...
stop
```

These lines are read by the BTS bug control interface. The 'user' line must be set to '  debian-multimedia@lists.debian.org ' so that the usertag can be set. The 'usertags' marks the bug with the usertag 'dmo'. The 'tags' line will reset all tags on the bug so that it will only contain one tag, the 'unreproducible' tag. The 'notfound' lines removes all instances where the BTS says the bug is found. These lines need to be added for each version the BTS says the bug is found against. In case the BTS says there are versions of the package the bug is fixed, the 'notfixed' lines must also be added just like the 'notfound' lines to unmark all versions where the BTS says the bug is fixed. Finally, the 'stop' line stops processing for the BTS control interface.

After the 'stop' line, write a message explaining that this bug does not exist against the version of the package in Debian and is being close and marked as such. It's also helpful to recommend users to report the bug to dmo.

All such marked bugs can be viewed at  <http://bugs.debian.org/cgi-bin/pkgreport.cgi?tag=dmo;users=debian-multimedia@lists.debian.org>.

Work in progress : We're checking all the WNPP bugs and updating infos on this page.

-- [billitch](#)

[CategorySound](#) [CategoryPackaging](#)

DebianMultimedia/DevelopPackaging (last modified 2025-06-04 09:38:13)

- [Teams](#)

Translation(s): [English](#) - [indonesia](#) - [Italiano](#) - [Portuguese \(Portugal\)](#) - [Português \(Brasil\)](#) - [Русский](#) - [Українська](#)

This page aims to provide an entry point to the various Debian internal teams. The goal is to document everything that is important for people who want to help and maybe later join the corresponding teams. If you're part of such a team, you should check out the [guidelines for internal teams](#), which provides information on creating and keeping a successful team.

Contents

1. Teams

1. [Infrastructure and Core teams](#)
2. [Development Projects](#)
3. [Blends teams](#)
4. [Packaging teams](#)

2. Help out

3. About this page

Teams

For convenience, please keep the list alphabetically sorted.

Infrastructure and Core teams

- [Alioth Lists](#) - The Alioth Lists mailing list service team
- [Backports](#) - The Debian Backports FTP Masters
- [Community](#) - The ~~Anti-harassment~~ Community Team
- [DAM](#) - The Debian Account Manager(s)
- [Data Protection](#) - External contact and internal advice on how Debian handles personal data
- [Debbugs](#) - The bugs.debian.org and debbugs maintainers
- [DebConf](#) - organizers of the [DebConf](#) conference
- [Debian Buildd](#) - Wanna-build and build daemon administrators
- [Debian CD](#) - CD / DVD / image production
- [Debian Live](#) - Live images toolchain and production
- [Debian Mentors](#) - Helping new contributors to change Debian
- [Debian Wiki](#) - Admin and management of this wiki
- [Debian Women](#) - The Debian Women Project
- [Delegation Advisory](#) - the DPL's Delegation Advisory Helpers

- [DFSG](#) - The DFSG, Licensing & New Packages Team
- [DPL](#) - The Debian Project Leader
- [DSA](#) - The Debian System Administrators
- [Events & Merchandise](#) - Gather/organize events, including merchandise
- [FrontDesk](#) - The Debian front desk for prospective developers
- [FTPMaster](#) - FTP Masters
- [I18n](#) - Internationalization and localization
- [KeyringMaint](#) - Keyring maintainers
- [ListMaster](#) - Debian List Masters
- [Local Groups](#) - Debian Local Groups team
- [MIA](#) - Debian "Missing in Action" team
- [Mirrors](#) - Debian Mirrors Administrators
- [Networking](#) - Network configuration integration and interoperability
- [Outreach](#) - Debian Outreach Team
- [Policy](#) - The Debian Policy team
- [Publicity](#) - The publicity team
- [Press](#) - The press team (old, now merged with [Publicity](#) team)
- [QA](#) - The Quality Assurance team
- [Release Team](#)
- [RTC](#) - Real Time Communications team
- [Salsa](#)
- [SalsaCI](#) - Continuous Integration on Salsa team
- [Secretary](#)
- [Security](#) - The security team
- [DebianSocial](#) - The Debian Social team
- [Treasurer](#) - The Debian Treasurer(s)
- [Video](#) - Debconf Video team
- [Webmaster](#) - The web team
- [debci](#) - Debian Continuous Integration Team
- [LiveCoding](#) - The Debian Live Coding team for art, sound, music and visuals
- [Teams/DebianNet](#) - debian.net team - hosting services for "unofficial" projects
- [Teams/Welcome](#) - Help new users and aspiring contributors move across Debian.

Development Projects

- [Apt](#) - We maintain apt and python-apt.
-  [Adequate](#) - Debian package quality testing tool.
- [Boxer](#) - Package blending and deployment tool.
- [Cloud](#) - Usage and distribution of Debian on computer clouds.
- [Debian Documentation Project](#) - Documentation for the Debian system.

- [Debian-eeepc](#) - Debian Eee PC team
- [DebianInstaller](#) - The Debian Installer team
- [Debian LAN](#) - Debian Local Area Network team
- [DebTags](#) - The Debtags team
- [Dpkg](#) - Developing the Debian package manager
- [Embedded_Debian](#) - The Debian Embedded team
- [FAI](#) - FAI - Fully Automatic Installation for Debian and others
- [Lintian](#) - Lintian Debian package checker

Blends teams

- [Debian Accessibility](#) The Debian Accessibility team.
- [Debian Astro](#) The Debian Astro Team
- [Debian Design Team](#) - Debian for designers
- [DebianEdu](#) / Skolelinux - Debian Blend Distribution for education (which should become a Pure Blend Distribution in future)
- [Debian Games](#) - The Debian Games Team
- [Debian GIS](#) - The Debian GIS applications team
- [Debian Math](#) - The Debian Math Team
- [Debian Med](#) - The Debian Med Blend.
- [Mobian](#) - Debian Blend targeting mobile devices
- ?Debian Parl - Debian for parliamentarians
- [Debian Science](#) - The Debian Science Blend
- [Sugar](#) - Debian Sugar [Blend](#)
- [Teams/Tinker](#) - Debian for tinkerers/makers/hackers
- ?DebianWorking - Debian for those wanting a (fully/barely/arguably) Working Debian
- [DebiChem](#) - The Debian Chemistry Team
- [NeuroDebian](#) - Debian for neuroscience research project
- [FreedomBox](#) - Debian for home server appliance

Packaging teams

In addition to the general [guidelines](#) for teams there's a page collecting [resources](#) for packaging teams.

- [pkg-3dprinting](#) - 3D-printing packaging team
- [pkg-android-tools](#) - Android development tools and SDK packaging team
- [pkg-apparmor-team](#) - Debian [AppArmor](#) Team
- [pkg-ayatana](#) - The Ayatana Packagers
-  [pkg-bacula](#) - The Debian Bacula Team
- [pkg-bazaar](#) - Debian Bazaar Team

- [pkg-bluetooth](#) - The Debian bluetooth group
- [pkg-boinc](#) - Debian BOINC Maintainers
- [pkg-boost](#) - The Debian Boost Team
- ?pkg-cas - CAS packaging team
- [pkg-cinnamon](#) - Cinnamon packaging team
- [pkg-cli-apps](#) - Debian CLI (~ Mono/...) Applications Team
- ?pkg-cli-libs - Debian CLI (~ Mono/...) Libraries Team
- [pkg-clojure](#) - Clojure packaging team
- ?pkg-collab-maint - Collaborative package maintenance
- [pkg-common-lisp](#) - The Debian Common Lisp Team
- ?pkg-corba - CORBA packaging team
- [pkg-crosswire](#) - Debian Crosswire Packaging Team
- [pkg-cryptocoin](#) - Debian Cryptocoin Packaging Team
- [pkg-cryptsetup](#) - Debian Cryptsetup Packaging Team
- ?pkg-cups-devel - Debian CUPS Maintainers
-  [pkg-cyrus-sasl2](#) - The Debian Cyrus SASL Team
- ?pkg-devscripts - devscripts Devel Team
- [pkg-dns](#) - Debian DNS packaging team
- [pkg-electronics](#) - Debian Electronics Packaging Team
- [pkg-emacsen](#) - Debian Emacsen team
- [pkg-erlang](#) - Erlang Packaging Team
- ?pkg-eucalyptus - Debian Eucalyptus Maintainers
- [pkg-exim4](#) - Debian exim 4 Team
- [pkg-fedora-ds](#) - Debian 389 Directory Server Packaging Team
- [pkg-fonts](#) - The Debian Fonts Task Force
- [pkg-foo2zjs](#) - The Debian Foo2zjs Team
- [pkg-forensics](#) - Forensics and security related packaging team
- ?pkg-fso - The Debian FreeSmartphone.Org Team
- [pkg-gnome](#) - The Debian GNOME team
- [pkg-gnustep](#) - Debian GNUstep packaging team
- [pkg-gnupg](#) - Debian GnuPG packaging team
- [pkg-go](#) - Debian Go (golang) packaging team
- [debian-hams](#) - ham-radio packaging team
- [pkg-haskell](#) - Packaging Haskell libraries
-  [pkg-hebrew](#) - Hebrew support on Debian
- [pkg-heka](#) - Packaging Lua Sandbox, Hindsight and dependencies
-  [pkg-horde](#) - Debian Horde Packaging team
- [debian-hpc](#) - Debian HPC team
- [debian-hyprland](#) - Hyprland packaging team
- [pkg-ime](#) - IME Packaging Team (input method)

-  [pkg-in](#) - The Debian Indic Language Packaging Team
- [pkg-iproute](#) - The iproute (iproute2) packaging team.
- [pkg-ipsec-tools](#) - The ipsec-tools/racoon packaging team.
- [pkg-iot](#) - Debian IoT Packaging Team
- [pkg-java](#) - The Java packaging team
- [Javascript](#) - Debian Javascript Maintainers
-  [pkg-jed](#) - Debian JED Group
-  [pkg-kde](#) - Debian Qt/KDE maintainers and KDE extra teams
- [pkg-kernel](#) - The Debian Kernel Team
- [pkg-kolab](#) - Debian Kolab
- [pkg-libreOffice](#) Debian libreOffice maintainers (and openOffice.org)
- [pkg-libvirt](#) - Libvirt/Virt-manager/oVirt Packaging Team
-  [pkg-lua](#) - The Debian Lua Team
- [pkg-mate](#) - Debian MATE Packaging Team
- [?pkg-mono](#) - Debian Mono Group
- [pkg-mozext](#) - Mozilla ?WebExtensions packaging team
- [pkg-multimedia](#) - The Debian Multimedia Project
- [pkg-munin](#) - Debian Munin Maintainers
- [pkg-mutt](#) - Debian Mutt packaging
- [pkg-mysql-maint](#) - Debian MariaDB/MySQL packaging
- [pkg-nagios](#) - Debian Nagios Maintainer Group
- [pkg-netfilter](#) - Debian Netfilter packaging team
- [debian-niri](#) - Debian Niri Maintainers
- [pkg-obs](#) - Debian Open Build Service packaging team
- [pkg-ocaml-maint](#) - The Debian OCaml Maintainers Task Force
- [pkg-octave](#) - Debian Octave Group
- [pkg-openmpi](#) - The Debian OpenMPI Maintainers
- [pkg-opennebula](#) - Debian [OpenNebula](#) Packaging Team
- [pkg-openstack](#) - Debian [OpenStack](#) Packaging Team
- [pkg-otr](#) - Debian OTR Team
- [pkg-owncloud](#) - ownCloud for Debian Team
- [pkg-perl](#) - The Debian Perl Group
- [pkg-privacy-maintainers](#) - Debian Privacy Maintainers packaging team
- [pkg-php](#) - The Debian PHP Group
- [pkg-proftpd](#) - The Debian ProFTPD team
- [pkg-puppet](#) - Puppet packaging team
- [PythonTeam](#) - Debian Python Team (merged from the Python Applications Team and Python Modules Team)
- [r-pkg](#) - Debian R Packages Maintainers
- [pkg-rakudo](#) - The Debian Rakudo (Perl6) Group

- [Robotics](#) - Debian Robotics Team, which maintains ROS and other robotics related software
- [pkg-ruby](#) - The Debian/Ruby team, which maintains the ruby interpreters
- [pkg-ruby-extras](#) - The Debian/Ruby Extras team, which maintains ruby libraries and applications
- [pkg-rust](#) - Debian/Ubuntu Rust team
- [?pkg-s3fs](#) - S3FS (FUSE Filesystem for Amazon S3) Packaging Team
- [pkg-samba](#) - Debian Samba Team
- [pkg-salt](#) - Salt Packaging Team
- [Sass](#)
- [?pkg-scicomp](#) - Debian Scientific Computing Team
- [pkg-sdl](#) - The Debian SDL Group
- [pkg-security](#) - The Debian Security Tools Packaging Team
- [pkg-shibboleth](#) - The Debian Shibboleth Packaging Team
- [pkg-ssh](#) - Debian SSH Team
- [?pkg-sugar](#) - Debian Sugar Team
- [pkg-systemd](#) - Debian systemd Maintainers
- [pkg-tcltk-devel](#) - The Debian Tcl/Tk team
- [pkg-telepathy](#) - Debian Telepathy maintainers
- [pkg-Tex](#) - The Debian TeX maintainers
- [?pkg-trac](#) - Debian Trac Team
- [pkg-vim](#) - Debian Vim Maintainers
- [pkg-voip](#) - The Debian pkg-VoIP packaging team
- [pkg-wmaker](#) - The Debian ?WindowMaker packaging team
- [pkg-wxwidgets](#) - wxWidgets packaging team
- [pkg-X](#) - The X Strike Force (XSF) : Debian X11 Maintainers
-  [Debian Xen Team](#) - Packaging the Xen hypervisor and tools for Debian
- [pkg-xfce](#) - The Debian Xfce Group
- [pkg-xml-sgml](#) - Debian XML/SGML Group
- [pkg-xmpp](#) - Debian XMPP Maintainers
- [r-pkg-team](#) - Debian R Packages Maintainers
- [?pkg-zenoss](#) - The Zenoss packaging team
- [lxqt-team](#) - The Debian LXQT packaging team
- [Hams](#) - Maintains amateur-radio related packages for Debian
- [Debian Logging](#) - Debian Logging Daemons Packaging Team

There are far more  [teams](#)!

Help out

If you want to create a new page for a team, please enter the name of your team in [CamelCase](#), press the button and fill out the template. It's best when the page is created by a member of the team. However, when the team is known to be overloaded, and if you would like to join that team to help out, it's wise to create the page and put all your knowledge about the team in that page. Then ask someone from the team to review it.

Some teams that have not yet been added to this page are listed in [/TODO](#).

Add a new team

About this page

A somewhat similar effort was once started with ?TaskDescription but it was too theoretic and never got very far. This page aims to be much more concrete and lightweight so that people effectively keep it up to date.

[CategoryTeams](#) [CategoryPackaging](#)

Teams ([last modified 2026-04-26 04:57:13](#))

- [Fonts](#)
- [PackagingPolicy](#)

Contents

1. [Font Naming](#)

2. [Font Location](#)

3. [Packaging Example](#)

4. [Team Maintained Fonts](#)

1. [Git Repository](#)

5. [Further Discussion](#)

What follows is not meant to be a comprehensive specification or policy statement, but rather a quick introduction to font packaging in Debian. You are encouraged to ask on the [debian-fonts mailing list](#) if you have any questions regarding fonts in Debian that are not covered below.

Font Naming

Packages that contain fonts should be named in the form of a tuple, such as **fonts-name**. If desired, the foundry (a/k/a the creator/distributor of the font) may be included as well using the **fonts-foundry-name** syntax. If you build variable vf, postfix **-variable** to the binary package name.

Fonts licensed under the Open Font License (OFL) with a Reserved Font Name clause (RFN) can be distributed in Debian but must be renamed. The only exception to this is when the original font binary files can be reproduced bit-for-bit by the build process of the Debian package of the font.

Font Location

The files corresponding to a given font are installed in directories dependent on the type and name of the font.

Fonts that are meant to be discovered by fontconfig must be stored in a directory named as `/usr/share/fonts/fonttype/name/`, where **fonttype** is the container format (OpenType, [TrueType](#), Type1, etc).

As an example, the Linux Libertine fonts distributed in OpenType format should be put in the directory `/usr/share/fonts/opentype/linuxlibertine/`, while the Rufscript font distributed in TrueType format should be put in the directory `/usr/share/fonts/truetype/rufscript`.

Any general-purpose font meant to be used by X11/Wayland/XeLaTeX/GD/... programs should be put into the directories used by them.

This doesn't apply to other kinds of fonts, however. Obviously fonts for the Linux console (.psf and others) can't be used this way.

Likewise web fonts (WOFF/WOFF2), despite being supported by *some* GUI programs, cause problems when exposed via fontconfig and so they should be stored elsewhere. For example, the Hack font distributed in WOFF2 format can be put in the directory `/usr/share/fonts-hack/woff2` (currently there is no specific policy for packaging web fonts).

Even nominally TrueType fonts that are unusable by arbitrary programs, such as the Firefox emoji font which consists entirely of embedded non-standard images, or a font with game icons encoded at codepoints clashing with Unicode, should be put elsewhere. In the case of the former, the `/usr/lib/firefox/fonts` directory is used.

Packaging Example

What follows is an example of how to prepare a very simple font package. This has been made really easy thanks to **debhelper**. More complex packages can be found in the [🌐 Git repository of the fonts team](#). These existing packages can and should serve as examples.

For this example, we will look at the font package `fonts-nafees`. The font is distributed upstream as a single font file, `NafeesWeb.ttf`.

And we would like to see it installed as a TrueType font, at `/usr/share/fonts/truetype/nafees`.

Nothing is easier with `dh_install`. Just add this to your `debian/install` file:

```
*.ttf usr/share/fonts/truetype/nafees
```

Only a minimal `dh` style `debian/rules` file is needed.

```
%:
dh $@
```

The `debian/control` file for the package looks like this:

```
Source: fonts-nafees
Section: fonts
Maintainer: Debian Fonts Team <debian-fonts@lists.debian.org>
Uploaders: Christian Perrier <bubulle@debian.org>, Mohammed Adnène Tro
Build-Depends:
    debhelper-compat (= 13),
Standards-Version: 4.7.3
Homepage: https://fontlibrary.org/en/font/nafees-nastaleeq
Vcs-Browser: https://salsa.debian.org/fonts-team/fonts-nafees
Vcs-Git: https://salsa.debian.org/fonts-team/fonts-nafees.git

Package: fonts-nafees
Architecture: all
Multi-Arch: foreign
Depends: ${misc:Depends}
Description: nafees free OpenType Urdu fonts
    This is a free OpenType Urdu font (Nafees Web Naskh), designed and
    developed by the Center for Research in Urdu Language Processing
    (CRULP, http://www.crulp.org/) at National University of Computer and
    Emerging Sciences (http://www.nu.edu.pk/).
```

Note that the `Section` is set to `fonts`.

And you are done with the technical part of your package. Now be really careful about the font licensing and fill `debian/copyright` accordingly and accurately. Upstream fonts very often lack clear information about the font license so you might need to talk with the font author. Even be prepared to explain to them the differences between licences. When in doubt, or if you don't feel like you have the needed skills for this, please redirect them to the  debian-legal@lists.debian.org mailing list.

Team Maintained Fonts

You are encouraged to maintain font packages as part of the  [Debian Fonts Team](#). Some points of contact for them can be found at:

-  [Debian-Fonts Mailing List](#)
-  [Debian-Fonts Packages Dashboard](#)
-  [Debian Salsa Project Group](#)
- Source Code: `git@salsa.debian.org:fonts-team/$(package_name).git`

Git Repository

All new team maintained packages should be maintained in Salsa Git repository.

See [the Salsa page](#) for more information on how to work with Salsa Git repositories.

In October 2014, the team finished [transferring](#) all of its packages from the previous Subversion repositories to Git.

Further Discussion

There is currently discussion on the  [pkg-fonts-devel](#) mailing list that hopes to lead to a more comprehensive and explicit policy document.

Rogério Brito maintains a draft proposal for the  ["Fonts Policy of Debian"](#), though it was last reviewed in 2011.

Nicolas Spalinger has also contributed to the discussion: 

<http://anonscm.debian.org/viewvc/pkg-fonts/people/yosch/debian-font-packaging-policy.txt>

[CategoryPackaging](#) [CategoryFonts](#)

Fonts/PackagingPolicy (last modified 2026-02-25 21:39:00)

- [Repackage_srcrpm](#)

Repackage src.rpm

Here are some hints for creating a proper Debian source package while referencing the src.rpm package created for the [RPM](#) system.

Although I call this activity as repackage, this activity involves many manual alteration processes and packaging strategy choices which are different from the original src.rpm.

It is not easy to decide which part of the source code to activate and which binary sub packages to offer for the Debian package, the use of src.rpm (or the SPEC file) as the baseline expedites decisions for such issues. So use them if you can!

🚨 Pre-requisite: Read and understand everything in 🌐 [Debian New Maintainers' Guide](#).

🚨 This is not the same as repackaging of the binary RPM file using the 🌐 [alien](#) package.

Contents

1. [Repackage src.rpm](#)
2. [How to find src.rpm from Fedora project](#)
3. [How to find src.rpm from OpenSUSE project](#)
4. [How to download and extract src.rpm](#)
5. [How to read the build info](#)
 1. [spec file](#)
 2. [How to set up source tree](#)
 3. [option for ./configure](#)
 4. [install](#)
 5. [splitting into binary packages](#)
6. [Reminders for repackaging](#)
7. [Example of src.rpm repackaging](#)
8. [Other source of reference information](#)

How to find src.rpm from Fedora project

Redhat kindly publishes their latest src.rpm packages of the Fedora project to the public. They can be searched at:

- 🌐 [Fedora ARM buildsystem](#)

For example, searching on "ibus*" glob returns "Search Results for packages matching "ibus*" with 47 sets of ID and Name starting with a set "3103" and "ibus".

Clicking on package name "ibus" leads to "Information for package ibus" with many build results. Here, "NVR" value such as "ibus-1.5.3-1.fc19" roughly means:

- The upstream package name: ibus
- The upstream package version: 1.5.3
- The target Fedora release: 1FC19 (the first release for the FC19 distribution)

Clicking on NVR "ibus-1.5.3-1.fc19" leads to "Information for build ibus-1.5.3-1.fc19".

This page lists information on the src.rpm, the binary rpm (armv7hl and noarch) and Changelog.

The URL of the latest src.rpm of Fedora is linked from "RPMs" -> "src" -> "(download)".

The outline information page of src.rpm containing build dependency indicated in the SPEC file and files bundled is linked from "RPMs" -> "src" -> "(info)".

How to find src.rpm from OpenSUSE project

Novell kindly publishes their latest src.rpm packages of the open SUSE project to the public. They can be searched at:

-  [The next openSUSE distribution](#)

For example, searching on "ibus" returns "Search Results for packages matching "ibus" with 26 sets of packages.

Clicking on package name "ibus" leads to "Intelligent Input Bus for Linux OS". Clicking "Repositories" and "standard" leads you to a page with "ibus-1.5.4-5.1.src.rpm".

This is the URL for the latest src.rpm of open SUSE.

I also look for URL such as:  <http://download.opensuse.org/factory/repo/src-oss/suse/src/>

How to download and extract src.rpm

I have created a "rget" script as:

```
#!/bin/sh
FCURL=$1
FCSRPM=$(basename $FCURL)
```

```
mkdir ${FCSRPM}
cd ${FCSRPM}/
wget ${FCURL}
rpm2cpio ${FCSRPM} | cpio -dium
```

For Fedora project, using right click on "(download)" to "Copy link address" and pasting to the console, I run "rget" command as:

```
$ rget http://arm.koji.fedoraproject.org//packages/ibus-anthy/1.5.3/1
```

This performs the following:

- This creates the "ibus-anthy-1.5.3-1.fc19.src.rpm" directory and cd into it.
- This downloads the "ibus-anthy-1.5.3-1.fc19.src.rpm".
- This extracts contents of this src.rpm.
 - ibus-1.5.3.tar.gz
 - ibus-530711-preload-sys.patch
 - ibus-541492-xkb.patch
 - ibus-810211-no-switch-by-no-trigger.patch
 - ibus.spec
 - ibus-xinput
 - ibus-xx-setup-frequent-lang.patch

The "**debamke -a <URL_TO_src.rpm>**" command (from the [DebianPts: debmake](#) package) can download and unpack the source RPM file while adding template **debian/*** files.

How to read the build info

spec file

The build info for the src.rpm is found in the spec file. For "ibus", the spec file is "ibus.spec".

💡 Some upstream tar balls include a <packagename>.spec file in the source tree. That can be used as well.

Debian packaging system stores package information in multiple files under the "debian" directory.

Instead, the spec file stores them all in a file sectioned by lines starting with "%" such as "%package", "%description", "%patch", "%build", "%configure", "%install", "%files", ... Basically, each section embeds shell script lines and text data lines.

Their meanings are quite intuitive. Their exact meanings are explained elsewhere:

-  [Fedora Draft Documentation, RPM Guide](#)
-  [Maximum RPM](#)
-  [Mandriva wiki page for RPM specs file syntax](#) (short!)

For now, let's recall the very basics.

The "#" character is used to put comment like a shell program

The "%" character has many other functions other than sectioning the spec file. Let's see them by the example.

```
%if (0%{?fedora} > 18 || 0%{?rhel} > 6)
%global with_python_pkg 1
%else
%global with_python_pkg 0
%endif
```

The above example section of configuration is like CPP conditional section and means:

```
If (the target fedora version is larger than 18 or the rhel version is
    set global variable with_python_pkg to 1
else:
    set global variable with_python_pkg to 0
endif
```

The global variables are string and can be used as the argument of "%if" for this CPP like conditional section. The string value 1 means True and the string value 0 means False.

How to set up source tree

The most interesting sections of the spec file are sections describing how to setup the source tree. You need to read several sections to get the idea. For example in "ibus.spec":

```
%global with_preload_xkb_engine 1

... [skip lines]
```

```

Name:      ibus
Version:   1.5.3
Release:   1%{?dist}
Summary:   Intelligent Input Bus for Linux OS
License:   LGPLv2+
Group:     System Environment/Libraries
URL:       http://code.google.com/p/ibus/
Source0:   http://ibus.googlecode.com/files/%{name}-%{version}.tar.gz
Source1:   %{name}-xinput
# Upstreamed patches.
# Patch0:   %{name}-HEAD.patch
# https://bugzilla.redhat.com/show_bug.cgi?id=810211
Patch1:    %{name}-810211-no-switch-by-no-trigger.patch
# https://bugzilla.redhat.com/show_bug.cgi?id=541492
Patch2:    %{name}-541492-xkb.patch
# https://bugzilla.redhat.com/show_bug.cgi?id=530711
Patch3:    %{name}-530711-preload-sys.patch
# Hide minor input method engines on ibus-setup by locale
Patch4:    %{name}-xx-setup-frequent-lang.patch

... [skip lines]

%prep
%setup -q

# %%patch0 -p1
cp client/gtk2/ibusimcontext.c client/gtk3/ibusimcontext.c ||
%patch1 -p1 -b .noswitch
%if %with_preload_xkb_engine
%patch2 -p1 -b .preload-xkb
rm -f bindings/vala/ibus-1.0.vapi
rm -f data/dconf/00-upstream-settings
%endif
%patch3 -p1 -b .preload-sys
%patch4 -p1 -b .setup-frequent-lang

```

This means:

- set %{name} to "ibus"
- set %{version} to "1.5.3"
- set %{release} to "1FC19"
- URL of upstream site: <http://code.google.com/p/ibus/>

- URL to obtain the main Source0: <http://ibus.googlecode.com/files/ibus-1.5.3.tar.gz>
- Additional file Source1: ibus-xinput
- setup source tree as:
 - extract ibus-1.5.3.tar.gz
 - skip name-HEAD.patch (commented out)
 - copy client/gtk2/ibusimcontext.c to client/gtk3/ibusimcontext.c
 - apply ibus-810211-no-switch-by-no-trigger.patch
 - apply ibus-541492-xkb.patch
 - remove bindings/vala/ibus-1.0.vapi
 - remove data/dconf/00-upstream-settings
 - apply ibus-530711-preload-sys.patch
 - apply ibus-xx-setup-frequent-lang.patch

The file removals can be implemented in the `override_dh_auto_clean` target.

💡 There are many entries with `BuildRequires:`. They indicate the package build dependency of the source.

option for `./configure`

Another interesting section is `"%configure"`. For example in `"ibus.spec"`:

```
%global with_pygobject2 1

... [skip lines]

%configure \
    --disable-static \
    --enable-gtk2 \
    --enable-gtk3 \
    --enable-xim \
    --enable-gtk-doc \
    --with-no-snooper-apps='gnome-do,Do.*,firefox.*,.*chrome.*,.*chrom:
    --enable-surrounding-text \
%if %with_pygobject2
    --enable-python-library \
%endif
    --enable-introspection
```

The global variable `"with_pygobject2"` is 1 (True). So when building package, this spec file asks to execute:

```
./configure \
    --disable-static \
    --enable-gtk2 \
    --enable-gtk3 \
    --enable-xim \
    --enable-gtk-doc \
    --with-no-snooper-apps='gnome-do,Do.*,firefox.*,.*chrome.*,.*chrom:
    --enable-surrounding-text \
    --enable-python-library \
    --enable-introspection
```

This usually goes to `debian/rules` under "override_dh_configure:" as:

```
override_dh_configure:
[TAB] dh_configure -- \
    --disable-static \
    --enable-gtk2 \
    --enable-gtk3 \
    --enable-xim \
    --enable-gtk-doc \
    --with-no-snooper-apps='gnome-do,Do.*,firefox.*,.*chrome.*,.*cl
    --enable-surrounding-text \
    --enable-python-library \
    --enable-introspection
```

install

Just like Debian packaging system have `<bin_package>.install` to pick files to be installed into a particular binary package, `src.rpm` have a section `"%file <bin_package_suffix>"` section. Please note that only suffix is used to identify the binary package name in the spec file.

If you need to execute command to generate installed files, execute them in the `override_dh_auto_install` target etc..

For example, `libtool` archive files are removed in the "ibus.spec" file in install phase after the execution of the standard "make install" as:

```
%install
make install DESTDIR=$RPM_BUILD_ROOT INSTALL='install -p'
```

```
rm -f $RPM_BUILD_ROOT%{_libdir}/libibus-%{ibus_api_version}.la
rm -f $RPM_BUILD_ROOT%{_libdir}/gtk-2.0/%{gtk2_binary_version}/immodule
rm -f $RPM_BUILD_ROOT%{_libdir}/gtk-3.0/%{gtk3_binary_version}/immodule
```

This can be translated into debian/rules by removing such files before install as:

```
override_dh_install:
    dh_install --fail-missing -X .la
```

Please note the purpose of the manual repackaging is not the line-by-line conversion. It is to create a new Debian package in the most natural form while reusing the known-good-packaging-idea used in the RPM world.

splitting into binary packages

The main binary (=source) package name, its package dependency, and the main package description are defined in SPEC file as, e.g. `ibus`,:

```
...
Name:      ibus
...
Requires:  %{name}-libs = %{version}-%{release}
Requires:  %{name}-gtk2 = %{version}-%{release}
Requires:  %{name}-gtk3 = %{version}-%{release}
Requires:  %{name}-conf = %{version}-%{release}

Requires:  pygtk2
Requires:  iso-codes
Requires:  dbus-python >= %{dbus_python_version}
Requires:  im-chooser >= %{im_chooser_version}
Requires:  notify-python
Requires:  librsvg2
...
%description
IBus means Intelligent Input Bus. It is an input framework for Linux OS
...
```

The sub binary package name suffix, its package dependency, and its description are defined in SPEC file as, e.g. `ibus-libs`,:


```

...
%package libs
Summary:    IBus libraries
Group:      System Environment/Libraries

Requires:   glib2 >= %{glib_ver}
Requires:   dbus >= 1.2.4

%description libs
This package contains the libraries for IBus
...

```

💡 The corresponding Debian package name is `libibus-1.0-5` for this case. Package naming convention differences need to be noted.

Reminders for repackaging

Please note few things for this repackaging:

- You should use the best practices of the Debian packaging.
 - use the debhelper compat level 9.
 - use the "dh \$@" syntax with override in the `debian/rules`.
 - manage source in GIT repository with the `gbp` command (from `git-buildpackage` package) compatible repository format.
 - build package under cowbuilder (or pbuilder) with "gbp buildpackage".
- You must follow the package naming convention of Debian.
 - Some package name conversions are intuitive: (RH --> Debian)
 - `gobject-introspection` --> `gobject-introspection`
 - `foo-devel` --> `foo-dev`
 - Some package name conversions are not so intuitive: (RH --> Debian)
 - `pygobject2` --> `python-gobject-2`
 - `pygobject2-devel` --> `python-gobject-2-dev`
 - `pygobject3` --> `python-gi`
 - `gobject-introspection-devel` --> `libgirepository1.0-dev`
- You must pay attention to the [🌐 multiarch](#).
 - If the install path of files needs to be adjusted, do not think complicated patches to the source such as **configure.ac**.
 - Think about placing **./configure --libdir=/usr/lib/\$(DEB_HOST_MULTIARCH)** in **override_dh_autoconfigure**: target of the `debian/rules` file as the first option.

- Think about placing the source and destination entries in the **debian/*.install** file as the backup option.
- Debian does not use the **/use/libexec** directory. Use **/use/lib** (or **/use/libexec/<packagename>**)
- You must pay attention to the  [hardening](#).
 - For hardening, use the **debian/rules** file as (Read the  [hardening](#) page for details):

```
#!/usr/bin/make -f

# Uncomment this to turn on verbose mode.
#export DH_VERBOSE=1

# To enable all hardening options, uncomment following line
#export DEB_BUILD_MAINT_OPTIONS = hardening=+all

# To add extra build options
export DEB_CFLAGS_MAINT_APPEND  = -Wall -pedantic
export DEB_LDFLAGS_MAINT_APPEND = -Wl,--as-needed

# To ensure setting $(DEB_HOST_MULTIARCH)
DEB_HOST_MULTIARCH ?= $(shell dpkg-architecture -qDEB_HOST_MULTIARCH)

# For hardening problem under cmake, uncomment following line
#CFLAGS += $(CPPFLAGS)
#CXXFLAGS += $(CPPFLAGS)

%:
    [TAB]    dh $@

...
```

Example of src.rpm repackaging

I strongly recommend to see the actual example: `ibus 1.5.3-*` Debian package and its FC19 src.rpm.

This can be done as:

```
$ git clone git://anonscm.debian.org/pkg-ime/ibus.git
$ cd ibus
```

```
$ gitk --all
```

There are much more details needed to be handled than described in the above.

Other source of reference information

If you need reference information other than Fedora and OpenSUSE projects, see an extensive list found at:

-  [Open Source Software Security Wiki's distro patches](#)

[CategoryPackaging](#)

Repackage_srcrpm (last modified 2025-08-23 00:08:06)

- [RPM](#)

Translations: English - [Русский](#)

Contents

1. RPM Package Manager

1. [Why would anyone want to use the RPM within Debian?](#)

2. [rpm to dpkg/apt command reference](#)

3. [Redhat Package Manager \(RPM\)](#)

4. [Install a .RPM in Debian](#)

5. [See also](#)

RPM Package Manager

The RPM was developed at RedHat for keeping track of the files each program or package installed as well as noting what other packages it depended on or depended on it, some information about the source of the package and a brief synopsis.

One should avoid using `.rpm` package on a Debian system.

There are some good (and more not so good) write-ups comparing the two package management systems. A search for rpm vs deb or rpm vs dpkg will turn them up.
todo add some url's for some good ones

Why would anyone want to use the RPM within Debian?

Binary package

If the package only comes in binary `.rpm` format, you can use [DebPkg: alien](#) to convert it.

Source Package

In order to extract sources from an rpm archive, it may be necessary to use the rpm tool.

A developer may want to extract package source code that is only available within an rpm archive.

rpm to dpkg/apt command reference

This guide may help people switching to a dpkg based distribution like Debian from an RPM based one like RedHat.

A quick comparison of rpm and apt/dpkg command-line arguments

List all installed packages:

```
rpm -qa  
dpkg --get-selections
```

List information about an installed package:

```
rpm -qi pkgname  
dpkg --get-selections pkgname (prints a bunch of extra info too)
```

List files in an installed package

```
rpm -ql pkgname  
dpkg --get-files pkgname
```

List information about a package on the local hard drive

```
rpm -qip file.rpm  
dpkg --get-info file.deb
```

List files in a package on the local hard drive

```
rpm -qpl file.rpm  
dpkg --get-files file.deb
```

List files in an uninstalled package (depends on dist)

- grep through the Contents.arch file found in ftp.us.debian.org/debian/dists/frozen/

Extract files in a package without installing it

- (Open it in Midnight Commander (mc) and then enter CONTENTS.cpio(1))

```
dpkg-deb --extract file.deb dir-to-extract-to
```

- or Midnight Commander works on .debs too.

Install a package from a local file

```
rpm -i file.rpm  
dpkg --install file.deb
```

Remove a package from the system

```
rpm -e pkgname           (saves copies of modified config files  
dpkg --purge pkgname     (removes everything)  
dpkg --remove pkgname    (leaves config files behind)
```

Identify the package that owns a file

```
rpm -qf full-path-to file  
rpm -qf name-of-file-in-local-dir  
dpkg --search any-portion-of-file's-path (2)
```

Get information about a remote package

```
rpm -qpi <url>  
apt-cache show package
```

List all remote packages

```
(browse rpmfind.net or your other site)  
apt-cache dumpavail
```

(1) thanks to Adriaan Penning

(2) i.e. "dpkg --search /etc" will display all packages that have files in etc.

Derived from a page by Scott Bronson at <http://www.trestle.com/linux/rpmdeb.html>

see also :

https://wikitech.leuksman.com/view/OS_differences : Sysadmin cheat sheet

Redhat Package Manager (RPM)

RPM was originally designed to be a standalone binary installer for Redhat based GNU/Linux distros (Like Fedora, Mandrake and SUSE). RPM has since been ported to other operating systems such as IBM's AIX and Novell Netware.

Install a .RPM in Debian

To install a .rpm file in a Debian based system you'll need the program [Alien](#). Note that Alien can't work 100% of the time, because rpm and apt are very different.

See also

-  [OpenPkg FAQ](#).
- [repackage RPM source packages as Debian packages](#)

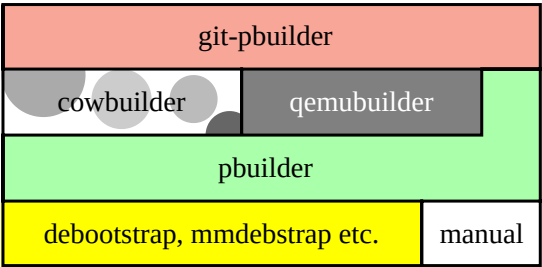
[CategoryPackageManagement](#) [CategoryPackaging](#)

RPM (last modified 2025-05-24 08:48:38)

- [pbuilder](#)

Translations: [English](#)

[DebianPkg: pbuilder](#) is the core of a software stack for building Debian packages. For common tricks in pbuilder-related tools, see [PbuilderTricks](#). For alternative tools, see [Package build tools](#). For more general package maintenance, see [Package maintenance tools](#).



The pbuilder stack

The pbuilder software stack has evolved over decades with no central planning. Each layer reused an existing program to provide a more targeted solution to a specific problem. This page discusses how to use the whole stack to create Debian packages, but see pages for individual programs for details.

The attached image shows the whole stack at the time of writing.

Contents

1. pbuilder vs. sbuild	
2. Layers of the stack	
1. debootstrap, mmdebstrap etc.	
2. pbuilder itself	
3. cowbuilder	
4. qemubuilder	
5. git-pbuilder	
3. Optional performance tweaks	
1. Use eatmydata	
2. Use ccache	
3. Add extra packages	
4. Mirror repositories locally with apt-cacher-ng	
5. Keep packages updated	
4. Create environments	
1. Create an environment with raw pbuilder	
2. Create an environment with cowbuilder	
3. Create an environment with git-pbuilder	
4. Advanced ways to create environments	
1. Create packages for non-default mirrors and components	
2. Create an Ubuntu environment	
3. Create an environment manually	

pbuilder vs. sbuild

`pbuilder` provides an easy-to-use packaging solution for people who spend most of their time solving other problems. The main alternative is [sbuild](#), which provides more granular control for people who spend the majority of their time dealing with the actual packaging process.

It's probably best to learn `pbuilder` first, then migrate to `sbuild` if you need the extra functionality. `sbuild` innovations are usually ported to `pbuilder` once they mature, so either solution is fine for most developers.

Layers of the stack

`pbuilder` is the central part of a software stack to create, update, and use environments. Layers below `pbuilder` itself create various types of environment, layers above are wrappers that improve the user experience.

debootstrap, mmdebstrap etc.

[debootstrap](#) and [DebianPkg: mmdebstrap](#) create a Debian environment in a subdirectory, which you can [chroot](#) into and treat more-or-less like a separate system. `pbuilder` uses them to create build environments, but for example you can also use them to create the filesystem for a machine with a different hardware architecture.

`debootstrap` is the standard way to create `pbuilder` environments, `mmdebstrap` is a more recent alternative - differences are listed in [DebianMan: the debootstrap section of the mmdebstrap man page](#). `mmdebstrap` is sometimes associated with `sbuild` for historical reasons, but it has been compatible with `pbuilder` since ~~Closed: #940188: compatibility with gnu debootstrap, pbuilder and cowbuilder: 940188~~. To switch to `mmdebstrap`, pass `--debootstrap=mmdebstrap` to `pbuilder` or add a line like `DEBOOTSTRAP="mmdebstrap"` in your `/etc/pbuilderrc`.

There are many other tools to create Debian environments. For example, [cdebootstrap](#) resembles `debootstrap` but supports a different set of target environments. For a list, see [Tools to create a build system](#).

You can even [create an environment manually](#). Just make sure to install all the [packages for building packages](#).

pbuilder itself

[DebianMan: pbuilder](#) creates build environments from existing "base" environments, then cleans up the build environment when it's finished. This ensures artifacts from one build never cause issues in the next build.

Using `pbuilder` directly creates environments from tarballs (e.g. `tar xf /var/cache/pbuilder/base.tgz`), which is highly flexible but requires a lot of I/O.

cowbuilder

[cowbuilder](#) is a wrapper around `pbuilder` that manages build environments with less disk I/O. It is a better choice for beginners.

`cowbuilder` creates environments with hardlinks (e.g. `cp -al /var/cache/pbuilder/base.cow/ . . .`). This is much faster by default, but makes some advanced uses more complicated (like [using a tmpfs for speed](#)).

For information about `cowbuilder` performance, see [cowbuilder benchmark](#).

qemubuilder

[qemubuilder](#) is a wrapper around `pbuilder` that builds environments in a `qemu` disk image. This was extremely useful when it was released, but modern [cross-compiling](#) and [user-mode emulation](#) have made it largely redundant.

`qemubuilder` creates environments by running a [QEMU](#) instance with a kernel and (optional) `initrd`, both provided by you. This is slower and more difficult than other alternatives, but may be necessary when compiling for a completely different operating system (like [Debian GNU/Hurd](#)).

 Because `qemubuilder` is hard to use and largely redundant, this page does not discuss how to use it.

git-pbuilder

[git-pbuilder](#) is a high-level wrapper around `cowbuilder`, which can also be configured to use `qemubuilder` or raw `pbuilder`. It focuses on making common operations easier - see [PackagingWithGit](#).

Examples on this page that include `sudo cowbuilder` can usually be replaced by `git-pbuilder` without `sudo` (`git-pbuilder` calls `sudo` internally).

 `git-pbuilder` is part of [DebianPkg: git-buildpackage](#), but there is no equivalent in e.g. [DebianPkg: svn-buildpackage](#) - please use `cowbuilder` directly.

Optional performance tweaks

The pbuilder stack has a number of optional tweaks that improve performance, at the cost of increasing complexity. If this is the first time you've created a package with Debian, you may want to skip these steps and come back later.

Use eatmydata

Linux calls [DebianMan: fsync](#) automatically when files are closed, to ensure data is actually written to disk. This massively reduces data loss from system crashes and unsafely removing disks, but slows down programs that write lots of small files. You can disable this safety measure with [DebianPkg: eatmydata](#).

Building packages involves writing lots of small files, and doesn't benefit from the safety provided by [DebianMan: fsync](#). Disable this safety measure by enabling eatmydata in your `/etc/pbuilderrc`:

```
# append eatmydata to $EXTRAPACKAGES:  
EXTRAPACKAGES="$EXTRAPACKAGES eatmydata"  
EATMYDATA=yes
```

You will need to manually install the eatmydata package in any environments you have already created - see [Add extra packages](#), below.

Note that running e.g. `sudo eatmydata pbuilder ...` instead of `sudo pbuilder ...` only affects the tiny minority of writes from *outside* the chroot, so doesn't provide any real performance benefit.

 [the "tmpfs" pbuilder trick](#) is faster than eatmydata but more complex to manage.

Use ccache

ccache caches compilations between runs, which can significantly reduce build times when you're trying to diagnose a packaging issue in a large compiled project.

Read [ccache](#) for general advice, then decide where to put your ccache directory. It's recommended to create a new directory like `/var/cache/pbuilder/ccache`, to avoid any risk of data from your local builds leaking into built packages. Wherever you put the directory, make it writeable by a user with UID 1234 and GID 1234.

Finally, add the following to your `/etc/pbuilderrc`:

```
export CCACHE_DIR="/path/to/your/ccache"  
export PATH="/usr/lib/ccache:$PATH"
```

```
EXTRAPACKAGES="$EXTRAPACKAGES ccache"  
BINDMOUNTS="$CCACHE_DIR"
```

You will need to manually install the `ccache` package in any environments you have already created, and should also run `update-ccache-symlinks`. See [Add extra packages](#).

Add extra packages

`pbuilder` environments start out with very few packages installed, then `pbuilder` installs your package's build-dependencies each time it rebuilds the package. This reduces the risk of unlisted dependencies, because your package will simply fail to build unless all dependencies are accounted for. But it significantly increases build times, because packages have to be reinstalled every single time.

All `pbuilder` wrappers support being called with `login --save-after-login` to run commands in the `chroot` then save the results. `pbuilder` and `cowbuilder` also support being called with `execute --save-after-exec` to do the same with a script specified on the command-line. Either option lets you `apt-get install` extra packages that will be available to all future builds.

The best way to manage extra packages depends on your personal workflow. For example, if you spend half your time fixing packaging errors in other people's [Go](#) packages, you might want to maintain both normal and `-go` variants of all your build environments. Or if you're packaging your own software and always compile your man pages from [DocBook](#) with [DebianPkg: pandoc](#), you might prefer to add `EXTRAPACKAGES="$EXTRAPACKAGES pandoc"` to your `/etc/pbuilderrc`.

 [the "temporary base environment" pbuilder trick](#) might provide some inspiration about ways to manage your process.

Mirror repositories locally with apt-cacher-ng

`pbuilder` runs several `apt-get` commands to upgrade and install packages. It maintains a package cache in `/var/cache/pbuilder/aptcache/` that significantly reduces downloads, but may not be enough in some cases. For example, you might want to cache Release files from `apt-get update`, or might want to share a cache between all the devices on your network.

Install [DebianPkg: apt-cacher-ng](#) then add this to your `/etc/pbuilderrc`:

```
APTCACHE=""  
  
export http_proxy=http://127.0.0.1:3142
```

This will switch from `pbuilder`'s built-in caching mechanism to `apt-cacher-ng`, both while building packages and while creating environments.

See also [AptCacherNg](#).

Keep packages updated

When `pbuilder` builds a package, it starts by running `apt-get update` and `apt-get upgrade`. This will only take a few seconds when you first create an environment, but becomes increasingly time-consuming as your base image gradually falls behind.

Updating an environment in place can cause weird bugs, like config files not being updated as expected. Instead of calling `pbuilder update`, it's better to just [delete](#) and [recreate](#) your environments.

The best schedule for updating your packages depends on your personal workflow. For example, if you're a Debian developer who builds packages all day, you might want to automatically recreate each of your environments overnight. Or if you maintain a mature project and want to check each year's release builds properly, you might prefer to just build new environments as part of the release process and delete them once it's done.

Create environments

`pbuilder` usually creates an environment by running a [tool to create a build system](#), and can require complicated configuration in some cases.

Create an environment with raw `pbuilder`

To create a Debian environment with `pbuilder`, do:

```
sudo pbuilder \
  create \
    --distribution <dist> --architecture <arch>

# Note: the command-line parser is a bit fussy...

# NO! '--key=value' is not supported by `pbuilder`:
#sudo pbuilder \
#  create \
#    --distribution=<dist> --architecture=<arch>

# NO! Must be 'create <options>', not '<options> create':
```

```
#sudo pbuilder \
# --distribution <dist> --architecture <arch> \
# create
```

This will create a .tgz file in /var/cache/pbuilder. If you previously added EATMYDATA=yes to your /etc/pbuilder.rc, you might see some lines like this:

```
ERROR: ld.so: object 'libeatmydata.so' from LD_PRELOAD cannot be preloa
```

These indicate eatmydata has not yet been installed, which is normal - it's an extra package, and is installed towards the end of the creation process. That message should stop appearing as soon as libeatmydata1 is unpacked.

 you may be interested in [a trick to support DIST= and ARCH= variables](#)

Create an environment with cowbuilder

cowbuilder passes most arguments through to pbuilder, so it can be called the same way:

```
sudo pbuilder \
create \
--distribution <dist> --architecture <arch>
```

This will create a .cow directory instead of a .tgz file, and with the cowedancer package added to the environment. cowbuilder supports --key=value command-line arguments, but it's better to stick with --key value for compatibility with pbuilder.

 you may be interested in [a trick to support DIST= and ARCH= variables](#)

Create an environment with git-pbuilder

git-pbuilder create passes many arguments through to sudo cowbuilder, but intercepts --distribution and --architecture, which must be passed via environment variables instead:

```
DIST=<dist> ARCH=<arch> git-pbuilder create
```

Note that the environment variable `DIST` maps to the command-line argument `--distribution - DISTRIBUTION=` and `--dist=` are not supported.

`git-pbuilder create` provides `git-buildpackage` syntax for `pbuilder` commands, so you will probably find this more intuitive than `cowbuilder` if your workflow involves other `git-buildpackage` commands. But you can call `cowbuilder` directly if you like, or even switch back and forth between them.

Advanced ways to create environments

Some types of environment need extra work to create properly.

Create packages for non-default mirrors and components

Build environments usually only include packages from the main component of the distribution's primary mirror. But you might need to add other locations to make your package compile. For example, you might want to compile a package for `stable-backports` that depends on other libraries in `stable-backports`, or coordinate the release of a `security` package that depends on other programs with security-related API changes.

When creating an environment, specify the components you want to include from your distribution's primary mirror by appending

`--components "main,<other-components>"` to your `create` command-line.

When creating an environment, specify extra lines to add to `sources.list` by appending `--othermirror 'deb <uri> <suite> <components>'` to your `create` command-line.

To modify an environment after creating it, use e.g.

`sudo pbuilder login --save-after-login` and edit your `/etc/apt/sources.list`.

 for more information, see the [Include specific Debian sources](#) trick

Create an Ubuntu environment

To create an Ubuntu environment with raw `pbuilder`, install `ubuntu-keyring` (and `ubuntu-archive-keyring` in [buster](#) and earlier), then append

`--mirror http://archive.ubuntu.com/ubuntu/` to the `pbuilder` command-line. To create an Ubuntu environment with any other wrapper, also append `--components "main,universe"` to the command-line. For example:

```
sudo apt install ubuntu-keyring
# Only needed in Buster and earlier:
```

```
#sudo apt install ubuntu-archive-keyring

# Use pbuilder directly:
sudo pbuilder \
    create \
    --distribution <dist> --architecture <arch> \
    --mirror http://archive.ubuntu.com/ubuntu/

# cowbuilder needs an extra argument:
sudo cowbuilder \
    create \
    --distribution <dist> --architecture <arch> \
    --mirror http://archive.ubuntu.com/ubuntu/ \
    --components "main,universe"
```

The `--components` argument is required because `cowbuilder` installs the `cowdancer` package, which is in Ubuntu's "universe" component.

Create an environment manually

Some distributions aren't supported by `debootstrap`, but do provide a downloadable root tarball. This is particularly useful for Raspberry Pi OS armhf (*Raspbian*), which is [subtly incompatible with Debian](#), so packages need to be compiled using a Pi toolchain.

First, learn how to [construct chroots for your target distribution and architecture](#), and make sure you can [run foreign binaries transparently](#).

The next steps depend on your preferred wrapper and which [packages for building packages](#) you need to add. Adapt this to suit your requirements:

```
# Pick a distribution name that won't conflict with any existing na
DIST=raspbian

# Unpack your tarball into an appropriate directory:
sudo mkdir -p /var/cache/pbuilder/base-"$DIST".cow
sudo tar \
    -C /var/cache/pbuilder/base-"$DIST".cow \
    -xf /path/to/tmp/root.tar.xz

# Install missing packages:
sudo chroot /var/cache/pbuilder/base-"$DIST".cow apt-get -yy \
```



```
install <packages for building packages>

# Update all packages:
sudo cowbuilder update --basepath=/var/cache/pbuilder/base-"$DIST".
```

You should now be able to use your environment in the normal way.

⚠ [Mirror repositories locally with apt-cacher-ng](#) if you use environments that aren't supported by `debootstrap`. They might use repositories with package filenames that conflict with Debian - for example, `pbuilder` might cache the Debian version of `libc6` in `/var/cache/pbuilder/aptcache`, then fail with a "wrong checksum" error when it tries to install that same package in an environment that ships a different version of `libc6` with the same filename.

Build packages

To build a package with `pbuilder` or `cowbuilder`, create a [dsc](#) file then run e.g. `sudo cowbuilder --build your-package.dsc`. The result will be placed in `/var/cache/pbuilder/result/`.

Because `git-pbuilder` is part of the `git-buildpackage` suite, it's usually called by a program that creates a `.dsc` file for you. For details, see [PackagingWithGit](#).

⚠ If you run multiple instances of `pbuilder` in parallel, either give them separate `aptcache` directories, or use [use apt-cacher-ng](#) and disable the `aptcache` directory with `--aptcache ""`.

Complete example

Go to [DebianPkg: the unstable source for GNU Hello](#) and download the files listed under **Download hello**.

Then in the directory where you downloaded the files, do `sudo cowbuilder build hello_*.dsc`.

When the build process finishes, look in `/var/cache/pbuilder/result/` - you should see a collection of build files, including an installable `.deb`.

Delete environments

To delete a `pbuilder` environment, first check there are no leftover mounts by doing `sudo mount | grep /var/cache/pbuilder`, then delete its associated file or directory (e.g. `/var/cache/pbuilder/base.tgz`). There is no other database of `pbuilder` environments that needs to be updated.

When `pbuilder` crashes, it usually leaves behind a build environment in `/var/cache/pbuilder/build/`. Anything in that directory can be safely deleted once the associated build job is complete and mount-points have been unmounted.

See also

- [UbuntuWiki: Ubuntu's pbuilder HOWTO](#)
- [Common pbuilder tricks](#)
- [🌐 pbuilder manual](#)
- [git-dpm](#) and [DebianPkg: gitpkg](#) - tools for maintaining Debian packages with git
 - these seem to create files to be passed to e.g. `cowbuilder`

[CategoryPackaging](#)

pbuilder (last modified 2025-07-17 14:45:45)

- [sbuild](#)

Translations: [English](#) - [Português \(Brasil\)](#) - [简体中文](#)

[DebianPkg: sbuild](#) is a tool to make and test binary packages (.deb) from debian source packages (.dsc). For alternatives, see [Package build tools](#). For more general package maintenance, see [Package maintenance tools](#).

Contents

1. [What is sbuild](#)
2. [Setup](#)
 1. ["Updating" chroot manually \(with unshare\)](#)
3. [Building packages](#)
 1. [Standalone](#)
 2. [Integration with git-buildpackage \(gbp-buildpackage\)](#)
4. [Speeding up build process](#)
 1. [Use apt-cacher-ng](#)
 1. [Hash sum mismatch errors](#)
 2. [Building on tmpfs \[pre-trixie\]](#)
 3. [Using ccache with sbuild](#)
 4. [use eatmydata](#)
 5. [Use schroot instead of unshare](#)
5. [Advanced Tips](#)
 1. [Cross-compiling packages](#)
 2. [External Commands](#)
 3. [Enabling experimental](#)
 4. [Build for experimental](#)
 5. [Enabling incoming.debian.org](#)
 6. [Using aliases](#)
 7. [Adding extra packages](#)
 8. [Remote build servers](#)
 9. [Validate package cleanup](#)
 10. [Using other virtualization servers for autopkgtests](#)
 1. [Using podman for autopkgtests](#)
 2. [Using qemu for autopkgtests](#)
 11. [Using /var/cache/apt/archives/ as package cache](#)

What is sbuild

What is [DebianPkg: sbuild](#) for:

- sbuild is used on the official [buildd](#) network to build binary and source packages for all supported architectures.
- sbuild can also be used by individuals to test that their packages build in a minimal Debian installation.
- sbuild can help ensure that you haven't missed any build dependencies.
- sbuild can test packages by running [lintian](#), [piuparts](#), [autopkgtest](#) or [other commands](#).
- sbuild can optionally produce a .changes file suitable for making a [source-only upload](#) using classic dput.
 - This last feature allows you to skip a separate `dpkg-buildpackage -S` execution after doing a final test binary build.
 - This last feature is irrelevant if you use modern `dgpgit push-source` for source-only uploads.

This main part of this page is intended as a short guide to sbuild. It documents how to set up sbuild and build packages with it. The later parts document optional enhancements to the simple setup described in the first section.

Setup

There are many ways to use sbuild. This section documents a modern configuration, using [DebianPkg: mmdebstrap](#) and unshare, which allows you to build and test packages without being root. In this configuration packages are built and tested in a clean and ephemeral chroot that is extracted from a tarball.

1. Install necessary packages:

```
sudo apt install sbuild mmdebstrap uidmap piuparts
```

1a. in case your user does not have subuids (old installation; adopt the range as needed):

```
sudo usermod --add-subuids 100000-165535 --add-subgids 100000-165535 <user>
```

2. Create a directory for the chroot tarball

```
mkdir -p ~/.cache/sbuild
```

3. Create/Update the tarball

```
mmdebstrap --include=ca-certificates --skip=output/dev --variant=build unstable ~/.cache/sbuild/unstable-amd64.t
```

Change `deb.debian.org` to any mirror if you need. `ca-certificates` is needed to support downloading packages over `https`: if you use `http` you can omit the `--include=ca-certificates`. See [packages for building packages](#) for a full list of packages you might need to include. The chroot is built in `/tmp`, if that's a `tmpfs` it might be too small. Set `TMPDIR` to a different place to remedy that.

You can use any `mmdebstrap` arguments such as `--include=gnupg,debhelper` to customise the chroot. The chroot is extracted into a temporary directory whenever it is used, ensuring builds and tests happen in a clean environment.

If you are using [apt-cacher-ng](#), specify `http://` instead of `https://` and enable the `apt-cacher-ng` proxy using `--aptopt='Acquire::http { Proxy "http://127.0.0.1:3142"; }'`. You can use any tarball compression extension you prefer.

```
mmdebstrap --skip=output/dev --variant=build unstable ~/.cache/sbuild/unstable-amd64.tar.zst http://deb.debian.c
```

You can choose the compression algorithm for the tarball by specifying the extension (`.tar.xz`, `.tar.gz` or plain `.tar`, etc). As of May 2024, ZST seems to provide the best size/time ratio. It certainly is the fastest on a Dell Precision 3800M, 16GB RAM, on an SSD drive (a computer from early 2015):

Format	Tarball size	Time
.xz	~100MB	179,60s user 7,09s system 75% cpu 4:07,49 total
.gz	~150MB	38,51s user 6,13s system 83% cpu 53,423 total
.zst	~139MB	22,68s user 6,28s system 74% cpu 38,868 total

4. Configure `~/.config/sbuild/config.pl`

The command below will create a new `~/.config/sbuild/config.pl` with suitable options. It will overwrite any existing file, so make sure you do not already have an `~/.config/sbuild/config.pl` or create a backup copy.



Use `~/.sbuildrc` with older sbuild

Before sbuild version 0.87.1, you must use `~/.sbuildrc` instead of `~/.config/sbuild/config.pl`. See [Closed in sbuild/0.87.1: #1087379: sbuild: Please add support for XDG Base Directory Specification: #1087379](#) and [0.87.1 changelog](#).

```
mkdir -p ~/.config/sbuild/
cat << "EOF" > ~/.config/sbuild/config.pl
# Set the chroot mode to be unshare.
$chroot_mode = 'unshare';

$external_commands = { "build-failed-commands" => [ [ '%SBUILD_SHELL' ] ] };

# Uncomment below to specify the distribution; this is the same as passing '-d unstable' to sbuild.
# Specifying the distribution is currently required for piuparts when the changelog targets UNRELEASED. See #
#$distribution = 'experimental';
#$distribution = 'unstable';
```

```

#$distribution = bookworm-backports';

# Specify an extra repository; this is the same as passing `--extra-repository` to sbuild.
#$extra_repositories = ['deb http://deb.debian.org/debian bookworm-backports main'];
#$extra_repositories = ['deb http://deb.debian.org/debian experimental main'];

# Specify extra local packages to make available to the build environment.
# The paths must be absolute paths from root; this is the same as passing `--extra-package` to sbuild.
#$extra_packages = ['/path/to/package1.deb', '/path/to/package2.deb'];

# Specify the build dependency resolver; this is the same as passing `--build-dep-resolver` to sbuild.
# When building with extra repositories, often 'aptitude' is better than 'apt' (the default).
#$build_dep_resolver = 'aptitude';

# Specify the exact version of a required dependency; this is the same as passing `--add-depends` to sbuild.
#$manual_depends = ['foo-dev (>= 1.2.3-4)', 'bar (>= 5.0-1)'];

# Build Architecture: all packages; this is the same as passing `-A` to sbuild.
$build_arch_all = 1;

# Build the source in addition to the other requested build artifacts.
# Without this, <BINARY>.changes files will not include the upstream source in
# their list and will fail uploads to Debian if they are for a -1 revision that
# includes a new upstream release; this is the same as passing `-s` to sbuild.
$build_source = 1;

# Produce a source.changes file suitable for a source-only upload;
# this is the same as passing `--source-only-changes` to sbuild.
$source_only_changes = 1;

## Run lintian after every build (in the same chroot as the build); use --no-run-lintian to override.
$run_lintian = 1;
# Display info tags.
$lintian_opts = ['--display-info', '--verbose', '--fail-on', 'error,warning', '--info'];
# Display info and pedantic tags, as well as overrides.
#$lintian_opts = ['--display-info', '--verbose', '--fail-on', 'error,warning', '--info', '--pedantic', '--show

## Run autopkgtest after every build (in a new, clean, chroot); use --no-run-autopkgtest to override.
$run_autopkgtest = 1;
# Specify autopkgtest options. The commented example below is the default since trixie.
#$autopkgtest_opts = ['--apt-upgrade', '--', 'unshare', '--release', '%r', '--arch', '%a' ];

## Run piuparts after every build (in a new, temporary, chroot); use --no-run-piuparts to override.
# this does not work in bookworm
$run_piuparts = 1;
# Build a temporary chroot.
$piuparts_opts = ['--no-eatmydata', '--distribution=%r', '--fake-essential-packages=systemd-sysv'];
# Build a temporary chroot that uses apt-cacher-ng as a proxy to save bandwidth and time and doesn't disable c
#$piuparts_opts = ['--distribution=%r', '--bootstrapcmd=mmdebstrap --skip=check/empty --variant=minbase --aptc

EOF

```

Make sure to [setup aliases](#) if building for UNRELEASED or experimental unless you want to manually specify the distribution.

When using this example `~/ .config/sbuild/config.pl`, Piuparts generates a temporary, minimal chroot with every run, because doing so gives [Closed: #1088308: sbuild: Piuparts fails when trying to use a pre-build tarball with unshare: more complete results](#) than using the one built with `--variant=buldd`. There are many options for sbuild: see the man pages for `sbuild(1)` and `sbuild.conf(5)` for details.

"Updating" chroot manually (with unshare)

To keep the chroot updated it is recommended to simply delete it and recreate it by re-running the mmdebstrap command (see above). This is fast and makes sure the build environment remains clean.

sbuid-update does not work with unshare. ([Open in sbuid/0.86.3: #1086703: sbuid: sbuid-update does not use chroot-mode from configuration file: 1086703](#)).

Building packages

Standalone

With properly configured `~/ .config/sbuid/config.pl` as above, you can build a package by running the following from the source directory of the package. This will build a source package (`.dsc`) and then build all the binary packages in it.

```
sbuid
```

If you have a `.dsc` (generated by `dpkg-buildpackage`, `git-buildpackage`, etc) you can use that:

```
sbuid package_*.dsc
```

If you have `deb-src` lines configured for `apt` you can build any package without downloading the source code first:

```
sbuid -d unstable hello
```

If you get an error about missing build-dependencies, that could be related to the default behaviour of sbuid since version 0.87.1 requiring build-dependencies for cleaning the build environment. See [Open in sbuid/0.87.1: #1088269: sbuid 0.87.1 fails on build dependencies missing from the host system: #1088269](#) for more context and discussion about this. One way to by-pass the issue is to take care of cleaning sources by yourself and launch sbuid with the `--no-clean` parameter.

Integration with git-buildpackage (gbp-buildpackage)

To build a package from a git repository managed with [DebianPts: git-buildpackage](#) run

```
gbp buildpackage --git-builder=sbuid
```

or put the following in `~/ .gbp.conf`:

```
[buildpackage]
builder = sbuid
```

This creates a source package from the git repository (on the host machine), and then runs sbuid to build and test the binary packages (in a chroot) There is more information about packaging with git at [PackagingWithGit](#).

Speeding up build process

Use apt-cacher-ng

```
sudo apt install apt-cacher-ng
```

If you use sbuid-generated chroots directly, append this line to `~/ .config/sbuid/config.pl`:

```
push @{$unshare_mmdebstrap_extra_args}, "*", ['--aptopt=Acquire::http { Proxy "http://127.0.0.1:3142"; }'];
```

If you want to create your own chroot with mmdebstrap, add this option:

```
--aptopt='Acquire::http { Proxy "http://127.0.0.1:3142"; }'
```

Using `auto-apt-proxy` is not recommended, as it will introduce non-essential packages into the build environment.

Hash sum mismatch errors

`apt-cacher-ng` clients may (regularly) report errors about hash sum mismatches that yield non-valid key signatures. One way to solve this is to clean the expired cache files, as documented at the [apt-cacher-ng](#) wiki page.

Building on tmpfs [pre-trixie]

Since trixie this is no longer needed because `/tmp` is now a tmpfs by default.

```
echo "\$unshare_tmpdir_template = '/dev/shm/tmp.sbuild.XXXXXXXXXX';" >> ~/.config/sbuild/config.pl
```

This requires `/dev/shm` to be large enough to unpack the chroot and run the build process: to temporarily enlarge it you can do (as root)

```
mount -o remount,size=2G /dev/shm
```

Using ccache with sbuild

[DebianPts: ccache](#) is a compiler wrapper that caches compilation results (object files) from `gcc` and `g++`. By retrieving object files from the cache instead of recompiling them, it can significantly reduce packaging time during development and for packages that frequently get small updates. For more information, see [ccache](#).

Add this to the `mmdebstrap` options when creating the chroot tarball:

```
--include=ccache --customize-hook='chroot "$1" update-ccache-symlinks'
```

(`update-ccache-symlinks` is needed due to [Open in ccache/3.1.5-2, ccache/3.2.4-1, ccache/4.6.1-1, ccache/4.2-1: #632779: cc and c++ symlinks aren't updated when installing gcc or g++ after ccache: 632779](#))

Add this to your sbuild configuration:

```
cat << "EOF" >> ~/.config/sbuild/config.pl
$build_environment = { "CCACHE_DIR" => "/build/ccache" };
$path = "/usr/lib/ccache:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games";
$build_path = "/build/package/";
$dsc_dir = "package";
$unshare_bind_mounts = [ { directory => "$HOME/.cache/ccache", mountpoint => "/build/ccache" } ];
$autopkgtest_opts = [ '--apt-upgrade', '--env=CCACHE_DIR=/build/ccache', '--env=PATH=/usr/lib/ccache:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games' ];
EOF
```

The sbuild user in the chroot needs to have access to `$HOME/.cache/ccache` which is on the host. [Open in sbuild/0.89.3: #1110942: sbuild: Possible improvement to security of shared .ccache: See also discussion here](#)

You can either:

1. `chmod a+X "$HOME" "$HOME/.cache"` and `chmod -R a+rwX "$HOME/.cache/ccache"` on the host
2. Ensure access to the `$HOME/.cache` and using ACL (access control lists). This allows finer grained access control:
 - Find out subuid that sbuild will use:
`BASE_SUBUID=$(grep $(whoami) /etc/subgid | cut -d ':' -f 2); echo "$BASE_SUBUID $((BASE_SUBUID + 999))"`
 - `chmod a+X "$HOME" "$HOME/.cache"`

- `setfacl -m u:<BUID1>:rwX -m u:<BUID2>:rwX "$HOME/.cache/ccache"` Where *<BUID1>* and *<BUID2>* are the ids returned in the first bullet

3. Use another cache location (note that files in `/tmp` are at risk of being wiped on reboot or by `systemd`'s periodic cleanup).

use eatmydata

It is unclear how much this helps if the chroot is extracted to a tmpfs, but running [DebianPts: eatmydata](#) may provide a speed boost by preventing data being written to the disk as often -- this is, obviously, with a risk of data corruption. It only affects the package build, not the rest of your system. To use it:

1. Include eatmydata in the chroot: add `--include=eatmydata` to the `mmdebstrap` options when creating the chroot tarball.
2. Use eatmydata in the build: add the following to the `sbuild` configuration: this chroot is amd64: you can adapt the path on other architectures.

```
# use it in the chroot
$external_commands = {
    'chroot-setup-commands' => [ "export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libeatmydata.so" ]
};

# use it in the build
$build_environment = {
    "LD_PRELOAD" => "/usr/lib/x86_64-linux-gnu/libeatmydata.so"
};

# add LD_PRELOAD to variables retained
$environment_filter=[Dpkg::BuildInfo::get_build_env_allowed(), '^LD_PRELOAD$'];
```

3. (You may want to add `--no-eatmydata` to `$piuparts_opts` since `piuparts` also enables `eatmydata` by default).
4. You can then call `sbuild` as usual: you may want to also install `eatmydata` on the host and call `sbuild` as

```
LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libeatmydata.so sbuild
```

which may help speed up the unpacking of the chroot tarball.

Use schroot instead of unshare

`sbuild` can use chroots created by [DebianPts: schroot](#), these use `overlayfs` which can be faster than `unshare`. However, this requires greater use of root than `unshare`: with `schroot` you can run the build, lintian and autopkgtests as a user (if root has granted access), but `piuparts` and creating a new chroot requires root (or permission to elevate privileges with [DebianPts: sudo](#) or similar).

Advanced Tips

Cross-compiling packages

`sbuild` also supports cross-compiling a package to build it for a different processor architecture. For example, you can build `arm64` or `ppc64el` packages on an `amd64` machine.

To build a package for `arm64` in an `amd64` chroot (as created above), you can use:

```
sbuild --host=arm64
```

NB: There's no need to cross-compile packages marked as `arch:all` as they are independent of architecture.

External Commands

`sbuild` supports running external commands at various stages of the build process. For example you can run a script inside the chroot after it has been setup.

To run `/usr/local/bin/myscript`, add it to the chroot tarball and include the following in your sbuild configuration:

```
$external_commands = {
    'chroot-setup-commands' => ['/usr/local/bin/myscript']
};
```

sbuild supports some percent-escaped keywords here. For example, `%SBUILD_CHANGES` is changed to the path of the `.changes` file for a successfully built package.

Here is an example of adding a post build command to run `/usr/local/bin/postbuildscript` with `%SBUILD_CHANGES` as an argument.

```
$external_commands = {
    'post-build-commands' => ['/usr/local/bin/postbuildscript', '%SBUILD_CHANGES'],
    'chroot-setup-commands' => ['/usr/local/bin/myscript'],
    'chroot-cleanup-commands' => [],
    'pre-build-commands' => []
};
```

Here is how to drop into a shell inside the ephemeral chroot if the build fails

```
$external_commands = {
    "chroot-update-failed-commands" => [ [ '%SBUILD_SHELL' ] ],
    "build-deps-failed-commands" => [ [ '%SBUILD_SHELL' ] ],
    "build-failed-commands" => [ [ '%SBUILD_SHELL' ] ],
};
```

See the 'EXTERNAL COMMANDS' section of the sbuild man page for more information on external commands.

Enabling experimental

To allow the build to use packages from the Debian experimental repository you can use the `--extra-repository` argument. You may also want to use the `aspcud` resolver:

```
sbuild --extra-repository='deb http://deb.debian.org/debian experimental main' --build-dep-resolver=aspcud mypkg.
```

To exactly mimic the resolver used on the official builddds you need to add aspcud preferences that minimise the number of packages from experimental. See the DSA puppet repository at <https://salsa.debian.org/dsa-team/mirror/dsa-puppet/-/blob/production/modules/builddd/templates/sbuild.conf.erb>

```
sbuild --extra-repository='deb http://deb.debian.org/debian experimental main' --build-dep-resolver=aspcud --aspcud
```

You can also use the aptitude resolver (used by `*-backports`), which for most packages should have the same effect.

```
sbuild --extra-repository='deb http://deb.debian.org/debian experimental main' --build-dep-resolver=aptitude mypkg
```

Another possibility is to have a dedicated `~/ .config/sbuild/config.experimental.pl` configuration file and run sbuild via

```
SBUILD_CONFIG=~/ .config/sbuild/config.experimental.pl sbuild mypkg.dsc
```

with `~/ .config/sbuild/config.experimental.pl` containing for example:

```
$extra_repositories = [ 'deb http://deb.debian.org/debian experimental main' ];
$build_dep_resolver = 'aptitude';
```

If you need to test a build against a versioned Build-Depends from experimental, you can use `--add-depends`:

```
sbuild --extra-repository='deb http://deb.debian.org/debian experimental main' --build-dep-resolver=aspcud --add-
```

Build for experimental

If you want to build for unstable but upload to experimental, use, for example,

```
sbuild -d experimental -c unstable-amd64-sbuild mypkg.dsc
```

Alternatively, if you know that you always want to build your packages for experimental in a unstable chroot, just add `experimental` as an alias for your unstable chroot:

```
cd ~/.cache/sbuild
ln -s {unstable,experimental}-.*.tar*
```

Enabling incoming.debian.org

If you want to build before [the next dinstall](#) you can add the `incoming` queue as an extra repository:

```
sbuild --extra-repository='deb http://incoming.debian.org/debian-build/ build-unstable main' mypkg.dsc
```

Using aliases

`sbuild` chooses the chroot based on the distribution in `debian/changelog`. You can change this with the `-d` option or you can use symlinks to set up aliases so that source package with `UNRELEASED` or `experimental` in `debian/changelog` are built in an unstable chroot (or so that `-d sid` can be used to build in unstable, etc):

```
cd ~/.cache/sbuild
for file in unstable-*.tar*; do
    ln -s "$file" "${file/unstable/UNRELEASED}"
done
for file in unstable-*.tar*; do
    ln -s "$file" "${file/unstable/experimental}"
done
```

Adding extra packages

It is often necessary to add extra binary packages as build dependencies for the package you are building. For example, you might want to provide a build dependency that is waiting in the NEW queue and so isn't available from the mirrors ; or you're precisely trying to assess if your new package breaks anything before uploading. To do this, use the `--extra-package=/path/to/foo.deb` option to `sbuild` ; and if there are several such packages, `--extra-package=/path/to/` will also scan this directory. **The path must be an absolute path on the host, and `~` is not automatically expanded to `$HOME`!**

You might find that the output from the resolver is not helpful in determining which extra package you need to make available. In this case, it can be useful to pass `--build-dep-resolver=aptitude` which tends to provide more useful output (though you should remove it once you've figured out the problem).

Remote build servers

One advantage of `cowbuilder` over `sbuild` is that it supports offloading builds to a remote server with `cowpoke`. One way to do this with `sbuild` is to create a source package locally and transfer it to the remote machine for building. The latter can be done with `dcmd`.

Example:

```
dpkg-buildpackage -S
dcmd scp ../foo-1.0.dsc example.net:build-area
ssh example.net sbuild build-area/foo-1.0.dsc
```

Validate package cleanup

Packages will fail to build twice in a row if the `clean` target of `debian/rules` does not restore the source directory to its initial state. Adding the following to your `sbuild` configuration will help you to detect modifications:

```
$external_commands = {
  "starting-build-commands" => [
    'bash -c \'find %SBUILD_PKGBUILD_DIR -print0 |
      sort -z |
      xargs -0 -I{} bash -c "
        stat --printf "%n %F %s \\'\'{}\'\'
        if [ -f \'\'{}\'\' ]; then sum -s \'\'{}\'\' | cut -d\'\' \'\' -f1; else echo; fi
      \'\' > /tmp/file-list.pre-build\'\'',
  ],
  "chroot-cleanup-commands" => [
    'cd %SBUILD_PKGBUILD_DIR && ./debian/rules clean',
    'bash -c \'find %SBUILD_PKGBUILD_DIR -print0 |
      sort -z |
      xargs -0 -I{} bash -c "
        stat --printf "%n %F %s \\'\'{}\'\'
        if [ -f \'\'{}\'\' ]; then sum -s \'\'{}\'\' | cut -d\'\' \'\' -f1; else echo; fi
      \'\' > /tmp/file-list.post-build\'\'',
    'diff /tmp/file-list.pre-build /tmp/file-list.post-build'
  ]
};
```

The check above assumes that the maintainer wants to restore the state preceeding the build, while some generated contents should ideally be removed before the build, so that their contents are never used unwantedly. The following recipe checks that `clean` results in the same state before and after the build. Most issues it reports are solved by copying the slightly modified `diff` output to `debian/clean`.

`dh_assistant restore-file-on-clean` may be useful for the rare files that really need to be restored.

[Toggle line numbers](#)

```
__1 my $sums_gen = q{find -type f -printf '%P %m ' -exec cksum -z '{}' ';' }
__2 . q{ -o -type l -printf '%P -> %l\\0' | sort -z | tr '\\0' '\\n'};
__3 my $sums_file = '/tmp/sums-preclean';
__4 $dpkg_buildpackage_user_options = [
__5   "--hook-build=$sums_gen > $sums_file",
__6   '--post-clean',
__7   "--hook-check=$sums_gen | diff $sums_file -",
__8 ];
```

Using other virtualization servers for autopkgtests

`autopkgtest` can run tests in a variety of environments. The following examples are optimised for `sbuild` - for general examples, see [autopkgtest](#).

Using podman for autopkgtests

```

sudo apt install podman passt
mmdebstrap --include=libpam-systemd,systemd-resolved --customize-hook='sed "s/^deb /deb-src /" "$1/etc/apt/sources.list"'
cat << "EOF" >> ~/.config/sbuild/config.pl
$autopkgtest_root_args = 'PATH=/usr/sbin:/usr/bin:/sbin:/bin';
$autopkgtest_opts = ['--shell-fail', '--apt-upgrade', '--', 'podman', '--init', 'debian-%r'];
EOF

```

Using qemu for autopkgtests

```

sudo apt install qemu-system
mmdebstrap-autopkgtest-build-qemu --boot=efi unstable $HOME/.cache/sbuild/unstable-amd64.img
cat << "EOF" >> ~/.config/sbuild/config.pl
$autopkgtest_opts = ['--shell-fail', '--apt-upgrade', '--', 'qemu', '--cpus', '4', '--ram-size', '8096', '--extra-repository="deb [trusted=yes] http://127.0.0.1:5678 ./" --chroot-setup-commands "rm /etc/apt/sources.list"'
EOF

```

Using /var/cache/apt/archives/ as package cache

```

sudo apt -o Dir::State::status=lock build-dep -d <pkg>
sudo apt -o Dir::State::status=lock install -d lintian
cd $(mktemp -d)
ln -s /var/cache/apt/archives/
apt-ftparchive packages . > Packages
apt-ftparchive release . > Release
python3 -m http.server 5678 --bind 127.0.0.1
sbuild --extra-repository="deb [trusted=yes] http://127.0.0.1:5678 ./" --chroot-setup-commands "rm /etc/apt/sources.list"

```

[CategoryPackaging](#) [CategoryPackaging](#)

sbuild (last modified 2026-05-11 21:06:08)

- [Lintian](#)

[DebianPkg: lintian](#) is a comprehensive **package checker** for Debian packages. For similar tools, see [Package-checking tools](#). For more general package maintenance, see [Package maintenance tools](#).

Lintian tries to check for

- Debian Policy violations and violations of various sub-policies,
- best practices,
- common mistakes,
- and problems that maintainers like to catch before uploads.

It is maintained by the [lintian team](#)

Derivatives (Vendors) use it. this [guide](#) describes how to setup a Lintian vendor profile and have Lintian recognise the release name of the distro.

See also

-  [Lintian User Manual](#)
- [DebianMan: Creating overrides with dh_lintian](#)
- [DebianPkg: lintian-brush](#) automatically fixes lintian problems

Lintian ([last modified 2025-06-03 16:54:47](#))

- [piuparts](#)

[DebianPkg: piuparts](#) tests `.deb` packages by installing, upgrading, and removing packages in a minimal Debian chroot. For more information, see [piuparts.debian.org](#). For similar tools, see [Package-checking tools](#). For more general package maintenance, see [Package maintenance tools](#).

`piuparts` is run constantly on all suites in the main Debian archive, the results are available at [piuparts.debian.org](#).

 [Open in piuparts/1.5.1: #1088928: piuparts: fails to convert UNRELEASED to unstable for apt: piuparts does not convert "UNRELEASED" to "unstable"](#). Developers often set the distribution to "UNRELEASED" in their changelog until it's time to release, because doing so avoids accidental uploads. If you want to pass that value from the changelog to `piuparts` (e.g. with `dpkg-parsechangelog`), handle "UNRELEASED" as a special case.

Contributing

If you are interested in contributing, that's great! Here's a list to get started:

- join [development](#) efforts. Every little pull request helps. Check the BTS for plenty of wishlist bugs to tackle.
- [file bugs](#) based on `piuparts.d.o` runs!
- improve the documentation, both these wiki pages and what's in GIT has been growing over the years and it's showing.

See also

- [piuparts README](#)
- [DebianMan: manpage](#)
- [Howto guides](#)
- [FAQ](#)
- [Resources about piuparts.debian.org & distributed testing](#)

[CategoryPermalink](#) [CategoryPackaging](#)

- [autopkgtest](#)

[DebianPkg: autopkgtest](#) runs test suites as defined in [DEP-8](#). For how to use it within Debian's continuous integration framework, see [ContinuousIntegration/autopkgtest](#). For similar tools, see [Package-checking tools](#). For more general package maintenance, see [Package maintenance tools](#).

Contents

1. Background

2. Add tests to your package

3. Run tests

1. Run tests in your development system

2. Run tests with unshare

3. Run tests with podman

4. Run tests with QEMU

4. See also

Background

Two of the most common ways to test software are [WikiPedia: unit testing](#) and [WikiPedia: integration testing](#) - i.e. checking whether (a component of) a system works correctly within itself, and checking whether (a component of) a system interacts properly with its surroundings.

Projects often provide a command like `make test` that runs unit tests, and sometimes also runs integration tests between units. But packagers also need to check the whole package integrates properly with the rest of the system. For example, does a program crash if its configuration file is missing? Does a library fail to load because a build-dependency should have been a runtime dependency?

To run `make test` during the build process, see [DebianMan: dh_auto_test](#). Or to create whole-package integration tests, continue reading this page.

Add tests to your package

First, add one of the following to your `debian/control` file:

```
# For Python packages:
Testsuite: autopkgtest-pkg-python

# For Perl packages:
Testsuite: autopkgtest-pkg-perl
```

```
# Other types of package:

Testsuite: autopkgtest-pkg-...

# Default value:

Testsuite: autopkgtest
```

For a complete list of `autopkgtest-pkg-` suites, see [DebianMan: EXAMPLES OF PRODUCED TEST CONTROL FILES in autodep8's man page](#).

Second, define the tests you want to run. If you used an `autopkgtest-pkg-` suite, you can probably use [DebianMan: the tests produced by autodep8](#). If you used the default `autopkgtest` suite, or need to extend the produced tests, create a new `debian/tests/control` file with a collection of blocks that look like this:

```
Tests: test1.sh, test2.py, test3.pl, ...

Depends: @builddeps@, pkg1, pkg2 [amd64] | pkg3 (>= 3), ...

Restrictions: needs-root, breaks-testbed, ...

...
```

For special dependency values, see [dependency field syntax](#). For a list of restrictions, see [defined restrictions](#). For the complete list of fields, see [the specification](#).

Finally, create any files required by your `autopkgtest-pkg-` suite, plus files in `debian/tests` named after values in the `Tests` field. For example, `debian/tests/test1.sh` might look like:

```
# exit with a non-zero value to indicate an error:
if ! my-program --version > /dev/null 2>&1
then exit 1
fi

# or print to STDERR to indicate an error:
grep '(rude words)' /usr/share/my-resource/polite-words.txt >&2

# to indicate success, exit successfully without printing to STDERR
exit 0
```

Run tests

`autopkgtest` checks that your package integrates with the wider system, so you need to pick a *virtualization server* to provide a wider system for it to run on.

Run tests in your development system

If you have installed all of a package's dependencies, you can run its tests on your main system with the `null` virtualization server:

```
# Test a source package:
autopkgtest /path/to/my_package.changes -- null

# Test the current directory:
autopkgtest --no-built-binaries -- null
```

Other common arguments to `autopkgtest` include `--shell-fail` (start a shell on test failure) and `--apt-upgrade` (update and upgrade your VM before running tests). For more information about command-line arguments and ways to specify a package, see [DebianMan: the autopkgtest man page](#).

Run tests with unshare

`unshare` runs commands in a new namespace, without needing `root` access. Use this if you want to run tests without creating an environment in advance or giving your test-runner elevated privileges.

The following example creates a new environment running the `i386` version of [unstable](#). Some architectures may require [QEMU user-mode emulation](#).

```
autopkgtest /path/to/my/package.changes \
-- unshare --arch i386 --release unstable
```

Run tests with podman

[podman](#) is an alternative to `docker` that does not require `root` access. Use this if you want to run tests without elevated privileges, and want to reuse an existing container.

You will need an appropriate `podman` image. For details, see [Build images](#) and especially [Build images from Debian environments](#).

```
autopkgtest /path/to/my/package.changes --apt-upgrade \
-- podman --init <your-podman-image>
```

Run tests with QEMU

[QEMU](#) lets you run commands in a virtual machine. This may be necessary if your test needs to e.g. modify kernel variables, but consider [user-mode emulation](#) instead if you just need to run tests on a different architecture.

First, make a disk image with [DebianMan: mmdebstrap-autopkgtest-build-qemu](#) or [DebianMan: autopkgtest-build-qemu](#). Adapt this to your requirements:

```
# Pick whichever of these best suits your workflow:
mmdebstrap-autopkgtest-build-qemu --efi --architecture=i386 unstable
sudo autopkgtest-build-qemu unstable /path/to/my_disk.img http://de
```

Then test it in the normal way:

```
# Only pass `--efi` if you used `mmdebstrap-autopkgtest-build-qemu`
autopkgtest /path/to/my/package.changes --apt-upgrade \
    -- qemu --efi /path/to/my_disk.img
```

See also

- [using autopkgtest with Debian's CI framework](#)
- [autopkgtest best practices](#)

[CategoryDeveloper](#)

autopkgtest (last modified 2025-06-03 17:03:24)

- [blhc](#)

[DebianPkg: blhc](#) checks build logs for missing hardening flags. For similar tools, see [Package-checking tools](#). For how to add hardening flags, see [Hardening](#).

"Hardening" is the process of telling your compiler to produce binaries that are better protected against certain types of attack. [Hardening](#) and [HardeningWalkthrough](#) should be enough for most packages, but complex build systems can sometimes run a compiler without setting all the necessary flags.

blhc scans your compilation log for lines that suggest missing flags. Run it like this:

```
blhc /path/to/log/file
```

See also

-  [homepage](#)
-  [how to use blhc to solve hardening issues when packaging](#)

blhc ([last modified 2025-06-03 17:03:58](#))

- [Packaging](#)
- [ruby-team-meta-build](#)

Debian Ruby team maintains a set of scripts that helps testing reverse dependencies of packages when updating an existing package in addition to running clean build with sbuild and autopkgtest with lxc. It runs autopkgtest for all reverse dependencies and rebuilds all reverse build dependencies.

You can clone  [this repo](#) from salsa and use build script. You will need to run setup script first to create all required root file systems and install required packages.

Note: Currently you need to follow  [these steps](#) if you got an error when creating lxc container or your autopkgtests always fail. You will need apparmor installed to have these settings to work.

Window Subsystem for Linux (WSL2)

In setting up ruby-team-meta-build script which makes testing reverse dependencies easy, by default it uses LXC as its default autopkgtests backend which doesn't work in WSL2 so you can configure to use schroot by setting AUTOPKGTEST_VIRT_SERVER to "schroot" and exporting it (`export AUTOPKGTEST_VIRT_SERVER="schroot"`) in `~/ .bashrc` file, autopkgtests will use schroot backend which is suitable for WSL2.

[CategoryPackaging](#) [CategoryRuby](#)

Packaging/ruby-team-meta-build ([last modified 2025-09-07 13:04:59](#))

- [UsingQuilt](#)

Translations: [English](#) - [Italiano](#) - [Português \(Brasil\)](#)

Manage Debian source packages using [DebianPkg: quilt](#) instead of a version control system. For an introduction to the program, see [/usr/share/doc/quilt/quilt.pdf.gz](#) in [DebianPkg: quilt](#). For a discussion of the "quilt" patch system, see [Projects/DebSrc3.0](#). For general packaging information, see [Packaging](#).

Contents

1. General information

1. documentation

2. Basic concepts

2. Using quilt with Debian source packages

1. Using .quiltrc configuration file

2. Environment variables

3. Encapsulated

3. Basic quilt tasks

1. Making a new patch

2. Editing an existing patch

3. Refresh a patch that failed to apply

4. Update the header comments of the patch on the top of the stack

4. Forwarding Patches to Upstream

General information

documentation

- There is an extensive guide on quilt and packaging:  [Debian New Maintainers' Guide](#).
- A comprehensive yet succinct tutorial on quilt,  [How to use quilt to manage patches in Debian packages](#), published by Raphaël Hertzog (coauthor of the New Maintainers' Guide above)
- A How-To from the Debian Perl Group:  [Quilt for Debian Maintainers](#).
- There is also a  [Quilt Tutorial](#) (PDF), though dated (2006)
- If using a git packaging workflow, consider using  [gbp-pq\(1\)](#) for quilt patch management.

Basic concepts

quilt works with some directories : it creates a **.pc/** and a **patches/** directory.

Those directories can be created when you do

```
$ quilt import some_package.diff.gz
```

Using quilt with Debian source packages

Situation: you have downloaded a Debian source package which uses quilt and want to fix a bug and then submit a patch to the maintainers.

Using .quiltrc configuration file

Place a .quiltrc configuration file in your home directory with the following lines.

```
QUILT_PATCHES=debian/patches
QUILT_NO_DIFF_INDEX=1
QUILT_NO_DIFF_TIMESTAMPS=1
QUILT_REFRESH_ARGS="-p ab"
QUILT_DIFF_ARGS="--color=auto" # If you want some color when using `quilt`
QUILT_PATCH_OPTS="--reject-format=unified"
QUILT_COLORS="diff_hdr=1;32:diff_add=1;34:diff_rem=1;31:diff_hunk=1;33"
```

Environment variables

Alternatively, you can set environment variables yourself.

Add these lines to your shell's init scripts (~/.bashrc), and restart your shell to apply the changes. Or, run them in your shell before using quilt.

```
export QUILT_PATCHES=debian/patches
export QUILT_REFRESH_ARGS="-p ab --no-timestamps --no-index"
```

Encapsulated

To apply quilt options only when inside a Debian source package, you can setup your ~/.quiltrc something like this:

```
d=. ; while [ ! -d $d/debian -a `readlink -e $d` != / ]; do d=$d/..; done
if [ -d $d/debian ] && [ -z $QUILT_PATCHES ]; then
    # if in Debian packaging tree with unset $QUILT_PATCHES
    QUILT_PATCHES="debian/patches"
```

```
if ! [ -d $d/debian/patches ]; then mkdir $d/debian/patches; fi
fi
```

Basic quilt tasks

Making a new patch

`apt-get source` decides whether to apply the patches based on the format of the package.

For some packages, you will have to apply the patches as specified here. For others, they will be automatically applied.

First step: apply existing patches to the source

```
quilt push -a
```

to "push" all existing patches onto the source tree (when you build a package, this is done by the build scripts)

Creating a new patch

```
quilt new myPatch.diff # this is the patch name
```

adding a file

```
quilt add README # Where 'README' is the name of the file you want to modify.
```

You have to do that for all files you modify before you change them.

One quilt patch can change multiple files

Now make changes to the source

Now, make your changes to the files added to the patch: edit it, or replace it with an already modified file stored in a different directory.

Updating a patch with changes made to its files

```
quilt refresh #you can do this as often as you like
```

Add description to the header

```
quilt header -e --dep3 #edits the header in $EDITOR starting with the DEP-3 header template
```

Finish your editing

```
quilt pop -a
```

This un-applies all patches so that the source returns to the downloaded condition

Editing an existing patch

quilt supports multiple patches, but you are only ever change one of them.

This is the patch last pushed.

To edit an existing patch, start by pushing it

```
quilt push myPatch.diff
```

Now edit it, and when ready save it

```
quilt refresh myPatch.diff
```

Command line completion may save you some typing

Refresh a patch that failed to apply

If the patch failed to apply (usually when updating to a new upstream release) when you `quilt push` like the example below,

```
$ quilt push
Applying patch CVE-2018-1000544_part1.patch
patching file lib/zip/entry.rb
Hunk #1 FAILED at 147.
1 out of 1 hunk FAILED -- rejects in file lib/zip/entry.rb
patching file test/data/absolutepath.zip
patching file test/entry_test.rb
Patch CVE-2018-1000544_part1.patch does not apply (enforce with -f)
```

```
$ quilt push -f
Applying patch CVE-2018-1000544_part1.patch
patching file lib/zip/entry.rb
Hunk #1 FAILED at 147.
1 out of 1 hunk FAILED -- saving rejects to file lib/zip/entry.rb.rej
patching file test/data/absolutepath.zip
```



```
patching file test/entry_test.rb
Applied patch CVE-2018-1000544_part1.patch (forced; needs refresh)
```

Now you should inspect the *.rej files (lib/zip/entry.rb.rej in this example) and adapt lib/zip/entry.rb to include the desired change.

If the change has been incorporated upstream, you can remove the change from your patch. For example:

```
$ quilt refresh
Diff failed on file 'test/data/absolutepath.zip', aborting
# In this case the change to absolutepath.zip has already been performed

# So let's remove the file from the patch:
$ quilt remove test/data/absolutepath.zip
File test/data/absolutepath.zip removed from patch CVE-2018-1000544_part1.patch
$ quilt refresh
```

Update the header comments of the patch on the top of the stack

```
quilt header -e
```

Now that we've saved our changes, remove all patches and return the source to its original state

```
quilt pop -a
```

Forwarding Patches to Upstream

Non Debian-specific patches, such as patches for bug fixes and feature enhancements, need to be forwarded upstream: [How to forward patches to upstream](#)

[CategoryPackaging](#)

UsingQuilt (last modified 2025-06-11 15:38:04)



Search

package names



devscripts

[all options](#)

Limit to suite: [\[bullseye\]](#) [\[bullseye-updates\]](#) [\[bullseye-backports\]](#) [\[bookworm\]](#) [\[bookworm-updates\]](#) [\[bookworm-backports\]](#) [\[trixie\]](#) [\[trixie-updates\]](#) [\[trixie-backports\]](#) [\[forky\]](#) [\[sid\]](#) [\[experimental\]](#)

Limit to a architecture: [\[alpha\]](#) [\[amd64\]](#) [\[arm\]](#) [\[arm64\]](#) [\[armel\]](#) [\[armhf\]](#) [\[avr32\]](#) [\[hppa\]](#) [\[hurd-i386\]](#) [\[i386\]](#) [\[ia64\]](#) [\[kfreebsd-amd64\]](#) [\[kfreebsd-i386\]](#) [\[loong64\]](#) [\[m68k\]](#) [\[mips\]](#) [\[mips64el\]](#) [\[mipsel\]](#) [\[powerpc\]](#) [\[powerpcspe\]](#) [\[ppc64\]](#) [\[ppc64el\]](#) [\[riscv64\]](#) [\[s390\]](#) [\[s390x\]](#) [\[sh4\]](#) [\[sparc\]](#) [\[sparc64\]](#) [\[x32\]](#)

You have searched for packages that names contain *devscripts* in all suites, all sections, and all architectures. Found **6** matching packages.

Exact hits

Package devscripts

- [bullseye \(oldoldstable\)](#) (devel): scripts to make the life of a Debian Package maintainer easier
2.21.3+deb11u1: amd64 arm64 armhf i386
- [bookworm \(oldstable\)](#) (devel): scripts to make the life of a Debian Package maintainer easier
2.23.4+deb12u2: amd64 arm64 armel armhf i386 mips64el mipsel ppc64el s390x
- [bookworm-backports](#) (devel): scripts to make the life of a Debian Package maintainer easier
2.25.15~bpo12+1: all
- [trixie \(stable\)](#) (devel): scripts to make the life of a Debian Package maintainer easier
2.25.15+deb13u1: all
- [trixie-backports](#) (devel): scripts to make the life of a Debian Package maintainer easier
2.26.7~bpo13+1: all
- [forky \(testing\)](#) (devel): scripts to make the life of a Debian Package maintainer easier
2.26.7: all
- [sid \(unstable\)](#) (devel): scripts to make the life of a Debian Package maintainer easier
2.26.8: all

Other hits

Package devscripts-el

- [bullseye \(oldoldstable\)](#) (lisp): Transition package, devscripts-el to elpa-devscripts
40.5: all
- [bookworm \(oldstable\)](#) (lisp): Transition package, devscripts-el to elpa-devscripts
40.5: all
- [trixie \(stable\)](#) (lisp): Transition package, devscripts-el to elpa-devscripts
40.7: all
- [forky \(testing\)](#) (lisp): Transition package, devscripts-el to elpa-devscripts
40.7: all
- [sid \(unstable\)](#) (lisp): Transition package, devscripts-el to elpa-devscripts
40.7: all

Package elpa-devscripts

- [bullseye \(oldoldstable\)](#) (lisp): Emacs wrappers for the commands in devscripts
40.5: all
- [bookworm \(oldstable\)](#) (lisp): Emacs wrappers for the commands in devscripts
40.5: all
- [trixie \(stable\)](#) (lisp): Emacs wrappers for the commands in devscripts
40.7: all
- [forky \(testing\)](#) (lisp): Emacs wrappers for the commands in devscripts
40.7: all
- [sid \(unstable\)](#) (lisp): Emacs wrappers for the commands in devscripts
40.7: all

Package haskell-devscripts

- [bullseye \(oldoldstable\)](#) (haskell): Tools to help Debian developers build Haskell packages
0.16.0: all
- [bookworm \(oldstable\)](#) (haskell): Debian tools to build Haskell packages (with hscolor)
0.16.29: all
- [trixie \(stable\)](#) (haskell): Debian tools to build Haskell packages (with hscolor)
0.16.34: all
- [forky \(testing\)](#) (haskell): Debian tools to build Haskell packages (with hscolor)
0.16.49: all
- [sid \(unstable\)](#) (haskell): Debian tools to build Haskell packages (with hscolor)
0.16.49: all

Package haskell-devscripts-minimal

- [bullseye \(oldoldstable\)](#) (haskell): Tools to help Debian developers build Haskell packages
0.16.0: all
- [bookworm \(oldstable\)](#) (haskell): Tools to help Debian developers build Haskell packages
0.16.29: all
- [trixie \(stable\)](#) (haskell): Tools to help Debian developers build Haskell packages
0.16.34: all
- [forky \(testing\)](#) (haskell): Tools to help Debian developers build Haskell packages
0.16.49: all
- [sid \(unstable\)](#) (haskell): Tools to help Debian developers build Haskell packages
0.16.49: all

Package mozilla-devscripts

- [bullseye \(oldoldstable\)](#) (devel): Development scripts used by Mozilla's addons packages
0.54.2: all
- [bookworm \(oldstable\)](#) (devel): Development scripts used by Mozilla's addons packages
0.54.2+nmu1: all

This page is also available in the following languages (How to set [the default document language](#)):

[Български \(Bǎlgarski\)](#), [Deutsch](#), [suomi](#), [français](#), [magyar](#), [日本語 \(Nihongo\)](#), [Nederlands](#), [polski](#), [Português \(br\)](#),
[Русский \(Russkij\)](#), [slovensky](#), [svenska](#), [Türkçe](#), [українська \(ukrajins'ka\)](#), [中文 \(Zhongwen, 简\)](#), [中文 \(Zhongwen, 繁\)](#)

See our [contact page](#) to get in touch.



Search

package names



dh-make

[all options](#)

Limit to suite: [\[bullseye\]](#) [\[bullseye-updates\]](#) [\[bullseye-backports\]](#) [\[bookworm\]](#) [\[bookworm-updates\]](#) [\[bookworm-backports\]](#) [\[trixie\]](#) [\[trixie-updates\]](#) [\[trixie-backports\]](#) [\[forky\]](#) [\[sid\]](#) [\[experimental\]](#)

Limit to a architecture: [\[alpha\]](#) [\[amd64\]](#) [\[arm\]](#) [\[arm64\]](#) [\[armel\]](#) [\[armhf\]](#) [\[avr32\]](#) [\[hppa\]](#) [\[hurd-i386\]](#) [\[i386\]](#) [\[ia64\]](#) [\[kfreebsd-amd64\]](#) [\[kfreebsd-i386\]](#) [\[loong64\]](#) [\[m68k\]](#) [\[mips\]](#) [\[mips64el\]](#) [\[mipsel\]](#) [\[powerpc\]](#) [\[powerpcspe\]](#) [\[ppc64\]](#) [\[ppc64el\]](#) [\[riscv64\]](#) [\[s390\]](#) [\[s390x\]](#) [\[sh4\]](#) [\[sparc\]](#) [\[sparc64\]](#) [\[x32\]](#)

You have searched for packages that names contain *dh-make* in all suites, all sections, and all architectures. Found **6** matching packages.

Exact hits

Package dh-make

- [bullseye \(oldoldstable\)](#) (devel): tool that converts source archives into Debian package source
2.202003: all
- [bookworm \(oldstable\)](#) (devel): tool that converts source archives into Debian package source
2.202301: all
- [trixie \(stable\)](#) (devel): tool that converts source archives into Debian package source
2.202503: all
- [forky \(testing\)](#) (devel): tool that converts source archives into Debian package source
2.202503: all
- [sid \(unstable\)](#) (devel): tool that converts source archives into Debian package source
2.202503: all

Other hits

Package dh-make-elpa

- [bullseye \(oldoldstable\)](#) (devel): helper for creating Debian packages from ELPA packages
0.19.1: all
- [bookworm \(oldstable\)](#) (devel): helper for creating Debian packages from ELPA packages
0.19.1: all
- [trixie \(stable\)](#) (devel): helper for creating Debian packages from ELPA packages
0.19.5: all
- [forky \(testing\)](#) (devel): helper for creating Debian packages from ELPA packages
0.19.10: all
- [sid \(unstable\)](#) (devel): helper for creating Debian packages from ELPA packages
0.19.10: all

Package dh-make-golang

- [bullseye \(oldoldstable\)](#) (devel): tool that converts Go packages into Debian package source
0.4.0-1+b6: amd64 arm64 armhf i386

- [bookworm \(oldstable\)](#) (devel): tool that converts Go packages into Debian package source
0.6.0-2+b5: amd64 arm64 armel armhf i386 mips64el mipsel ppc64el s390x
- [trixie \(stable\)](#) (devel): tool that converts Go packages into Debian package source
0.8.0-1: amd64 arm64 armel armhf i386 ppc64el riscv64 s390x
- [trixie-backports](#) (golang): tool that converts Go packages into Debian package source
0.8.3-1~bpo13+1: amd64 arm64 armel armhf i386 ppc64el riscv64 s390x
- [forky \(testing\)](#) (devel): tool that converts Go packages into Debian package source
0.8.3-1: amd64 arm64 armhf i386 ppc64el riscv64 s390x
- [sid \(unstable\)](#) (devel): tool that converts Go packages into Debian package source
0.8.3-1: amd64 arm64 armhf i386 loong64 ppc64el riscv64 s390x
0.0~git20180410.bcf5bf-1 [[debports](#)]: alpha ppc64 sparc64

Package dh-make-golang-dbgsym

- [sid \(unstable\)](#) (debug): debug symbols for dh-make-golang
0.8.1-1 [[debports](#)]: loong64
0.6.0-2+b5 [[debports](#)]: riscv64
0.0~git20180410.bcf5bf-1 [[debports](#)]: ppc64

Package dh-make-perl

- [bullseye \(oldoldstable\)](#) (perl): helper for creating Debian packages from perl modules
0.116: all
- [bookworm \(oldstable\)](#) (perl): helper for creating Debian packages from perl modules
0.122: all
- [trixie \(stable\)](#) (perl): helper for creating Debian packages from perl modules
0.128: all
- [forky \(testing\)](#) (perl): helper for creating Debian packages from perl modules
0.132: all
- [sid \(unstable\)](#) (perl): helper for creating Debian packages from perl modules
0.133: all

Package dh-make-raku

- [bookworm \(oldstable\)](#) (devel): Manage Debian package files of Raku modules
0.5: all
- [trixie \(stable\)](#) (devel): Manage Debian package files of Raku modules
0.8: all
- [forky \(testing\)](#) (devel): Manage Debian package files of Raku modules
0.8: all
- [sid \(unstable\)](#) (devel): Manage Debian package files of Raku modules
0.8: all

This page is also available in the following languages (How to set [the default document language](#)):

[Български \(Bǎlgarski\)](#), [Deutsch](#), [suomi](#), [français](#), [magyar](#), [日本語 \(Nihongo\)](#), [Nederlands](#), [polski](#), [Português \(br\)](#), [Русский \(Russkij\)](#), [slovensky](#), [svenska](#), [Türkçe](#), [українська \(ukrajins'ka\)](#), [中文 \(Zhongwen, 简\)](#), [中文 \(Zhongwen, 繁\)](#)

See our [contact page](#) to get in touch.

Content Copyright © 1997 - 2026 [SPI Inc.](#); See [license terms](#). Debian is a [trademark](#) of SPI Inc. [Learn more about this site](#).

- [CheckInstall](#)

[Translation\(s\)](#) : English - [Français](#) - [Русский](#)

 ?Discussion

[DebianPkg: checkinstall](#) keeps track of all the files created or modified by your installation script, builds a standard binary package (.deb, .rpm, .tgz) and installs it in your system giving you the ability to uninstall it with your distribution's standard package management utilities.



Note that for most useful actions, checkinstall must be run as [root](#)

checkinstall is really useful if you've got a tarball with software that you have to compile with the usual

```
./configure
make
make install
make clean
```

To use it, you just build the software according to its instructions, and then use the install command in the instructions through checkinstall. I usually boils down to

```
tar -zxvf source-app.tar.gz
cd source
./configure
make
checkinstall
```

checkinstall will build a .deb package and install it.

If you want to remove the package, just use your favorite [package management tool](#).

If you add the --install=no option to checkinstall, the program will generate a .deb package without installing it.

You can now install the generated .deb on your computer or other machines of the same architecture (and the same version of Debian system), with
dpkg -i source-app_version_amd64.deb or [DebianPkg: gdebi](#).

The default configuration is read from `/etc/checkinstallrc`

See also

- [DebianMan: checkinstall manpage](#)

External Links

-  [Checkinstall homepage](#)
-  [Installing from tarballs using checkinstall](#) (2004)

[CategoryPackageManagement](#) | [CategoryPackaging](#) | [CategorySoftware](#)

CheckInstall ([last modified 2025-08-24 08:36:32](#))

- [Renaming Packages](#)

Contents

1. [Why?](#)
2. [Transition package method](#)
 1. [Compatibility symlinks](#)
3. [Clean slate method](#)
4. [Disappearing package method](#)

Why?

Sometimes, it is necessary for a package to get a new name. Although this should rather seldom be the case, there are some situations when it makes sense, for example when the name of the upstream application changes. Of course an *apt-get dist-upgrade* should still seamlessly upgrade the package, best by completely removing the old package and installing the new one as a replacement.

Transition package method

Basically, the solution is to define a binary dummy package with the same name as the old package in the control file of the new package. The new source package takes over the binary dummy package, and the old source package, which is then binaryless, will be cleaned up by *rene*, an archive cleanup tool.

Assume that the last upstream version of the old package "oldname" was 1.5 and the package was renamed to "newname" for version 2.0.

Then, the dummy package is defined like this in *debian/control*:

```
Package: oldname
Depends: newname, ${misc:Depends}
Architecture: all
Priority: optional
Section: oldlibs
Description: transitional package
    This is a transitional package. It can safely be removed.
```

This entry defines the binary dummy package. It will get version 2.0-1 and automatically be pulled in by an *apt-get dist-upgrade* if an earlier version of oldname was already installed. Be sure to use the term "transitional package" in the

long description as various tools ([DebianPkg: lintian](#), [DebianPkg: deborphan](#), [DebianPkg: debfoster](#), etc.) look for exactly those two words.

The package only installs the mandatory files in `/usr/share/doc/oldname` and possibly some compatibility symlinks, see below.

Since it depends on newname, it also installs the new package.

Note that the package does not contain any architecture specific files anymore and therefore the Architecture is set to **all**, even if it was **any** before.

If you set the section of the dummy package to oldlibs, tools like [DebianPkg: deborphan](#) will suggest removal of the package once there aren't any more reverse-dependencies installed. All transitional, including non-library, packages should be put in this section, even though it's pretty much a misnomer, but there is currently no better section for this (see <https://lintian.debian.org/tags/transitional-package-should-be-oldlibs-optional.html>).

The new package is defined like this:

```
Package: newname
Replaces: oldname (<< 2.0-1~)
Breaks: oldname (<< 2.0-1~)
...
```

You can also add a versioned **Provides:** entry if it is appropriate (for example when all interfaces are provided), which will help to satisfy any versioned reverse dependencies, and will allow users to remove the transitional package.

Please make sure that you use the proper version to break against. You must make sure that you break against all versions which still have the files in the old package. Especially if you split it between two Debian revisions, you must add a Debian revision and not only conflict against older upstream versions. (Or you will stumble across bugs like [Closed in gtk-qt-engine/1:0.7-3: #397993: gtk-qt-engine: File conflict: #397993](#)).

Rationale: from "conflicts-with-version" Lintian tag: breaks is a weaker requirement that provides the package manager more leeway to find a valid upgrade path. Conflicts should only be used if two packages can never be unpacked at the same time, or for some situations involving virtual packages (where a version clause is not appropriate). In particular, when moving files between packages, use Breaks plus Replaces, not Conflicts plus Replaces.

Compatibility symlinks

If your new package also changes some filename-based interfaces to users or other packages, for example the package name changed because the program name changed, you might want to add compatibility symlinks. Those can either be placed in the old or the new package. If the symlink is there to stay, place it in the new package (so users can remove the transitional package and still have it). If the symlink is supposed to go away, placing it in the old package has the advantage that users can choose to have it or not (of course make sure to give the old package a proper description stating that it contains that compatibility) and that there is an easy way to see which other packages still need to change (things depending on the old name may need the compatibility symlink, while things depending on the new name must call it with the new name). As packages that have been in a release will need to have the transitional package in the next release, that extra information is especially useful if the package never released so the transitional package can be removed soon or if there are chances the transition to the new name will take more than one release to have all reverse-dependencies fixed).

Clean slate method

This method consists in switching to the new name without using a transitional package. It can imply renaming the binary package within the existing source (and possibly renaming the source package) or creating a new source package (which would require a request removal of the old source package). And let the new package **Provides**, **Replaces** and **Conflicts** the old package, ideally by using versioned Provides to fulfill versioned dependencies when targeting $\geq$ Debian Stretch.

This should be used for example for any non-leaf packages that are only ever installed by other packages depending on it. Because in that case the only thing needed will be for the reverse-dependencies to start depending on the new package.

But this method has disadvantages:

- Cannot be used for packages that are used in build dependencies, as several build tools (like [DebianPkg: sbuild](#)) do not support virtual packages in those dependencies by design, to guarantee deterministic builds.
- Most package managers (including APT, as of 2011-02-21) do not know to replace the old with the new one and will only use the new package if it is installed for some other reason.

Disappearing package method

This page used to describe a method trying to get rid of the transitional package automatically by removing all its files. If you are interested why this does not work at all (yet?), take a look at the history of this page.

- [PackageTransition](#)

[Translation\(s\)](#): none

This page summarizes several Debian package transition scenarios which require some coordinated tweaks on debian/control file of newly uploaded packages.

Recent Debian Policy (> 3.8.0) updates on "Breaks" field is accounted here.

Contents

- 1. [Package Transition](#)
- 2. [A Note on Package Transitions in The Backports Suite](#)
- 3. [See also](#)

Package Transition

Basic scenarios

Let's assume package version to be

- old package: 1.0
- new package: 2.0

case	Package (transition) situation	Flags for new A package	Flags for new B package
#1	Conflicts: never work together	Conflicts: B	Conflicts: A
#2	Resolved conflicts: old A and B conflicted with each other and with new ones; only newer A and B can coexist	Breaks: B (<<2)	Breaks: A (<<2)
#3	Virtual package: normal virtual package which avoids Conflicts using update-alternatives(8) etc.	Provides: virt	Provides: virt
#4	Exclusive virtual package: exclusive virtual package such as mail-transport-agent	Provides: virt Conflicts: virt Replaces: virt	Provides: virt Conflicts: virt Replaces: virt
#5	Rename: only A existed; move everything from A into B; make A a	Depends: B	Breaks: A (<<2)

	transitional package without real contents. See RenamingPackages for an in-depth explanation.		Replaces: A (<<2) optional* -- Provides: A
#6	Merge: A and B existed; merge everything from A into B; make A a transitional package without real contents	Depends: B (>=2)	Breaks: A (<<2) Replaces: A (<<2) optional* -- Provides: A
#7	Split: only A existed; move some files from A to B; new A does not require new B		Breaks: A (<<2) Replaces: A (<<2)
#8	Split: only A existed; move some files from A to B; new A always requires new B	Depends: B	Breaks: A (<<2) Replaces: A (<<2)
#9	Reorg: A and B existed; move some files from A to B; new A does not require new B	Breaks: B (<<2)	Breaks: A (<<2) Replaces: A (<<2)
#10	Reorg: A and B existed; move some files from A to B; new A always requires new B	Depends: B (>=2)	Breaks: A (<<2) Replaces: A (<<2)
#11 ??	Merge and remove: A and B existed; merge everything from A into B; A does not exist in new archive; B is the functioning package	(non-existing)	Breaks: A (<<2) (for backport ease) Replaces: A (<<2) optional* -- Provides: A
#12 ??	Remove transitional: A existed as a transitional package without real contents in previous release; A does not exist in new archive; B is the functioning package	(non-existing)	optional* -- Provides: A (?? is this enough ??)

*) **Provides** are needed only when there are some packages which depend on A.

🚩 ?? marks needs to be checked. #11 and #12 may be wrong. For #11, unless something else forces apt to remove package A, apt will be unwilling to remove it even on dist-upgrade, leaving a set of packages kept-back. Doing #6 (with a transitional package) is needed instead.

A Note on Package Transitions in The Backports Suite

?: If has stable A, but does not have B, it seems the backport must include B, because packages NEW to testing will most likely depend on B and not A. Thus, backports of packages NEW to testing may be blocked if stable-backports does not have B.

When a package in stable has followed "#5 Rename", and the stable release has both A (dummy transitional) and B (new package name), the A package may be removed from the src:package in testing following #12. Let X be the src:package in testing and Y be the src:package in stable-backports. When Y is updated from X at #12, A must be resurrected in Y. If A in stable has optional Provides, or if A is at #3, then these should be retained in Y, resurrecting them if necessary.

?: It seems like the question of "if B Provides: A has been added to ...should Y merge these new Provides or reject them" might be up to the iscretion of the backporter.

Unsolved cases

Can you provide solutions for these cases?

- **Files recently added, then moved:** File path */lorem/ipsum* did not exist; then, package B version 3 installs that path. Later, package A installs that path instead, and package B version 6 stops installing it.
 - **Too broad: Breaks:** B (< 6) incorrectly breaks earlier versions of B that would work fine (e.g. with a backport of A).
 - **Too broad: Breaks:** B (>= 3), B (< 6) incorrectly declares breakage of *all* versions of B, because the conditions are applied individually.
 - **Invalid: Breaks:** B (>= 3, < 6) would be good, but it is invalid syntax.

Terminology

Replaces is used when a new package replaces/overwrites files, and Provides is package provides functionality. Do not use Replaces to say that a package replaces functionality when it does not replace/overwrite files. During a package rename+split, let A be the old package name, let B contains the bulk of the functionality, and let C be something like a library component or server-back-end for B. New A becomes a dummy transitional package that depends on B, and B depends on C. If only C replaces/overwrites files from old-A, then C receives the the Breaks and Replaces flags, and neither A nor B receives Breaks and Replaces flags; however, if B also replaces/overwrites files from old-A then B must also have Breaks and Replaces flags. --NEEDS EDIT: is it ever appropriate for a transitional dummy package to have Breaks and Replaces? Also, please clarify the timeline for migrations, eg: stable+1 has transitional dummy package, stable+2 has transitional dummy package and B Provides A. Stable+3 has no transitional dummy package. In Stable+4 package B can drop "Provides A"?

See also

This document is based on following references and mailing list messages:

-  [Debian Policy Manual, Chapter 7 - Declaring relationships between packages](#)
-  [Developers Reference, 5.9.3. Replacing or renaming packages](#)
- manual page: deb-control(5)
-  [Package renaming, and transitional dependencies](#) (before introduction of **Breaks**)
-  <http://lists.debian.org/debian-ctte/2010/05/msg00012.html> and following messages.
-  <http://lists.debian.org/debian-policy/2010/06/msg00222.html> explaining why **Breaks** is needed for **dist-upgrade**.
-  [conflicts/replaces/provides vs. breaks/replaces/provides under policy 3.9.1](#) and following messages.
- guessing ... (Replacing as much versioned **Conflicts** with **Breaks**)

[CategoryPackaging](#)

PackageTransition ([last modified 2026-03-03 22:12:28](#))

- [DpkgDiversions](#)

Diversions allow files from a package *pkg-A* to be temporarily replaced by files from another package *pkg-B*. When *pkg-B* is uninstalled, the files from *pkg-A* are put back into place.

For example, the `vim-tiny` package includes a small stub file in `/usr/share/vim/doc/help.txt`, which is renamed to `help.txt.vim-tiny` to make room for a fuller version when `vim-runtime` is installed. When `vim-runtime` is uninstalled, the `help.txt.vim-tiny` is renamed back to `help.txt`.

Diversions are a bit finicky. Consider whether a `Conflicts` relation or `update-alternatives` might be a better way to achieve what you are trying to accomplish.

Adding a diversion to a package

See [Appendix G](#) in the Debian policy manual, but take into account the information within is currently quite out-of-date.

In `preinst`, make sure the diversion is added in all cases. If you want to avoid spurious messages on upgrades, you can check for versions known to already include the diversion, or you can explicitly check whether the diversion already exists.

```
diversion_added_version=1.0.9-1
this_version=3.8-2

if
    test "$1" = install ||
    dpkg --compare-versions "$2" lt "$diversion_added_version" ||
    dpkg --compare-versions "$this_version" lt "$2"
then
    dpkg-divert --add --rename /usr/share/foo
fi
```

In `post rm`, remove the diversion if and only if the package is being either removed or downgraded to a version before the diversion was added. On upgrades to a newer version, do nothing: the diversion is the new version's responsibility now.

```

diversion_added_version=1.0.9-1
this_version=3.8-2

losing_diversion=n

if test "$1" = failed-upgrade
then
    dpkg --compare-versions "$2" le-nl "$this_version" ||
    # An upgrade from a newer version failed.
    # There is no way for us to know enough to take over from here
    # so abort the upgrade.
    exit 1
elif dpkg --compare-versions "$2" lt-nl "$diversion_added_version"
then
    losing_diversion=y
fi

case "$1,$losing_diversion" in
remove,*)|abort-install,*)|disappear,*)|*,y)
    dpkg-divert --remove --rename /usr/share/foo
    ;;
esac

```

Removing a diversion from a package

In *postinst* and *prerm*, remove the diversion. Since failed upgrades can change the version number without configuration succeeding, the last version configured is not a reliable indication of whether the diversion is still around. If you want to avoid spurious messages, you can check for the diversion before removing it.

```

if dpkg-divert --list | grep -F "/usr/share/foo.distrib"
then
    dpkg-divert --remove --rename /usr/share/foo
fi

```

Without perfect foresight, the *prerm* surely cannot recover from failed attempts to upgrade from a newer version. (Nor can any other package; arguably this is a flaw in policy.) So supporting such downgrades reliably requires a check.

```
this_version=7.1
```

```
if test "$1" = failed-upgrade
```

```
then
```

```
    dpkg --compare-versions "$2" le "$this_version" ||
```

```
    # An upgrade from a newer version failed.
```

```
    # There is no way for us to know enough to take over from here
```

```
    # so abort the upgrade.
```

```
    exit 1
```

```
fi
```

Diverting configuration files

Diversions can also be used to provide [ConfigPackages](#) without throwing dpkg into an interactive conflict resolution system.

See also

- [DebianMan: dpkg-divert.1](#) manpage

[CategoryDeveloper](#) [CategoryPackaging](#)

DpkgDiversions ([last modified 2021-10-24 14:29:18](#))

- [ConfigPackages](#)

One way to manage local configuration of multiple systems is to put the configuration files and other mechanisms into the files and install scripts of a Debian package. Some advantages of doing this:

- configuration data and methods are encapsulated in a single object
- global configuration can be systematically removed and reversed by the more local administrators if desired
- configuration can be tested before deployment with `dpkg -i`
- configuration can be distributed securely along with the packages being developed
- configuration can be staged out exactly like packages when using a system like  <http://code.google.com/p/debmarshal>

The disadvantage of building configuration packages:

- Basic package setup as described below takes about half an hour for each new package
- Existing packages have to be signed and uploaded to the local repository, which may take some time if not already set up or keys need to be generated, found, or authorized.
- The first time a particular file is taken over, significant research may need to go into how the other system that delivers it handles letting something else take over. See the examples table below.

Setup basic package

1. `mkdir _packagename_-1`
2. `cd _packagename_-1`
3. `dh_make --native`
 1. `s`
4. `debian/changelog`
 1. `unstable -> company`
 2.  _username_@company.com
5. `debian/control`
 1. Maintainer:  _x_-team@company.com
 2. Section: `_same as package it configures_`
 3. Depend: on the package and versions appropriate
 4. Section and Priority to match upstream package
 5. Description: fill in both single line and longer description. List files being configured.
6. Makefile
 1. `debian/rules` calls this without args to "build". Do nothing on the first target (eg. `all:`)
 2. calls `clean`. Do nothing here either.
 3. calls `install`. `cp` and `mkdir` relative to `$(DESTDIR)` to put files in the package.

conffiles

Roughly 20% of the packages in Debian and Ubuntu ship default configuration files. If these are simply replaced, an upstream update later that modifies the same configuration file will throw `dpkg` into an interactive conflict resolution system. This is best avoided to make updates non-interactive. To avoid `dpkg` handling, the upstream package is [diverted](#) to a non-active file, and restored on removal of the

config package. Placing this diversion and replacement package in its own config package allows the package to be installed by debian-installer before first boot.

The replacement file is best provided as a regular package file (not a conffile) somewhere other than the original location, and symlinked from /etc. This avoids making the replacement file also a conffile. There are complex interaction cases where a package may be removed but its configuration files remain on the system. If the replacements are also configuration files, there are twice as many cases of package installation states to deal with, and no preinst or postrm scripts to execute any logic to handle the additional cases. conffiles are listed in /var/lib/dpkg/info/*.conffiles for each package.

The recommended method to assemble -config packages is to divert and symlink in the postinst, and remove symlinks and diversions in the prerm script. The symlinks are only created if the path is either not present or is already a symlink, and only removed if the path is a symlink. One suggested location is /etc/site/. This requires a purge of the conffiles in the package build, and will generate a linitian error.

The config-package-dev package provides CDBS rules files that help automate much of the work of creating Debian configuration packages using the divert-and-symlink technique with careful error checking and support for apply simple modifications to a Debian upstream configuration file in a way that is easy to maintain over time. It is available in Debian lenny or later. You can read the config-package-dev documentation at <http://debathena.mit.edu/config-package-dev> for details on how to use it.

Another option is to replace both the file and the checksum so dpkg is unaware of a change, though this would result in new upstream configuration files replacing the locally customized one.

debian/postinst

```
#!/bin/sh
set -e
PKG=company-service-config
if [ "$1" = configure ] ; then
    for f in auto.master gssapi_mech.conf
    do
        dpkg-divert --add --package ${PKG} --rename \
            --divert /etc/$f.distrib /etc/$f
        [ \! -e /etc/$f -o -L /etc/$f ] && ln -sf /etc/site/$f /etc/$f
    done
fi
#DEBHELPER#
exit 0
```

debian/prerm

```
#!/bin/sh
set -e
PKG=company-service-config
if [ "$1" = remove ] ; then
    for f in gssapi_mech.conf auto.master
```

```

do
    [ -L /etc/$f ] && rm /etc/$f
    dpkg-divert --remove --package ${PKG} --rename \
        --divert /etc/$f.distrib /etc/$f
done
fi
#DEBHELPER#
exit 0

```

To prevent files in `/etc/site` in the `-config` package from becoming conffiles themselves, in the `-config` `debian/rules` file, remove or purge the automatically generated `DEBIAN/conffiles` file after `dh_installdeb` runs.

debian/rules

```

binary-arch: build install
...
dh_installdeb
rm debian/company-service-config/DEBIAN/conffiles
...

```

debconf-generated configuration files

Roughly 5% of Debian packages use the debconf database and custom scripts using that data to generate configuration files. These scripts are located in `/var/lib/dpkg/info/*.config`. While there aren't a large number of these packages, they tend to be the most critical and important packages, and each one is a custom script that must be understood before you take over control of its configuration file. If `dlocate /etc/filename` doesn't locate a package, there's a good chance the configuration file is a debconf-generated file.

In the ideal case, there are values you can put in the debconf system that will generate the correct file. These can be done using `debconf-set-selections` and `dpkg-reconfigure` in the `-config` `postinst` script or in the installer `preseed`. Many `.config` scripts are rudimentary and are incapable of generating the required configuration file.

Typically, you should start by diverting the existing file using the method described above for conffiles, but also take some action (read the `package.config` script) that prevents that script from regenerating a configuration file on the real path. Ideally the script would respect the diversion, but none of them do at present. Some typical methods that disable particular `.config` scripts are:

1. Presence or absence of a comment or directive in the configuration file itself (eg. `##DEBCONF##`)
2. A debconf variable (eg. `package/override`)
3. Test whether configuration file is really a symlink now

Diverting the `/var/lib/dpkg/info/package.config` script in advance of package installation doesn't work.

ucf

Debian/etch has 113 packages (about 1% of the source archive) using ucf for configuration file handling, up from just autofs in sarge and dapper. ucf attempts to provide conf file behavior for scripts that are autogenerated. It is not yet in wide use. A real file exists somewhere else on the system, and on first install this is copied to the /etc pathname. On upgrades, the checksum of the /etc configuration file is compared with the checksum of the original, and is upgraded if no end-user changes were made. Otherwise the /etc configuration file is left unchanged.

The ucf system should be turned off for files that are provided by a -config package, and turned back on if the -config package is removed. The -config package would need to have encoded inside it the same ucf command that took over management in the first place.

debian/preinst

```
ucf --purge /etc/auto.master
```

debian/postrm

```
ucf /usr/share/autofs/conffiles/auto.master /etc/auto.master
```

This doesn't work, as autofs re-ucf's its config files and then asks whether to overwrite. If you answer yes, ucf will follow your symlink back to the /etc/site config file and overwrite it.

Solution: divert the usr/share/autofs/conffiles/auto.master file as well, either leaving it empty or making it a symlink to /etc/site as well. Doing both seems to be sufficient to preserve the local changes.

cme and configuration upgrade

 [lcdproc](#) configuration is managed by  [cme](#) to setup a basic working configuration and to upgrade the configuration during package upgrade without requiring user assistance. See [PackageConfigUpgrade](#) for more details.

fully custom

There are also packages with scripts that fully automate the generation and long term maintenance of their configuration files. Each one of these is a special case, without even debconf's generalized input model.

bind mount

Some configuration files may need to be taken over at specific stages of bootup, but left unmodified until then. If there is no good mechanism otherwise to disable a package from rewriting a configuration file, or if it has to be returned to unmodified state before the next boot, a read-only bind mount in a bootup script may be the only way.

Example config files and recommended takeover methods

<u>file</u>	<u>package</u>	<u>method</u>	<u>disabled by</u>
/etc/nsswitch.conf	base-files	conf file	dpkg-divert and bind-mount
/etc/ldap/ldap.conf	libldap2	conf file	dpkg-divert
/etc/libnss-ldap.conf	libnss-ldap	debconf	libnss-ldap/override

/etc/pam_ldap.conf	libpam-ldap	debconf	libpam-ldap/override
/etc/krb5.conf	krb5-config	debconf	symlink
/etc/pam.d/common-auth	libpam-runtime	conf file	dpkg-divert
/etc/pam.d/common-account	libpam-runtime	conf file	dpkg-divert
/etc/ssh/sshd_config	ssh-krb5	debconf	ssh/new_config
	openssh-server	debconf	
/etc/security/access.conf	libpam-modules	conf file	dpkg-divert
/etc/sudoers	sudo	custom script	existence of file
/etc/krb5.keytab			
/etc/resolv.conf	resolvconf	dpkg-divert	/etc/resolvconf/resolv.conf.d/{tail head}
/etc/resolv.conf	resolvconf	hierarchical	echo searchline > /etc/resolvconf/run/interface/zzzinterface
/etc/postfix/main.cf	postfix	debconf	postfix/main_mailer_type
/etc/cron-apt/action.d/5-install	cron-apt	hierarchical	~3-download without -d flag. Other options as necessary.
/etc/syslog-ng/syslog-ng.conf			
/etc/inittab	debootstrap	debian-installer	
/etc/lsb-release	base-files	conf file	dpkg-divert
/etc/hosts	netcfg	debian-installer/DHCP	unmanaged after install
/etc/apt/apt.conf	apt-setup	debian-installer	preseed file
	base-installer		
/etc/motd	base-files	custom script	
/etc/profile.d/_fixprofile.sh			
/etc/profile.d/_fixprofile.csh			

Alternatives to config packages

Any alternative configuration file handling method still has to inform the native Debian/Ubuntu configuration handling systems that the native system (dpkg) should leave the new files alone and ignore changes to them. Most documentation on the web does not mention this. The problems arise later when there are updates to the packaged systems.

- slack - A simple packaging system published by Google to drop configuration files on systems. Unaware of native package configuration handling, no removal capability. slack roles need to divert or otherwise wedge native configuration methods as described above.
- cfengine2 - Like slack, configuration rules must still use dpkg-divert to cleanly handle configuration files.
- puppet - A configuration management tool that hides the details of implementation so that you can easily describe policy. Has no understanding of Debian's conffiles system.
- FAI - An installer. Configurations are changed by reinstalling the system with the new configurations. Could avoid using its native configuration file system and use as a transport for configuration packages to install systems that don't require reinstall to reconfigure.
- bcfg2 - A configuration management tool that can 'bundle' configuration files with their respective packages so that verification can succeed despite file changes from the default package installation. Uses debsums to verify installed package consistency.

References

- <http://www.eyrie.org/~eagle/notes/debian/server.html>
- <https://debathena.mit.edu/config-packages/>
- <https://wiki.ubuntu.com/NetworkWideUpdates>
- <https://wiki.debian.org/DebianInstaller/Preseed>

Using patch files

Description of the Problem

Some changes to the configuration files are small and simple. So simple that can be represented by a patch. When the change isn't needed anymore the configuration file should return to the original state. It should respect the modifications made by the local administrator.

Solution

A patch management system to patch the configuration files. This is similar to the patch system used to manage source packages in Debian, like *dpatch* or *quilt*. The system should keep record of the patches installed or not installed, permit a easy way to install or remove the configuration and upgrading the configured packages without conflicts in the configuration files.

Implementation

At the core of the implementation is the command *dpatch*. It's called by a wraparound script, so it can apply the patches to the root of the Linux system and the patches files are inside the configuration directory of the package. This wraparound script is called *spatch*, is placed in */usr/share/\$packname* and is customized to the configuration package. This script *spatch* is only called by the local administrator if the patch system is broken in anyway. The script *gen-spatch* inside the non-official package *cal-scripts* can generate *spatch* commands.

To configure the others packages the command *update-package* do one of the three things:

- update - install or updates the installed customizations and reload the daemons,
- clean - remove all customizations and reload the daemons, --revert - remove all customizations but not reload the daemons, to permit upgrades without conflicts on the configurations packages.

It have the option *--silent* to allow the command to run unattended.

When *update-package* changes the configurations files it notifies the daemons to use the new configuration.

Example

The command *gen-spatch* is:

```
#!/bin/bash
cat > spatch <<EOF
#!/bin/bash
```

```

PACKNAME=$1
APPLYDIR=$2
PATCHESDIR=$3
pushd \${PATCHESDIR} > /dev/null
OPT="--workdir \${APPLYDIR}"
while [ \ $# -gt 0 ]; do
    case \$1 in
        help)
            dpatch \$*
            exit 1
            ;;
        -*)
            OPT="\${OPT} \$1"
            ;;
        *)
            break
            ;;
    esac
    shift || true
done
case \$1 in
    list*)
        dpatch \${OPT} \$*
        ;;
    patch-template)
        dpatch \${OPT} \$*
        ;;
    version)
        dpatch \${OPT} \$*
        ;;
    *)
        CMD=\$1
        shift
        dpatch \${OPT} \$CMD --stampdir=\${PATCHESDIR}/debian/patched \$*
    esac
popd > /dev/null
EOF

```

As an example the update-package will have commands like:

```

CONFIG=/etc/\${PACKNAME}/\${PACKNAME}.conf
source ${CONFIG}
function update-clean-patches () {
    deapply-patches $*
    if [ $CLEAN = no ] ; then
        ${SPATCH} apply $1
    fi
}

```

```

    fi
}
# Doing updates
update-clean-patches 10_bootlogd_v00
if [ -r /boot/grub/menu.lst ] ; then
    update-clean-patches 10_grub_menu.lst_v00 10_grub_menu.lst
fi
# openssh-server on etch don't need patch
deapply-patches 10_sshd_config_v00 10_sshd_config
if egrep "[[:space:]]*X11Forwarding[[:space:]]+yes" /etc/ssh/sshd_config > /dev/null
    echo "openssh-server on etch don't need patch"
else
    update-clean-patches 10_sshd_config_v00 10_sshd_config
    if [ ${RESTART} = "yes" ] ; then
        invoke-rc.d ssh reload
    fi
fi
#Patching /etc/modules
ModulesAllPatches="20_modules_firewalls_v00 20_modules_routers_v00 \
20_modules_tagus_policy_v01 20_modules_tagus_policy_v00 \
10_modules_prerequisites_v00"
deapply-patches $ModulesAllPatches
update-clean-patches 10_modules_prerequisites_v00
update-clean-patches 20_modules_tagus_policy_v01 20_modules_tagus_policy_v00
case $TpSrvConfNetOptionsRouter in
    Yes|yes|YES)
        update-clean-patches 20_modules_routers_v00
        ;;
esac
case $TpSrvConfNetOptionsFirewall in
    Yes|yes|YES)
        update-clean-patches 20_modules_firewalls_v00
        ;;
    *)
esac

```

Problems

Is responsibility of the local administrator to run:

```
update-package --revert before upgrades update-package --update after upgrades
```

It's possible to automate this tasks by using apt options, by dropping a file inside /etc/apt/apt.conf.d with something like the following example:

```

DPkg::Pre-Invoke { "/usr/sbin/update-package --revert --silent;" };
//DPkg::Pre-Install-Pkgs { "/usr/bin/update-tp-conf-srv --apt"; };

```

```
DPkg::Post-Invoke { "/usr/sbin/update-package --silent;" };
```

References

deb  <http://debian.tagus.ist.utl.pt/debian> etch/updates main contrib

deb  <http://debian.tagus.ist.utl.pt/debian> etch/proposed-updates main contrib

dpsyco approach

The dpsyco package implements a system to define and apply a local policy. This policy define users, groups that automatically exists or are removed from the systems. Permits to auto-configure the users that can have access to mysql database or samba. Have a system to automatically apply this changes or using patches to changes configuration files. The packages already in Debian etch: dpsyco, dpsyco-*

On the reverside it lacks documentation on its design or capabilities. Help is needed to understand how it works.

References

 <http://www.opal.dhs.org/programs/dpsyco/>

External links

- [DebianMan: dpkg-divert manual page](#)

[CategoryDeveloper](#) | [CategorySystemAdministration](#) | [CategoryPackaging](#)

ConfigPackages ([last modified 2021-10-24 14:29:58](#))

- [SimpleBackportCreation](#)

Translations: [English](#) - [Español](#) - [Français](#) - [Italiano](#) - [Português \(Brasil\)](#) - [Русский](#)

How to build private backports. To create a personal repo for your backports, see [DebianRepository/Setup](#). For backports suitable for uploading to [backports.debian.org](#), see [BuildingFormalBackports](#). For general packaging information, see [Packaging](#).

Here we take the example of the package coreutils, from which we want to install a newer release available in testing. If the package you're looking for is not available in Testing, but it is in a Ubuntu PPA, you can have a look at [CreatePackageFromPPA](#).

We don't need to be root here except the first and last steps.

Contents

1. [Install Debian packaging tools](#)
2. [Find out which version is available in the Debian archive](#)
3. [Add source package entries for the testing distribution](#)
4. [Install build dependencies](#)
5. [Indicate in the changelog a backport revision number](#)
6. [Test if we can successfully build the package](#)
7. [Build a package properly, without signing the package](#)
8. [Install and enjoy!](#)
9. [Go further](#)



Describes a specific packaging workflow

This page describes one possible workflow to solve a specific problem. Parts of it may inspire you to find a workflow that suits your situation.

For more pages like this, see [Packaging](#).

Install Debian packaging tools

```
sudo apt install dpkg-dev debian-keyring devscripts
```

Find out which version is available in the Debian archive

```
$ rmdison coreutils --architecture amd64
```

coreutils	8.23-4	oldstable	amd64
coreutils	8.26-3	stable	amd64
coreutils	8.30-3	testing	amd64
coreutils	8.30-3	unstable	amd64

Add source package entries for the testing distribution

If the package you are backporting isn't yet in testing, then use the suite it is currently in instead, such as unstable or experimental, instead of testing, in the instructions below.

Add a testing **deb-src** entries to your [apt sources](#):

```
$ cat /etc/apt/sources.list.d/testing-src.sources
```

Types: deb-src
URIs: https://deb.debian.org/debian
Suites: testing
Components: main
Signed-By: /usr/share/keyrings/debian-archive-keyring.gpg

Update your packages index:

```
apt update
```

Download the package source:

```
apt source coreutils/testing
```

Install build dependencies

```
cd coreutils-*/  
sudo mk-build-deps --install --remove
```

This will install a package named `coreutils-build-deps` depending on the listed build dependencies. If you remove this package later, the actual build dependencies will be marked as "automatically installed and no longer needed" and can be cleared with `apt autoremove`.

Indicate in the changelog a backport revision number

```
dch --bpo
```

This will add something like **~bpo9+** to the package version number. The tilde ~ makes the package inferior in version, which should allow a proper package upgrade when you upgrade to the next Debian release (i.e. your package will be replaced with the official Debian package).

Test if we can successfully build the package

```
fakeroot debian/rules binary
```

If this should fail with a missing file, [apt-file](#) may be useful in locating the dependency you require.

Build a package properly, without signing the package

```
dpkg-buildpackage --build=binary --unsigned-changes
```

Install and enjoy!

```
sudo apt install ../coreutils_*.deb
```

Go further

You could have a look [Building Formal Backports](#) and contribute your backport to Debian as explained here: 🌐 <https://backports.debian.org/Contribute/>

[CategoryPackaging](#)

SimpleBackportCreation ([last modified 2026-02-19 15:39:45](#))

- [Building Formal Backports](#)

Compile a backport while following  [the rules](#) for the backports. For personal backports, see [Simple Backport Creation](#). For general packaging information, see [Packaging](#).

Contents

1. Retrieve the Package Source

2. Modify the package as needed

3. Build in a minimal environment

1. Using `sbuild`

1. Preparation for first use

2. Building your backported package

3. Building multi-dependency packages

4. Correct distribution name

2. Using `pbuilder`

1. Installing the toolchain

2. Building for different distributions

3. Advanced: Building multi-dependency packages

4. Other tricks

5. Self-contained example for Fossil 2

4. bookworm-backports

1. `time64`

2. `/usr-move`

5. Uploading the backport

6. See also



Describes a specific packaging workflow

This page describes one possible workflow to solve a specific problem. Parts of it may inspire you to find a workflow that suits your situation.

For more pages like this, see [Packaging](#).

Retrieve the Package Source

Backports for a release, for almost all cases, must be prepared from the subsequent release. There are multiple ways to retrieve a package source. A simple way is to use `dget` (from the `devscripts` package) on the URL for the desired package's dsc file:

If you find your package listing by searching on  <https://packages.debian.org> for the appropriate source distribution (Testing, in the case of Stable backports), you will see a

link to its dsc file. The following command will retrieve the source package and extract it for you:

```
dget -x <url-of-dsc-file>
```

More efficient workflows for ongoing maintenance of the backport will make use of the maintainer's packaging VCS and/or `dg` instead.

Modify the package as needed

To make a proper backport, you need to modify the package according to the [rules](#). A simpler version:

- Use `stretch-backports`, `jessie-backports`, `wheezy-backports-sloppy` or `wheezy-backports` as distribution and append `~bpo${debian_release}+${build_int}` to the version number (**consider using `dch --bpo`**), e.g. `1.2.3-4` now becomes `1.2.3-4~bpo8+1` for jessie, `1.2.3-4~bpo7+1` for wheezy-backports-sloppy or `1.2.3-4~bpo70+1` for wheezy.

This is just a summary of the technical aspects of the rules. It is **really important** that you read up on [the rules](#) and follow them.

Build in a minimal environment

Using sbuild

Preparation for first use

The [sbuild](#) page provides guidance for setting up `sb`. Some backports-specific notes are mentioned here.

The [Examples section of sbuild-createrepo\(8\)](#) specifically shows how to create a chroot for the Stable release that also allows access to Stable-backports (which may be needed to fulfill dependencies in the backport you intend to prepare).

Building your backported package

From the top-level directory of the backported source package, you should be able to build as follows:

```
sbuid --build-dep-resolver=aptitude --debbuildopts="-v<last-main-version>"
```

Where `<last-main-version>` is the version of the most recent backport or Stable released version (as recommended in the [Backports rules](#))

The `aptitude` build dependency resolver is needed in order to be able to pull packages from Stable-backports as needed (which will only happen if the build-dependencies cannot be met from Stable).

Building multi-dependency packages

If your package requires others to be backported first, you can build them as described above and then use them from your local file system without having to manually set up a temporary APT repository. Simply pass

`--extra-package <path-to-deb-file>` in your call to `sbuild`.

Correct distribution name

The distribution field in the `.changes` file must be correct for the backports upload to process into the correct queue. For example, for bookworm this field must be `bookworm-backports`. `Sbuild` automatically populates this field with the name of the chroot, so it is important that the chroot have the correct name and not simply be something like `bookworm` or `stable`.

Using pbuilder

 [ToDo](#): give this section a major update - it has a lot of mistakes and thus isn't about official backports anymore.

Your first step is to configure `pbuilder` and, optionally, `cowbuilder`. The remaining of this howto will assume you will use `cowbuilder`, as this is what is in use here and is more efficient. We also like to use [git-buildpackage](#) to avoid building a `.dsc` file first, so to build straight from the source.

A full `pbuilder` or `cowbuilder` is beyond the scope of this manual, but there are specific issues you should not miss here. See [git-pbuilder](#) for details of use of `cowbuilder/pbuilder` with `git-buildpackage`.

Installing the toolchain

```
apt-get install cowbuilder pbuilder
```

Building for different distributions

Then enable `pbuilder` to build for different distributions easily, by following [those instructions](#).

At this point you should be able to build a package using:

```
DIST=jessie ARCH=amd64 pbuilder build --debbuildopts "-sa -v1.0" foo.d
```

With git-buildpackage this becomes:

```
cd package
DIST=buster ARCH=amd64 git-buildpackage --git-builder=git-pbuilder -s
# or
DIST=buster ARCH=amd64 gbp buildpackage --git-builder=git-pbuilder -s
```



Note that this option can be written to your ~/.gbp.conf file as such:

```
builder = /usr/bin/git-pbuilder.
```

Another example, with cowbuilder:

```
DIST=wheezy ARCH=i386 pdebuild --pbuilder cowbuilder --debbuildopts "-s
```

Advanced: Building multi-dependency packages

At this point you are able to build and upload backports of existing packages easily. The tricky part comes up when you have multiple dependencies to upload at once.

pbuilder chroots do not use packages from backports by default, so that's the first thing we need to fix. Add this to your **pbuilder**rc:

```
OTHERMIRROR="deb http://deb.debian.org/debian/ $DIST-backports main"
```

Then update the chroot with the new sources.list line:

```
sudo DIST=jessie ARCH=amd64 cowbuilder --update --override-config
```

Then you *could* build the packages one at a time: backport one, upload it, wait for it to show up in the backports archive, build the second one, etc. But this is really time consuming and could take a long time for big package suites. It could also mean a lot of dependent packages be uploaded to backports while the package that needs it is not there, which is bad practice.

What you need is to be able to use the packages you're building locally in the chroot. This is again described in the [PbuilderTricks page](#), but we'll show our own way here.

Add this to your pbuilderrc:

```
OTHERMIRROR="deb http://deb.debian.org/debian/ $DIST-backports main|del
HOOKDIR=/usr/lib/pbuilder/hooks
BINDMOUNTS="/home"
```

The above assumes you are building your packages in `/home/anarcat/dist/package` and the build results end up in `/home/anarcat/dist/build-area/`. This can be freely changed, but it will only work if it is under the BINDMOUNTS.

Then you need to add a hook that will run `apt-get update` before the package is built, in `/usr/lib/pbuilder/hooks/D90update`:

```
/usr/bin/apt-get update
```

The HOOKDIR variable can be changed if you want to put your hook file somewhere else, but the file name is important.

Then the only bit missing is making sure the `/home/anarcat/dist/build-area/` directory is a valid archive. Other howtos make that part of the hook, but since we are not running as root (see below), this is not practical for us. We simply use the following command on a as-needed basis:

```
dpkg-scanpackages . /dev/null | bzip2 -c > Packages.bz2
```

It is in the following makefile to make things easier:

```
all: Packages.bz2 Sources.bz2

Packages.bz2::
    dpkg-scanpackages . /dev/null | bzip2 -c > Packages.bz2

Sources.bz2::
    dpkg-scansources . /dev/null | bzip2 -c > Sources.bz2
```



Note that the above will fail because it's missing the [SecureApt](#) key. See [AutomateBackports](#) for another example which may resolve this.

We have found the following options to be useful in pbuilderrc:

```
PDEBUILD_PBUILDER="cowbuilder"

BUILDUSERID="500"
BUILDUSERNAME="anarcat-pbuilder"

AUTO_DEBSIGN=yes
```

This is the `~/ .gbp.conf` (git-buildpackage config):

```
[DEFAULT]
# tell git-buildpackage howto clean the source tree
cleaner = fakeroot debian/rules clean
postbuild = lintian $GBP_CHANGES_FILE
# this is how we invoke pbuilder, arguments passed to git-buildpackage
# passed to dpkg-buildpackage in the chroot
builder = /usr/bin/git-pbuilder

[git-buildpackage]
export-dir = ../build-area/

[git-import-orig]
dch = False
```

Self-contained example for Fossil 2

```
STABLE=stretch
TESTING=buster

# Prepare local repo
mkdir -p /usr/src/backports/$STABLE/
touch /usr/src/backports/$STABLE/Packages
cat <<EOF > /usr/src/backports/$STABLE/Release
NotAutomatic: yes
ButAutomaticUpgrades: yes
Date: $(date -R -u)
```

```

EOF
cat <<EOF > /usr/src/backports/$STABLE/D70update
#!/bin/bash
# Take previous builds into account
apt-get update
EOF
chmod 755 /usr/src/backports/$STABLE/D70update
cat <<EOF > /usr/src/backports/$STABLE/I70scanpackages
#!/bin/bash
# Update repo after successful build
cd /usr/src/backports/$STABLE/
dpkg-scanpackages . /dev/null > Packages
EOF
chmod 755 /usr/src/backports/$STABLE/I70scanpackages

# Create initial environment
sudo pbuilder --create --basetgz /var/cache/pbuilder/base-$STABLE-bpo.
--distribution $STABLE \
--othermirror "deb http://security.debian.org/ $STABLE/updates main|
--bindmounts /usr/src/backports/$STABLE/

# Update regularly:
# sudo pbuilder --update --basetgz /var/cache/pbuilder/base-$STABLE-bpo

# Add source for 'apt source'
echo "deb-src http://ftp.fr.debian.org/debian/ $TESTING main" \
| sudo tee /etc/apt/sources.list.d/$TESTING-src.list
sudo apt update

# Setup identity
export DEBEMAIL="you@debian.org"
export DEBFULLNAME="Your Name"

# Configure build
#export DEB_BUILD_OPTIONS="parallel=$(nproc) nocheck"
export DEB_BUILD_OPTIONS="parallel=$(nproc)"

# Create a working directory for sources
mkdir /usr/src/backports/sources/
cd /usr/src/backports/sources/

# Dependencies to add in our local repo
apt source sqlite3/$TESTING

```

```

(
    cd sqlite3-3.25.2/
    dch --bpo "No changes."
    pdebuild --debbuildopts '-v3.16.2-5+deb9u1' \
        --buildresult /usr/src/backports/$STABLE/ \
        --pbuilderdepends /usr/lib/pbuilder/pbuilder-satisfydepe
        -- --basetgz /var/cache/pbuilder/base-$STABLE-bpo.tgz \
        --bindmounts /usr/src/backports/$STABLE/ \
        --hookdir /usr/src/backports/$STABLE/
    lintian -i
)

# Fossil 2 itself
apt source fossil/$TESTING
(
    cd fossil-2.8/
    dch --bpo "Note: depends on backported sqlite3."
    pdebuild --debbuildopts '-v1:1.37-1' \
        --buildresult /usr/src/backports/$STABLE/ \
        --pbuilderdepends /usr/lib/pbuilder/pbuilder-satisfydepe
        -- --basetgz /var/cache/pbuilder/base-$STABLE-bpo.tgz \
        --bindmounts /usr/src/backports/$STABLE/ \
        --hookdir /usr/src/backports/$STABLE/
    lintian -i
    lintian -i /usr/src/backports/$STABLE/fossil_2.8-1~bpo9+1_amd64.ch
)

# Test
sudo --preserve-env=DEB_BUILD_OPTIONS pbuilder --login --basetgz /var/c
apt install fossil ...

# Publish to official archive
cd /usr/src/backports/$STABLE/
debsign xxx.changes -k YOUR_KEYID
dput xxx.changes

```

If something goes wrong and you need to test manually:


```
sudo pbuilder --login --basetgz /var/cache/pbuilder/base-$STABLE-bpo.t
apt update
cd /usr/src/backports/sources/fossil-2.8/
apt install pbuilder devscripts fakeroot
/usr/lib/pbuilder/pbuilder-satisfydepends-experimental
debuild ...
```

Remember to subscribe to the packages in the Debian Package Tracker, so you get notified of new versions, especially security fixes!

Notes:

- Not using cowbuilder, since it complexifies the commands, and since a few packages fail to build in hard-links corner-case situations [Closed: #475181: cowbuilder: fail to build qt-x11-free: 1](#)

bookworm-backports

Two transitions in trixie/sid have a significant effect on backports and require special attention.

time64

In trixie and later, dpkg and gcc inject compiler flags causing the use of 64bit time types. Refer to [ReleaseGoals/64bit-time](#) for details. These flags are not injected in bookworm (nor bookworm-backports) or earlier. Therefore, a backport may unexpectedly break ABI.

If you backport a package that carries a t64 suffix. The package rename needs to be reverted for the backport. The *t64 packages in trixie/sid carry Breaks and Replaces that are compatible with such backports (as they use (<< \${source:Version})). However, renaming a package back to its previous form may interact with the /usr-move transition (see below).

If you backport a package that does not carry a t64 suffix, it may have lost a t64 suffix in an soname bump. If the library in question actually is affected by the time64 transition, the ABI of a backport may differ from the ABI in trixie on 32bit architectures. There are two ways to handle this situation. One option is to restrict the backport to 64bit architectures by adding a build dependency on architecture-is-64bit. The other is performing a different rename in your backport adding a t32 suffix. If you add such a suffix, the version in trixie and sid must declare Breaks and Replaces regarding these new t32 packages before uploading to bookworm-backports. As in the other case, package renames may interact with the /usr-move transition (see below).

/usr-move

In `trixie`, all files from aliased locations such as `/bin`, `/lib` and `/sbin` shall be moved to their corresponding location below `/usr` ([UsrMerge](#)). In `bookworm` (and `bookworm-backports`) such moves are not performed and forbidden by the file move moratorium issued by the CTTE. Therefore any file moves performed in `trixie` must be reverted when doing a backport. If the package in question does not ship any files below `/bin`, `/lib` and `/sbin` in both `bookworm` and `trixie`, this section can be ignored. Additionally, any mitigations for problems caused by `/usr-move` must be reverted. Some mechanisms have been implemented in a way that is safe for backports use:

- `dh-sequence-movetouser / dh_movetouser`: This sequence addon and helper has been ported to `bookworm-backports` such that it will not actually perform any move in `bookworm-backports`. It can be kept and will automatically turn itself into a noop.
- `dh_installsystemd / dh_installudev`: The installation location for `systemd` units and `udev` rules has been changed for `trixie`. When backporting a package, such units will automatically move back to the aliased locations. Note that this does not apply to files installed using `dh_install` or an upstream build system.
- `pkg-config` files `systemd` and `udev`: The `pkg-config` files for `systemd` and `udev` identify the location for units and rules respectively via variables `systemdsystemunitdir` and `udevdir`. A number of upstream build systems rely on these variables to determine the installation location. When performing a backport, relying on these will work, but files may additionally be listed in `debian/*.install` and may need to be updated there.

A number of `/usr-move` mitigations rely on `Breaks` or `Conflicts` declarations. As such it may be necessary to update the version constraint in `unstable` when performing a backport. Since backports incur new upgrade paths, these may also come with `/usr-move` problems that did not exist at all earlier. Therefore, some packages such as `glibc`, `base-files`, `cryptsetup`, `isc-dhcp`, or `molly-guard` should not be backported to `bookworm-backports` at all. If in doubt, please request assistance from DEP17 transition drivers e.g. via `irc.oftc.net` channel `#debian-usrmerge` or by contacting them via email.

Uploading the backport

Use [DebianPackage: dput](#) ($\geq 0.9.6.3+nmu2$), [DebianPackage: dput-ng](#) or [DebianPackage: dupload](#) to upload the resulting package (just like a regular package upload).

See also

- [PbuilderTricks](#)
- [cowbuilder howto](#)

- [PackagingWithGit](#) - git-buildpackage and friends
 - [Backports](#) - the backports user documentation
 - [SimpleBackportCreation](#) - your own unofficial backports
 -  [Backports guide by gitlab maintainers](#) - covers best practices when the package is maintained in git and its complementary  [video tutorial/demo](#)
 - [AutomateBackports](#) - more elaborate: your own unofficial backport archive
 -  [Official backports documentation](#)
 - [HowToSetupADebianRepository](#)
 -  [The original trick of how to automatically adding dependencies to the pbuilder chroot](#)
 -  [d-d-a: Backports integrated into the main archive](#)
-
-

[CategoryPackaging](#)

BuildingFormalBackports (last modified 2025-06-10 13:07:25)

- [CreatePackageFromPPA](#)

Rebuild and install a package that's in an 3rd party repository but not in Debian. To create a personal repo for your packages, see [DebianRepository/Setup](#). For more guides, see [Packaging](#).

The example uses the  [Pogo](#) music player in an Ubuntu PPA.

Contents

1. [Install the Debian SDK](#)
2. [Get the Package Source](#)
3. [Install the Package build-dependencies and Build the Package](#)
4. [Install the Resulting Package](#)
5. [Problems? Stuck?](#)
6. [Going further](#)



Describes a specific packaging workflow

This page describes one possible workflow to solve a specific problem. Parts of it may inspire you to find a workflow that suits your situation.

For more pages like this, see [Packaging](#).



Is the package really missing in Debian ? The package might not be available in your Debian version, but it may be available in testing/unstable.

You can check this with the command `rmadison`. If you're interested in a package called **vagrant**, try if the command `rmadison vagrant` returns something like:

```
vagrant | 1.6.5+dfsg1-2 | sid | source, all
```

If you see the package is available in *sid*, it means you should instead do a [SimpleBackportCreation](#) of the **vagrant** package.

Now if `rmadison` does not return anything, you can build the PPA package in the following 5 easy steps.

Install the Debian SDK

```
sudo apt install devscripts build-essential
```



A previous version of these instructions advised adding the PPA to `/etc/apt/sources.list` with the `apt-add-repository` tool. The following section has been rewritten to avoid problems that can be caused by this approach.

Get the Package Source

On the page  <https://launchpad.net/~pogo-dev/+archive/ubuntu/stable>, click on "view package details", or go directly to  <https://launchpad.net/~pogo-dev/+archive/ubuntu/stable/+packages>, which is the same url but with `/+packages` added to the end.

From this page, copy the url to the *changes file* from the top link marked **(changes file)**. (If you want to rebuild a previous version of the package, use the corresponding changes url instead.)

The url in this example is

`https://launchpad.net/~pogo-dev/+archive/ubuntu/stable/+files/pogo_1.0.1-0~717~ubuntu20.10.1_source.changes`.

From a temporary working directory, run:

```
dget --extract --allow-unauthenticated https://launchpad.net/~pogo-dev/+archive/ubuntu/stable/+files/pogo_1.0.1-0~
```



The `--allow-unauthenticated` option to `dget` means this does not verify the source against developer OpenPGP keys. Use at your own risk.

You should now have a directory called `pogo-1.0.1`.

Install the Package build-dependencies and Build the Package

As normal user:

```
cd pogo-1.0.1
sudo mk-build-deps --install --remove
```

This step generates and installs a dummy package that tells apt what packages are needed to build pogo. If you remove this package later, the actual build dependencies will be marked as "automatically installed and no longer needed" and can be cleared with `apt autoremove`.

As long as this completed with no errors, the package is ready to build. In the pogo-1.0.1 directory, run:

```
dpkg-buildpackage --build=binary --no-sign
```

Install the Resulting Package

You should now have a package in your top level working directory called something like `pogo_1.0.1-0~717~ubuntu20.10.1_all.deb`

Install it with:

```
sudo apt install ../pogo_1.0.1-0~717~ubuntu20.10.1_all.deb
```

Problems? Stuck?

Join us in  [#debian](#) and tell us you are following these instructions, what step you are stuck on, and what happened. Read [GettingHelpOnIrc](#) for more information about debian IRC support.

Going further

If you find that the program you used would be of interest to other Debian users, check it if an "ITP:IntentToPackage" or a "RFP:RequestForPackage" bug report has been used. If no one has done this before, create a new bug report. For instance this the RFP for Pogo: ~~Closed: #666008: RFP: pogo — Probably the simplest and fastest audio player for Linux: <https://bugs.debian.org/cgi-bin/bugreport.cgi?bug=666008>~~

If you feel brave, have a look at what's inside the `pogo-1.0.1/debian/` directory, and read [IntroDebianPackaging](#)

[CategoryPackaging](#)

CreatePackageFromPPA ([last modified 2025-06-10 13:09:48](#))

- [SourceOnlyUpload](#)

Contents

1. Source-only uploads

1. Summary

2. Restrictions

3. Rationale

4. How to make a source-only upload

1. Generating _source.changes with dpkg-buildpackage and debuild

2. Generating _source.changes with sbuild and pbuilder

3. Generating _source.changes with git-buildpackage tool chain

4. Generating source-only upload with dgit

5. Sources

Source-only uploads

Summary

Source-only are uploads to the [Debian archive](#) that do not include a binary build of the Debian package, the [builddd](#) network handles the build and distributes it to the archive.

Since circa August 2014, the Debian archive accepts source-only uploads and since August 2015, Architecture: all packages are built on the builddds.

Since July 2019, binaries uploaded by maintainers are 🌐 [not allowed](#) to migrate to testing. In other words, packages must have source-only uploads before they can reach the next release.

Restrictions

NEW uploads and uploads with NEW binary packages currently cannot be source-only. This is also true for backports-NEW. This means you need to upload a source+binary package in these situations.

After your package has passed the checks and is in the archive, your uploaded binary packages will be thrown away if the upload was targeting Debian Sid/Unstable. There is no need to do another explicit source-only upload to allow the package to migrate to testing 🌐 [after August 2025](#).

[DebianPkg: dupload](#) has various checks for when source-only or sourceful uploads are appropriate, and will warn and prompt on how to proceed in those cases.

Rationale

Historically, the uploads of packages required them to be built on the developer's machine. The binary, arch-specific packages would be taken as-it-is by the archive after the upload (along with *all* binary packages, which are architecture agnostic). This means in particular that when you install an *amd64* package it is very likely the exact version that was compiled by the maintainer.

There are a few problems with this approach:

- it happens sometimes that a developer uploads a miscompiled package by a mistake (e.g., for another suite); this needs then to be fixed with one-time [binNMU](#)
- often the developer compile in an out-of-date environment (e.g., old chroots), and the package might end up linking against old libraries; this is particularly annoying during transitions as it requires investigating why it happens and issuing binNMUs
- if the machine of a developer is compromised, the package may contain maliciously compiled code

Source-only uploads address these issues by essentially having a central authority to compile all code. However, the disadvantage is that if *the archive* is compromised then *every package* is compromised. This is true, but it is true even now for more than 90% packages that are built by the archive.

A complementary project is [ReproducibleBuilds](#) which aims to make builds fully, bit-per-bit, reproducible. In the future one can imagine that packages are built both by the developer and the archive, the results are tested for equality and the package is only accepted if they match. All this work is about distributing trust and detecting problems early.

How to make a source-only upload

Generating `_source.changes` with `dpkg-buildpackage` and `debuild`

If the package just cleared NEW, you can add a new changelog entry for source-only upload (`dch -i` and `dch -r`). See  [this commit](#) and the resulting changelog file is given below.

```
ruby-cose (1.2.0-2) unstable; urgency=medium

  * Source only upload for migration to testing

-- Pirate Praveen <praveen@debian.org> Sun, 08 Nov 2020 15:13:34 +05:30

ruby-cose (1.2.0-1) unstable; urgency=medium

  * Initial release (Closes: #973864)
```

```
-- Pirate Praveen <praveen@debian.org> Fri, 06 Nov 2020 14:43:15 +05.
```

The *dpkg-buildpackage* program accepts the *--changes-option=-S* flag which builds the packages as always, but the final *.changes* file will contain only the source code. You can then use *dupload* or *dput* to upload the *.changes* file (see [this thread](#)). Example:

```
$ cd nghttp2
$ dpkg-buildpackage --changes-option=-S
$ cd ..
$ ls *nghttp2*.*
libnghttp2-doc_1.3.4-1_all.deb  nghttp2_1.3.4-1_amd64.changes  nghttp2_
nghttp2_1.3.4-1_amd64.build      nghttp2_1.3.4-1.debian.tar.xz  nghttp2_
$ egrep '^ \S{32} ' nghttp2_1.3.4-1_amd64.changes
01c9325805a6fe7fc444c890cf43e0fa 2008 httpd optional nghttp2_1.3.4-1.
cce2f954f27981191e539f43066e939a 1504585 httpd optional nghttp2_1.3.4
e0be575279e76a872eac15708374499e 10060 httpd optional nghttp2_1.3.4-1
```

You can also use the *-S* flag which only creates the source-only upload (the package is not even built in this case). Please make sure that the package builds properly before.

If you like *debuild* command, you can do:

```
$ debuild -S
```

Generating *_source.changes* with *sbuild* and *pbuilder*

While you should not build binary packages to upload directly with *dpkg-buildpackage*, it is fine to use *dpkg-buildpackage -S* to prepare source-only uploads.

If you build binaries for testing with *sbuild* and *pbuilder*, as a convenience feature, they can be made to emit a *_source.changes* using the *--source-only-changes* option.

Typical build commands are:

```
sbuild -A -d unstable --source-only-changes --run-lintian
pbuilder --build --source-only-changes package.dsc
```


Alternatively, you can set it globally in the configuration and avoid needing to give the option on the command-line:

- [DebianMan: pbuilder](#): `SOURCE_ONLY_CHANGES=yes`
- [DebianMan: sbuild.conf](#): `$source_only_changes = 1;`

Generating `_source.changes` with `git-buildpackage` tool chain

If you are using `git-buildpackage` without `pbuilder` or `sbuild`, it will happily accept `-S` or `--changes-option=-S` switch:

```
gbp buildpackage --changes-option=-S
```

This also works with `--git-pbuilder` too in `jessie` and below.

If you are using `--git-pbuilder` in `stretch` or later, you should use `--git-pbuilder-options=--source-only-changes`:

```
gbp buildpackage --git-pbuilder --git-pbuilder-options=--source-only-cl
```

This will give a `binary+source .changes` file and a `source-only .changes` file. Naturally, you must discard the `.changes` file that includes information about binary packages (`.deb` files) and only sign and upload the `source-only .changes` file in order to make a `source-only` upload.

When using `git-buildpackage` with `sbuild` ($\geq 0.70.0-1$), specify `--source-only-changes` in order to get both a `binary .changes` file and a `source-only .changes` file, e.g.:

```
gbp buildpackage --git-builder='sbuild --source-only-changes -v -As --c
```

You can also set it globally in the builder configuration (see above).

Generating `source-only` upload with `dggit`

Simply:

```
dggit push-source unstable
```

For `dggit-maint-gbp(7)`

```
dggit push-source --gbp
```

Sources

- "First steps towards source-only uploads" thread ( <https://lists.debian.org/debian-devel/2014/08/threads.html#00010>)
- "Auto-building arch:all packages" in "  [Bits from the Wanna Build team](#)" (2015-08-21)
- "dggit push-source" ( <https://spwhitton.name/blog/entry/pushsource/>)
- "Are you a DD or DM doing source-only uploads to Debian out of a git repository?" ( <https://spwhitton.name/blog/entry/pushsourcedropin/>)

SourceOnlyUpload (last modified 2025-11-10 13:44:24)

- [Services](#)
- [Debian Archive](#)

Debian Archive

- **Service Name:** Debian Archive
- **Service Short Description:** official Debian repositories for source and binary packages
- **Service Documentation:**  <https://ftp-master.debian.org/>
- **Source Code:** git clone  <https://salsa.debian.org/ftp-team/dak.git>
- **Service Status :** Working
- **Service Maintainer(s):**  ftpmaster@debian.org
- **Service Hosting:** debian.org
- **Service Authentication:** OpenPGP | public

Services/Debian Archive ([last modified 2020-01-02 06:00:23](#))

- [Packaging](#)
- [EmbeddedCopies](#)



[Debian Policy Manual: 4.13. Embedded code copies](#)

Some software packages include in their release distributions "convenience" copies of code from other software packages, generally so that users compiling from source don't have to download multiple archives. Debian packages should not make use of these copies unless the included package is explicitly intended to be used in this way. If the included code is already in the Debian archive in the form of a library, the Debian packaging should ensure that binary packages reference the libraries already in Debian and not the embedded copy. If the included code is not already in Debian, it should be packaged separately as a prerequisite dependency, if possible.

Embedded Copies

Debian discourages embedded copies (vendoring) where possible

It is recommended that Debian packages do not ship embedded copies of **code, data, fonts or other things**. Instead, package dependencies should be kept separate, and dependencies used to ensure the needed items are installed.

Shipping embedded copies (also known as 'vendoring') is discouraged because:

- It makes it more likely that users are exposed to known security issues. Debian considers it is better to have a single place to make security and other fixes. Fixing issues in vendored items requires more manual work, and embedded copies may be missed.
- Embedded items tend to be older versions that are no longer supported by the author of the embedded item. This leads to unfixed bugs.
- Multiple copies of the embedding items are needless duplication on user systems, and in the debian archive.

In practice, some upstreams explicitly design their software to vendor huge numbers of packages and removing the vendoring is impractical. Vendoring is reluctantly tolerated for non-libraries in some circumstances: see  [Debian Policy Manual: 4.13. Embedded code copies](#).

Packaging with embedded copies

When packaging software that has embedded copies, you should ask upstream to consider removing them from the upstream VCS and source tarballs. If upstream removes the embedded items, the Debian package can then be updated to the fixed version. Alternatives for upstream include:

- using dependencies. many ecosystems have package managers that support dependencies;
- only embedding the copy in the binary tarballs they distribute;
- scripting the install dependencies; or
- bundling dependencies into a single, separate, tarball instead of embedding.

If upstream refuse to remove the embedded copies, then Debian should either:

- repack the upstream tarball using [Files-Excluded](#). This is particularly appropriate if there is a [DFSG](#) or size issue.
- remove the files when building the package.

This can be done in the [debian/rules](#)' `clean` target, or early in the `build` target, to ensure the copy is not used in the build process.

Tracking embedded copies

The [list of packages that embed copies \(including unused ones\) of other projects](#) is maintained in the security-tracker git repository. This list also contains information about forks so that the security team can check if all forks contain the same vulnerabilities.

All Debian members have commit access to the security-tracker repository and others can send suggestions or additions to the [DebianList: debian-security-tracker mailing list](#).

Tools

Lintian

[Lintian](#) detects embedding of

- Common libraries written in
 - [C/C++](#)
 - [JavaScript](#)
 - [PHP](#) ([PEAR](#))
- [Fonts](#)
- PostScript: [copyrighted Adobe font fragments](#) ([without credit](#))
- [feedparser](#)

Others

- [check-all-the-things](#) has a couple of tests (`embed-readme`, `embed-dirs`) for finding embedded copies via heuristics, and several ideas for new tests.
- These [Gobby](#) pages mention embedded copies: [Embedded modules in inc](#) (by the [Debian Perl Team](#))

- The [Debian duplication detector](#) detects duplicate files in binary packages and may be useful for detecting verbatim duplication of files across multiple binary packages.
- [Clonewise](#) is a tool not yet in Debian that could be used to [find unfixed vulnerabilities because of embedded code copies](#).
- [SourcererCC](#) is another tool for detecting embedded code copies.
- [Socrates](#) can also do [duplication detection](#).
- [JPlag](#) finds pairwise similarities among a set of multiple programs

The [Debian Sources](#) service allows [searching](#) for specific hashes and ctags throughout all Debian source code, which may be useful for detecting duplication of source code and data.

If you have a particular file with some interesting aspect (security issue, etc.), you can likely find other copies using [Debian Code Search](#) or similar external service, such as [Black Duck Open Hub](#), [SourceGraph Public Code Search](#) or [GitHub Search](#).

If a file has a fairly unique name, you can often find copies of that file by searching the contents of Debian binary or source packages using apt-file:

```
apt-file search uniqueness.py
or
apt-file search -I dsc uniqueness.c
```

Tracking

Various Debian folks keep track of embedded copies they found via usertags:

[rbritto@ime.usp.br](#) [jwilk@debian.org](#) [mbehrle@debian.org](#) [pabs@debian.org](#) [sramacher@debian.org](#) [dr@jones.dk](#)

See also

These wiki pages mention embedded copies: [arc4random](#)

External links

- Fedora Packaging Guidelines: [Bundling policy](#)
- Fedora Wiki: [Bundled libraries](#)
- Gentoo Wiki: [Bundled dependencies policy](#)
- Homebrew Documentation: [Acceptable Formulae § Vendored dependencies policy](#)

- [LanguageVersionedDevPackages](#)

This page documents why and how Debian maintainers for packages written in enlightened programming languages include a version or hash in the `-dev` package name for each library, similar to the Shared Object version in the library package name.

Even if no one-size-fits-all solution ever emerges, sharing good ideas will be beneficial.

Languages

Ada

The  [Debian Policy for Ada](#) documents the existing process. Inspired by this page, the packaging style will probably change for gnat-13.

The  [dh_ada_library_tool](#) implements some parts of the policy.

 <https://wiki.debian.org/Ada> adds some details.

Problem

The language requires that an object is recompiled when a source changes in its *recursive* dependencies. The compiler maintains a `.ali` file for each compilation unit in order to detect such changes (ignoring comments and spaces). When a library is updated, the `.ali` file in its `-dev` package change, all its recursive, reverse dependencies must be recompiled in a correct order. Meanwhile, they FTBFS with sometimes cryptic error messages, or worst may produce buggy executables.

Dependencies

The name of an Ada `-dev` package describes the state of all sources of all recursive dependencies.

The dependencies between `-dev` packages are accurate enough for `dpkg` to prevent the installation of two package directly or indirectly using two versions of the same source.

For example, when several packages are uploaded at once, the build daemons figure a correct build order on each architecture because a (direct) build-dependency can only be installed once all its (recursive, versioned) dependencies can.

- currently The maintainer has to manually edit the Break/Replace/Package fields.


```
Source: foo
Build-Depends-Arch: libbar1.2-dev

Package: libfoo3.4-dev
Breaks: libfoo3.1-dev, libfoo3.2-dev, libfoo3.3-dev
Replaces: libfoo3.1-dev, libfoo3.2-dev, libfoo3.3-dev
Depends: ${ada:Depends}          # libbar1.2-dev, concrete package
```

- planned for gnat-13:

```
Source: foo`
Build-Depends-Arch: ada-bar-dev

Package: ada-foo-dev
Provides: ${ada:Provides} # ada-foo-dev-1a2b3c4d
Depends: ${ada:Depends}   # ada-bar-dev-5e6f7a8b the one available at I
```

Transitions

- currently

During an update, either of the library or of a direct or indirect build-dependency, the maintainer must (but sometimes forgets to) rename the `-dev` package, then upload to the NEW queue. The standard library cannot afford a passage through NEW and is handled specially (with error-prone manual work). Removing the exception is one of the motivation for the planned change.

- planned for gnat-13

Each `ada-* -dev` package provides a virtual packaged named after the state of the sources of the library and all its recursive dependencies. In practice, the name is computed from the `.ali` checksums for the library and the name of the `ada-* -dev -*` virtual packages for each `ada-* -dev` build-dependency.

An update of a library automatically renames the `-dev` package, breaking all reverse dependencies. The reverse dependencies are fixed by a simple rebuild. This is similar to the `+b1` rebuilds during SO transitions.

Transitions require no migration script (at least for libraries),

Recursive reverse dependencies fail to build from source (with an explicit message instead of either a cryptic message or a buggy result) during the transition.

Guidelines for any language

Language-specific conventions or tools should take care not assume that a given source package only deals with one programming language (see plplot for an example).

Naming scheme for the dev packages

In the absence of other constraints (like existing practice), please name the packages **LANGUAGE - LIBRARY - dev** and **LANGUAGE - LIBRARY - dev - VERSION**. This improves the consistency across the archive and automatic detection of **-dev** build dependencies or built binary packages for a specific language.

LanguageVersionedDevPackages ([last modified 2023-07-15 15:34:35](#))

- [Writing Debian Package Descriptions](#)

Writing Debian package descriptions



Written in 2007

This is based on  [an essay by Colin Walters from 2007](#). The theory hasn't changed, but the some details may no longer apply.

Guidelines

Debian package descriptions are basically a form of advertising, although of an informative, useful sort. They're a way to draw users to your package, and in addition convey important details about the package. In advertising, the key idea is to know your audience. This is the overriding principle you should use when writing Debian package descriptions. For example, think about what kinds of people use your package (computer programmers, biologists, musicians, genealogists), the level of computer sophistication they'll have, and the kinds of key words they will look for.

Historically, Debian has had a very technical audience. This has been reflected in all sorts of things in the project, but package descriptions are an obvious aspect. As a consequence, our package descriptions are almost all targeted at technical users, who know GNU/Linux well. They're filled with technical jargon like "GTK+", "X", "binaries", and "GUI". They assume a high level of computer understanding.

In some cases, this isn't a bad thing. Let's take the glade package as an example. Its synopsis line is currently "GTK+ User Interface Builder", and the long description contains "Glade is a RAD tool...". Now, the audience for Glade is definitely going to be mainly composed of people who know what GTK+ and RAD are; computer programmers, who are generally going to have a high level of technical understanding. For them, expanding RAD as Rapid Application Development isn't necessary, although it might not be an altogether bad idea. They'll probably know what GTK+ is.

In contrast, let's examine a package like rhythmbox. In this case, the targeted audience is very general; pretty much anyone who wants to play music on their computer. An example of a bad description would be: "GTK+/GNOME music player like xmms and iTunes". It may help to imagine a friend or relative who isn't very technical looking at the package description. To them, "xmms", "GTK+", and "iTunes" are most likely going to be terms that are just completely meaningless. It's quite possible "GNOME" is too, but we'll visit that issue in a minute. A novice computer user reading this might just skip the package entirely after reading that description, because it looks too challenging. If you're an experienced computer user, this may seem silly, but it really happens!

So how can we better target these users? The answer is to rewrite the description so it makes sense to most users, while maintaining a minimal amount of technical jargon so that it is still useful to advanced users. Here's the current description: "music player for GNOME". In this, the "music player" is featured first. That should draw in all the novices. Now, the intermediate and advanced users can see the "GNOME" at the end, and know that that means it will be well-integrated into their desktop environment. The novice users might not know this term, but they can learn to ignore it (although they will hopefully learn it, like almost all computer users have a vague idea what "Windows" is).

So far we've mainly discussed the synopsis line; all of these same principles apply to the rest of the description as well. However, the long description should take into account that the user has already read the synopsis, and was interested enough to read the rest of the description. That makes it a good place to explain what features it has, how your program is different from the alternatives, etc. In doing this, you should gradually get into more and more detail, just like any other good advertisement would. Again, generally avoid technical terms in the first sentence or first paragraph, but feel free to use them after that.

Again, the above are just guidelines; the principle of know your audience should be the most important guide. For example, the `fastdnaml` package is quite obviously targeted at biologists. Its synopsis line contains terms like "phylogenetic trees" which are totally meaningless to me, but to a biologist they probably mean something. Thus it's perfectly acceptable for its description to use these terms prominently.

One other note is that while we make the analogy that package descriptions are a form of advertisement, they're also supposed to be as useful as possible. This means that saying things like "This package is the best!!!" isn't really useful. It's doubtful that it is an effective form of advertising either. If another package supports a specialized feature that you don't have yet, it might not be a bad idea to mention it in your description. Remember, the end goal is to help the user.

A description checklist

Take a look at the following checklist, and make sure your description satisfies the criteria [1](#):

- Does your description conform to all the guidelines in the Debian Policy?
- What does it do (in nontechnical terms)?
- Do I need it? (look at the last sentence in the `Linux Configure.help` descriptions -- they're really helpful if you don't want to understand stuff because you don't need it but need to pick a choice now)
- What are outstanding features and deficiencies compared to other packages (if you need X, use Y instead)?
- Is this package related to other packages in some way that is not handled by the package manager (this is the client to the foo server)?

A description template

Now, let's put all of these guidelines together into a very generic sort of template. You should use creativity here; obviously the below is not even close to a good, real description. It needs to be fleshed out quite a bit.

```
Package: foo
Description: <perform some function, do some task> <for GNOME/KDE/Windows/etc>
    This is a <function> program, designed to help
    you <task>. <more simple details about task>. Written for
    the <environment>, it supports <feature1> and <feature2>.
    .
    Other features are <feature3> and <feature4>.
    <more advanced details for technical users>. See the <other package>
    for more details.
    .
    <This package is currently alpha/beta, other stuff...>
    .
    <You can find more information about foo at http://www.foo.org.>
```

Unsolved issues

There are a few stylistic issues which are still unsolved in the project today; some of them (especially the synopsis line) have been debated at length, with no apparent consensus.

- The number of spaces after a full stop - one or two? It seems to me that in general Americans use two spaces, and Europeans use one. However, the consensus reached on debian-devel is that the two spaces have been used to simulate extra space after the end of sentences in fixed-width fonts.
- Whether or not the synopsis line should end with a full stop (.); see bug #139957.
- Whether or not the beginning of the synopsis line should be capitalized.
- Whether or not the description should contain a URL. We really need a separate URL or Homepage field, but until that happens, some people like to have it in the description. However, since the debian/copyright file should contain a URL, other people prefer the description not contain a URL, and instead that we have software automatically extract it from debian/copyright.

[CategoryPackaging](#)

1. Thanks to Simon Richter <[✉ sjr@debian.org](mailto:sjr@debian.org)> for most of the checklist. ([1](#))

- [ManageUpstreamDifferences](#)

Manage differences between a package you have created and the upstream project. For general packaging information, see [Packaging](#).

Contents

1. If upstream has a /debian directory or file

1. Politely ask upstream to move the directory (recommended)
2. Pretend it doesn't exist
3. Move it before importing it

2. If upstream has content that is disallowed

3. If upstream content needs patching

1. If the patches are small or temporary

1. Managing debian/patches with git-buildpackage
2. Maintaining packages with quilt

2. If the patches are large and permanent

 This page assumes you have specified the 3.0 (quilt) format in `debian/source/format`. If you are using the simplified 3.0 (native) or deprecated 1.0 format, please switch to 3.0 (quilt) before continuing.

The quilt patch system was first popularised by [DebianPkg: the package of the same name](#), but nowadays that program is only recommended for packages without a version control system. Use [DebianPkg: gbp-buildpackage](#) instead for git-based projects.

If upstream has a /debian directory or file

In rare cases, your upstream's top-level directory might have a subdirectory (or even a file!) called `debian`. Debian uses that directory to store its data, so here are some possible solutions.

Politely ask upstream to move the directory (recommended)

Upstreams that release their own `.deb` packages often provide a `debian/` directory in their repo or tarballs. It's up to you whether to use this as the basis of your own work, it just can't have the specific filename `debian` upstream. Most upstreams should respond to a polite request, and [the relevant section of the upstream guide](#) discusses alternative solutions.

Pretend it doesn't exist

3.0 (quilt)  [requires the removal of any pre-existing debian directory](#), so programs that import from a tarball (e.g. `gbp import -orig`) will do this automatically. Or if you import with `git pull`, adapt the following solution instead:

```
# Run the following commands in debian/latest:
git checkout -b debian/latest -t upstream/latest

# Delete the upstream directory in the debian branch:
git rm -r debian
git commit debian -m 'Delete upstream debian'

# Define a custom "ours" merge-driver that ignores upstream changes
git config merge.ours.driver /usr/bin/true

# Use the custom merge-driver for files in debian/:
# See https://git-scm.com/docs/gitattributes#\_performing\_a\_three\_way\_merge
echo '/debian/** merge=ours' > .git/info/attributes
```

 Document your solution in your `debian/README.Debian`

Move it before importing it

In very rare cases, an upstream could have a `debian` that has nothing to do with Debian packaging. For example, a program to detect the operating system of a network host might put heuristics to detect a Debian host in a file called `/debian`.

An actual upstream file called `debian` probably can't be ignored or deleted, so it needs to be fixed the same way as an upstream with content that is disallowed (see next section).

If upstream has content that is disallowed

Some upstream tarballs contain files that need to be modified before they can even go in an upstream tarball. For example, Debian will refuse to host an upstream tarball with contents that violate the [Debian Free Software Guidelines](#), so those files need to be deleted altogether. This is quite rare - the vast majority of upstream issues can simply be [patched](#).

If you import tarballs with `uscan` (or `gbp import -orig --uscan`) and just need to delete some files, you can [use uscan to strip files from upstream tarballs](#).

If you import branches with `git pull`, the recommended solution is to pull into `upstream/latest`, clean data and commit to a `dfsg_clean` branch, then merge the

clean branch into debian/latest. For details, see [Handling non-DFSG clean upstream sources](#) (also available in `/usr/share/doc/git-buildpackage/manual-html/gbp.special.html`).

If merging into a `dfsg_clean` branch generates merge conflicts, but the conflicts are the same every time, you might be able to automate the resolution process with [git rerere](#).

If merging into a `dfsg_clean` branch generates merge conflicts that can't be fixed with `git rerere`, you may need to use git "plumbing" commands to automate your workflow. Adapt the following to suit your needs:

```
#!/bin/sh

#
# Skeleton script to pull and apply custom modifications
#
# Before resorting to this, please try:
# * recommended solution:
#   https://honk.sigxcpu.org/projects/git-buildpackage/manual-html/
# * reuse recorded resolutions:
#   https://git-scm.com/book/en/v2/Git-Tools-Rerere

set -e

UPSTREAM_BRANCH="$(gbp config DEFAULT.upstream-branch)"

# Download recent changes:
git fetch

# Switch to an anonymous branch based on upstream:
git checkout "$UPSTREAM_BRANCH"@{upstream}

# ... make your changes ...

# Add any changes you made:
git add -u

# Make a merge commit using low-level plumbing commands:
DFSG_CLEAN_COMMIT="$(
    git commit-tree "$(git write-tree)" \
        -p "$UPSTREAM_BRANCH" \
        -p "$UPSTREAM_BRANCH"@{upstream} \
```



```
-m "Merge upstream into upstream/latest"

)"

# Merge the cleaned commit into upstream/latest:
git checkout "$UPSTREAM_BRANCH"
git merge "$DFSG_CLEAN_COMMIT"
```

If your `.orig.tar.gz` doesn't match the upstream tarball, you may need to [use a special version number](#).

 Document your solution in your `debian/README.Debian`, and commit any scripts you create.

If upstream content needs patching

It's quite common to make changes in your `debian` branch but outside your `debian/` directory. For example, [lintian](#) might complain about a spelling mistake in a man page, or you might have a build failure because of [an outdated autoconf](#). Debian needs these changes to be accounted for, so people can replicate every step from the upstream source to the binary package.

If you have a good relationship with upstream, you might prefer to fix issues there and come back to Debian when your changes have been released. But you may well need to develop patches in Debian first and send them upstream when the time comes.

If the patches are small or temporary

Debian supports maintaining a small or temporary series of patches against upstream. For example, you might prefer to wait until your new package is ready before submitting fixes, or you might [backport](#) a security patch for your mature package.

Patches are stored using a *patch system* called `quilt`. A patch system is a minimal alternative to a *version control system* like `git` - it just manages a collection of *patches* (like individual commits in `git`) and an ordered *patch-series* (like a  [topic branch](#) in `git`). The patch-series is always based on the current contents of the project, so changing the project implicitly  [rebases](#) the patch-series.

`quilt` patches are stored in `debian/patches/*.patch` and the patch-series is stored in `debian/patches/series`. The format of individual patch files is defined in  [DEP-3](#), and resembles [DebianMan: git-format-patch](#).

`quilt` can be easier than `git` for some simple problems - for example, you can reorganise patches by editing `debian/patches/series` in text editor. But if your changes are complicated enough to need a full version control system, you may need to [fork the upstream project](#) instead.

Managing debian/patches with git-buildpackage

`git-buildpackage` can convert a `quilt` patch-series to a `git` branch and back again using the [DebianMan: gbp pq](#) command. A typical workflow is to `gbp pq import` a patch-series into a branch, make your changes in the normal way, `gbp pq export` the branch back to `quilt`, then `git add debian/patches`. The first time you call `gbp pq import`, it will notice you don't have a `debian/patches/series` file and create a new branch with no extra commits.

A `git` branch that's associated with a `quilt` series is called a *patch-queue branch*, because its commits are queued up waiting to become patches.

To submit a patch (or a series of patches) to an upstream forge, create a normal branch and [DebianMan: cherry-pick](#) some commits from your patch-queue branch into it. Or to submit a patch (or a series of patches) to an upstream mailing list, copy the relevant files from `debian/patches` into a new directory, then send them with [DebianMan: git-send-email](#). You may want to do a `--dry-run` or set `--to=<your-address>` first, to check you've got everything right. When your patches are accepted upstream, get rid of any patch-queue branches then delete the patches by editing `debian/patches/series`.

See [DebianMan: the gbp pq command](#) for a complete list of actions, and  [the official documentation](#) for a more detailed explanation.

Maintaining packages with quilt

If your project doesn't use version control (very rare nowadays), you can manage patches with the original [DebianPkg: quilt](#) program. For details, see [UsingQuilt](#).

If the patches are large and permanent

If your upstream project has been abandoned or is unwilling to accept changes, and your changes are becoming hard to manage in Debian, you may need to fork the upstream project.

Before forking the project, reach out to the maintainer - they may be willing to officially hand the project over to you, and at minimum they deserve the opportunity to make their opinion known. Also reach out to anyone that might help, like maintainers for other distributions or contributors to open bug reports - they may have something useful to add, and at minimum they need to know your fork is available.

This document talks about "the upstream project", but you may want to make a so-called "downstream fork". That is, a project that is downstream from some other project (so you routinely accept changes from them), but upstream from distributions (so distributions routinely accept changes from you). That makes it easier to merge your project back upstream if the situation improves.

It's OK to remain the Debian maintainer of a project you fork, especially for an abandoned project. But consider asking someone else to take over if there's a potential conflict of interest. For example, if the maintainer is still active but taking the project in a direction you think is wrong, it's best to avoid the appearance that Debian is backing one side of a dispute.

[CategoryPackaging](#)

ManageUpstreamDifferences (last modified 2025-06-18 19:31:37)

- debian
- [changelog](#)

debian/changelog (last modified 2025-01-03 07:41:35)

- debian
- [copyright](#)

How to write a debian/copyright file. For guidance about *what* to put in the file, see [CopyrightReviewTools](#). For general packaging information, see [Packaging](#).

Contents

1. [Background](#)
2. [Write the Header stanza](#)
3. [Write the Files stanzas](#)
4. [Write the License stanzas](#)

Background

A debian/copyright file contains copyright statements for each file in a package. This page discusses one way to write the file. For the formal specification, including alternatives not covered here, see  [Machine-readable debian/copyright file](#).

The file consists of one *Header stanza* that defines general information about the project, one or more *Files stanzas* that define groups of files with the same copyright information, and one or more *License stanzas* contain text for each license used in the package.

For real-world examples, see  [debian/copyright on codesearch.debian.net](#). A simple example might look like this:

```
# Header stanza:
Format: https://www.debian.org/doc/packaging-manuals/copyright-form
Source: https://www.example.com/upstream-project-name
Upstream-Name: upstream-project-name
Upstream-Contact: Upstream Maintainer <maintainer@upstream.com>

# Files stanza:
Files: *
Copyright: 1975-2025, Upstream Maintainer <maintainer@upstream.com>
License: GPL-2.0-only
Reference: COPYING
```

Write the Header stanza

Start with a Header stanza like this:

```
Format: https://www.debian.org/doc/packaging-manuals/copyright-form
Source: https://www.example.com/upstream-project-name
Upstream-Name: upstream-project-name
Upstream-Contact: Upstream Maintainer <maintainer@upstream.com>
#Disclaimer: (this package is non-free/contrib - ... long explanati
#License: (general information about the licensing regime)
#Copyright: (copyright statement that doesn't apply to any specific
#Comment ...
```

1. leave `Format` alone - it must be the same in all `debian/copyright` files
2. set `Source` to the URL of your upstream source
3. set `Upstream-Name` to the name of the upstream project
4. set `Upstream-Contact` to the name and e-mail address people should use when asking about legal issues
5. delete `Disclaimer` if this package goes in the [main archive area](#) (most packages), otherwise uncomment it and explain why
6. uncomment `License` and/or `Copyright` in the unlikely event your package has some licensing complexity that isn't otherwise captured, otherwise delete them
7. delete `Comment`, or uncomment it and add any other comments

 If it's hard to describe your upstream this way, see [the specification](#) for alternatives.

Write the Files stanzas

A `Files` stanza maps a set of files to a copyright statement and a license. Files with the same license but different copyright statements must go in separate stanzas. A simple `Files` stanza looks like this:

```
Files: file1
      file2
      ...
Copyright: 1975-2025, Upstream Maintainer <maintainer@upstream.com>
License: GPL-2
```

1. use [copyright review tools](#) to decide how to group packages
 - this may take a long time, especially if the upstream project doesn't specify the copyright statement at the top of each file

2. specify one file (e.g. `file1`) or wildcard (e.g. `src/*`) per line in `Files`
3. copy the `Copyright` statement from one of the files
4. put the appropriate short name in `License`
 - if your license is in the list of [standard short names](#), use that keyword
 - use e.g. `GPL-2` and `GPL-2+` instead of [GPL-2.0-only](#) and [GPL-2.0-or-later](#)
 - if your project uses the MIT license, use `Expat`
 - otherwise use the [SPDX license identifier](#)

 You can merge the file and license stanza for a license that only appears once.

Write the License stanzas

For each unique `License` in the `Files` stanzas, you need a `License` stanza with the full text of that license. A simple `License` stanza might look like this:

```
License: 0BSD
Copyright (C) YEAR by AUTHOR EMAIL
.
Permission to use, copy, modify, and/or distribute this software f
.
THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARR
```

If possible, use the short license text in `/usr/share/perl5/Software/LicenseMoreUtils/debian-summaries.yml` (part of [DebianPkg: libsoftware-licensemoreutils-perl](#)). Otherwise, include the complete text of your license.

Here are some commands to help generate `License` stanzas.

```
# Print short text for `GPL-2.0-or-later`:
sed -n \
    -e '/^GPL_2/,/^[^ ]/ { s/^ *$/ ./ ; s/{\{$or_later_clause\}}/, \
    /usr/share/perl5/Software/LicenseMoreUtils/debian-summaries.yml

# Print short text for `LGPL-2.1-only`:
sed -n \
    -e '/^LGPL_2_1/,/^[^ ]/ { s/^ *$/ ./ ; s/{\{$or_later_clause\}}
    /usr/share/perl5/Software/LicenseMoreUtils/debian-summaries.yml
```

```
# Append your upstream's COPYING file to debian/copyright:
sed -e 's/^/ /' -e 's/^ $/ ./' COPYING >> debian/copyright

# Download a copyright statement, reformat it, and put it in your p
curl https://raw.githubusercontent.com/spdx/license-list-data/main/
  | sed -e 's/^/ /' -e 's/^ $/ ./' \
  | xclip
```

[CategoryPackaging](#)

debian/copyright ([last modified 2025-08-03 13:50:45](#))

- [debian](#)
- [patches](#)

Translations: [English](#) - [Português \(Brasil\)](#)

Patch systems for Debian source packages

This page stemmed from the following discussion on debian-devel in January 2008:
<alpine.DEB.1.00.0801250818360.5187@wr-linux02>.

Note: The patch format itself is now a formal [DEP](#),  [DEP-3: Patch Tagging Guidelines](#).

Supported and obsolete patch systems

Patch system	Status	Location of the patches	How to deactivate a patch	Accepts diff -u output	Advantage
3.0 (quilt)	supported since dpkg 1.14.17	debian/patches in .debian.tar.gz file	Rename to include a non-word non-hyphen character	Yes	Native to  dpkg as 3.0 (quilt) format
 quilt	supported	debian/patches	Remove its name from debian/patches/series	Yes	Suitable for generating patches on any size codebase. Advanced VCS-like features. Simple
DebPkg: cdb simple-patchsys 2.0 a.k.a. wig&pen	 obsolete	debian/patches in .debian.tar.gz file	Remove its .diff or .patch suffix	Yes	Native to  dpkg as 2.0 format
 dpkg	obsolete (use 3.0 quilt)  obsolete	debian/patches	Remove its name from debian/patches/00list	Header needs to be added	Can do scripting
 dbs	 obsolete	debian/patches	Remove it from the directory	Yes	Patches applied in ASCIIbetical order, no series file. Tarball-in-tarball (if you're in to that).

[FixMe](#): rename page to *DebianPatches*? The address "debian/*" seems not to fit the wiki hierarchy.

debian/patches ([last modified 2025-03-28 23:50:22](#))

- [debian](#)
- [upstream](#)

[Translation\(s\):](#) [English](#) - [Português \(Brasil\)](#)

debian/upstream

This directory contains files describing the upstream project that is packaged.

debian/upstream/metadata

This is a file in YAML format where information about Upstream is recorded for the [UpstreamMetadata](#) gatherer.

This syntax is being formalised as  [DEP 12](#).

debian/upstream/signing-key.asc

This file is used by `uscan` to authenticate the source archive downloaded at the URL indicated by the [debian/watch](#) file.

debian/upstream/edam

This is a file in YAML format to categorise and describe the Debian package with terms of the  [EDAM ontology](#) ( [browse](#)). Together with the `debian/upstream/metadata` and `debian/control` files, the `upstream` folder then contains every information to auto-create a decent entry in the  [bio.tools](#) database of the European  [ELIXIR](#) project. If more than a single binary package is created, the file may be decided to name `packagename.edam`, instead. See [debian/upstream/edam](#) for details.

[FixMe](#): rename page to *DebianUpstream*? The address "debian/*" seems not to fit the wiki hierarchy.

[CategoryPackaging](#)

- [debian](#)
- [upstream](#)
- [edam](#)

Translations: [English](#) - [Português \(Brasil\)](#)

Tagging biomedical packages with EDAM

Formal categorisation of a package with concepts from the  [EDAM ontology](#) ( <https://bioportal.bioontology.org/ontologies/EDAM?p=classes>). For general categorisation, see [UpstreamMetadata](#).

It is formatted in YAML like the other files in the debian/upstream folder. Use commandline  [YAML Lint](#) for consistent validation. NB. The commandline version is not as strict as the (ugly) formatting produced by the online version!

For source packages with multiple binary packages that all need different EDAM annotation and/or for which the main binary package is not named like the source package, it is suggested to name the edam file *packagename.edam* .

The context of this development is the emerging  [bio.tools](#) database of the European  [ELIXIR](#) project. A set of scripts for an automated upload a bio.tools-ready description from the Debian database (edam, control, copyright, changelog) has already been implemented and is available:

-  <https://github.com/bio-tools/biotoolsConnect/blob/master/DebianMed/edam.sh> in the biotoolsConnect github repository
-  <https://salsa.debian.org/med-team/community/infrastructure/tree/master/edam/registry-tool.py> in the Debian Med subversion repository

These link the Debian package archive to entries in bio.tools. Those in search of a particular tool may thus become more quickly aware of Debian-provided binaries. Particularly in biological and medical sciences the confidence to use the same binary as others do, is of a particular value. Alternatives may be web services, but for many of today's high-throughput data, I/O is a bottleneck.

Issues

- There is yet no automated update of entries in bio.tools - today a second entry would be created. This is not what we want which is why there are e.g. no weekly

reuploads or for instance a hook in the Debian Med git repository to automate was has already been automated

- Licensing - the data in bio.tools is of a creative commons license, which needs to be mentioned in debian/control
- The bio.tools folks have established a range of EDAM annotations already, in part with renowned community efforts like SeqAnswers.com. How do we exchange information properly? The Debian package maintainer freely decides to peek a boo at those resources and describes so in the debian/copyright file?

Format description

Borrowing from the [debian/upstream/edam](#) file of the aspiring Debian package condetri, which again borrowed from trimmomatic, the first line identifies the ontology and version the file refers to. Typical for the EDAM ontology the whole package then has a single topic. That topic may have several scopes, but typically there is just one, i.e. a summary such.

```
---
ontology: EDAM (1.12)
topic:
  - Sequencing
scopes:
  - name: summary
    function:
      - Sequence trimming
      - Sequencing quality control
    inputs:
      - data: Sequence
        formats: [FASTQ]
    outputs:
      - data: Sequence
        formats: [FASTQ]
```

For some softwares suites, like for instance EMBOSS, it may be suitable to have several scopes to separate binaries. A scope has functions, with inputs and outputs.

Examples

A series of packages already features an EDAM annotation. You may decide to adopt terms from a similar program as a head start:

- [aegean](#)
- [andi](#)

-  [arden](#)
-  [artemis](#)
-  [barrnap](#)
-  [bowtie2](#)
-  [clustalo](#)
-  [condetri](#)
-  [dindel](#)
-  [fastag](#)
-  [fastqc](#)
-  [fastx-toolkit](#)
-  [jalview](#)
-  [mothur](#)
-  [mummer](#)
-  [muscle](#)
-  [picard-tools](#)
-  [pymol](#)
-  [rna-star](#)
-  [snpmatic](#)
-  [sra-sdk](#)
-  [tophat](#) - lost?
-  [trimmomatic](#)
-  [uc-echo](#)

This list is not complete.

Tools helping to organise EDAM annotation

- Andreas interlinked the EDAM files with the UDD and provides  [this script](#) to access the information. Perform the following for an overview on tools that feature an EDAM annotation in Debian:

```
wget -O edam_query.sh https://raw.githubusercontent.com/bio-tools/biotools/master/edam_query.sh
chmod +x edam_query.sh
# install postgresql client if not already installed
[ -x /usr/bin/psql ] || sudo apt-get install postgresql-client-9.5
./edam_query.sh
```

This produces a file named edam.txt with everything Debian today knows about EDAM and more - feels almost like worthy to upload to biotools 😊

```
$ head -n 3 edam.txt | tail -n 1
abacas | debian | sid | main
```

You can also create json output when calling the script with -j option:

```
$ ./edam_query.sh -j
```

This script is actually not intended as a fully qualified tool but rather as an example for an UDD query that can be turned into a tool.

See also

- [!\[\]\(e662c6fdc679f154c0e75d901761d894_img.jpg\) YAML](#)
- [!\[\]\(e0657301a840725a62b5d9c03de7d165_img.jpg\) http://bio.tools](#)
 - The [Common workflow language](#) aims at providing the means to help inter-connecting tools in the bio.tools database and beyond.
- [UltimateDebianDatabase](#) (UDD)
- [UpstreamMetadata](#)

[CategoryPackaging](#)

debian/upstream/edam (last modified 2025-08-02 16:47:20)

- [UpstreamMetadata](#)

Optional metadata about a package's upstream project, which can be queried from the [Ultimate Debian Database](#). For a formal categorisation, see [debian/upstream/edam](#). For the specification, see  [DEP 12](#).

Contents

1. [Create debian/upstream/metadata](#)
2. [Upstream METadata GAttered with YAMl \(UMEGAYA\)](#)
3. [History](#)
 1. [First proof of principle](#)
 2. [Discussion](#)
 1. [Problems](#)
 3. [Lintian check](#)
 4. [Deprecated features](#)
 1. [Deprecated fields](#)
 2. [Hyphen shortcut for mappings](#)
4. [Similar technologies](#)
 1. [Edam files](#)
 1. [Fields](#)
 2. [AppStream](#)
5. [See Also](#)

Create debian/upstream/metadata

debian/upstream/metadata is a  [YAML](#) document described in  [DEP 12](#). Information can be left out if it isn't relevant for the current project, and the file doesn't need to be included if no relevant information is available.

Copy the following example, then fill in the relevant fields:

```
# Metadata about the upstream project.
# See https://wiki.debian.org/UpstreamMetadata

# Uncomment these and fill them out to help users interact with ups
#Bug-Database: https://github.com/<user>/<project>/issues
#Bug-Submit: https://github.com/<user>/<project>/issues/new
#Changelog: https://github.com/<user>/<project>/blob/master/CHANGES
#Documentation: https://github.com/<user>/<project>/wiki
#Repository-Browse: https://github.com/<user>/<project>
#Repository: https://github.com/<user>/<project>.git
```



```

#FAQ: https://github.com/<user>/<project>/blob/main/FAQ.md
#Webservice: https://<project>.github.io/live-demo/
#Donation: https://github.com/sponsors/<project>
#Registration: https://github.com/signup
#Archive: PyPI # or CPAN, boost, etc.

# Uncomment these and fill them out to help users evaluate upstream
#Gallery: https://github.com/<user>/<project>/wiki/pictures-made-wi
#Screenshots: # pictures *of* the program, as opposed to pictures *
# - https://github.com/<user>/<project>/wiki/login-screen.png
# - https://github.com/<user>/<project>/wiki/help-menu.png
# - ...

# Uncomment these and fill them out to help resolve security issues
#Security-Contact: <how to send security-related messages>
#CPE: <space-separated Common Platform Enumerator values - see http

# Uncomment these and fill them out to help researchers cite the pr
#ASCL-Id: <Astrophysics Source Code Library ID>
#Cite-As: <how the authors want their software be cited in publicat
#Funding: <sources of funding>
#Registry: # See DEP-12 for valid values
# - Name: <software registry>
#   Entry: <entry in registry>
# - ...

#Reference:
# -
#   Author: <please use full names and separate multiple author by
#   Title: "remember to quote strings: colon characters confuse YA
#   Type: <BibTeX entry type (e.g. "book", "inproceedings", or "ar
#   Year: <year of publication>
#   Booktitle: <title of the book the article is published in>
#   Editor: <editor of the book>
#   ISBN: <International Standard Book Number of the book>
#   Publisher: <Publisher of the book>
#   Journal: <abbreviated journal name>
#   ISSN: <International Standard Serial Number of the periodical>
#   Number: <issue number>
#   Volume: <journal volume>
#   Pages: <article page number(s)>
#   DOI: <digital object identifier of the publication describing
#   PMID: <PubMed ID number>

```

```
# URL: <link to article's abstract, if different from eprint and
# eprint: <link to the article's full PDF>
# -
# ...
#Other-References: <link to a page with more references>
```

For real-world examples, see codesearch.debian.net.

This file must be valid [YAML](#). Check the validity of your file online (e.g. with [Online YAML Parser](#)), or in a terminal (e.g. with [DebPkg: yamllint](#)).

The most common error is that you must put quotes around strings containing " : ":

```
Title: ✗ invalid YAML: colon outside of quoted string
Title: "✓ valid YAML: colon inside of quoted string"
```

(YAML disallows the first line because the colon might be an attempt to define a key called "invalid YAML")

Upstream MEtadata GAttered with YAMl (UMEGAYA)

⚠ This section is an archive of the original discussion. If you just want to write a `debian/upstream/metadata` file, you can safely ignore it.

History

UMEGAYA (Upstream MEtadata GAttered with YAMl) was an attempt effort to collect meta-information about upstream projects in a file called `debian/upstream.metadata` in the source packages maintained in a publicly accessible version control system (at that time Subversion or Git). Since this information can be directly accessed from the VCS, the idea was that it could be updated without uploading the source packages to the Debian archive.

This experiment evolved into [DEP-12](#), where the file collecting the meta-information is now called [debian/upstream/metadata](#).

First proof of principle

The [DebianMed](#) [web sentinels](#) use the [UltimateDebianDatabase](#) (UDD) to display bibliographic information about which academic article to cite when using our packages. This was previously done by collecting the information from the central file

used to create the med-* metapackages (See the [Bits from Debian Pure Blends of October 2012](#)).

A UDD importer was developed, consisting of a [gatherer](#) and a [UDD module](#).
The current importer is in https://salsa.debian.org/ga/udd/-/blob/master/udd/upstream_reader.py.

The data about bibliographic information is loaded in the [bibref](#) table of the [UltimateDebianDatabase](#). The following UDD query outputs all source packages featuring bibliographic information. (The join is needed to exclude those references of packages that are not yet uploaded to Debian package pool but used in so called blends prospective packages.)

```
SELECT distinct s.source from bibref b join sources s on s.source = b..
```

Discussion

Discussion took place on the wiki, and on the [DebianList: debian-med](#) and [debian-ga](#) mailing lists.

The data is not really Debian-specific, let's put it outside Debian and use mechanisms for [Mapping package names across distributions](#).

To do: formalise the above using [Config::Model::Backend::Yaml](#), and generate docs as explained in <http://ddumont.wordpress.com/2011/04/08/configuration-doc-generation-with-configmodel/>.

** In addition to ?DOAP, other Semantic Web ontologies/namespaces/schemas should be reused in order to not reinvent the wheel, and enable such metadata to participate to the ?Semantic Web (see also Open Linked Data matters). As such, [SPDX](#) would be an interesting standard to link to, as well as ADMS.F/OSS, for packages description, IMHO. Syntactically, any form of [RDF](#) would be interesting to explicitly convey the prefixes in the field names... and I'm not sure it can be done in ?YAML -- [OlivierBerger](#)*

[BenFinney](#) asks (2019-01-21): Which version of the YAML specification? Please link to the exact YAML specification that rules this format.

Problems

BibTeX: Currently there is no way to let {} that might be used to force capitalisation in BibTeX entries slip through from debian/upstream/metadata into BibTeX

It seems that the python library which is used to parse debian/upstream/metadata files for inclusion into UDD has a bug when values are of the form <d:d> (decimal_number

colon decimal_number). You should include strings like this into single quotes. (see [Discussion on Debian Med mailing list](#))

Lintian check

Simon Kainz has written some preliminary [lintian check](#) to verify the syntax of debian/upstream/metadata files (see also [Closed in 2.5.81: #731340: lintian: Check if debian/upstream files are valid YAML: 731340](#)). Any testing is welcome. A simple lintian check for YAML syntax was implemented by Petter Reinholdtsen (see [Closed in lintian/2.5.41: #813904: lintian: Check if upstream/metadata files are well formed: 813904](#)). Andrius Merkys is working on [validator for values](#) in debian/upstream/metadata files.

Deprecated features

Deprecated fields

According to [DEP5](#) these fields belong to *debian/copyright* and should not be duplicated in *debian/upstream/metadata*:

Name *

Upstream name of the packaged work.

Contact

Which person, mailing list, forum,... to send messages in the first place.

Yet it was objected that these fields must still be allowed, as not all packagers wish to use DEP 5.

Hyphen shortcut for mappings

Only a subset of YAML is used: sequences are only expected to contain scalars and mappings are only expected to contain a scalar or a mapping, but with only one level of imbrication.

In addition, two conventions that are not part of the YAML format were proposed and used in the [umaegaya gatherer](#), but have been abandonned since and are not used anymore:

- Field names are case-insensitive.
- Nested mappings are shortcuts for longer field names composed of both mapping field names separated by a dash. The following two examples are equivalent:

```
Foo:
  Bar: baz
```

Similar technologies

Edam files

The  [EDAM ontology](#) provides some means to classify software used in bioinformatics. The Debian Med team intends to link all bioinformatic tools with the EDAM ontology. To approach this the YAML file [debian/upstream/edam](#) can provide extra information.

Fields

ontology

EDAM (1.13) (currently version 1.13 is the latest EDAM version)

topic

EDAM topic

scopes

EDAM scopes

AppStream

[AppStream](#) was initially conceived to provide a user-visible app store for the desktop, as such there is overlap in the  [project metadata](#), but not all the fields listed above are supported.

See Also

- [CPEtagPackagesDep](#)
- [debian/upstream/edam](#)

[CategoryPackaging](#) [CategoryPermalink](#)

UpstreamMetadata (last modified 2025-08-02 16:45:13)

- `debian`
- [watch](#)

Translations: [English](#) - [Português \(Brasil\)](#)

Contents

1. `debian/watch`

- 1. [Common mistakes](#)
- 2. [Version number gotchas](#)
- 3. [Cryptographic signature verification](#)
- 4. [Upstream source sites](#)

1. [GitHub](#)

1. [Version 4 Samples](#)

- 2. [GitLab](#)
- 3. [Salsa \(Gitlab\)](#)
- 4. [Gitea / Forgejo / Codeberg.org](#)
- 5. [Bitbucket](#)
- 6. [Launchpad](#)
- 7. [PyPI](#)
- 8. [SourceForge](#)
- 9. [KDE](#)
- 10. [Uncommon sites](#)
- 11. [Autogenerated](#)

2. [Further information](#)

3. [Other distros](#)

debian/watch

Warning: the official documentation for `debian/watch` is provided by  [debian-watch\(5\) manpage](#). This documentation may be outdated.

The `watch` file located in the `debian` directory serves the purpose of monitoring for updates of upstream software and, if available, initiating its download. This downloading process is facilitated by the [DebianMan: uscan](#) program, which is part of the [DebianPkg: devscripts](#) package. `uscan` can be invoked with the path to the `debian` directory containing the `watch` file as an argument, or it can traverse through directories below the current working directory to locate the necessary files.

Please see also the  [Debian Policy Manual](#) about `debian/watch`. Updating to version 5 changes the format to `rfc822`-style, but offers many templates to simplify checks, e.g. towards Github and other forges. For more information regarding templates take a look at the  [uscan-templates\(5\) manpage](#).

Basically a watch file will have this format:

```
Version: 5
Source: https://somesite.com/dir/
```

and `uscan` will try to figure out the correct file and version.

To specify the file name to look for in hrefs of the web page you can define the `Matching-Pattern` field.

```
Version: 5
Source: https://somesite.com/dir/
Matching-Pattern: filename_(.+).tar.gz
```

The uscan program will then execute a "dir" command and check all the files in that directory for the highest version number.

uscan also supports the FTP protocol. Unfortunately, when working with HTTP, HTML pages do not always contain directory listing with files. The complete (or relative) path of the .tar.gz file to be downloaded will appear as a hyperlink within the web page. We hence need two types of information, the path to the page announcing the file and a regular expression to "grep" for the right link:

```
version=4
https://somesite.com/path link_from_href-1.0.tar.gz
```

"https://somesite.com/path" is the site from where you are downloading the source and "link_from_href.1.0.tar.gz" is obtained from the HTML source code (from the "<a href=" tags).

If https://somesite.com/path HTML code has "" inside it, for example, you will use

```
https://somesite.com/dir foo/program-(.+)\.tar\.gz
```

Watch files with errors are generally wrong on this second part.

Generally you want a slightly more flexible regex for the tarball name so that if upstream switches tarball naming schemes or compression formats you are covered:

```
https://somesite.com/dir foo/program-(.+)\.(?:zip|tgz|tbz|txz|(?:tar\.(?:gz|bz2|xz)))
```

For FTP sites it changes a little, but it's basically the same thing ([DebianMan: man uscan](#) will help you).

But the nice part is to test it:

```
uscan --no-download --verbose
```

If it's not working as expected, you can use debug to see what it's fetching and what it's (not) matching:

```
uscan --no-download --verbose --debug
```

Nice write-up by João Eriberto

 <http://eriberto.pro.br/blog/?p=1459>

Common mistakes

- Not escaping dots, which match any character. The solution is `\.` instead of `.` in the regex.
- A file extension regex that is not flexible enough. The solution is `\.(?:zip|tgz|tbz|txz|(?:tar\.(?:gz|bz2|xz)))`
- Not anchoring the version group at the right place. The solution is to include something before `(\d\S+)` like `fooproj-(\d\S+)\.tar.gz`
- Not starting the version part of the regex with a digit. The solution is to use `\d` instead of `.`
- Not being flexible enough in the path to the file. The solution is to use `https://example.com/someproject/.*/program-(\d\S+)\.tar.gz` instead of `https://example.com/someproject/ /path/to/program/downloads/program-(\d\S+)\.tar.gz`
- Not mangling upstream versions that are alphas, betas or release candidates to make them sort before the final release. The solution is to use `uversionmangle` like this `opts=uversionmangle=s/(\d)[_\. \- \+]?((RC|rc|pre|dev|beta|alpha)\d*)$/$1~$2/`
- Not mangling Debian versions to remove the `+dfsg.1` or `+dfsg1` suffix. The solution is to use `dversionmangle` like this `opts=dversionmangle=s/\+(debian|dfsg|ds|deb)(\.\d+)?$/`
- Not enabling [cryptographic signature verification](#) when your upstream signs their releases with OpenPGP.

Version number gotchas

For upstreams that release alpha, beta, release-candidate versions it's common to use versions like 1.0-alpha, 1.0-rc1, ... before 1.0 Without proper mangling this will break `dpkg` version comparisons:

```
dpkg --compare-versions '1.0-1' gt '1.0-alpha-1' # false
dpkg --compare-versions '1.0-1' gt '1.0~alpha-1' # true (notice the ~)
```

The solution is to use `uversionmangle` like this

```
opts=uversionmangle=s/(\d)[_\. \- \+]?((RC|rc|pre|dev|beta|alpha)\d*)$/$1~$2/
```

before importing new releases.

If you already uploaded a "broken" version number you can sometimes use the `ds<n>` repack suffix e.g. `'1.0-ds1-1'` or `'1.0.ds1-1'`

See <https://codesearch.debian.net/search?q=repacksuffix%3D%2Bds1&literal=1>

Cryptographic signature verification

If your upstream provides cryptographic signatures for their packages in the same place that the packages are available for download, and you know what OpenPGP key or keys will be used to sign these packages, `uscan` can verify these signatures for you if you give it the `pgpsigurlmangle` option.

For example, for OpenSSH (which uses a `.asc` suffix for the package signatures), you'd place [Damien Miller's public key](#) ascii-armored in `debian/upstream/signing-key.asc`:

```
gpg --armor --export-options export-minimal --export '59C2 118E D206 D927 E667 EBE3 D3E5
```

and then update `debian/watch` to say:

```
version=4
opts=pgpsigurlmangle=s/$/.asc/ ftp://ftp.openbsd.org/pub/OpenBSD/OpenSSH/portable/openssh
```


This lets you ensure that the package was not tampered with in transit, and that it came from the developer you expected it to come from. A valid signature does not mean that the contents of the package are somehow magically perfect and DFSG-free and policy compliant, of course!

Often, when upstream uses git, it doesn't sign its releases, but signs its tags. Here is an example of a debian/watch file using mode=git to retrieve releases and checking tags signatures:

```
version=4
opts="mode=git,pgpmode=gittag" \
https://mygitlabinstance.tld/myusername/myproject.git refs/tags/v([\d\.]+)
```

Upstream source sites

GitHub

Watch files in version 5 introduce templates, which simplifies the watch file usually to something like this:

```
Version: 5
Template: Github
Owner: <owner or github org name>
Project: <reponame>
```

If you need to pull maintainer created release tarballs from Github, you can not use the template. A version 5 watch file, using the API endpoint for version and download information, can look like this:

```
Version: 5
Search-Mode: plain
Pgp-Mode: auto
Source: https://api.github.com/repos/<owner or github org name>/@PACKAGE@/releases?per_page=1
Matching-Pattern: https://github.com/[^/]+/[^/]+/releases/download/(?:\d\.)+/@PACKAGE@
```

Version 4 Samples

If the project using version tags:

```
version=4
https://github.com/<REPONAME>/tags .*/v?(\d.*)@ARCHIVE_EXT@
```

If the project offer a *Release* section:

```
version=4
https://github.com/<REPONAME>/releases/download/v(\d+).(\d+).(\d+).tar.gz
```

If upstream releases beta/alpha tarballs, you will need to make use of uversionmangle.

```

version=4
opts="searchmode=plain,\
filenamemangle=s%v?@ANY_VERSION@@@PACKAGE@-$1.tar.gz%,\
uversionmangle=s/(\d)[_\.\\-\\+]?((RC|rc|pre|dev|beta|alpha|a|b)\d*)$/$1~$2/" \
https://api.github.com/repos/<user>/<project>/releases?per_page=50 \
https://github.com/<user>/<project>/releases/download/@ANY_VERSION@/@PACKAGE@-@ANY_VERSION@

```

If your d/watch is failing to find the tarballs, you can analyze the page at:

 https://api.github.com/repos/<user>/<project>/releases?per_page=50

And identify a regex that can be used to fetch the right tarball.

As of 2022-10-02, ?GitHub change the way its release/tags page is rendered. The links are dynamically loaded in ?JavaScript instead of static text. See thread for the discussions: 

<https://lists.debian.org/debian-devel/2022/09/msg00224.html>

Example using the Github API:

Although it is possible to obtain a tarball it another way, the API also works and it is up to the maintainer's discretion which of the options to use.

```

version=4
opts="searchmode=plain,\
filenamemangle=s%v?@ANY_VERSION@@@PACKAGE@-$1.tar.xz%" \
https://api.github.com/repos/<user>/<project>/releases?per_page=100 \
https://api.github.com/repos/[^/]+/[^/]+/tarball/v?@ANY_VERSION@

```

Manipulation using v1.0.0 (example) and signed tarballs (for security reasons as integrity check, it is highly recommended and see  [how can I verify tor source code?](#)):

```

version=4
opts=downloadurlmangle=s%archive/refs/tags/v(.*)\.tar\.gz%releases/download/v$1/@PACKAGE@
pgpsigurlmangle=s/$/.asc/ \
https://github.com/<user>/@PACKAGE@/tags \
(?:.*?/)?v?@ANY_VERSION@@@ARCHIVE_EXT@

```

GitLab

From **2024/03** the packages that receive tarballs from [GitLab](#) need to be changed (showing the error `debian/watch nomatching files for watch`), the solution below has been tested and other previous solutions will not work. [GitLab](#) changed the tag order and `debian/watch` will have to be updated and suggestions below:

```

version=4
opts="searchmode=plain" \
https://gitlab.com/<user>/@PACKAGE@/tags?sort=updated_desc -/archive/v?\d[\d.]+/@PACKAGE@

```

Or

```
version=4
opts="searchmode=plain" \
https://gitlab.com/<user>/@PACKAGE@/tags?sort=updated_desc -/archive/v?\d[\d.]+\@PACKAGE@
```

You can also use `filenamemangle` following the Salsa example. See <https://lists.debian.org/debian-mentors/2024/03/msg00259.html>

Fay Stegerman figured out how to get tarball using upload URL instead of archive URL and she explains about it here: <https://lists.debian.org/debian-mentors/2024/04/msg00013.html> and example [Switch to new version scheme](#). See also [uscan \(wget\) no longer presented the same HTML as a normal browser for releases](#).

If upstream uses only tags and not releases, one has to forge the URLs from the API page:

```
version=4
opts="pagemangle=s!"name": "(v?@ANY_VERSION@)"!<a href="https://gitlab.com/@PACKAGE@/@PACKAGE@/api/v4/projects/@PACKAGE@%2F@PACKAGE@/repository/tags \
https://gitlab.com/@PACKAGE@/@PACKAGE@/.*/@PACKAGE@-v?@ANY_VERSION@@@ARCHIVE_EXT@
```

Salsa (Gitlab)

Please ensure that the tags are strictly in numeric format. For example, the version tag should be written as 1.4.5.

```
version=4
opts=filenamemangle=s%.*/(\d\S+)/archive\.\tar\.\gz%<project>-$1\.\tar\.\gz% \
https://salsa.debian.org/<user>/<project>/tags .*/(\d\S+)/archive\.\tar\.\gz
```

Gitea / Forgejo / Codeberg.org

If the project use release tags like v1.2.3.

```
version=4
https://codeberg.org/<user>/<project>/tags .*/v@ANY_VERSION@@@ARCHIVE_EXT@
```

Bitbucket

```
version=4
https://bitbucket.org/<user>/<project>/downloads?tab=tags .*/(\d\S+)\.\tar\.\gz
```

The 2nd regexp matches most often used tags like '0.1.2' or 'v0.1.2'. It may need to be adapted if upstream uses other version tags.

Launchpad

```
version=4
opts=pgpsigurlmangle=s/$/.asc/ \
https://launchpad.net/<project>/ \
https://launchpad.net/. *download/<project>- ([\d]+)\.tar\.xz
```

PyPI

The PyPI site is hard to work with directly as it includes md5sums for the downloads in the URLs which makes for truly horrible regexps. The pypi.debian.net redirector makes it much easier to work with PyPI:

```
version=4
opts=pgpsigurlmangle=s/$/.asc/ \
https://pypi.debian.net/<project>/<project>- (.)\.(?:zip|tgz|tbz|txz|(?:tar\.(?:gz|bz2|xz
```

There's also (a bit more advanced) autogenerated watch file that you can download from  <http://pypi.debian.net/<project>/watch>

If you do want to work directly with PyPI, then make sure you use the new Simple API:

```
https://pypi.python.org/simple/<project>/
```

Note that direct use of the PyPI listing pages for packages (such as <https://pypi.python.org/pypi/<package>/>) is discouraged by the PyPI administrators. Automated use of package listing pages is not supported and is liable to break; the Simple API is provided for precisely that reason.

SourceForge

```
version=4
# qa.debian.org runs a redirector which allows a simpler form of URL
# for SourceForge based projects. The format below will automatically
# be rewritten to use the redirector.
http://sf.net/audacity/audacity-src-([\d\S+)]\.(?:zip|tgz|tbz|txz|(?:tar\.(?:gz|bz2|xz)))
```

See also:  <https://salsa.debian.org/qa/qa/tree/master/wml/watch/>

KDE

```
version=4
opts=pgpsigurlmangle=s/$/.sig/ https://download.kde.org/stable/release-service/([\d.]+)
```

Uncommon sites

If you have a site that has version numbers in some form but doesn't have hrefs containing them and the URL mangling capabilities of uscan are not enough, you can create a redirector. The Debian QA folks run

one called [!\[\]\(f4912148590488019602cab6e009e597_img.jpg\) `fakeupstream.cgi`](#) ([!\[\]\(d7a8eaa1c5d6eb8f857fe636176c5d31_img.jpg\) `git`](#)) for lots of different upstream sites. If you want to add a new one, please submit a wishlist [!\[\]\(374c4e35f0de7d5605f992f908c4a567_img.jpg\) `bug report assigned to qa.debian.org`](#) with or without a patch.

Files hosted on various big project hosting sites can be specified with the URLs below. SourceForge has a special "redirector" URL (see more details in 'man uscan'). This allows the upstream URLs to change without the need to adapt watch files of affected packages.

Autogenerated

A scripted [!\[\]\(bd1a142de767a21e5362c595f844a4ff_img.jpg\) `debian/watch.generator`](#) is available

Further information

- Explaining blog post how to write a useful regexp line: [!\[\]\(8c4dca64662d21542001ca0ed7eeb688_img.jpg\) `http://eriberto.pro.br/blog/2013/10/07/how-to-write-a-good-debianwatch-easily/`](#)
- Ignore some upstream versions, such as upstream changed from v20190101 to v1.2.3: [!\[\]\(3de35c640e7147a3fb61ee393128d2ae_img.jpg\) `https://lists.debian.org/debian-mentors/2020/08/msg00025.html`](#)

Other distros

Fedora also has an [!\[\]\(830769b31eeeaca920791081939ff8ba_img.jpg\) `upstream release monitoring project`](#) ([!\[\]\(198f559926258ddfad814817bda0ffbc_img.jpg\) `more info`](#)).

[CategoryPackaging](#)

debian/watch ([last modified 2026-03-10 18:45:13](#))

- [Mentors](#)

 [Debian Mentors](#) are Debian Developers that are willing to upload packages to Debian repositories on behalf of the actual maintainer. The Debian Developer will also check the package for technical correctness and help the maintainer to improve the package if necessary. Mentors are also called *sponsors*.

The individuals listed below are available to sponsor and/or mentor packages in specific areas of Debian.

If you're a Debian member with upload rights, feel free to sign up and indicate in which areas you are willing to sponsor packages (or add additional categories). If you're unable to sponsor packages temporarily, just remove your name (or comment it out.)

Categories

The sponsors listed below are experienced in the following categories; if you want your package sponsored, you should consider talking to one of them in addition to using  debian-mentors@lists.debian.org or  [#debian-mentors](#)

If there are no sponsors listed below for the appropriate categories, take a look at the [FAQ](#) for how to get your packages sponsored via debian-mentors, [teams](#) or other means.

In addition to package sponsorship, there are one-on-one mentoring opportunities [available](#).

Perl

- [DonArmstrong](#)

Python

- [OttoKekalainen](#)

Gnome

Go

- [OttoKekalainen](#)

KDE

Libraries

General

- [OttoKekalainen](#)

Mentors

- [DonArmstrong](#) -- perl packages
- [OttoKekalainen](#)

[CategoryDeveloper](#) [CategoryPackaging](#)

Mentors ([last modified 2025-07-30 03:28:54](#))

- [SponsorChecklist](#)

Contents

1. Checklist for Package Sponsors

1. Checklists from individual DDs/groups
2. Determine if the package actually belongs in Debian
3. Determine if the package can actually get into Debian
4. Determine if the maintainer can actually maintain the package
5. Determine if you can back up the maintainer
6. Make sure that the package can be distributed by Debian
7. Make sure that the packaging is up to par
 1. control
 2. rules
 3. changelog
 4. general
8. Check out the upstream codebase

Checklist for Package Sponsors

The checklist below is not exhaustive; you should be applying all of the normal checks you do to your own packages in addition to these.

(If there are additional checks that you think should be added, feel free to add them.)

Checklists from individual DDs/groups

- [Checklist for NEW](#),  [REJECT-FAQ](#),  [FTP-Master Drinking Game](#)
- [Debian games team](#)
- [Debian Ruby Team](#)
-  [Axel Beckert](#)
- [Daniel Kahn Gillmor](#)
-  [Eriberto Mota](#)
- [KilianKrause](#)
-  [Mattia Rizzolo](#)
-  [Michal Čihař](#)
-  [Neil McGovern](#)
-  [Paul Tagliamonte](#)
- [Paul Wise](#)
-  [Piotr Ożarowski](#)

-  [Raphaël Hertzog](#)
- [Sergio Durigan Junior](#)

Determine if the package actually belongs in Debian

- Is there a significant use case for the package?
- Are there pre-existing packages that do a better job?
- Would you use the package?
- Does the maintainer use the package?
- What is the maturity level of the upstream codebase?
- Is upstream active (alive)? If not, is the maintainer capable of handling upstream problems? Are you?

Determine if the package can actually get into Debian

- Does the package conform to all the items in the ftpmasters  [REJECT-FAQ](#) and [NEWChecklist](#) documents?

Determine if the maintainer can actually maintain the package

- What is the skill level of the maintainer?
- Are they familiar with the package and its languages?
- How active are they?
- Do they have existing packages?
- Do they have too many packages or too many bugs?
- How do they interact with users? [Check out their existing bugs.]

Determine if you can back up the maintainer

- Is the package one that you are comfortable maintaining if the maintainer disappears?
- Do you have time to assist the maintainer if they get stuck?

Make sure that the package can be distributed by Debian

- Does debian/copyright list all of the copyrights of portions of the code?
- Are all pieces licensed under licenses that Debian can distribute?
- Does the proposed section (main, contrib, non-free) match the license?
- Are there significant patents which the work infringes which are known to be enforced?

Make sure that the packaging is up to par

control

- Is the description good?
- Have they sent an ITP with the description to -devel? [Have they incorporated any comments?]
- Are the dependencies correct? Are there missing recommends? Suggests?

rules

- Are the rules sane?
- Are all of the targets there?
- Are they using existing tools correctly? (debhelper? quilt? etc?)
- Have they removed (or commented out) useless debhelper calls?
- Does the maintainer understand what their rules file does?

changelog

- Are the changes to the packaging described?
- Are ITP/ITA bugs closed?
- Have the comments in the ITP/ITA bug been addressed?
- Are existing bugs in the package fixed?
- If a security issue is mentioned is a CVE id mentioned too?
- Are bugs closed by the upstream code just closed or are the fixes described too?

general

- Is the package lintian clean?
- Does it build in a buildd pbuilder chroot?
- Does it build correctly a second time after cleaning?
- Does the resulting package install correctly?
- Does it uninstall cleanly?
- Does it pass piuparts tests?
- Do the resultant packages work? Have you used them?
- Does the package contain what it's supposed to contain?
- Does the .diff.gz look sane?
- Is the upstream source pristine?

Check out the upstream codebase

- Is the upstream source sane?
 - If upstream provides md5sum/shasums for the source, does the signature still match?
 - Are you building against Debian distributed libraries and not against internal copies?
 - Are there potential security issues? (Daemons, tempfile vulnerabilities, setuid binaries?)
 - If building against internal code copies, has the maintainer notified the security team that there are embedded code copies?
-

SponsorChecklist (last modified 2025-08-23 00:21:09)

- [Software that can't be packaged](#)

Software ineligible for Debian packaging



The purpose of this article is to avoid unnecessary efforts to analyze software that has already been determined as incompatible with Debian.

Please, maintain this table in alphabetical order by name.

Name	Version	Description	Reason	Debian Bug	Repology Entry
 Bypass Paywalls Firefox Clean	3.3.5.0	Firefox browser plugin to bypass various paywalls	Potentially illegal under German law	Closed: #1053256: ITP: bypass paywalls- firefox clean Firefox browser plugin to bypass various paywalls: #1053256	—
 NXLog CE	2.7.1191	Universal log collector and forwarder	DFSG-incompatible	Closed: #739338: RM: nxlog-ce RoM; new license is DFSG-incompatible: #739338	 nxlog
 sock	0.3.2	Richard Stevens' sock program	DFSG-incompatible	Closed: #486127: ITP: sock make a customized traffic over network: #486127	 sock
 urlcrazy	0.5	Generate and test domain typos and variations	DFSG-incompatible	—	 urlcrazy
 VeraCrypt	1.16	Cross-platform on-the-fly disk encryption	DFSG-incompatible	Closed: #814352: ITP: veracrypt Cross-platform on-the-fly encryption: #814352	 veracrypt

See also

-  [wnpp.debian.net](#) lets you search for requests/intents to package software
- [Packaging](#) discusses the packaging process in general



Debian NEW and BYHAND Packages

Summary for: NEW

Click to toggle all/binary-NEW packages

Package	Version	Arch	Distribution	Age	Upload info	Closes
pymatgen-core-test-files	2026.4.16+dfsg-1	source all	unstable	3 weeks	Maintainer: Debichem Team Changed-By: Drew Parsons Fingerprint: 23C9A93E585819E9126D0A36573EF1E4BD5A01FA	
ipscan	3.9.3+ds-1	source arm64	unstable	1 week	Maintainer: Jonathan Bergh Changed-By: Jonathan Bergh Sponsor: tar@debian.org Fingerprint: B60A1BF363DC1319FF0A8E89116852BCDF7515C0	#1122631
rust-bitstring	0.2.1-1	source amd64	unstable	4 days	Maintainer: Daniel Baumann Changed-By: Daniel Baumann Fingerprint: 2698683880B6A84A3D044602FBB4F0E80A80222F	
optking	0.5.0-1	source all	unstable	4 days	Maintainer: Debichem Team Changed-By: Michael Banck Fingerprint: 9CA877749FAB2E4FA96862ECDC686A27B43481B0	
rust-axum-extra	0.12.6-2 0.12.6-1	source amd64	unstable	4 days	Maintainer: Daniel Baumann Changed-By: Daniel Baumann Fingerprint: 2698683880B6A84A3D044602FBB4F0E80A80222F	
rust-axum-macros	0.5.1-1	source amd64	unstable	4 days	Maintainer: Daniel Baumann Changed-By: Daniel Baumann Fingerprint: 2698683880B6A84A3D044602FBB4F0E80A80222F	
rust-typed-json	0.1.1-1	source amd64	unstable	4 days	Maintainer: Daniel Baumann Changed-By: Daniel Baumann Fingerprint: 2698683880B6A84A3D044602FBB4F0E80A80222F	
rust-password-hash-0.5	0.5.0-1	source amd64	unstable	4 days	Maintainer: Debian Rust Maintainers Changed-By: kpcyrd Fingerprint: 64B13F7117D6E07D661BBCE0FE763A64F5E54FD6	
rust-rrule	0.14.0-1	source amd64	unstable	4 days	Maintainer: Debian Rust Maintainers Changed-By: Dominik George Fingerprint: A4EB3C5160961C85E80191310AE554E5460E1BDD	
zimfw	1.20.0-1	source all	unstable	4 days	Maintainer: Felipe Sateler Changed-By: Felipe Sateler	#1135885

Package	Version	Arch	Distribution	Age	Upload info	Closes
					Fingerprint: 218EE0362033C87B6C135FA4A3BABAE2408DD6CF	
rust-mediatype	0.21.0-1	source amd64	unstable	4 days	Maintainer: Debian Rust Maintainers Changed-By: kpcyrd Fingerprint: 64B13F7117D6E07D661BBCE0FE763A64F5E54FD6	
rust-rw-stream-sink	0.4.0-1	source amd64	unstable	3 days	Maintainer: Debian Rust Maintainers Changed-By: kpcyrd Fingerprint: 64B13F7117D6E07D661BBCE0FE763A64F5E54FD6	
rust-copy-dir	0.1.3-1	source amd64	unstable	3 days	Maintainer: Debian Rust Maintainers Changed-By: Jelmer Vernooij Sponsor: jelmer@debian.org Fingerprint: DC837EE14A7E37347E87061700806F2BD729A457	
rust-isal-sys	0.5.3+dfsg-1	source amd64	unstable	3 days	Maintainer: Debian Rust Maintainers Changed-By: Jelmer Vernooij Sponsor: jelmer@debian.org Fingerprint: DC837EE14A7E37347E87061700806F2BD729A457	
rust-cx448	0.1.1-1	source amd64	unstable	3 days	Maintainer: Debian Rust Maintainers Changed-By: Alexander Kjäll Sponsor: capitol@debian.org Fingerprint: 440B7638ED88D68D5F351BCFB95B0493F5AB0C47	
rust-isal-rs	0.5.3-1	source amd64	unstable	2 days	Maintainer: Debian Rust Maintainers Changed-By: Jelmer Vernooij Sponsor: jelmer@debian.org Fingerprint: DC837EE14A7E37347E87061700806F2BD729A457	
rust-cpubits	0.1.1-1	source amd64	unstable	2 days	Maintainer: Debian Rust Maintainers Changed-By: kpcyrd Fingerprint: 64B13F7117D6E07D661BBCE0FE763A64F5E54FD6	
gcc-16-cross-mipsen	1+c1	source all amd64	unstable	1 day	Maintainer: Debian GCC Maintainers Changed-By: YunQiang Su Fingerprint: 816790FE0A75677E2A6C22C814135D277B88D7E5	
rust-pyo3-introspection	0.28.3-1	source amd64	unstable	1 day	Maintainer: Debian Rust Maintainers Changed-By: Martin Sponsor: debacle@debian.org Fingerprint: 9FDAD37B09F913B996AA86ADB490E3DF337F3213	

Package	Version	Arch	Distribution	Age	Upload info	Closes
aiokem	1.1.4-1	source all	unstable	1 day	Maintainer: Home Assistant Team Changed-By: Edward Betts Fingerprint: FB8ACFA78C726089C38AD0269605A1098C63B92A	#1136397
aiontfy	0.8.5-1	source all	unstable	1 day	Maintainer: Home Assistant Team Changed-By: Edward Betts Fingerprint: FB8ACFA78C726089C38AD0269605A1098C63B92A	#1136398
rust-cached	0.59.0-1	source amd64	unstable	1 day	Maintainer: Debian Rust Maintainers Changed-By: kpcyrd Fingerprint: 64B13F7117D6E07D661BBCE0FE763A64F5E54FD6	
aiowebdav2	0.6.2-1	source all	unstable	1 day	Maintainer: Home Assistant Team Changed-By: Edward Betts Fingerprint: FB8ACFA78C726089C38AD0269605A1098C63B92A	#1136413
aioymaps	1.2.5-1	source all	unstable	1 day	Maintainer: Home Assistant Team Changed-By: Edward Betts Fingerprint: FB8ACFA78C726089C38AD0269605A1098C63B92A	#1136418
enchant	2.8.16+dfsg-1	source amd64	unstable	19 hours	Maintainer: Debian GNOME Maintainers Changed-By: Jeremy Bícha Sponsor: jbicha@debian.org Fingerprint: 4D0BE12F0E4776D8AAACE9696E66C775AEBFE6C7D	#1120314
pytorch	2.12.0+dfsg-1~exp1	source amd64	experimental	14 hours	Maintainer: Debian Deep Learning Team Changed-By: Aron Xu Fingerprint: 3A9E7D149697510A3E37CD95C38E8160A17841FE	
mkdocs-rss-plugin	1.19.0-1	source all	unstable	11 hours	Maintainer: Debian Python Team Changed-By: Josenilson Ferreira da Silva Sponsor: merkys@debian.org Fingerprint: 772292F6F7AC85FAE041D41EE5F43F9C2734F287	#1136269
python-airly	1.1.0-1	source all	unstable	10 hours	Maintainer: Home Assistant Team Changed-By: Edward Betts Fingerprint: FB8ACFA78C726089C38AD0269605A1098C63B92A	#1136635
mstpd	0.2.0-1	source amd64	experimental	6 hours	Maintainer: Debian Networking Team Changed-By: Nadzeya Hutsko Sponsor: werdahias@debian.org Fingerprint: 14593BFF4A5EBF6FE0E9716EECBEDBB607B9B2BE	#767013

Package	Version	Arch	Distribution	Age	Upload info	Closes
python-automower-ble	0.2.8-1	source all	unstable	5 hours	Maintainer: Home Assistant Team Changed-By: Edward Betts Fingerprint: FB8ACFA78C726089C38AD0269605A1098C63B92A	#1136659
python-ayla-iot-unofficial	1.5.0-1	source all	unstable	5 hours	Maintainer: Home Assistant Team Changed-By: Edward Betts Fingerprint: FB8ACFA78C726089C38AD0269605A1098C63B92A	#1136661
python-london-tube-status	0.7-1	source all	unstable	5 hours	Maintainer: Home Assistant Team Changed-By: Edward Betts Fingerprint: FB8ACFA78C726089C38AD0269605A1098C63B92A	#1136662
python-compit-inext-api	0.8.0-1	source all	unstable	5 hours	Maintainer: Home Assistant Team Changed-By: Edward Betts Fingerprint: FB8ACFA78C726089C38AD0269605A1098C63B92A	#1136665
apt-suggest-auto	0.1	source all	unstable	2 hours	Maintainer: Lucas Nussbaum Changed-By: Lucas Nussbaum Fingerprint: FEDEC1CB337BCF509F43C2243914B532F4DFBE99	#1113974
glew	2.3.1+dfsg2-4	source arm64	unstable	2 hours	Maintainer: Alastair McKinstry Changed-By: Alastair McKinstry Fingerprint: 82383CE9165B347C787081A2CBE6BB4E5D9AD3A5	
golang-forgejo-forgejo-actions-proto	0.7.0-1	source all	unstable	57 minutes	Maintainer: Daniel Baumann Changed-By: Daniel Baumann Fingerprint: 2698683880B6A84A3D044602FBB4F0E80A80222F	
apertium-oci-spa	1.0.9-1	source all	experimental	57 minutes	Maintainer: Debian Science Maintainers Changed-By: Tino Didriksen Fingerprint: 2CAEAF9987DB153FB4BCF1C3C2DD086F4523F9D	
fonts-arcade	1.0-1	source all	unstable	27 minutes	Maintainer: Debian Fonts Task Force Changed-By: Alex Myczko Fingerprint: B60A1BF363DC1319FF0A8E89116852BCDF7515C0	#1136676

Package count in **NEW**: 38 | Total Package count: 39

Timestamp: 14.05.2026 / 19:02:25 (UTC)

There are [graphs about the queues](#) available. You can also look at the [RFC822 version](#).

Hint: Age is the youngest upload of the package, if there is more than one version.
You may want to look at [the REJECT-FAQ](#) for possible reasons why one of the above packages may get rejected.

- [LucaFalavigna](#)
- [NEWChecklist](#)

Checklist for NEW queue processing

This is a quick checklist of major points to check while processing the  [NEW queue](#). It is not meant to be an exhaustive reference.

New source package

- Is it worth including it in the archive?
 - Is it maintained upstream?
 - Is it buggy, or does it represent a security threat?
 - Can it be provided by other source packages?
 - Is it really useful?
 - Is it "big" enough to be included in the archive?
- Is source name valid?
 - Does it respect naming convention of similar packages?
 - Does it replace other software in the archive?
- Is version valid?
- Is Distribution field correct?
- Are dependencies and build-dependencies satisfied?
- Are descriptions accurate and exhaustive?
- Are binary package names valid?
 - Do they respect naming convention of similar packages?
 - Do they supersede other software in the archive?
- Are binary packages empty?
- Are binary architectures valid?
- Do files in binary packages respect  [FHS](#)?
- Does lintian emit noteworthy errors/warnings?
- Is source code DFSG-free?
 - Does it contain binaries of any kind?
 - Does it provide PDF files without corresponding source code?
 - Does it contain compressed code without corresponding plain-text version?
- Is copyright file accurate?
 - Does copyright file mention correct copyright holders list?
 - Does copyright file mention every license for every file in the source code?
 - Does it comply with  [machine readable copyright file](#), if applicable
- Is it QA safe?
 - Is source code of good quality?
 - Does it contain convenience copy of system libraries?
 - Does it violate  [Reject FAQ](#)?

- Is debian/rules correct?
- Are maintainer scripts correct?
- Are overrides correct?

New binary package

- Is there a valid reason to provide a new binary package?
- Is version valid?
- Is Distribution field correct?
- Are dependencies and build-dependencies satisfied?
- Are descriptions accurate and exhaustive?
- Is binary package name valid?
 - Does it respect naming convention of similar packages?
 - Does it supersede other software in the archive?
- Is binary package empty?
- Is binary architecture valid?
- Do files in binary packages respect  [FHS](#)?
- Does lintian emit noteworthy errors/warnings?
- Is source code DFSG-free?
- Is copyright file accurate?
- Is it QA safe?
 - Does it need missing Conflicts/Breaks/Replaces fields?
 - Does it violate  [Reject FAQ?](#)
- Are overrides correct?
- Does this package introduce new transitions?

LucaFalavigna/NEWChecklist ([last modified 2022-09-12 17:53:09](#))

Not Found

The requested URL was not found on this server.

Apache Server at ftp-master.debian.org Port 443

- [DEX](#)

Welcome to DEX - improving Debian and its derivatives through cross-community teamwork

Why DEX?

Debian is a great general-purpose operating system itself, but Debian's collection of modular packages is also an excellent platform for building  [other operating systems](#). There are  [over 300](#) distributions which are, directly or transitively, based on Debian.

 [Some of these](#) are developed within the Debian project, while many others are managed as separate projects. These child projects are self-directed, and their goals are generally focused on a specific category of users. For example,  [Ubuntu](#) aims to provide an easy-to-use operating system for PCs.

In the pursuit of their goals, derivatives make various changes and improvements to Debian packages. Often, these changes would benefit Debian and other derivatives, but do not make their way back into Debian proper.

It is hard work to coordinate a single community project. Working across projects is even harder. Although there is often a desire to cooperate, both in Debian and in derivative projects, this can be difficult in practice.

Our Approach

The  [Debian derivatives front desk](#) helps derivatives cooperate with Debian by providing informational resources, a point of contact and discussion forum for derivatives, as well as an ongoing  [census](#).

DEX works with the front desk and complements it by organizing cross-distribution working groups to monitor and merge changes from derivatives into Debian proper. In DEX, derivative developers and Debian developers work side-by-side to merge changes into Debian.

Who is DEX?

The DEX project is organized by:

- Stefano Zacchiroli
- Matt Zimmerman

Historic teams include:

- [Ubuntu](#) DEX team

Get Involved

- Join the  [debian-derivatives mailing list](#)
- Join the debian-derivatives channel on OFTC
- Read through the project pages above to see what other people are working on

Links

-  [Derivatives front desk](#)
 -  [Derivatives census](#)
-

[CategoryPackaging](#)

DEX ([last modified 2025-02-26 00:49:24](#))

- [SoftwarePackaging](#)

Unix/Linux Software Development: be Friendly to Your Packagers

On this page, 'Best Practices' for Unix / GNU/Linux software development are collected that make the packager's life easier. Software authors are not necessarily aware of the issues of software packaging, since the two are quite distinct activities - communication between the two group is therefore necessary.

User-editable vs. constant files

Everything that should be modified by the local sysadmin goes into `/etc`, everything else goes into `/usr` - so much is obvious (but see below). But what with scripts that need to be edited by the user in some cases (resolvconf package), or (HTML/mail) templates that are often edited by the user (request-tracker, mailman)?

request-tracker clearly has the best approach to this: put these files into `/usr/share` by default, but try to load them **also** from `/etc/F00`. In this way, I don't have to deal with ucf prompts on every package upgrade, but still don't have to wait for the much longer installation times for conffiles in `/etc` (mailman shows that this can take quite a while - though admittedly it's much better now than in earlier mailman versions.)

Make system directories configurable

Autconf supports a number of flags to control the location of special directories, - `-sysconfdir` for `/etc` is one well-known example. Please support these flags, and add your own if appropriate, instead of hard-coding them

Likewise, please use `TMPDIR` if it is set, instead of hard-coding `/tmp` or `/var/tmp`.

Compile time options

Every setting that has to be picked at compile time forces the packager to make a choice that rightfully should belong to the user (in most cases.) Making as much of the program behaviour as possible configurable at run time makes the packager's life easier since he doesn't have all the users telling him that he should be picking different compile time switches.

This is especially true for compile time options that cause library dependencies: for instance, instead of linking to a database library, use a database abstraction layer that can be configured at run time. Obviously, writing modular code in that sense is not trivial, and is overkill in many cases, so it's not always an obvious choice.

Installation targets / DESTDIR mechanism

While most Makefiles today are generated using autoconf/automake (or other more or less common build tools, such as ant for Java applications), there are various projects which use hand-crafted build systems. These often exhibit a problem which makes packaging pretty hard for whoever is doing the packaging.

Namely they lack support for the `DESTDIR` variable which is prepended to the intended installation location. If a file should finally reside in `/usr/bin` for example, the Makefile (in whichever way it was created) should provide a mechanism to install the file into `${DESTDIR}/usr/bin` instead. A file intended for `/etc` is likewise installed to `${DESTDIR}/etc` instead. The important part is that the program should still look for its configuration in `/etc`, the `DESTDIR` mechanism is only used so that the packager can simply bundle up anything below `${DESTDIR}` instead of locating any installed file and picking it out of the fully installed system.

Of course it is not important whether the makefile actually uses `DESTDIR` to achieve the goal or some other easily available mechanism, it is only important that it provides some means to have any file which needs to be installed to make the package work copied to a mirror of the final directory layout below some specified location.

Please don't change anything that's not within your build tree in your 'build' target. Your 'install' target, on the other hand, should not change anything within your build tree; otherwise there'll be some root-owned files lying around.

Be careful with autoconf/automake

If your package uses autoconf/automake for configuration, there are few tips you should know:

- Use `AM_MAINTAINER_MODE` in `configure.ac`: this prevents automake and autoconf from being automatically rerun by make when timestamps are broken (e.g. by applying a patch).
- Be sure to have shipped recent versions of `config.sub` and `config.guess` (though Debian packages should only use the one in the 'autotools-dev' package).
- Always try 'make distcheck' before doing a release. This catches most problems packagers would stumble upon.

Use Sensible Version Numbers

Ideally, sorting the version numbers alphanumerically should result in the correct ordering of the versions; that way, packaging tools can easily compare versions when a package is to be up- or downgraded.

Using a consistent numbering schema that makes it easy to distinguish between snapshot, beta, release candidates and final releases, and using version numbers in a

way that makes it easy to distinguish major releases from minor (bugfix) releases is a good idea, too.

"make clean" and generated files

The "make clean" and "make distclean" commands should work as intended: make clean should delete all files that are generated by "make all", and after "make distclean" everything should be like it was when the tarball was originally unpacked.

Generated files - be they object files, or generated source files - don't belong in a distributed tarball. Instead, it is vital that the Makefile includes rules to build them. Usually the only exception to this are the configure script and related files: users should be able to build the software without having the autotools installed, especially since the autotools are extremely version sensitive. Another reason for exceptions of this rule is if the tools to generate the files are either complex to set up correctly or if it is extremely time-consuming to regenerate those files.

Please use 'rm -f' in your cleanup targets so that it may proceed without user interaction, in case some files are owned by different users, or are read-only.

Communication issues

A public mailing list is a big bonus as soon as the software has more than a few users, or as soon as more than one person is developing the software - and for many small programs, when somebody decides to package it for Debian (or whatever OS), this is the first time that a second developer joins the original author. Further mailing list that could be created when the project grows: translators, announcements, packagers (the latter as a closed list with hand-picked subscribers, so that security issues can be discussed before public disclosure.) For smaller projects, it usually suffices for this purpose when a README in the software project includes an email address of the software author (keeping in mind that not only the packager may need to contact the author about security problems, but maybe also the security team of the software distribution that the packager targets.)

Use a public version control system

There are a number of free version control systems to choose from (cvs, arch, svn, ...) as well as some not-so-free ones (bitkeeper). You can get free hosting for all of them.

Version control is important because:

- You can track who contributed which part of your code.
- The Debian maintainer can extract bug fixes more easily.
- The availability of a public version control archive, even if read-only, makes for easier collaboration and encourages wider testing of new versions.
- ... you can probably think of some more reasons.

If you do use a version control system, please do not package their metadata (SCCS subdirectories, .cvsignore files, ...) in the tarball.

Version control systems generally don't deal with links well. For this and other reasons, if you do need to have links in your sources when compiling, generate them early and delete them in your "clean" target.

External libraries

Many programs make use of external libraries - after all, there's no need to reinvent the wheel every time. The distribution tar ball should not include those libraries (source code or object code), or at least should also allow an external copy of the library to be used. On one hand, including dozens if not hundreds of copies of the same library wastes a lot of space, and on the other hand, if this library has a security flaw, each copy needs to be fixed (the classical example was a security flaw in the zlib library.)

For the same reasons, dynamic linking should be preferred over static linking whenever possible: less wasted disk and memory, and a dynamic library can just be replaced on a security upgrade, while applications with statically linked libraries must be recompiled.

Persistent files

When adding a file, don't remove it with the next release only to add another one.

For example, man-pages have the bad habit of including man-pages-?version.lsm and man-pages-?version.Announce files that are removed and recreated for each and every release.

Links

- A similar page, though mostly specific to Java, is here: 
<http://java.debian.net/index.php/What%20is%20required%20from%20upstream>
- Another similar page, by ESR, targetted to the broader open-source community: 
<http://en.tldp.org/HOWTO/Software-Release-Practice-HOWTO/>
- A Rant based on experiences compiling and packaging physics software: 
<http://www.starplot.org/articles/physics-software-rant.html> (TODO: some of that could be folded into this document. Content is BSD licensed, the author tells me.)
- An excellent article on the topic was written by ?ONLamp.com at 
<http://www.onlamp.com/pub/a/onlamp/2005/03/31/packaging.html>

[CategoryPackaging](#)

- [WNPP](#)

Translation(s): [English](#) - [Português \(Brasil\)](#)



<http://www.debian.org/devel/wnpp/> - Official *Work-Needing and Prospective Packages* page.

Work-Needing and Prospective Packages (WNPP) holds list of :

- Packages in need of a new maintainer:
 - Packages up for adoption (RFA)
 - Packages up for adoption, by maintainer
 - Orphaned packages (O)
 - Packages currently being adopted
- Packages in need of help (RFH)
- Prospective packages:
 - Packages being worked on (ITP)
 - Requested packages ([RFP](#))

The difference between "packages up for adoption" and "orphaned packages" is the fact that the latter no longer has an active maintainer.

For more information on the above items, read  <http://www.debian.org/devel/wnpp/>

BTS templates

RFS template

Please note that mentors.debian.net will generate a  [template](#) for your uploaded package.

ITP template

Please note that reportbug will generate a  [template](#) (click the first result) for your ITP. Run `reportbug wnpp` and follow the prompts.

See also

-  <http://wnpp.debian.net/> - WNPP package search
-  <https://www.debian.org/devel/wnpp/prospective> - full list of WNPP packages
-  <https://wnpp-by-tags.debian.net/index.html> - Use debtags to browse WNPP packages

- [BTS](#) - The Debian Bug Tracking System
 - [Diaspora/Packaging/RFS](#) - Example of simple Request For Sponsor
-

[CategorySoftware](#) [CategoryPackaging](#)

WNPP ([last modified 2024-12-25 20:47:59](#))

- [Salsa](#)

Translation(s): [English](#) - [Italiano](#) - [Português](#) - [Português \(Brasil\)](#) - [Українська](#)

Contents

1. [What is Salsa?](#)

2. [Documentation](#)

3. [Salsa support and maintenance](#)

4. [History](#)

What is Salsa?

Salsa is a collaborative development server for Debian available at [🌐 `salsa.debian.org`](https://salsa.debian.org) (using [🌐 `GitLab`](#)). *Salsa* is supposed to provide the necessary tools for package maintainers, packaging teams and other Debian related individuals and groups for collaborative development.

Additionally `sa``lsa` is also the name of a script contained in [🌐 `devscripts`](#) package which is an indirect dependency of [🌐 `git-buildpackage`](#).

Documentation

- *Salsa* is an instance of [GitLab](#) [🌐 `community edition`](#), and offers all [🌐 `free tier features of GitLab`](#). Please see [🌐 `GitLab documentation`](#).
- [Debian specific Documentation for Salsa](#)
- [Using Salsa as identity provider](#)
- [FAQ about Salsa](#)
- [Salsa CI](#) - Continuous Integration for regression testing

Salsa support and maintenance

Translations: [English](#) - [Italiano](#) - [Português \(Brasil\)](#)

In case you encounter any problems with *Salsa*, to get support you may want to join us:

- first, check the [Frequently Asked Questions](#)
- then you can ask for help at [🖥️ `#salsa on OFTC`](#) ([🌐 `webchat`](#)) which is bridged to [🌐 `#salsa:matrix.debian.social on Matrix`](#)
- or create an issue in our [🌐 `support tracker`](#)
- or send a mail to [📧 `salsa-admin@debian.org`](mailto:salsa-admin@debian.org).

... they may help you.

[CategoryDeveloper](#) [CategoryPackaging](#)

Salsa has these administrators (IRC nick in parenthesis):

- Alexander Wirt ( [formorer](#))
- Joerg Jaspert ( [ganneff](#))
- Thomas Goirand ( [zigo](#))
- Antonio Terceiro ( [terceiro](#))

Any maintenance windows will be announced on the [DebianList: debian-infrastructure-announce](#) mailing list (low-volume).

History

After various discussions about the future of [Alioth](#), the [Alioth Sprint](#) in August 2017 gave birth to the initial setup of the upcoming *Salsa* service. The productive weekend resulted in a working prototype and was launched as a  [beta](#) in December 2017. It  [left](#) its beta status in January 2018.

Related news about that migration process:

- 2018-04-14:  [Alioth mailing lists switched to a new host](#), see [Alioth/MailingListContinuation](#) for details
- 2018-01-27:  [service left beta](#)
- 2018-01-06:  [Updates on the ongoing beta](#)
- 2017-12-15:  [service in beta](#)
- 2017-08-20: [Alioth Sprint 2017](#) results in a prototype instance of the new development platform

[CategoryDeveloper](#) [CategoryPackaging](#) [CategoryForge](#)

Salsa ([last modified 2025-06-26 09:48:19](#))

- [DebianWomen](#)

[Home](#)[Profiles](#)[History](#)[Projects](#)

[Translation\(s\):](#) [English](#) - [Français](#) - [Italiano](#) - [Português \(Brasil\)](#)

The  [Debian Women](#) project seeks to balance and diversify the Debian Project by actively engaging with interested women and encouraging them to become more involved with the Debian Project.

We promote women's involvement in Debian by

- increasing the visibility of active women,
- providing mentoring and role models,
- and creating opportunities for collaboration with new and current members of the Debian Project.

Infrastructure

- **Website:**  <https://www.debian.org/women/>

Interacting with the team

- **Read the FAQ first:**  <https://www.debian.org/women/faq>
- **Email contact:** [DebianList: debian-women@lists.debian.org](mailto:debian-women@lists.debian.org) (publicly archived)
- **IRC channel:**  [#debian-women on OFTC](#) (unregistered tor users are blocked, so create an account and set cloak on 😊)
- **Matrix:**  <https://app.element.io/#/room/#debian-women:matrix.debian.social>
- **Telegram:**  https://t.me/debian_women

DebianWomen (last modified 2025-07-18 20:57:26)

- [DebianWomen](#)
- [Projects](#)
- [Events](#)
- [TrainingSessions](#)



[Home](#)

[Profiles](#)

[History](#)

[Projects](#)

[Translation\(s\)](#): none

Contents

1. [Debian Women Training Sessions](#)
2. [Ideas and Requests for IRC Training Sessions](#)
 1. [Series : ask the ...](#)
 2. [Series : one day with the team](#)
 3. [Series : How it works in Debian](#)
 4. [packaging issues](#)
3. [Previous Training Sessions](#)
 1. [2011 Schedule](#)
 2. [2010 Schedule](#)
 3. [Training Sessions prior to 2010](#)

Debian Women Training Sessions

This page discusses both future and past training sessions organised by the Debian Women project.

In accordance with the aims of the Debian Women project, the aim of this initiative is to encourage more women to contribute to Debian while introducing them to different aspects of the Debian Project. However, the sessions are open to *all* who wish to attend, irrespective of age, gender or experience level. The Debian Women project expects attendees behave in a respectful manner to each other and to please bear in mind minors might be present.

The training is led by experienced community members and provides useful information for people new to Debian, as well as helping more experienced Debian members brush up on their skills or improve their skill set. Particular emphasis is however given to the trainee.

The training sessions are always held on our IRC channel, `#debian-women` on `irc.debian.org`, where female attendees, both trainee and experienced, can be assured the same respect as their male counterparts within the Debian community.

The language of the training sessions is English. The sessions are logged: you can find the logs in a subdirectory of [here](#).

Questions can be asked and are encouraged, but, please, **do not** ask them in the main `#debian-women` channel. Please **do** ask them in the separate question channel, `#dw-question`. Your question will then join a queue with others and be presented to both the trainer and the main channel one by one. The use of `#dw-question` in this manner reduces disruption to the main channel and allows the trainer to be organised when answering questions.

Sessions are open to everybody, regardless of gender or previous involvement in the community.

Training session dates and times will be announced here and in the `#debian-women` channel. Any times stated will be UTC unless stated otherwise.

Please add your requests for training sessions, or indicate your willingness to conduct a training session [here](#)

Ideas and Requests for IRC Training Sessions

Please either add here your request for new training topics or to sign up yourself as a trainer: we are always looking for more people to share knowledge and complete the schedule: so, don't be shy!

✓ = session confirmed by trainer ⓘ = trainer already contacted, but she/he hasn't confirmed yet

- The Debian project, organization and history. ⓘ
 - Trainer: Biella Coleman

Series : ask the ...

- "Ask the DPL": what exactly the [DebianProjectLeader](#) does: according to constitution, folklore, and personal experience; question/answer session ✓
 - Trainer: [StefanoZacchioli](#)
- "Ask the DSA" : what exactly Debian System Administrators do: according to constitution, folklore, and personal experience; question/answer session
 - Trainer: *we could ask zobel*
- Ask the DAM,
- Ask the (president of) CTTE,
- Ask the secretary

- Ask the FTP-*

Series : one day with the team

- "One Day in the Debian Perl Group" ✓
 - Trainer: Gregor Herrmann
- Something from the debian-games team
 - Trainer: *we could ask Miriam Ruiz*
- Kde team
- Webmaster team
- Publicity team
- DSA
- i18n
- QA team

Series : How it works in Debian

- How internationalization works in Debian (for translators as well as developers) ⓘ
 - Trainer: Christian Perrier (*Christian can't do it for now, but maybe he can after the release*)

programming practices

- Secure programming practices
- Using quilt
- Advanced use of Git
 - Trainer: *we could ask Martin Krafft (madduck)* (suggested by Enrico Zini)

bug triage

- Simple things to begin with
- Dealing with upstream

packaging issues

- Common packaging issues
 - Maintaining packages with git-buildpackage
 - Java packaging
 - Moreutils, debhelper
 - Trainer: Joey Hess (suggested by Enrico Zini)
-

Previous Training Sessions

Details of previous training sessions now follow.

2011 Schedule

Topic	Trainer	Date/Period	Hour	Duration
<i>Build It</i>	Margarita Manterola	Saturday, 7th May	11:00 UTC & 22:00 UTC	1.5-2 hours

Build It

- Trainer: [Margarita Manterola](#)
- Duration: 1.5-2 hours
- Date: Saturday, 7th May
- IRC organisation : Margarita Manterola
- Log:  <http://meetbot.debian.net/debian-women/2011/debian-women.2011-05-07-11.00.log.html>
 <http://meetbot.debian.net/debian-women/2011/debian-women.2011-05-07-22.02.log.html>
- Tutorial: [Building Tutorial](#) from log by ?MonicaRamirezArceda and [Margarita Manterola](#)

2010 Schedule

Here is a list of the training sessions held in 2010 with further information, including links to the logs of and wiki tutorials made from the sessions, following below the table.

Topic	Trainer	Date/Period	Hour	Duration
<i>Introduction to Debian</i>	Lars	Thursday, 18th	20:00	2 hours
<i>Packaging</i>	Wirzenius	November	UTC	
<i>Using Git</i>	David	Thursday, 25th	21:00	2.5
	Paleino	November	UTC	hours
<i>Python</i>	Piotr	Thursday, 2nd	20:00	2 hours
<i>libraries/application packaging</i>	Ożarowski	December	UTC	
<i>How to use the Bug Tracking System</i>	Gerfried	Thursday, 9th	19:00	3 hours
<i>Debian package information</i>	Fuchs	December	UTC	
	Enrico Zini	Thursday, 16th	20:00	3 hours
		December	UTC	

Packaging

- Trainer: [LarsWirzenius](#)
- Duration: 2 hours
- Date: Thursday, 18th November

- IRC organisation : MadameZou and dapal
- Log:  <http://meetbot.debian.net/debian-women/2010/debian-women.2010-11-18-20.05.log.html>
- Tutorial: [Introduction to Debian Packaging](#) from log by [FrancescaCiceri](#)

Using git

- Trainer: [DavidPaleino](#)
- Duration: 2.5 hours
- Date: 25th November
- IRC organisation : MadameZou and aghisla
- Log:  <http://meetbot.debian.net/debian-women/2010/debian-women.2010-11-25-21.00.log.html>
- Tutorial: [Using git](#) from log by ?MonicaRamirezArceda

Python libraries/application packaging

- Trainer: [Piotr Ożarowski](#)
- Date: Thursday, 2th December
- IRC organisation : MadameZou
- Log:  <http://meetbot.debian.net/debian-women/2010/debian-women.2010-12-02-20.02.log.html>
- Tutorial: [Python modules and applications packaging](#) from log by ?MonicaRamirezArceda

How to use the BTS (Bug Tracking System)

- Trainer: [GerfriedFuchs](#)
- Date: Thursday, 9th December
- IRC organisation : MadameZou and aghisla
- Log:  <http://meetbot.debian.net/debian-women/2010/debian-women.2010-12-09-19.06.log.html>
- Tutorial: [How to use the Bug Tracking System](#) from log by ?MonicaRamirezArceda and [FrancescaCiceri](#)

Debian Package Information

- Trainer: [EnricoZini](#)
- Date: Thursday, 16th December
- IRC organisation : MadameZou and dapal
- Log:  <http://meetbot.debian.net/debian-women/2010/debian-women.2010-12-16-20.09.log.html>
- Tutorial: [Debian Package Information](#) from log by ?MonicaRamirezArceda

- Note: the session covers various aspects about Debian packages as packages file, debtags, popcon, Debian Weather, etc

Training Sessions prior to 2010

Sessions prior to 2010 have included :

- [AdvancedBuildingTips](#)
- [BuildingTutorial](#)
- [Courses2005/BuildingWithoutHelper](#)
- Create a dummy package to [hack dependencies](#)
- [BTS/HowTo](#)
- [PackagingTutorial](#)

DebianWomen/Projects/Events/TrainingSessions ([last modified 2012-11-25 12:03:55](#))